PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Programa de Pós-graduação em Informática

Matheus Alcântara Souza

WAY-REPLACEMENT ALGORITHMS FOR MULTICORE PROCESSORS BASED ON COHERENCE AND SHARING STATES WITH REINFORCEMENT LEARNING

Belo Horizonte
2021

Matheus Alcântara Souza

WAY-REPLACEMENT ALGORITHMS FOR MULTICORE PROCESSORS BASED ON COHERENCE AND SHARING STATES WITH REINFORCEMENT LEARNING

Thesis presented in partial fulfillment of the requirements for the degree of Doctor in Informatics.

Advisor: Prof. Henrique Cota de Freitas, Dr. - PUC Minas (Advisor)
Research field: Computer architecture.

Belo Horizonte
2021

FICHA CATALOGRÁFICA
Elaborada pela Biblioteca da Pontifícia Universidade Católica de Minas Gerais

Souza, Matheus Alcântara
S729w Way-replacement algorithms for multicore processors based on coherence and sharing states with reinforcement learning / Matheus Alcântara Souza. Belo Horizonte, 2021.
181 f. : il.

Orientador: Henrique Cota de Freitas
Tese (Doutorado) – Pontifícia Universidade Católica de Minas Gerais. Programa de Pós-Graduação em Informática

1. Arquitetura de computador. 2. Multiprocessadores. 3. Memória cache. 4. Gerenciamento de memória (Computação). 5. Computação de alto desempenho. 6. Algorítmos computacionais. 7. Aprendizado do computador. I. Freitas, Henrique Cota de. II. Pontifícia Universidade Católica de Minas Gerais. Programa de Pós-Graduação em Informática. III. Título.

SIB PUC MINAS

CDU: 681.3-11

Ficha catalográfica elaborada por Fabiana Marques de Souza e Silva - CRB 6/2086

Matheus Alcântara Souza

WAY-REPLACEMENT ALGORITHMS FOR MULTICORE PROCESSORS BASED ON COHERENCE AND SHARING STATES WITH REINFORCEMENT LEARNING

Thesis presented in partial fulfillment of the requirements for the degree of Doctor in Informatics.

Research field: Computer architecture.

Prof. Henrique Cota de Freitas, Dr. - PUC Minas (Advisor)

Prof. Alba Cristina M. A. de Melo, Dr. - UnB (Examining Committee)

Prof. Ricardo dos Santos Ferreira, Dr. - UFV (Examining Committee)

Prof. Luís Fabrício Wanderley Góes, Dr. - Univ. Leicester (Examining Committee)

Prof. Carlos Augusto P. S. Martins, Dr. - PUC Minas (Examining Committee)

Belo Horizonte, October 26th, 2021

Sempre por minha família, por tudo, em especial à Valéria, meu porto seguro.

AGRADECIMENTOS

Em primeiro lugar, agradeço a toda a minha família. Aos meus pais, Anelize e João, agradeço por sempre estarem presentes ao meu lado com atenção e carinho, com suporte imensurável, me mostrando que agir com humildade e razão é possível. À minha irmã, Mariane, obrigado por compartilhar comigo bons momentos e ser tão especial, me apoiando desde sempre. À Véia Coroca, valeu por todas as horas de descontração e por ter ajudado meus pais em nossa criação, você é eterna! Ao meu eterno amor, Valéria, obrigado por me amar, me apoiar, me suportar! Te agradeço por fazer parte disso tudo, e por ser parte de mim! Aos bichinhos queridos, em especial Tartufo, Jaina, Leka e Moa – os animais nos ensinam muito sobre lealdade e perdão, coisas que somos incapazes de reproduzir igual a eles. Amo vocês!

Agradeço demais ao Professor Henrique, que tive o privilégio de ter como orientador. Pela orientação em todos os sentidos, pelo profissionalismo e por compartilhar toda sua sabedoria, mas principalmente por se tornar um grande amigo! Obrigado, mais uma vez, pelos diálogos e discussões, e pelo tratamento igualitário e completo à todos seus alunos.

Agradeço aos membros da banca, que contribuíram com valorosas considerações e críticas. Aos membros do PPGINF e funcionários da PUC Minas, obrigado por trabalharem com tanta dedicação em prol do bem estar dos alunos. Agradecimento especial à "mamãe" Giovana, que sabe como reduzir as tensões do dia a dia dos alunos! Também aos amigos e colegas do PPGINF e do CArT (Pedro Penna, Pedro Ruas, Lucas, João Paulo, e vários outros nomes que não caberiam aqui), obrigado pela parceria de sempre. Desejo sucesso à todos!

À CAPES, FAPEMIG e CNPq, instituições sem as quais a pesquisa e desenvolvimento no país não seria possível. À UGA, Grenoble INP, INRIA, e TIMA-SLS, instituições francesas que colaboraram para a internacionalização do trabalho. Aos professores Frédéric Pétrot e Jean-François Méhaut, pelo apoio concedido durante os períodos na França. À Câmara Municipal de Belo Horizonte e aos colegas de trabalho. Agradeço pela compreensão e apoio durante toda a minha trajetória.

Por fim, agradeço a superioridade sobrenatural que nos governa, seja ela qual for, pela capacidade de prosseguir pelas etapas da vida, com saúde e resiliência para seguir em frente, ultrapassando as dificuldades, e também a tudo e todos que, de alguma forma, fizeram parte e contribuíram com mais essa jornada.

PREFACE

This thesis presents the research I have done during my Ph.D. (2016-2021) studies at the Computer Architecture and Parallel Processing Team (CArT), PUC Minas. My research path in this area began when I started my M.Sc. (2013-2015) and, under the supervision of Professor Henrique C. Freitas, we have been reaping the fruits of our work so far. Among other projects, we participated in the Intel Hardware Accelerator Research Program (HARP), in which I attended the program workshop in San Jose, USA, in February 2017. At that time, I personally also discussed some research with Professor Viktor Prasanna at the University of Southern California. Later, starting in August 2018, I did a six-month doctoral internship in Grenoble, France. It was carried out within the System Level Synthesis (SLS) group at the Laboratoire TIMA, with the collaboration of Professor Frédéric Pétrot. These were crucial elements that allowed me to complete my Ph.D. thesis and made an essential contribution to improving my expertise in computer architecture. Additionally, I participated in other conferences and workshops, such as CSBC (2014), SBAC-PAD (2014, 2015, 2017, 2018, 2019), WSCAD (2014, 2015, 2017, 2019), Euro-Par (2016), and ISCA (2020). These works have yielded some direct and indirect publications, which are listed next.

Publications on Computer Architecture directly related to the subject of the thesis

- **Coherence State Awareness in Way-Replacement Algorithms for Multicore Processors.** Matheus A. Souza, Henrique C. Freitas, Frédéric Pétrot. *Simpósio de Sistemas Computacionais de Alto Desempenho (WSCAD).* Campo Grande, 2019.
- **Design Space Exploration of Energy Efficient NoC- and Cache-based Many-Core Architectures.** Matheus A. Souza, Henrique C. Freitas, Jean-François Méhaut. Workshop on Applications for Multi-Core Architectures (WAMCA). Lyon, 2018.
- **Energy consumption improvement of shared-cache multicore clusters based on explicit simultaneous multithreading.** Matheus A. Souza, Túlio T. Cota, Matheus M. Queiroz, Henrique C. Freitas. International Symposium on Computer Architecture and High Performance Computing Workshops (SBAC-PAD). Campinas, 2017.
- **CAP Bench: a benchmark suite for performance and energy evaluation of low-power many-core processors.** Matheus A. Souza, Pedro H. Penna, Matheus M. Queiroz, Alyson D. Pereira, Luís F.W. Góes, Henrique C. Freitas, Márcio Castro, Philippe O.A. Navaux, Jean-François Méhaut. Concurrency and Computation: Practice and Experience. 2017.
- **Avaliação de Topologias de Redes-em-Chip usando Simulação de Sistemas Completos e Aplicações Paralelas.** Daniel A.S. Carmo, Matheus A. Souza, Henrique C. Freitas. Simpósio em Sistemas Computacionais de Alto Desempenho (WSCAD-SSC). Aracajú, 2016.
- **A low-cost energy-efficient Raspberry Pi cluster for data mining algorithms.** João Saffran, Gabriel Garcia, Matheus A. Souza, Pedro H. Penna, Márcio Castro, Luís F. W. Góes, Henrique C. Freitas. Euro-Par: Parallel Processing Workshops. Paris, 2016. Lecture Notes in Computer Science. 2017.

- **Avaliação do Consumo Energético em Arquiteturas Multi-Core com Memória Cache Compartilhada.** Matheus A. Souza, Marco A. Z. Alves, Henrique C. Freitas, Philippe O. A. Navaux. Workshop em Desempenho de Sistemas Computacionais e de Comunicação (WPerformance). Brasília, 2014.

Publications on Computer Architecture and Machine Learning

- **Reconfigurable FPGA-based K-means/K-modes Architecture for Network Intrusion Detection.** Lucas A. Maciel, Matheus A. Souza, Henrique C. Freitas. Transactions on Circuits and Systems II: Express Briefs. 2019.
- **Arquitetura Heterogênea CPU+ FPGA para Análise Formal de Conceitos.** Lucas A. Maciel, João P.P. Novais, Matheus A. Souza, Mark A.J. Song, Henrique C. Freitas Simpósio de Sistemas Computacionais de Alto Desempenho (WSCAD) Campo Grande, 2019.
- **Energy Efficient Parallel K-Means Clustering for an Intel® Hybrid Multi-Chip Package.** Matheus A. Souza, Lucas A. Maciel, Pedro H. Penna, Henrique C. Freitas. High Performance Machine Learning Workshop (HPML). Lyon, 2018.
- **Projeto e Avaliação de uma Arquitetura do Algoritmo de Clusterização K-means em VHDL e FPGA.** Lucas A. Maciel, Matheus A. Souza, Henrique C. Freitas. Anais do XVIII Simpósio em Sistemas Computacionais de Alto Desempenho. Campinas, 2017.

Publications not directly related to the thesis, in partnership with CArT members

- **RMem: An OS Service for Transparent Remote Memory Access in Lightweight Manycores.** Pedro H. Penna, Matheus A. Souza, Emmanuel P. Junior, João Souto, Márcio Castro, François Broquedis, Henrique C. Freitas, Jean-François Mehaut. International Workshop on Programmability and Architectures for Heterogeneous Multicores (MultiProg). Valencia, 2019.
- **An OS Service for Transparent Remote Memory Accesses in NoC-Based Lightweight Manycores.** Pedro H. Penna, Matheus A. Souza, Emmanuel P. Junior, Bruno Nascimento, Márcio Castro, François Broquedis, Henrique C. Freitas, Jean-François Mehaut. International Symposium on Networks-on-Chip (NOCS). Torino, 2018.
- **Proposta e Avaliação de uma Rede-em-Chip Programável.** João Paulo P. Novais, Matheus Alcântara Souza, Henrique Cota Freitas. Simpósio em Sistemas Computacionais de Alto Desempenho - Workshop de Iniciação Científica (WSCAD-WIC). Campinas, 2017.
- **Projeto em VHDL de um processador de rede intra-chip.** João Paulo P. Novais, Matheus Alcântara Souza, Henrique Cota Freitas. Simpósio em Sistemas Computacionais de Alto Desempenho - Workshop de Iniciação Científica (WSCAD-WIC). Aracajú, 2016.

Título da tese: ALGORITMOS DE SUBSTITUIÇÃO PARA PROCESSADORES MULTICORE BASEADOS EM COERÊNCIA E APRENDIZADO POR REFORÇO

RESUMO

O uso de Computação de Alto Desempenho (HPC) está cada vez mais difundido em muitas áreas, inclusive pelo tamanho crescente dos dados a serem processados. Os supercomputadores empregam aceleradores, sistemas de memória híbridos, sistemas de interconexão altamente aprimorados e outras tecnologias para alcançar a escala de computação em exaflops. No entanto, o caminho para esse marco contém vários desafios, como restrições de consumo energético, adaptação de algoritmos e as restrições do sistema de memória. Mais precisamente, as tecnologias de memória não acompanharam o avanço dos processadores.

O conceito de hierarquia de memória e caches foi proposto anos atrás para reduzir essa lacuna. O uso de caches permitiu uma computação mais rápida, com dados providos em caches menores e mais rápidas, reduzindo assim a necessidade de acesso a memórias subjacentes mais lentas. No entanto, uma cache não pode armazenar todos os dados ao mesmo tempo. Portanto, uma estratégia para substituir os blocos de cache é necessária: a política de substituição. O ideal é atingir as maiores taxas de acerto possíveis, o que requer conhecimento prévio do que será processado para a tomada de decisões. O algoritmo de Belády se propõe a isso, porém, os sistemas atuais não podem implementá-lo justamente pela necessidade de previsão futura do uso dos dados.

Outro desafio encontrado nos sistemas de memória é o problema de coerência, em razão do cenário multiprocessado. Infelizmente, essa solução leva a uma nova categoria de conflitos: *coherence misses*. Ou seja, é o despejo de um bloco da cache como consequência das transições de coerência. Apesar disso, o comportamento relacionado à coerência apresenta uma oportunidade para tomar melhores decisões ao manipular dados em caches.

Nesta tese, o objetivo é investigar diversos algoritmos de substituição em caches, aprimorando-os com diferentes estratégias de despejo. A primeira contribuição é uma estratégia que modifica esses algoritmos para incluir a informação do estado de coerência em seu processo de decisão. A segunda é uma abordagem que altera esses algoritmos considerando o número de núcleos que compartilham um bloco de cache antes do despejo. Por fim, uma vez que a substituição é naturalmente um processo de tomada de decisão, propõe-se o uso do Aprendizado por Reforço para aprender com despejos anteriores e decidir novos.

As propostas apresentadas foram avaliadas usando o simulador Sniper, executando aplicações do CAP Benchmarks. A análise realizada contém diferentes perspectivas, levando em consideração, por exemplo, a associatividade e estratégia de compartilhamento de caches. Os resultados coletados mostraram que nossas propostas alcançam um desempenho comparável ou superior em relação as políticas de substituição originais em vários casos. Por exemplo, alguns dos primeiros resultados foram 69.04% e 68.07% de redução no *miss rate* na cache L2.

Além disso, a abordagem usando Aprendizado por Reforço e também o vetor de bits de compartilhadores reduziu o *miss rate* na cache L1 por 11.50% em média, alcançando até 41.20% de redução. Esses resultados apresentaram até 27.33% de ganhos em Instruções por Ciclos e 20.36% menos energia consumida do que a versão original das políticas. Com essas considerações, conclui-se que informações relacionadas a coerência do sistema de memória podem ser usadas para melhorar o desempenho de políticas de substituição em cache.

Palavras-chave: multicore; simulação; caches; substituição; aprendizado por reforço.

ABSTRACT

The use of High-Performance Computing (HPC) has become increasingly widespread across many areas. HPC must keep up with the ever-increasing sizes of data to be processed to offer quick responses. Supercomputers employ accelerators, hybrid memory systems, highly improved interconnection systems, and other technologies to achieve Exascale Computing. However, the road to this milestone contains multiple challenges, such as the power wall, the need to adapt algorithms, and the memory system constraints. More precisely, memory technologies in computer systems could not follow the advance of processors.

The memory hierarchy, whose main components are caches, was proposed years ago to reduce this gap. The use of a cache hierarchy enabled faster computation due to data being provided first in small but fast caches, thus, reducing the need to access the underlying slower memories. However, the use of memory hierarchies brings new challenges, one of which is how to use caches efficiently. As is common in technological research, these improvements bring new challenges. For example, the cache cannot store all the data at the same time. Thus, data requests might not be satisfied in a specific time, which is a cache miss. Therefore, a strategy to replace cache blocks is required: the way-replacement policy. The ideal one would achieve the highest possible hit rates, which requires future knowledge to make decisions. Béládyʼs algorithm achieves this. However, current systems cannot implement it.

Another challenge in the memory system, mainly due to the multicore scenario, is the coherence problem. The architecture should guarantee that data is updated while being used by multiple cores through coherence protocols. Unfortunately, this solution leads to a new category of conflicts: coherence misses. That is the eviction of a block as a consequence of coherence transitions. Despite this, coherence-related behavior presents an opportunity to make better decisions when manipulating data in caches.

This way, we understand that way-replacement algorithms can use the coherence state and the sharers' bit-vector of a cache block as input for better decisions in way-replacement algorithms. In this thesis, our goal was to investigate several way-replacement algorithms in caches, improving them with different eviction strategies. The first contribution is a strategy that modifies these algorithms to include the coherence state awareness in their decision process. The second is an approach that changes these algorithms, considering the number of cores that share a cache block before the eviction. Third, since way-replacement is naturally a decision-making process, we propose using Reinforcement Learning to learn from past evictions and decide new ones.

We evaluate these proposals using the Sniper multicore simulator, running workloads from a set of kernels in CAP Benchmarks. Our analysis contains different perspectives, taking into account, for instance, the associativity and sharing strategy of caches. Nevertheless, the results we gathered showed that our proposals could achieve comparable performance or outperform traditional way replacement policies in several cases. For instance, some of our first results reached 69.04% and 68.07% miss rate reduction in the L2 cache.

Moreover, the Reinforcement Learning-based approach using the sharers' bit-vector reduced the L1 cache miss rate by 11.50% on average, reaching a 41.20% reduction. These results led to up to 27.33% IPC gain and 20.36% less energy consumed over the baseline. With those remarks, we concluded that we could use the coherence information of cache blocks to enhance the performance of way-replacement policies.

Keywords: multicore; simulation; caches; way-replacement; reinforcement learning.

LIST OF FIGURES

FIGURE 2.1 - Shared memory 26
FIGURE 2.2 - Distributed memory 26
FIGURE 2.3 - Directed mapped cache 30
FIGURE 2.4 - Block search in cache mapping strategies 31
FIGURE 2.5 - Coherence problem in a dual-core processor 33
FIGURE 2.6 - State transition diagram for the MESI protocol 35
FIGURE 2.7 - Cache coherence transitions 36
FIGURE 2.8 - Cache coherence transitions 37
FIGURE 2.9 - Cache coherence transitions 37
FIGURE 2.10 - Reinforcement Learning diagram 38

FIGURE 3.1 - A taxonomy of cache way-replacement policies 41

FIGURE 4.1 - CAP Bench kernels - cache accesses and misses 60

FIGURE 5.1 - Overview of the coherence state-based eviction 63
FIGURE 5.2 - Cache miss rate vs. associativity – FAST 64
FIGURE 5.3 - Cache miss rate vs. associativity – FN 65
FIGURE 5.4 - Cache miss rate vs. associativity – GF 66
FIGURE 5.5 - Cache miss rate vs. associativity - IS 67
FIGURE 5.6 - Cache miss rate vs. associativity - KM 67
FIGURE 5.7 - Cache miss rate vs. associativity - LU 68
FIGURE 5.8 - Cache miss rate vs. associativity – TSP 69
FIGURE 5.9 - Cache miss rate vs. block size - FAST 70
FIGURE 5.10 - Cache miss rate vs. block size - FN 70
FIGURE 5.11 - Cache miss rate vs. block size - GF 71
FIGURE 5.12 - Cache miss rate vs. block size - IS 71
FIGURE 5.13 - Cache miss rate vs. block size - KM 72
FIGURE 5.14 - Cache miss rate vs. block size - LU 73
FIGURE 5.15 - Cache miss rate vs. block size - TSP 73
FIGURE 5.16 - L2 cache miss rate vs. sharing strategy - FAST 74

FIGURE 5.17 - L2 cache miss rate vs. sharing strategy - KM 74
FIGURE 5.18 - L2 cache miss rate vs. sharing strategy - FN 75
FIGURE 5.19 - L2 cache miss rate vs. sharing strategy - GF 75
FIGURE 5.20 - L2 cache miss rate vs. sharing strategy - IS 76
FIGURE 5.21 - L2 cache miss rate vs. sharing strategy - LU 76
FIGURE 5.22 - L2 cache miss rate vs. sharing strategy - TSP 77
FIGURE 5.23 - L1 cache miss rate decrease 77
FIGURE 5.24 - L2 cache miss rate decrease 78
FIGURE 5.25 - Coherence invalidates decrease 79
FIGURE 5.26 - Coherence write-backs decrease 80
FIGURE 5.27 - IPC increase 81
FIGURE 5.28 - Cycle decrease 81
FIGURE 5.29 - Energy consumption decrease 82

FIGURE 6.1 - Overview of the sharer count-based eviction strategy 85
FIGURE 6.2 - Cache miss rate vs. associativity - FAST 86
FIGURE 6.3 - Cache miss rate vs. associativity - FN 87
FIGURE 6.4 - Cache miss rate vs. associativity - GF 87
FIGURE 6.5 - Cache miss rate vs. associativity - IS 88
FIGURE 6.6 - Cache miss rate vs. associativity - KM 88
FIGURE 6.7 - Cache miss rate vs. associativity - LU 89
FIGURE 6.8 - Cache miss rate vs. associativity - TSP 90
FIGURE 6.9 - Cache miss rate vs. block size - FAST 90
FIGURE 6.10 - Cache miss rate vs. block size - FN 91
FIGURE 6.11 - Cache miss rate vs. block size - GF 92
FIGURE 6.12 - Cache miss rate vs. block size - IS 92
FIGURE 6.13 - Cache miss rate vs. block size - KM 93
FIGURE 6.14 - Cache miss rate vs. block size - LU 94
FIGURE 6.15 - Cache miss rate vs. block size - TSP 94
FIGURE 6.16 - L2 cache miss rate vs. sharing strategy - FAST 95
FIGURE 6.17 - L2 cache miss rate vs. sharing strategy - FN 95
FIGURE 6.18 - L2 cache miss rate vs. sharing strategy - GF 96
FIGURE 6.19 - L2 cache miss rate vs. sharing strategy - IS 96

FIGURE 6.20 - L2 cache miss rate vs. sharing strategy - KM 97
FIGURE 6.21 - L2 cache miss rate vs. sharing strategy - LU 97
FIGURE 6.22 - L2 cache miss rate vs. sharing strategy - TSP 98
FIGURE 6.23 - L1 cache miss rate decrease 98
FIGURE 6.24 - L2 cache miss rate decrease 99
FIGURE 6.25 - Coherence invalidates decrease 100
FIGURE 6.26 - Coherence write-backs decrease 101
FIGURE 6.27 - IPC increase 101
FIGURE 6.28 - Cycle decrease 102
FIGURE 6.29 - Energy consumption decrease 103

FIGURE 7.1 - Overview of the RL-based eviction strategy 105
FIGURE 7.2 - Cache miss rate vs. associativity - FAST 107
FIGURE 7.3 - Cache miss rate vs. associativity - FN 108
FIGURE 7.4 - Cache miss rate vs. associativity - GF 108
FIGURE 7.5 - Cache miss rate vs. associativity - IS 109
FIGURE 7.6 - Cache miss rate vs. associativity - KM 110
FIGURE 7.7 - Cache miss rate vs. associativity - LU 110
FIGURE 7.8 - Cache miss rate vs. associativity - TSP 111
FIGURE 7.9 - Cache miss rate vs. block size - FAST 111
FIGURE 7.10 - Cache miss rate vs. block size - FN 112
FIGURE 7.11 - Cache miss rate vs. block size - GF 112
FIGURE 7.12 - L1 cache miss rate vs. block size - IS 113
FIGURE 7.13 - L1 cache miss rate vs. block size - KM 114
FIGURE 7.14 - L1 cache miss rate vs. block size - LU 114
FIGURE 7.15 - L1 cache miss rate vs. block size - TSP 115
FIGURE 7.16 - L2 cache miss rate vs. sharing strategy - FAST 115
FIGURE 7.17 - L2 cache miss rate vs. sharing strategy - FN 116
FIGURE 7.18 - L2 cache miss rate vs. sharing strategy - GF 116
FIGURE 7.19 - L2 cache miss rate vs. sharing strategy - IS 117
FIGURE 7.20 - L2 cache miss rate vs. sharing strategy - KM 117
FIGURE 7.21 - L2 cache miss rate vs. sharing strategy - LU 118
FIGURE 7.22 - L2 cache miss rate vs. sharing strategy - TSP 118

FIGURE 7.23 - L1 cache miss rate decrease 119
FIGURE 7.24 - L2 cache miss rate decrease 120
FIGURE 7.25 - Coherence invalidates decrease 121
FIGURE 7.26 - Coherence write-backs decrease 122
FIGURE 7.27 - IPC increase 122
FIGURE 7.28 - Cycle decrease 123
FIGURE 7.29 - Energy consumption decrease 124
FIGURE 7.30 - Simple cache example 126
FIGURE 7.31 - Distribution of the best cache miss results 129
FIGURE 7.32 - Distribution of the best coherence operations results 130
FIGURE 7.33 - Distribution of the best IPC and energy consumption results 131

FIGURE B.1 - L2 cache miss rate vs. associativity - FAST 175
FIGURE B.2 - L2 cache miss rate vs. associativity - FN 176
FIGURE B.3 - L2 cache miss rate vs. associativity - GF 177
FIGURE B.4 - L2 cache miss rate vs. associativity - IS 178
FIGURE B.5 - L2 cache miss rate vs. associativity - KM 179
FIGURE B.6 - L2 cache miss rate vs. associativity - LU 180
FIGURE B.7 - L2 cache miss rate vs. associativity - TSP 181

LIST OF TABLES

TABLE 3.1 - Comparison between related work on ML-based way-replacement . . . 53

TABLE 4.1 - Cache hierarchy setup 57

TABLE 4.2 - Complexity comparison of way-replacement policies 57

TABLE 4.3 - Characteristics of each application available in CAP Bench. 60

TABLE 4.4 - Simulation parameters 62

TABLE 7.1 - Additional bits required by the strategies 125

TABLE 7.2 - Total size of the cache using the strategies 127

TABLE 7.3 - Summary of cache miss rate results 129

TABLE 7.4 - Summary of coherence operations results 130

TABLE 7.5 - Summary of performance results 131

TABLE A.1 - L1 cache miss rate vs. associativity - FAST 147

TABLE A.2 - L2 cache miss rate vs. associativity - FAST 147

TABLE A.3 - L1 cache miss rate vs. associativity - FN 148

TABLE A.4 - L2 cache miss rate vs. associativity - FN 148

TABLE A.5 - L1 cache miss rate vs. associativity - GF 149

TABLE A.6 - L2 cache miss rate vs. associativity - GF 149

TABLE A.7 - L1 cache miss rate vs. associativity - IS 150

TABLE A.8 - L2 cache miss rate vs. associativity - IS 150

TABLE A.9 - L1 cache miss rate vs. associativity - KM 151

TABLE A.10 - L2 cache miss rate vs. associativity - KM 151

TABLE A.11 - L1 cache miss rate vs. associativity - LU 152

TABLE A.12 - L2 cache miss rate vs. associativity - LU 152

TABLE A.13 - L1 cache miss rate vs. associativity - TSP 153

TABLE A.14 - L2 cache miss rate vs. associativity - TSP 153

TABLE A.15 - L1 cache miss rate vs. block size - FAST 154

TABLE A.16 - L2 cache miss rate vs. block size - FAST 154

TABLE A.17 - L1 cache miss rate vs. block size - FN 155

TABLE A.18 - L2 cache miss rate vs. block size - FN 155

TABLE A.19 - L1 cache miss rate vs. block size - GF 156
TABLE A.20 - L2 cache miss rate vs. block size - GF 156
TABLE A.21 - L1 cache miss rate vs. block size - IS 157
TABLE A.22 - L2 cache miss rate vs. block size - IS 157
TABLE A.23 - L1 cache miss rate vs. block size - KM 158
TABLE A.24 - L2 cache miss rate vs. block size - KM 158
TABLE A.25 - L1 cache miss rate vs. block size - LU 159
TABLE A.26 - L2 cache miss rate vs. block size - LU 159
TABLE A.27 - L1 cache miss rate vs. block size - TSP 160
TABLE A.28 - L2 cache miss rate vs. block size - TSP 160
TABLE A.29 - L2 cache miss rate vs. sharing strategy - FAST 161
TABLE A.30 - L2 cache miss rate vs. sharing strategy - FN 161
TABLE A.31 - L2 cache miss rate vs. sharing strategy - GF 162
TABLE A.32 - L2 cache miss rate vs. sharing strategy - IS 162
TABLE A.33 - L2 cache miss rate vs. sharing strategy - KM 162
TABLE A.34 - L2 cache miss rate vs. sharing strategy - LU 163
TABLE A.35 - L2 cache miss rate vs. sharing strategy - TSP 163
TABLE A.36 - L1 cache miss rate - Overall comparison - FAST 164
TABLE A.37 - L2 cache miss rate - Overall comparison - FAST 164
TABLE A.38 - L1 cache miss rate - Overall comparison - FN 164
TABLE A.39 - L2 cache miss rate - Overall comparison - FN 164
TABLE A.40 - L1 cache miss rate - Overall comparison - GF 165
TABLE A.41 - L2 cache miss rate - Overall comparison - GF 165
TABLE A.42 - L1 cache miss rate - Overall comparison - IS 165
TABLE A.43 - L2 cache miss rate - Overall comparison - IS 165
TABLE A.44 - L1 cache miss rate - Overall comparison - KM 166
TABLE A.45 - L2 cache miss rate - Overall comparison - KM 166
TABLE A.46 - L1 cache miss rate - Overall comparison - LU 166
TABLE A.47 - L2 cache miss rate - Overall comparison - LU 166
TABLE A.48 - L1 cache miss rate - Overall comparison - TSP 167
TABLE A.49 - L2 cache miss rate - Overall comparison - TSP 167
TABLE A.50 - Coherence operations - FAST 167

TABLE A.51 - Coherence operations - FN 168
TABLE A.52 - Coherence operations - GF 168
TABLE A.53 - Coherence operations - IS 169
TABLE A.54 - Coherence operations - KM 169
TABLE A.55 - Coherence operations - LU 170
TABLE A.56 - Coherence operations - TSP 170
TABLE A.57 - Instructions per cycle - FAST 171
TABLE A.58 - Instructions per cycle - FN 171
TABLE A.59 - Instructions per cycle - GF 171
TABLE A.60 - Instructions per cycle - IS 171
TABLE A.61 - Instructions per cycle - KM 172
TABLE A.62 - Instructions per cycle - LU 172
TABLE A.63 - Instructions per cycle - TSP 172
TABLE A.64 - Energy consumption (Joules) - FAST 173
TABLE A.65 - Energy consumption (Joules) - FN 173
TABLE A.66 - Energy consumption (Joules) - GF 173
TABLE A.67 - Energy consumption (Joules) - IS 173
TABLE A.68 - Energy consumption (Joules) - KM 174
TABLE A.69 - Energy consumption (Joules) - LU 174
TABLE A.70 - Energy consumption (Joules) - TSP 174
TABLE B.1 - L2 cache miss rate vs. associativity - FAST 175
TABLE B.2 - L2 cache miss rate vs. associativity - FN 176
TABLE B.3 - L2 cache miss rate vs. associativity - GF 177
TABLE B.4 - L2 cache miss rate vs. associativity - IS 178
TABLE B.5 - L2 cache miss rate vs. associativity - KM 179
TABLE B.6 - L2 cache miss rate vs. associativity - LU 180
TABLE B.7 - L2 cache miss rate vs. associativity - TSP 181

SUMMARY

1 INTRODUCTION . . . 19
1.1 Thesis statement and hypothesis . . . 21
1.2 Objectives . . . 21
1.3 Contributions . . . 22
1.4 Thesis outline . . . 23

2 BACKGROUND . . . 25
2.1 Parallel architectures . . . 25
2.1.1 Shared and distributed memories . . . 25
2.2 Interconnection systems . . . 28
2.3 Cache memories . . . 29
2.3.1 Cache mapping . . . 29
2.3.2 Way-replacement policies . . . 31
2.3.3 Cache coherence . . . 33
2.4 Reinforcement Learning . . . 38
2.5 Concluding remarks . . . 40

3 STATE OF THE ART . . . 41
3.1 Way-replacement strategies . . . 41
3.1.1 Coarse-grained policies . . . 42
3.1.1.1 Recency-based policies . . . 42
3.1.1.2 Frequency-based policies . . . 44
3.1.1.3 Hybrid policies . . . 45
3.1.2 Fine-grained policies . . . 45
3.1.2.1 Reuse distance . . . 45
3.1.2.2 Classification . . . 46
3.1.2.3 Value-function-based policies . . . 47
3.1.3 Architecture-aware policies . . . 49
3.2 Related work . . . 50
3.2.1 Coherence-based policies . . . 50
3.2.2 Machine learning applied to computer architecture . . . 50
3.2.3 Machine learning-based policies . . . 52
3.3 Concluding remarks . . . 52

4 DESIGN AND EVALUATION METHODOLOGY . . . 55
4.1 Evaluation methods . . . 55
4.2 Experimental setup . . . 55
4.2.1 Simulated architecture and preliminary design . . . 56
4.2.1.1 The next attempt strategy . . . 58
4.2.2 Workloads . . . 58
4.2.3 Performance metrics and evaluation perspectives . . . 61

5 COHERENCE STATE-BASED EVICTION . . . 63
5.1 Experimental results . . . 64
5.1.1 Cache miss rate . . . 64

5.1.2 Coherence operations 79
5.1.3 Instructions per cycle and energy consumption 80

6 SHARER COUNT-BASED EVICTION **85**
6.1 Experimental results 86
6.1.1 Cache miss rate 86
6.1.2 Coherence operations 100
6.1.3 Instructions per cycle and energy consumption 101

7 REINFORCEMENT LEARNING-BASED EVICTION **105**
7.1 Experimental results 106
7.1.1 Cache miss rate 106
7.1.2 Coherence operations 120
7.1.3 Instructions per cycle and energy consumption 122
7.2 Hardware overhead 125
7.3 Concluding remarks 128

8 CONCLUSIONS **133**
8.1 Future work 134

REFERENCES **137**

APPENDIX A - DATA FROM EXPERIMENTAL RESULTS **147**

APPENDIX B - SHARED L2 CACHE MISS RATE RESULTS **175**

1 INTRODUCTION

The use of High-Performance Computing (HPC) has become increasingly widespread across many areas. Scientists are solving problems that impact the daily life of society applying HPC, besides in the traditional science applications in academia and industry. For instance, data science emerged not long ago to improve customers' experience (KIM et al., 2021). Similarly, Leung et al. (2020) used HPC in healthcare and Pourhabibi et al. (2020) in fraud detection. HPC must keep up with the ever-increasing sizes of data to be processed to offer quick responses. Due to this context, research efforts in Computer Architecture focus on achieving the next step: Exascale Computing.

Supercomputers employ accelerators, hybrid memory systems, highly improved interconnection systems, and other technologies to achieve Exascale Computing. However, the road to this milestone contains multiple challenges. First of all, the high power consumption makes it necessary to rethink architectural designs. Second, applications and algorithms will have to adapt to these designs, dealing with resilience, parallelism, and scalability. Last but not least, the memory system, which is one of the most critical aspects of computer architecture.

For the last decades, memory technologies in computer systems could not follow the advance of the Central Processing Unit (CPU). This fact widened the gap between the CPU cycle time capabilities and memory access latency (WULF; MCKEE, 1995). However, in 1946, scientists already knew what would be ideal:

> Ideally one would desire an indefinitely large memory capacity such that any particular ... word would be immediately available ... We are ... forced to recognize the possibility of constructing a hierarchy of memories, each of which has greater capacity than the preceding but which is less quickly accessible.
> (A. W. Burks, H. H. Goldstine, J. von Neumann.) [1]

This statement is still valid nowadays. To reduce the gap, modern processors use a memory hierarchy whose main components are caches. A memory hierarchy's primary purpose is to increase data access performance. The idea is to provide data first in small but fast caches, thus, reducing the need to access the underlying slower memories. Furthermore, the use of caches can follow spatial and temporal locality principles. That is, faster layers contain data that the processor will likely reference again soon or adjacent data from accessed memory addresses.

The use of memory hierarchies brings new challenges, one of which is how to use caches efficiently. The system must put relevant data in the levels close to the processor to reduce cache misses, which are data requests that cannot be satisfied in a specific time. We can broadly classify cache misses into three categories (HENNESSY; PATTERSON, 2019): (1) Compulsory misses, due to first access to the data, (2) Capacity misses, because of the cache reduced size compared to the working set size, and (3) Conflict misses because even though the program

[1] Preliminary Discussion of the Logical Design of an Electronic Computing Instrument, 1946.

may not occupy all cache blocks, the function that maps a memory address to a cache cannot target an empty block in this cache.

Although cache misses are inevitable in such memory hierarchies, one can employ strategies to optimize the system. For example, Bélády (1966) published his work describing an optimal replacement policy for standard caches. Bélády's algorithm achieves the highest possible hit rates. However, it is an off-line algorithm, which requires future knowledge to make decisions. In addition, current caches cannot implement this optimal algorithm.

Reducing conflict misses can be done by increasing the associativity of the cache, i.e., having mapping functions targeting more blocks per set. Unfortunately, searching for the data then requires more hardware and takes more time to conclude. The solution used to allow resources vs. performance trade-offs is using set-associative caches: a block may be placed anywhere in a set whose elements, or ways, can be searched in parallel. As a result, the search in a few ways fastens computation.

We trace back the invention of set-associative caches to 1968. After that, way-replacement policies have been extensively studied (CONTI et al., 1968). Later on, in 1978, Smith (SMITH, 1978) and Rao (RAO, 1978) independently published surveys and analyses of replacement algorithms. Since then, researchers and the industry have devised new replacement policies and proposed many optimizations to fit different hardware-related constraints, such as area, power, and associativity. In addition, there are well-known heuristic-based algorithms: Least Recently Used, Most Recently Used, Least Frequently Used, and even a random selection of ways. In this thesis, we explain some of them in detail.

Current computing architectures contain several processors that share a single memory address space. As already mentioned, these multicore processors include caches, which have copies from shared data in the main memory. Thus, the processing units are constantly reading and writing shared data on cache blocks. This parallel computing scenario raises a new concern: ensuring cache coherence. The idea is to guarantee that cache blocks are updated while being used by multiple cores. Unfortunately, cache coherence leads to a new category of conflicts: coherence misses, i.e., transitions in the coherence protocol forcing the eviction of cache blocks. And this eviction may interfere with the local way-replacement policy.

The primary motivation of the present thesis is to move forward on the State of the Art concerning way-replacement algorithms for multicore processors. Future computer architectures will continue to use parallelism and shared memory hierarchies; thus, new designs will still deal with cache coherence. Therefore, there is an opportunity in coherence-related behavior to make better decisions when manipulating data in caches. This way, we understand that way-replacement policies can use the coherence state and the sharing set of a cache block to provide better eviction decisions. For that purpose, our strategy relies on architectures which use the directory-based MESI coherence protocol at the two first levels of caches in the hierarchy.

Furthermore, reinforcement learning is a technique with potential capabilities that can help this process. Reinforcement Learning (RL) is an area of Machine Learning (ML) that models

a system using two entities: an agent and an environment. Its purpose is to observe this environment and agent actions to provide the system with a near-optimal decision-making process. That purpose fits well the eviction decision process in way-replacement algorithms, which we detail in this thesis. The following sections present a more resounding thesis statement with our hypothesis and its main goals and contributions.

1.1 Thesis statement and hypothesis

Although caches solve a significant problem in parallel architectures, we must guarantee their efficient use. Choosing the best option for block eviction is a challenge that numerous way-replacement policies have addressed. As it is impossible to implement Bélády's algorithm in current caches, past works presented many heuristics. Currently, most general-purpose processors are multicore, requiring strategies to keep the coherence. Among the extensive corpus of research, we almost did not find replacement policies that consider information relative to the sharing state a cache block is.

Due to their performance impact on program execution, we must study way-replacement policies in set-associative caches in great depth. Thus, we find open research issues addressed by this thesis. The first one concerns if way-replacement policies use coherence information to improve the eviction decision. Another issue is if these policies use Machine Learning techniques to dynamically improve currently well-known algorithms for better eviction decisions. With those open issues and the State of the Art, our main hypothesis follows:

- Way-replacement policies can use the sharing state and the number of cores that share a cache block to enhance their performance.
- These policies can use Reinforcement Learning to improve currently well-known way-replacement policies for better eviction decisions.

1.2 Objectives

This thesis aims to investigate several way-replacement algorithms in caches, improving them with different strategies. We seek approaches that consider: (a) the presence of multiple cores sharing data in the architecture; (b) the use of different way-replacement algorithms; (c) the presence of cache coherence mechanism and its impacts; (d) the use of machine learning approaches.

The main goal is to propose and evaluate algorithms attached to current way-replacement policies, to achieve better eviction decisions in caches. To complete this primary goal, we consider the following sub-goals:

- To design algorithms considering the number of cores that share a cache block or its coherence state.

- To extend those algorithms using static and dynamic approaches.
- To extend those algorithms using reinforcement learning.
- To evaluate parallel applications behavior when executed on architectures with those strategies, considering cache misses, energy efficiency, and performance.

One can observe several aspects in experiments involving computer architectures. For instance, the influence of cache sizes, sharing strategy, number of cores, among others. In this way, we took evaluation and design decisions to delimit our proposal.

Our goal is to improve current way-replacement policies using a *"next attempt"* approach. We explain this strategy further, but the idea is to use the results of traditional algorithms and then decide whether or not to follow their directions. Moreover, to evaluate our architectural proposal, the objective is to use the CAP Bench suite (SOUZA et al., 2017). It comprises several algorithms implemented with several parallel patterns, as we will show in our methodology.

1.3 Contributions

This thesis provides several modified versions of traditional way-replacement algorithms to tackle the goals mentioned above. Furthermore, we carried out a comparative analysis of our proposals when combined with well-known way-replacement policies: Least Recently Used (LRU), Most Recently Used (MRU), Not Recently Used (NRU), Pseudo-LRU (PLRU), Least Frequently Used (LFU), and Static Re-Reference Interval Prediction (SRRIP). Hence, we highlight the following main contributions:

Coherence state-based eviction strategy: we propose modified way-replacement algorithms aware of the coherence state of cache blocks (SOUZA et al., 2019). We divided this proposal into a static and a dynamic selection of the coherence state to be verified. The eviction decision is based on the state of coherence previously determined (static) or on the most recently used state in a cache set (dynamic). Our experiments will show that branches of this strategy can reduce the cache miss rate by 24.84% and 69.04% in L1 and L2 caches, respectively. Moreover, it can reduce the energy consumption by 14.87% compared to an unmodified policy and improve IPC by 20.31%.

Sharer count-based eviction strategy: the same algorithms are modified considering the number of cores that share a given data (SOUZA et al., 2019). During the computation, the cache keeps tracking how many cores use the data. The eviction decision considers this number, checking if it is greater or lower than a previously defined threshold. The use of this information in the strategy can reduce the L1 cache miss rate by 22.94%. Likewise, up to 69.04% reduction in L2 cache miss rate is achieved. Furthermore, it improved the IPC over the unmodified version by up to 8.75%. About energy, it consumes 10.17% fewer Joules in the best case.

Reinforcement learning-based eviction strategy: in short words, we remodeled our previous proposals by applying the machine learning feature to them. The coherence state and sharing count to be compared against the current status of a cache block are not previously determined. Reinforcement Learning observes eviction decisions taken in the past, and it uses this information for future decisions. This ML-based approach reduces the cache miss rate by 41.20% and 27.30% in L1 and L2 caches, respectively. Besides that, it can improve the IPC by 27.33% in the best case and reduce the energy consumption by 20.36% compared to an unmodified policy.

1.4 Thesis outline

We organized the remaining chapters of the thesis as follows:

- Chapter 2 reviews concepts and topics that are relevant throughout this thesis. We briefly discuss the evolution of parallel architectures, as well as their interconnection systems and memory hierarchies. Besides that, we present an overview of cache coherence and way-replacement problems. Furthermore, we cover a basic theoretical introduction about Reinforcement Learning.

- Chapter 3 presents the State of the Art on way-replacement policies. First, we give an overview of the way-replacement policies that are in evidence nowadays. Besides that, we show past works using three perspectives: coherence-based way-replacement policies, machine learning applied to computer architecture, and machine learning-based way-replacement policies. Then, we argue on what points these solutions lack for answering the problem stated in this thesis. Additionally, we summarize why our work advances the state-of-the-art strategies for carrying out the way-replacement in multicores.

- Chapter 4 presents the experimental methodology we use to design our solutions and to evaluate them. We also present the *"next attempt"* approach that we use in all strategies.

- Chapters 5, 6, and 7 contain an in-depth discussion on the contributions of this thesis. First, we describe the coherence state-based approach for way-replacement algorithms. Then, we show our way-replacement proposal that considers the number of cores that share a given data. Finally, we propose the use of Reinforcement Learning to improve these algorithms.

- Chapter 8 presents our conclusions and future perspectives.

2 BACKGROUND

In this chapter, we present concepts that comprise the foundations of this thesis. First, we take a look at parallel architectures and their history (Section 2.1). Next, we introduce concepts on interconnection systems (Section 2.2) and caches in the memory hierarchy (Section 2.3). Then, we present a brief introduction about Reinforcement Learning (RL), a technique that we use in one of our proposals (Section 2.4). Finally, we conclude the background by positioning our work on it (Section 2.5).

2.1 Parallel architectures

Performance improvements of processors mainly were based on advances in instruction-level parallelism and the increase in frequency. However, these strategies have reached a point of diminishing returns due to barriers related to the chip material (VILLA et al., 2014). Thus, there has been a paradigm shift in building computer architectures (ASANOVIC et al., 2009). Nevertheless, the parallel approach has prevailed in computer architecture, giving rise to multicore and manycore chips from this point forward.

The parallel execution of computing tasks was first introduced in 1966 when Michael J. Flynn proposed his taxonomy (FLYNN, 1966). Flynn's Taxonomy groups architectures according to their instruction and data streams:

- **Single Instruction, Single Data (SISD):** a single processor executes a single instruction stream over data that comes from a single stream too.
- **Single Instruction, Multiple Data (SIMD):** a single instruction is executed simultaneously by multiple processing elements over different data streams.
- **Multiple Instructions, Single Data (MISD):** different processing units execute several instruction streams over the same data sequence.
- **Multiple Instructions, Multiple Data (MIMD):** a set of processors simultaneously execute different instructions on different data streams.

Flynn's Taxonomy is still valid. All its architectural categories, but SISD, are parallel by nature. Among them, MIMD is the most flexible and used nowadays. Therefore, it can be evaluated by itself, introducing different architectural characteristics.

2.1.1 Shared and distributed memories

First, we organize MIMD architectures using two aspects (HENNESSY; PATTERSON, 2019). The former is related to how data streams are accessed, which is the memory sharing strategy. The latter considers physical aspects, that is, how data streams are available.

Figure 2.1 – Shared memory

Source: Elaborated by the author

In a shared memory model, tasks running on different processing units can simultaneously access the same memory address range. Thus, all processing units that share this memory see data as if they were in a single accessible source. This model improves the development of parallel applications, making communication between multiple threads transparent to the programmer. However, one drawback of this model is the communication bottleneck. For instance, scalability is strongly affected in architectures with several processing units concurrently accessing the same address. The opposite of shared memory is private memory, in which the memory address space is unique for each processing unit set. Figure 2.1 depicts the shared memory model, in which processors denoted by P share the same memory (a). It also shows the communication between tasks t_i and t_j through memory (b).

Figure 2.2 – Distributed memory

Source: Elaborated by the author

In a distributed model, the memory is divided into separated memory modules. This model has greater scalability than its opposite, supporting a more significant number of processing units. Communication between tasks running in each set of processors and memory occurs through an interconnection network. This need for communication makes the distributed memory model more complex for program development. Centralized memory is the alternative organization of the distributed model. Figure 2.2 shows the distributed memory model (a) in separated groups with each processor P with its memory module M. An interconnection system connects these groups. In (b), the figure shows how communication occurs between two groups executing tasks t_i and t_j, respectively.

Considering these characteristics, we have two MIMD architectures. First, the multiprocessor is a system with several processing units capable of performing multiple tasks, sharing the same memory address space. Second, the multicomputer is a system with multiple processors connected by an interconnection network with different memory address spaces.

Multiprocessors can be further classified accordingly to the distance between memory and processing units:

Uniform Memory Access (UMA): in this type, the time for accessing data in memory is the same for all processing units and memory locations. The most common form of interconnection is the bus. Furthermore, this type of architecture usually implements memory as a single module.

Non-Uniform Memory Access (NUMA): memory is usually distributed and therefore implemented in multiple modules. Each processing unit is associated with a module, but access to other modules is possible. Thus, the address space is common to all processors, and the access time is different depending on the memory module being accessed (local accesses are faster).

NUMA type has its subclasses (ROSE; NAVAUX, 2008):

Cache Only Memory Architecture (COMA): every local memory modules in the machine is used only as a cache in this type.

Cache Coherent NUMA (CC-NUMA): pertains to multiprocessors based on caches with coherence guaranteed by the hardware. We present the coherence problem in Section 2.3.3.

Non-Cache Coherent NUMA (NCC-NUMA): this one is a multiprocessor with no guarantee of consistency in cached data or without caches.

Software Coherent NUMA (SC-NUMA): there is a software-based protocol to guarantee coherence.

Multicomputers comprise the **No Remote Memory Access (NORMA)** classification, which we can also divide into more categories (WILKINSON; ALLEN, 2004):

Massively Parallel Processors (MPP): those are multicomputers made up of many processors, tightly coupled through a high-speed network. They are usually architectures with higher costs and specific purposes.

Cluster of Workstations (COW): also known as **Network of Workstations (NOW)**, it is built from ordinary computers or servers linked by cabled interconnecting networks.

Parallel architectures have their efficiency strongly linked to the memory organization. Moreover, the interconnection system has an essential role beyond multi and manycore processors and memory. Thus, to take an in-depth look at the foundations of this thesis, the following sections present and discuss these topics.

2.2 Interconnection systems

Initially, it is essential to trace back the evolution of parallel architectures. In past years, it has been possible to reach a level of performance at the petaflops scale, thanks to the advent of more complex parallel architectures. For instance, Graphics Processing Units (GPUs), Field-Programmable Gate Arrays (FPGAs), and multicore processors have been used in this context. However, to achieve computational performance towards exascale, it is necessary to increase further the scalability and performance of parallel architectures (SHALF et al., 2011; SIMON, 2012). Thus, the community can exploit manycore processors, architectures with many interconnected cores (e.g., more than 64).

Since then, manycore processors have been studied, and new platforms have emerged (DINECHIN et al., 2013; ROSSI et al., 2014; MOTTIN et al., 2014). The design of these platforms has considered groundbreaking technologies to offer high performance and efficiency. Notwithstanding, they face new obstacles, such as memory limitations and communication bottlenecks (CASTRO et al., 2013; FRANCESQUINI et al., 2015).

Interconnection systems must exist in parallel architectures as a natural consequence of using multiple cores. Examples of traditional interconnection systems are buses and crossbar switches (ANDERSON; JENSEN, 1975; AHMADI; DENZEL, 1989; KUMAR et al., 2005). A bus is a transmission medium shared between the devices in the architecture. It can be seen as a wire, in which only a pair of devices can communicate over it at a given time. A crossbar switch is a connection matrix in which multiple buses are interconnected by switches, allowing simultaneous communication between different devices.

Despite being used in some situations, these types of interconnections have limitations. As an example, the restrictions imposed by electronic circuits, such as low bandwidth, signal attenuation, high latencies, and competition for resources (HO et al., 2001; DALLY; TOWLES, 2001). Therefore, only a more robust intra-chip communication system can efficiently interconnect many components (memory, cores, and others).

In this way, one can apply an efficient communication network, the Network-on-Chip (NoC) (BENINI; MICHELI, 2002; BENINI; BERTOZZI, 2005). The principle of a NoC is to reduce the size of interconnections by adding intermediate routers for communication between cores. Thus, the chip's scalability increases, allowing it to include more processing cores and other devices.

In manycore architectures there are many processors, so the number of memory accesses also tends to increase. This scenario causes more intensive use of interconnection networks and competition for available resources or contention. Furthermore, the organization of the architecture and the type of interconnection can impact cache access latency (YE et al., 2013; SOUZA et al., 2018). Although the interconnection affects the performance of the architecture, our work does not directly involve this system. We set up the interconnection system as a constant so that it does not influence the evaluation of our proposal in different architectures.

2.3 Cache memories

Some of the parallel architecture classifications discussed in the last section make use of caches. Caches are the main components of a memory hierarchy commonly found in most processors in the market. They are memory elements between the processing units and the main memory, e.g., the Dynamic Random Access Memory (DRAM).

Unfortunately, current processing units cannot process all the data available in the main memory at once. Thus, they need to perform instructions over smaller chunks of data, accessing memory repeatedly. These load and store operations in the main memory take time, hampering computation. The primary purpose of a cache organization is to allow data to be read and written by the processor in a faster way (PATTERSON; HENNESSY, 2020).

Caches can be organized in levels, for instance, in a three-level design. The upper level (L1) is closer to the processor. L1 caches have small sizes and, thus, shorter access times, due to better but expensive materials in their construction In this design, the lower level (L3) is close to the main memory, also called the Last Level Cache (LLC). Although LLC caches have bigger sizes and higher access times than L1, it is still faster than the main memory. Within L1 and L3, there is an intermediate level (L2).

The cache organization requires an inclusion policy. This policy defines whether or not cached data in upper levels will exist in lower ones. For instance, if a given cache block in L2 is also present in L1, the L2 cache is said to be inclusive. On the other hand, suppose that a given cache block is not in L1 nor L2. When the processor requests this block, the system will store it only in the L1 cache. The L2 cache will store this block only when it gets evicted from L1. In this example, the L2 follows the exclusive policy.

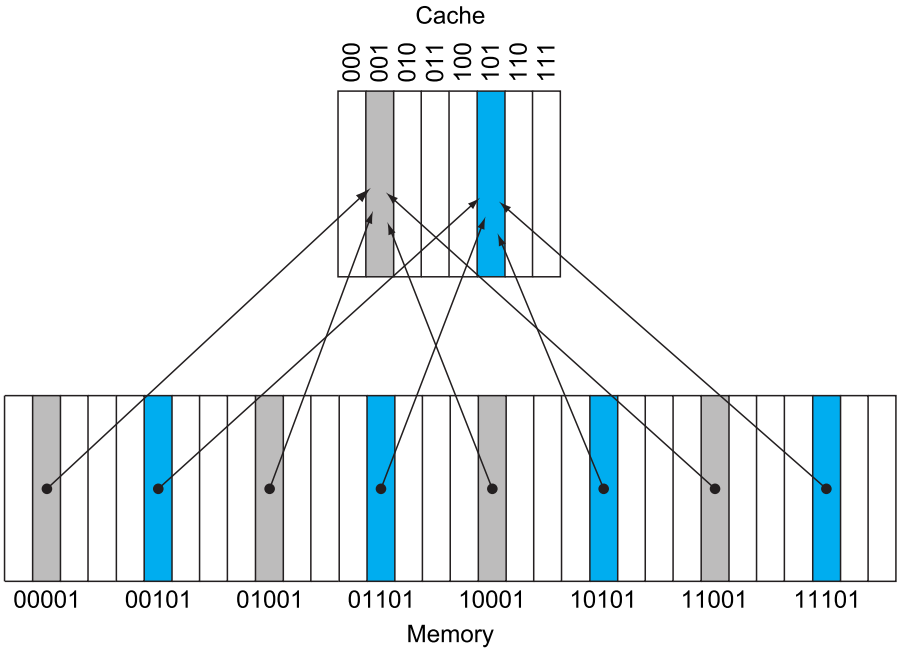
Bigger caches can handle more data; however, larger memory spaces imply more time to load and store data. Therefore, caches are disposed of at different levels, with different latencies and sizes, to solve this trade-off. This hierarchy favors the principles of temporal and spatial locality. The **temporal locality** principle says that the program repeats the access to a memory address after the first access. Likewise, the **spatial locality** principle states that the system will reference nearby addresses after referencing a specific address.

We can measure the effectiveness of caches by using the **hit rate**, which is complementary to the **miss rate**. The hit rate is the portion of data requests made to the cache that is satisfied. The use of caches requires strategies to allocate incoming data in the reduced space. Furthermore, there is a need to ensure that shared data is consistent across all cache levels. We discuss these topics in the following sections.

2.3.1 Cache mapping

The minimum unit of information that we can have in a cache is called a **line** or a **block**. As caches are much smaller than the main memory, it is impossible to allocate all their data

Figure 2.3 – Directed mapped cache



Source: (PATTERSON; HENNESSY, 2020)

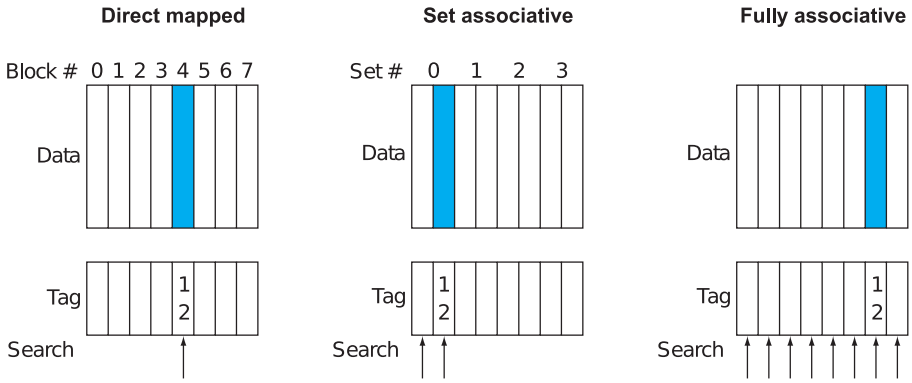
blocks in different caches. Therefore, when the processor requests data to some cache level, we call it a **hit** if this data is present. Otherwise, we say that a **miss** occurred. In this case, the system searches the data in the following levels of the hierarchy. If it is still not found, the system copies the main memory blocks to the cache in locations according to the cache mapping strategy. There are three different techniques to perform cache mapping:

Direct Mapping: a particular block of the main memory maps to exactly one line in the cache. The system will calculate the cache line number (index) to receive this block following the equation $cache\ line\ number = (main\ memory\ block\ address)\ \%\ (number\ of\ lines\ in\ cache)$. Figure 2.3 shows how this modulo operation works between the main memory and cache. Besides the line number, the memory address contains a tag number. Tags are part of the memory address used to identify whether the requested data corresponds to the one in the cache block.

Fully Associative Mapping: a block of the main memory maps to any available line in the cache. In this technique, there is no specific equation to decide where to put the copies of data. Nevertheless, the system needs to traverse the cache comparing block tags against the address tag until it finds the requested data. Figure 2.4 (on the next page) shows the fully associative search in its last frame, where the memory block for address 12 can be in any of the cache blocks.

Set Associative Mapping: the cache has its lines grouped into **sets**. Sets are a fixed number of locations to which a particular main memory block can map. This

Figure 2.4 – Block search in cache mapping strategies



Source: (PATTERSON; HENNESSY, 2020)

block can map any available entry in the set. The system will calculate the set number to receive this block following the equation: $cache\ set\ number = (main\ memory\ block\ address)\ \%\ (number\ of\ sets\ in\ cache)$. The pair $(cachesetnumber, cachewaynumber)$ identifies a cache block. The memory system uses the tag to identify the different memory data within the same cache set. Figure 2.4 also shows the set-associative cache search process, performed within a single set instead of the whole cache.

If a cache miss occurs in a direct-mapped cache, it will bring the block to a location determined simply by the mapping strategy. On the other hand, associative caches require more attention. Thus, in the next section, we discuss cache way-replacement strategies.

2.3.2 Way-replacement policies

Recalling some concepts about cache misses, we can classify it into three categories (PATTERSON; HENNESSY, 2020): (1) Compulsory misses, due to first access to the data, (2) Capacity misses, which occur when the working set size does not fit in the cache, and (3) Conflict misses, when the function that maps a memory address to a cache set cannot target an empty block in this set.

We can reduce conflict misses by increasing the associativity of the cache, i.e., having cache mapping targeting more blocks per set. But unfortunately, the search for the data requires more hardware and more time, for instance, to compare tags. Besides this trade-off, the cache must exchange some blocks with the new ones retrieved from the main memory when a set is complete. Way-replacement policies carry this out by choosing which block to evict to make room in the cache for the new one. **Replacement** is the process that takes place when the cache becomes full, then to make space for new blocks, we must remove old ones, replacing it. **Eviction** is the term used when blocks are released from the cache, which might be due to a replacement process (PATTERSON; HENNESSY, 2020; SORIN et al., 2011;

FERNANDEZ-PASCUAL et al., 2017). In summary, a replacement policy is the algorithm which manages how the evictions of blocks should happen in caches. For simplicity, we use the terms “replacement” and “eviction” interchangeably along this thesis.

We trace back the invention of set-associative caches and way-replacement policies to 1968 (CONTI et al., 1968). In summary, these policies intend to decide which way contains the cache entry to evict if the cache set is complete. In the seventies, Smith (1978) and Rao (1978) independently published surveys and evaluations of replacement algorithms. Since then, past efforts have resulted in new replacement policies and many optimizations proposed to fit different hardware-related constraints. It is worth recalling that Bélády’s algorithm is the most efficient way-replacement strategy since it always discards the information that it will not need for the longest time in the future. However, current processors cannot implement this algorithm due to the impossibility of predicting how far they will use the data in the future.

Several way-replacement policies are based on simple but sometimes effective strategies, while the others employ more complex behaviors (JAIN; LIN, 2019). A simple algorithm that the cache system can use is **Random**, which, as the name says, randomly selects the cache block to be evicted among the ones in a cache set. This strategy does not require complex logic, and we can implement it using an LFSR register [1]. Another simple algorithm is the **First In First Out (FIFO)**, which chooses to evict the oldest cache block, i.e., the one that first entered the set. The principle here is to assume that the oldest entries tend to be less accessed. The opposite stand, called **Last In First Out (LIFO)**, evicts the cache block that last entered the set. FIFO increases the complexity because it maintains the order in which it inserts the blocks.

Random and FIFO are more straightforward than the others because they do not care about the cache blocks’ *recency* (age) or the *frequency* of access. Thus, they are not able to properly exploit the spatial and temporal locality principles. There is the **Most Recently Used (MRU)** algorithm, which is also simple to implement. However, it is against the temporal locality principle when it evicts the most recently used block. Using these algorithms can get counter-intuitive results: larger caches or higher associativity may lead to higher miss-rates (BÉLÁDY, 1966).

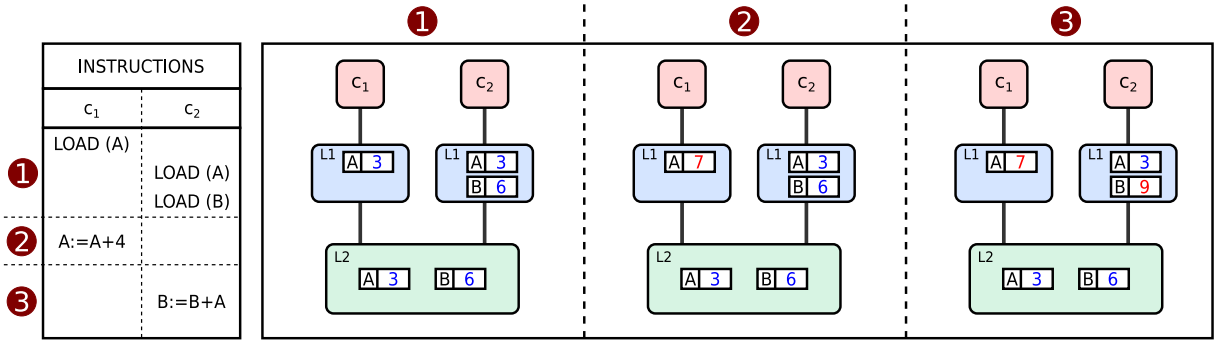
More sophisticated strategies, such as the **Least Recently Used (LRU)** and **Pseudo-LRU**, the **Least Frequently Used (LFU)**, the **Not Recently Used (NRU)**, and the **Re-Reference Interval Prediction (RRIP)**, care about the principles. They used the frequency of accesses or a recency measure, or even a prediction interval, to produce the eviction decisions. However, there is the disadvantage of hardware complexity which rapidly increases with the number of ways. In Chapter 3, we present these algorithms and others to provide an overview of the State of the Art in the way-replacement policies, which is the main subject of this thesis.

[1] **Linear-Feedback Shift Register (LFSR)**: a shift register that can produce a sequence of deterministic random values. In computer science, this sequence approximates the properties of sequences of random numbers.

2.3.3 Cache coherence

Current computing systems contain several processors that share a single memory address space. Furthermore, a cache hierarchy can have varying characteristics in these systems, such as the sharing and distribution strategy. For example, a hierarchy could have distributed private caches and a second-level centralized shared cache. A core could then change data that another one also accesses, raising a new concern: ensuring cache coherence (GHARACHORLOO, 1995; SORIN et al., 2011; HENNESSY; PATTERSON, 2019).

Figure 2.5 – Coherence problem in a dual-core processor



Source: Elaborated by the author

Consider the scenario presented in Figure 2.5, which comprises two cores in a processor accessing shared data in the L2 cache. The figure shows a simple instruction flow for each core. Cores c_1 and c_2 have private L1 caches and share a single L2. The figure shows an example of when the loss of coherence happens, which we need to handle carefully.

- First, c_1 loads the entry A to its L1 cache, and c_2 loads both A and B.
- Next, c_1 updates the value of A; however, c_2 is not aware of this operation.
- Finally, c_2 updates the value of B using the old value of A.

The architecture can use cache coherence protocols when it is necessary to guarantee that all copies of the same data in several caches have the same value. These protocols synchronize the operations in caches through a finite-state controller, defining the behavior of reads and writes to shared memory resources at the same address. For simplicity, we only discuss in this thesis the cache coherence protocols implemented in hardware, specifically in the cache controller. Two main techniques ensure cache coherence: **snooping** and **directory-based** protocols.

Snooping protocol: all cache controllers monitor the bus to check if their cache has or not a copy of the block. The cache controller from the processor writing in that block sends a broadcast message to all other cache controllers. There are two methods to implement this protocol. One is the **write-invalidate**, which invalidates the copies of the block in other caches. The other is the **write-update**, which updates all the blocks in other caches.

These protocols have no scalability, being a shortcoming mainly with a high number of processors. Goodman (1983) described this protocol at first.

Directory-based protocol: each cache has a local directory, which stores information such as the sharing status of a given block, a list of sharers for it, and its current owner. This protocol emerged in the seventies (TANG, 1976; CENSIER; FEAUTRIER, 1978). It selectively sends messages to the cache controllers with copies of the same shared block to keep consistency. Since there are no regular broadcasts, this protocol has increased scalability by reducing the number of messages required for coherence requests (AGARWAL et al., 1988).

The finite-state controller in each cache can deal with the following states:

- **Invalid (I)** – the block is invalid in the cache, thus, cannot be used.
- **Shared (S)** – the processor only read the block, and there might be copies in other caches.
- **Modified (M)** – only this cache has a valid copy of the block; however, the block in the main memory is not up to date.
- **Exclusive (E)** – only this cache has a valid copy of the block, and the main memory data is updated.
- **Owned (O)** – this cache can provide the data if a failure occurs while reading at the bus since the memory is outdated. There might be copies in other caches too.

There are many combinations of these states in several coherence protocols. The higher the number of states, the more complex the protocol is, due to more transactions being performed. Protocol names follow the state they use. For instance, **MSI** deals with Modified, Shared, and Invalid states, and so do **MESI** and **MOESI** with further states. This thesis uses the MESI protocol (SORIN et al., 2011).

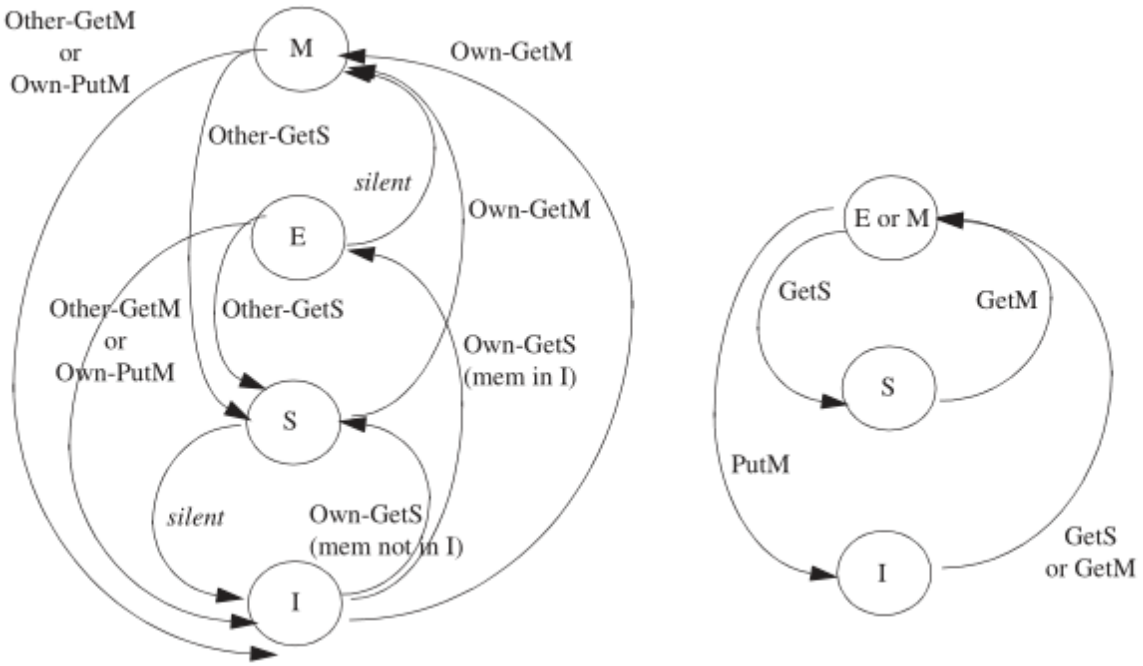
We briefly explain how these protocol works next, but first, we must understand how transactions occur in coherence protocols. Transactions in the protocol occur when there are memory operations in the system. We can classify operations on the memory into two categories (JERGER et al., 2017; SORIN et al., 2011):

Processor-generated requests consist of typical memory read/write or load/store operations. L1-cache accesses immediately result in cache hits, whereas a cache miss requires bringing the entry from lower levels of cache. The L2-cache receives read/write requests to it due to a cache miss in the L1-cache. The flow of these requests happens from processor to memory or from an upper level to lower levels of the memory hierarchy.

Interconnection Generated Requests originate from the cache controllers or the interconnection (e.g., the bus). The main goal of these operations is to maintain

coherence. They can vary with the implementation of the coherence protocol. The flow of requests is from the lower level to the upper level of the memory hierarchy or between caches at the same levels.

Figure 2.6 – State transition diagram for the MESI protocol



Source: Sorin et al. (2011)

The MESI protocol is a four-state invalidation based coherence protocol. These states are Modified, Exclusive, Shared and Invalid. Figure 2.6 shows the state transition diagram for this protocol, and how transitions occur on it. The "Own" prefix means that the processor that owns that cache is the one performing the operation. On the other hand, the "Other" prefix means that other component is doing it. The terms "Get" and "Put" mean, respectively, read and write operations. Meanwhile, the additional "M" and "S" are used to identify in the diagram if the block state is Modified or Shared, before the operation. For instance, if the cache block is "Exclusive" for this processor, and other processor reads it, then the protocol changes the state to "Shared", as depicted in the figure.

The operations on memory imply a group of events that occur in the cache hierarchy. Concerning the MESI protocol, these events can be the following, which might or might not mean a state transition in cache blocks (SORIN et al., 2011):

Upgrade: a processor reads a cache block to its cache in the Shared state because it is already in another one and then writes to it (the state changes to Modified or Exclusive).

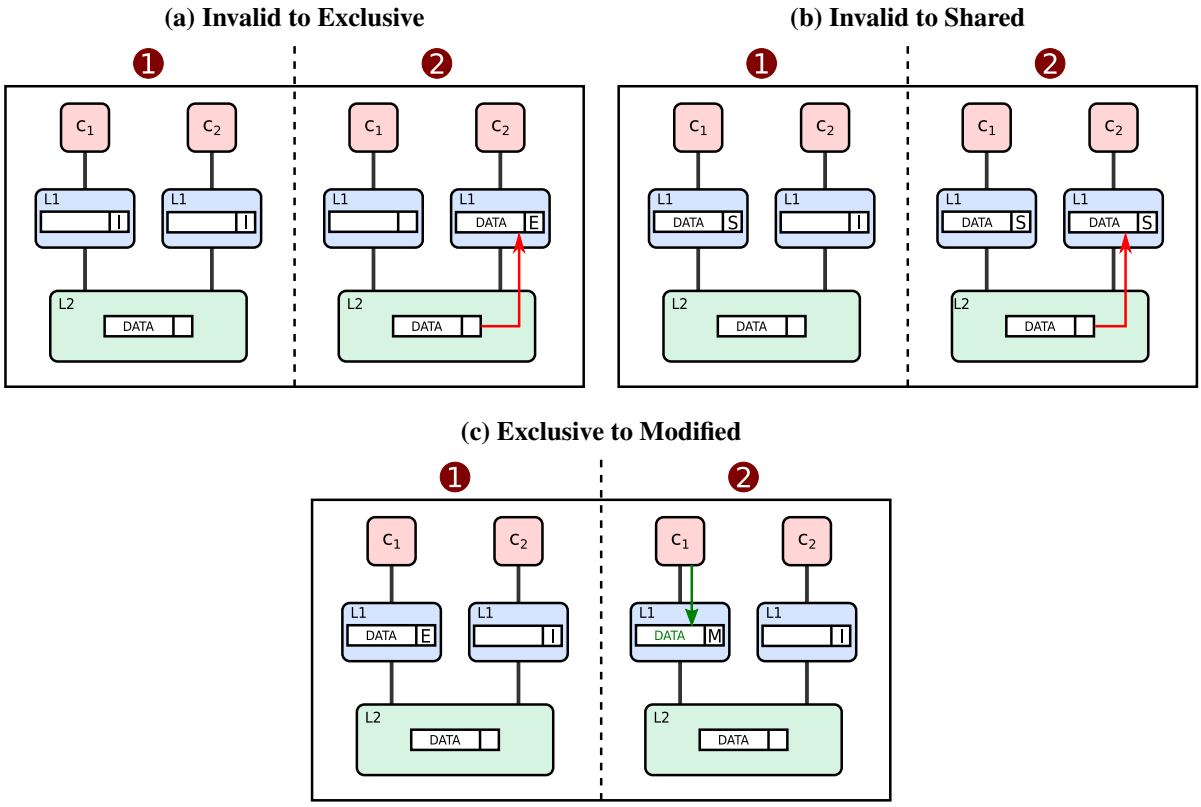
Downgrade: a processor has the cache block in its cache in Modified or Exclusive state, but it downgrades the block to Shared due to a read from another processor.

Invalidate: after the upgrade, the protocol must invalidate copies of the block in other caches.

Write-back: memory access forces a cache line in another cache in the Modified state to be written back to lower levels.

One can use this information to measure how the coherence protocol is affecting performance. In other words, the lower the number of coherence traffic, the better. Following, we show some examples of how coherence occurs in a dual-core processor sharing a single L2 cache, each with its private L1 cache.

Figure 2.7 – Cache coherence transitions



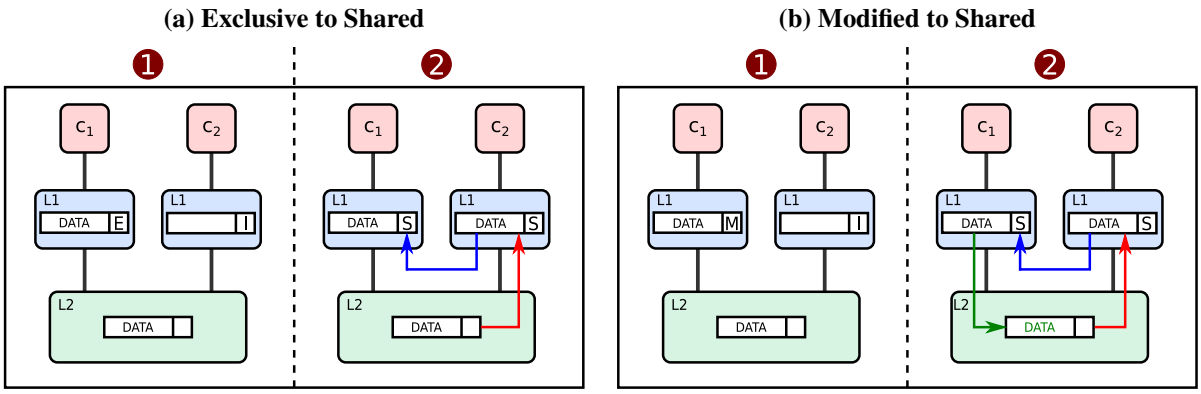
Source: Elaborated by the author

Figures 2.7a and 2.7b show examples where there is no need to inform other cores about the read/write operations. In 2.7a, both c_1 and c_2 have cache blocks in the Invalid state, for instance, due to the absence of data loading. If core c_2 reads the data, it will fetch the line into its cache in the Exclusive state because there are no copies of this cache block in other caches at the same level. Likewise, in 2.7b, c_2 reads the data from a cache block in the Shared state in c_1's cache. The cache block in c_1 stays in the Shared state, as well as the one in c_2. Thus, there are fewer costs related to the coherence than in other transition types in these examples.

Another low overhead transition is the one depicted in 2.7c. Core c_1 writes data in the cache block at its L1 cache, in the Exclusive state. Due to this update, the cache block changes its state to Modified since it knows that no other core cache has a copy at the same level. If that

were the case, the protocol would invalidate these copies. The same behavior occurs when the core writes to a cache block already in the Modified state that is not in nearby caches.

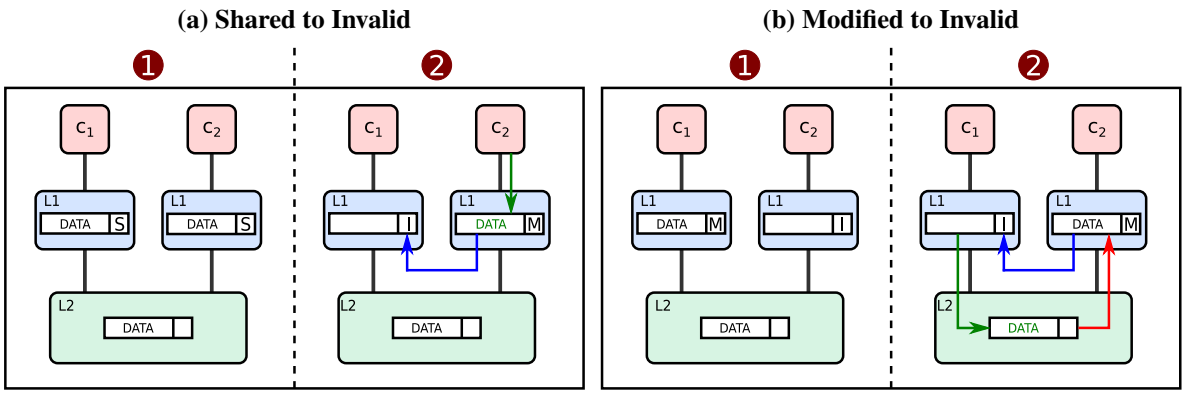
Figure 2.8 – Cache coherence transitions



Source: Elaborated by the author

To exemplify downgrade events, 2.8a presents a scenario in which a cache block is in the Exclusive state in c_1's L1 cache. If c_2 reads the same cache block from L2, the protocol fetches it in the Shared state (red arrow) and downgrades it in c_1's cache from Exclusive to Shared (blue arrow). In the same way, Figure 2.8b shows a similar case, but the cache block in c_1's cache is in the Modified state. Before carrying out the downgrade from Modified to Shared (blue arrow), c_1 must first write back the data (green arrow). Then, c_2 has its cache written with another copy of the block in the Shared state (red arrow).

Figure 2.9 – Cache coherence transitions



Source: Elaborated by the author

Last but not least, Figures 2.9a and 2.9b present two scenarios in which invalidation events occur. In the former (a), both c_1 and c_2 have their L1 with copies of Shared-state blocks. When c_2 writes data in the Shared cache block, it must upgrade it to the Modified state (green arrow). Hence, it must invalidate any block copies in neighbor caches (blue arrow) so that they do not retain an outdated copy of the block. The latter (b) shows c_1's L1 cache with a Modified cache block. Again, c_2 writes to the cache block that it does not have in its L1. Besides that, another

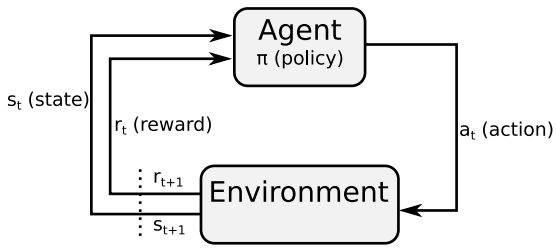
copy of this block is in the Modified state in c_1's cache. Thus, the protocol must first guarantee that the data is written-back (green arrow) and then invalidate the other copy (blue arrow).

2.4 Reinforcement Learning

Reinforcement Learning (RL) (SUTTON; BARTO, 2018) is a Machine Learning (ML) technique that involves training an agent based on repeated actions and their rewards. RL is fundamentally different from ML supervised or unsupervised methods. The agent in RL learns from its own experience created during the interaction with the environment. Thus, it does not depend on previous training based on labeled nor unlabeled data or patterns. The RL **agent** is a decision-making entity, and complementary to its universe, there is the **environment**. Due to the decision-making characteristic, we usually model RL as a Markov Decision Process (MDP) (BELLMAN, 1957; PUTERMAN, 2005).

An RL system has three other fundamental elements besides the agent and its interaction with the environment: a policy, a reward signal, and a value function. Figure 2.10 shows the diagram that represents how traditional RL generally works.

Figure 2.10 – Reinforcement Learning diagram



Source: Elaborated by the author

At each step t, the agent receives a set of observations, the **state** $s_t \in S$, from the environment. Then, it executes an action $a_t \in A$ according to a given **policy**. A policy assigns the state s to a value denoted by $\pi(s, a)$. This value represents the probability of taking an action a while in the state s in some time step t. The environment then provides feedback to the agent in the form of a **reward signal** $r_{t+1} \in \mathbb{R}$. During this process, there is a transition $s_t \rightarrow s_{t+1}$, which provides the agent with the new observations $s_{t+1} \in S$.

RL systems aim to maximize the **expected return**, the cumulative reward the agent receives over a long time. Given a sequence of rewards $R_{t+1}, R_{t+2}, R_{t+3}, ..., R_T$ within time steps t and T, we can say that the return G_t is a **reward function** of this sequence. The simplest case comprises the sum of the rewards in this sequence, as in Equation 2.1:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + ... + R_T. \tag{2.1}$$

Many problems modeled in RL have continuing tasks, with unlimited interactions between the agent and the environment, i.e., $T = \infty$. Thus, it is difficult to obtain the reward function

since it also tends to ∞. To solve that, we can use a discount factor, denoted by $0 \leq \gamma \leq 1$, which represents the difference in importance between short and long-term rewards. Thus, reward functions can have different degrees of complexity. Nevertheless, their goal is to provide agents with positive and negative feedback about past events.

Most RL methods aim to estimate *value functions* to verify if the agent is learning to choose positive rewards (SUTTON; BARTO, 2018). A value function is a map between a state s, or a pair state-action (s, a), and a value obtained from current and future rewards. A Markov Decision Process defines the state value function as in Equation 2.2:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\{R_t | s_t = s\} = \mathbb{E}_{\pi}\left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \middle| s_t = s \right\} \tag{2.2}$$

where $V^{\pi}(s)$ is the expected return for the current state $s_t = s$, i.e., the sum of rewards and the discount factor γ. Likewise, the value function of both *state and action* is the expected return after taking an action $a_t = a$ in the state $s_t = s$. Equation 2.3 defines the value function:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\{R_t | s_t = s, a_t = a\} = \mathbb{E}_{\pi}\left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \middle| s_t = s, a_t = a \right\} \tag{2.3}$$

There are three classes of fundamental methods to solve RL problems: Dynamic Programming (DP) (BELLMAN, 1958), Monte Carlo Methods (MC) (RUBINSTEIN, 1981), and Temporal-Difference Learning (TD). The last method is a combination of Monte Carlo and Dynamic Programming (SUTTON; BARTO, 2018). DP methods are model-based; that is, they assume that the dynamics of the MDP are known. MC and TD, on the other hand, are model-free methods, which learn directly from experience and do not assume any knowledge of the environment's dynamics.

In model-free methods, the challenge is to find an optimal policy $\pi^*(s, a)$ that leads to the highest value function for all states. The methods commonly use policy-based and value-based approaches to learn this optimal policy. Policy Gradient methods are examples of policy-based techniques (WILLIAMS, 1992). Q-learning algorithms and their extensions are well-known value-based algorithms (WATKINS; DAYAN, 1992).

As mentioned before, the RL agent seeks to maximize the reward value, choosing actions that it knows have produced the most effective rewards. However, for the agent to have this previous knowledge of actions, it had to choose actions that it had not selected before. Thus, to keep the discovery for better actions in the future, the agent must not stop looking for alternatives. This fact constitutes the trade-off between **exploration** and **exploitation**. Thus, several techniques addressed this dilemma (KAELBLING et al., 1996).

This section presented a brief background to the concepts of Reinforcement Learning, although it is a topic where there is much more to discuss. We explain in-depth the specific strategy used in our proposals in future sections.

2.5 Concluding remarks

In this chapter, we reviewed the basic concepts and topics relevant to this thesis's context. First, we discussed parallel architectures, showing that they are the primary approach in modern processors, perhaps the only viable one. However, these architectures have their performance increasingly tied to how fast memories can perform. This scenario becomes attractive due to peculiarities such as how architectures organize their computing units and the memory system.

Next, we briefly showed memory concepts to understand essential design decisions that we may take to produce parallel systems. For example, there is no memory technology with enough space to sustain heavy workloads while providing access as fast as arithmetic operations inside the processing core. Thus, computer architectures can use caches at the price of making the architecture more complex to deal with problems inherent in this technology.

We then discussed way-replacement strategies and cache coherence, which can impact the performance of the processor. Good way-replacement strategies are the ones that can approximate predictions of the future, thus, improve the hit rate. Higher hit rates favor the average memory access time since there is no need to search for the data in slower levels of caches or the main memory.

More precisely, this thesis is interested in way-replacement strategies that concern information from the coherence state of cache blocks. There is still a range of opportunities to improve way-replacement algorithms. We believe that Machine Learning fits this scenario well. Specifically, we can use Reinforcement Learning (RL) to improve decisions for the future, based on the evictions of the past and the implications they have caused. That is why we propose way-replacement algorithms for multicore processors with an RL-based approach.

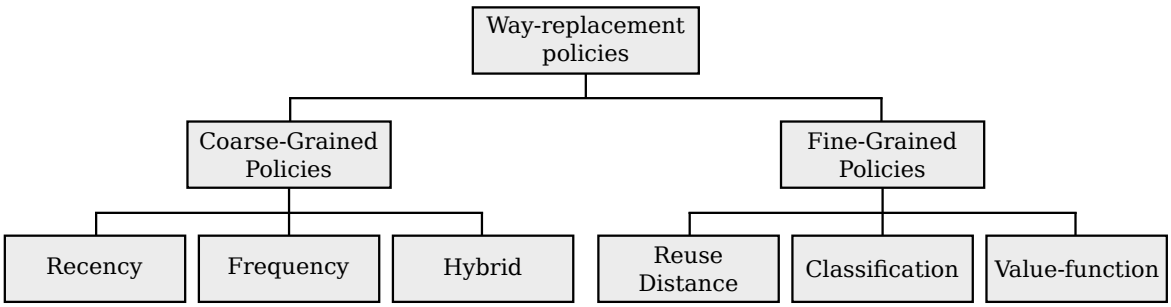
3 STATE OF THE ART

Since 1966, when Bélády presented its algorithm (BÉLÁDY, 1966), there have been considerable efforts to develop viable way-replacement algorithms. This chapter briefly reports the State of the Art in such way-replacement algorithms (Section 3.1). Besides that, we also present previous works related to ours (Section 3.2). Finally, in Section 3.3, we place our proposals in the context of past research since the State of the Art comprises the most relevant works related to the subject explored in this thesis.

3.1 Way-replacement strategies

Choosing the best cache block for eviction is a challenge that numerous way-replacement policies have addressed. Thus, we consider the State of the Art in this topic to be vast and continually increasing. Nonetheless, some aspects can organize this space of research (BECKMANN, 2015; JAIN; LIN, 2019). More precisely, Jain and Lin (2019) claimed that way-replacement policies consider a *replacement state* of the cache block. Thus, the policy must first consider initializing this state when inserting a block in the cache. Next, it needs to update the state during the life cycle of the cache block, using some criteria. Finally, it chose a cache block to evict, given the state of each candidate block.

Figure 3.1 – A taxonomy of cache way-replacement policies



Source: Adapted from Jain and Lin (2019, p. 8)

This view of the replacement state of cache blocks during some period led to a taxonomy of way-replacement policies, which we show in Figure 3.1. The first level of categorization comprises Coarse-Grained and Fine-Grained policies. This granularity-based categorization considers the treatment given to cache blocks at insertion and when they reside on the cache In the following sections, we briefly discuss these two categories and present the strategies and related works which fit each classification.

3.1.1 Coarse-grained policies

Coarse-Grained policies treat cache blocks this way: (i) the replacement state is the same for every cache block inserted into the cache, and (ii) it differentiates the blocks based on the replacement state and updates it during their presence into this cache. The taxonomy divides this group into three categories: recency-based, frequency-based, and hybrid policies.

3.1.1.1 Recency-based policies

Recency-based policies evict cache blocks based on their age, that is, how recently the system used a given cache block. Many approaches use this strategy, as follows.

Least Recently Used (LRU) is one of the most used policies. It assumes that cache blocks recently used will be used again soon. Thus, it chooses to evict the block that has not been accessed for the longest time. The algorithm maintains the order of accesses to cache blocks in a cache set in a linked list. Each access to the cache requires an update to this linked list, which increases the policy cost. Despite its advantage in performance (AL-ZOUBI et al., 2004; HENNESSY; PATTERSON, 2019), the policy does not scale with greater associativity. This way, it has several variants.

Most Recently Used (MRU) is the opposite of LRU. It chooses to evict the recently used entries. This algorithm is more straightforward than LRU since there is no need to use a linked list and keep the access order up to date. It only tracks the most recently used entry, regardless of associativity.

Pseudo-LRU (PLRU) implements the LRU strategy through approximate measures of how long the cache block is in the cache, rather than maintaining the exact order. The idea is to improve performance, reducing the search, and maintaining costs, at the cost of possible sub-optimal LRU returns. This policy can use the bit-based approach (MALAMY et al., 1994) or the tree-based (SO; RECHTSCHAFFEN, 1988) one. The former uses one status bit (MRU bit) for each cache block. Access to a block sets its status bit to 1, and when the last 0-bit changes to 1, all the others reset to 0. The algorithm chooses the leftmost status bit equal to 0 within a cache set. The latter uses a binary search tree, with each node being a single bit. The algorithm traverses the tree (0 to the left and 1 to the right) until it finds the leaf element corresponding to the victim block. During traversal, it inverts the node bits to update the access order.

Early Eviction LRU (EELRU) is an adaptive replacement algorithm that uses the same recency information as LRU (SMARAGDAKIS et al., 1999). According to the authors, the improvement of this algorithm relies on the effective use of the LRU information. This process occurs using an online cost/benefit analysis to decide whether or not to replace a cache block. The policy observes the working set and cache sizes and evicts

the LRU cache block when the working set fits in the cache. When the opposite occurs (the working set is bigger than the cache), EELRU evicts the block in an *early region* concerning the LRU stack.

Segmented LRU (SLRU) is a policy that logically divides the LRU stack into two segments (KAREDLA et al., 1994). First, the probationary segment receives the requested cache blocks as the most recently used (MRU). When these blocks receive a hit, they are then promoted to the protected segment, as the MRU too. Next, the replacement policy chooses to evict the LRU cache block from the probationary segment. Finally, the respective LRU block is demoted to the probationary segment as the MRU when the protected segment is full.

LRU Insertion Policy (LIP) emulates MRU by inserting new blocks in the LRU position instead of the MRU one (QURESHI et al., 2007). This algorithm performs precisely as the LRU policy and the only difference relies on the insertion of the block.

Bimodal Insertion Policy (BIP) modifies the LIP policy (QURESHI et al., 2007). This policy inserts new cache blocks into the cache at the LRU or the MRU position. It carries this out by using a previously set probability value. According to the authors, the idea is to unify insertion policies that lie between LRU and MRU.

Not Recently Used (NRU) is a policy that can be considered an approximation of LRU. It considers the *Re-Reference Interval Prediction (RRIP)* concept, the predicted access order of cache blocks (JALEEL et al., 2010). The algorithm uses one bit (NRU bit) for each cache block in the set. The NRU-bit set to 0 means that the block was recently used and is predicted to be used again in the *near-immediate* future. On the other hand, 1 means the block is predicted to be used in the *distant* future because it is not used recently. Thus, NRU evicts the first cache block with the NRU-bit equal to 1. If all cache blocks are predicted to be used again in the *near-immediate* future, the algorithm resets all bits to 1.

Static Re-Reference Interval Prediction (SRRIP) has the idea of addressing mixed access patterns to caches (JALEEL et al., 2010). It uses Re-Reference Prediction Values (RRPV) to represent more than two RRIPs. RRIP is not limited to the *near-immediate* (0) or *distant* (1) future. Instead, it uses $r - bits$ per block to represent 2^r possible RRPVs, comprising more intervals. If $r = 1$, the RRIP algorithm performs identically to NRU. Differently, if $r > 1$, there will be more intervals between *near-immediate* and *distant*.

Bimodal Re-Reference Interval Prediction (BRRIP) is an extended version of SRRIP. Jaleel et al. (2010) extended the BIP policy by inserting most cache blocks with a high RRPV (e.g., 3). Besides that, it inserts the rest of the cache blocks with a lower RRPV (e.g., 2). The number of cache blocks that follow each RRPV follows a previously set probability value, as in BIP.

Protecting Distance-Based Policy (PDP) follows the RRIP principles generally (DUONG et al., 2012). The algorithm works by dynamically estimating a Protecting Distance (PD), a continually updated value for each cache block. The PD is calculated using the distribution of reuse distances observed within a specific time. With this mechanism, cache blocks are protected for access before eviction. This is done by observing the PD value, thus, resulting in unprotected blocks being evicted first.

Genetic Insertion and Promotion for Pseudo-LRU Replacement (GIPPR) is a policy that uses the concept of Insertion/Promotion Vector (IPV) (JIMéNEZ, 2013). The IPV specifies the position of a cache block in the recency stack at the moment of its insertion into the cache and when a cache hit occurs in this block. Thus, like in RRIP policies, cache blocks are inserted in the stack in dynamic positions, according to the IPV values.

Shepherd Cache is a proposal that emulates Bélády's optimal way-replacement policy (RAJAN; RAMASWAMY, 2007). This strategy separates cache blocks into two logical caches: Main Cache (MC) and Shepherd Cache (SC). The SC provides MC with a look-ahead window; thus, MC can emulate the optimal policy. Furthermore, the SC implements a simple FIFO policy. The requested cache block is cached in SC when a miss occurs before MC starts the replacement decision process. Meanwhile, reuse information is gathered in MC to choose which block to evict.

3.1.1.2 Frequency-based policies

Frequency-based policies choose to evict blocks concerning how many times the system used these blocks. Three policies are in this class.

Least Frequently Used (LFU) is concerned with the frequency of access to a cache block and uses a stack of frequency to decide which block to evict. Thus, it will replace the block which was used the fewest times in the cache.

Frequency-Based Replacement (FBR) was proposed by Robinson and Devarakonda (1990), which observed that frequency-based strategies could lead to high counter values. These values appear when there are, for instance, short bursts of temporal locality. The policy solves this problem by dividing the stack into two sections: *new* and *old*. It updates the frequency counters selectively in the old section and does not update them in the new section. Thus, the short bursts of temporal locality do not impact the frequency counters.

Least Recently/Frequently Used (LRFU) mixes LRU and LFU policies (LEE et al., 2001). It uses the Combined Recency and Frequency (CRF) metric, which represents a value between a spectrum of policies between recency and frequency information. The CRF value is a sum of a weighing function, which considers past references of cache blocks. In other words, it quantifies how likely the cache block is to be referenced in the future. LRFU chooses to evict the block with the lowest CRF value.

3.1.1.3 Hybrid policies

Hybrid policies are the ones that balance the use of frequency and recency information. Although promising, hybrid policies bring new challenges (JAIN; LIN, 2019). First, they need to implement multiple policies, increasing the costs. Second, the policies must accurately identify which policy to use to choose cache blocks.

Adaptive Replacement Cache (ARC) carries out the recency/frequency balance using four structures (MEGIDDO; MODHA, 2003). The strategy uses T1 for recently used cache blocks and T2 for frequently used ones. B1 contains blocks recently evicted from the T1 list, and similarly, B2 contains the blocks evicted from T2. Each pair T1/B1 and T2/B2 forms a directory, named L1 and L2, respectively. New cache blocks enter T1 and move towards B1 unless it is re-referenced (moving towards T2). ARC dynamically changes T1 and T2 sizes, according to the hits in B1 and B2 lists, and the candidate for eviction is also dynamically changed during this process.

Dynamic Insertion Policy (DIP) alternates between the BIP policy and the LRU one (QURESHI et al., 2007). Thus, it combines the benefits of a recency-based policy and a trash-resistant one. It dynamically tracks the hit rate of each policy and uses Set Dueling to choose which policy to use. More precisely, the strategy chooses the policy which produces more cache hits.

Dynamic Re-Reference Interval Prediction (DRRIP) also follows the RRIP concepts. However, Jaleel et al. (2010) proposed the hybrid use of SRRIP and BRRIP policies. To decide between these policies, the authors used the same strategy employed in DIP.

3.1.2 Fine-grained policies

Fine-Grained policies differ from Coarse-Grained ones because they deal with the replacement state at the insertion of a cache block. Additionally, they also differentiate these blocks during their life cycle into the cache. These policies rely on historical information about the blocks to determine their state. Jain and Lin (2019) divided Fine-Grained policies into two main categories: reuse distance and classification. Besides that, we renamed a Fine-Grained class that had only one policy. We call this third category as Value-function-based policies.

3.1.2.1 Reuse distance

The *reuse distance* measures the distance (or time) between two consecutive accesses to a cache block. That is, how far these accesses are from each other. It can be given, for instance, by the number of accesses to the cache or the number of cycles spent. Following are some policies that use the reuse distance.

Timekeeping is a technique proposed by Hu et al. (2002), which considers the *live time* of cache blocks. More precisely, this time is the number of cycles the cache block stays cached. Thus, when a cache block is inserted again in the cache, its lifetime is already known. If the cache block stays in the cache twice its previous lifetime, the strategy predicts it as a dead block. A dead cache block is a block that will not be used again until being evicted; thus, it is an excellent candidate to be removed from the cache.

Access Interval Predictor (AIP) is similar to Timekeeping (KHARBUTLI; SOLIHIN, 2005). The difference is in the metric used to define the lifetime of the cache blocks. AIP uses how many cache accesses occur between consecutive references to the same cache block.

Estimated Time of Access (ETA) also uses a predictor to carry out the replacement strategy (KERAMIDAS et al., 2007). It learns the reuse distance from load instructions and incoming cache blocks to compute the ETA. The ETA is the sum of the current time and the expected reuse interval. The policy evicts the block with the highest ETA value.

Leeway is the policy proposed by Faldu and Grot (2017), which uses the *live distance* to identify dead blocks. The live distance of a cache block is how far this block went into the recency stack. Thus, the strategy monitors the live distances from previously accessed cache blocks and uses a predictor to decide if new blocks are dead or not. Leeway uses a Live Distance Predictor Table (LDPT) for each PC to predict the live distance of incoming cache blocks. The strategy uses set dueling to decide between a strategy that favors bypassing and another which favors caching for updating the recency stack.

Reusability Monitor (BANG et al., 2018) Another adaptive cache replacement policy selects blocks by monitoring their reuse characteristics. The authors categorize the blocks as reuse or non-reuse and as dirty or clean. Then, a priority between those categories is defined to select blocks in such situations for eviction. The IPC has improved by 13% compared to LRU, according to the authors.

3.1.2.2 Classification

A group of policies can classify cache blocks into cache-friendly or cache-averse. Cache-friendly blocks tend to result in cache hits; thus, they receive high priority to stay in the cache. On the other hand, cache-averse blocks sometimes need to be in the cache, but the processor will not use them again in the future; thus, the policy should evict them.

Evicted Address Filter (EAF) is the classification-based policy proposed by Seshadri et al. (2012). It predicts the reuse behavior of missed cache blocks, using a probabilistic, space-efficient structure called bloom filter to track evicted addresses. If the search address is present in the filter on a cache miss, the respective cache block is predicted as reuse-friendly, with high priority to stay in the cache. The strategy inserts the blocks according to the BIP policy if the address is not in the filter.

Sampling-Based Dead Block Prediction (SDBP) is a predictor-based policy for way-replacement problems (KHAN et al., 2010). The idea is to sample the caching behavior of program counters (PCs) and classify them according to the cache block, i.e., if this cache block will be dead or not. Then, the SDBP verifies the PCs that may insert dead blocks in the cache and do not cache them, bypassing the access. Conversely, other cache blocks follow traditional replacement policies at the insertion, e.g., the LRU.

Signature-Based Hit Prediction (SHiP) is another predictor proposed by Wu et al. (2011). It works similarly to SDBP; however, it focuses on the PC that first inserted the dead block in the cache rather than other PCs that might access the block. Furthermore, the strategy uses the SHiP predictor only on cache misses; thus, it does not update the LRU stack on every cache access.

Hawkeye is the policy proposed by Jain and Lin (2016). The authors used Béládyʼs MIN algorithm to learn the optimal solution from past accesses. The strategy uses a trainer that receives cache access history and computes the optimal algorithm. Next, the Hawkeye predictor uses this information and the PC to classify cache blocks into cache-friendly or cache-averse. The policy consults the predictor in every cache insertion and promotion.

Multiperspective Placement, Promotion, and Bypass (MPPPB) is a technique that uses a multiperspective reuse predictor (JIMéNEZ; TERAN, 2017). This predictor captures several characteristics from the application in execution, producing many perspectives. Furthermore, the predictor also uses information from the LRU stack to perform the training steps, besides the program counter (PC) information. Thus, with those inputs, the predictor produces highly accurate results for eviction decisions.

3.1.2.3 Value-function-based policies

We extended the taxonomy from Jain and Lin (2019) to add value-function-based policies. The idea is to group the way-replacement policies that use a measure of cost based on some function. The following policies are examples of such strategies.

Economic Value Added (EVA) is the strategy proposed by Beckmann and Sanchez (2017). The authors claimed that way-replacement could be modeled as a Markov Decision Process due to how it works. Thus, they adopted the idea and proposed to use the Economic Value Added (EVA) of cache blocks. This EVA is given by the equation $EVA = expected_hits - (cache_hit_rate/cache_size) \times expected_time$. Thus, it is the difference between the number of hits a cache block should yield and its average hit count and occupancy in the cache.

Adaptive Demotion Policy (ADP) is an adaptive policy that detects the application behavior to carry out the decision process (TADA et al., 2017). The idea is to calculate a global

fluctuation of priority values on each cache set based on the Adaptive Demotion Policy (ADP). ADP is another replacement policy that uses 2-bit priority values for each cache block. The value increases when a cache hit occurs; otherwise, it is compared against other blocks to be decreased or not. The strategy presents a 0.37% IPC improvement over LRU, with cache miss rate reductions.

Expected Hit Count (EHC) proposes a correlation between the number of hits in a last-level cache block and its reuse distance (Vakil-Ghahani et al., 2018). Furthermore, the authors suggested a block selection strategy based on the number of hits stored in a table. Thus, the strategy evicts the block with the lowest value in this table. As a result, they obtained up to 12.2% performance improvements over Pseudo-LRU.

Cost-Aware Replacement Policy (CSOPT) is a policy that measures the costs of a single miss and uses this information (JEONG; DUBOIS, 2006). In other words, it does not try to minimize the number of misses but to avoid the ones with higher costs. A high-cost miss is the one that results in higher overall latency to the system. Thus, CSOPT explores multiple options for eviction using a branch-and-bound approach, measuring the costs based on Bélády's MIN policy. However, to perform these steps, the algorithm requires knowledge of future cache access, thus impractical.

MLP-aware Linear (LIN), proposed by Qureshi et al. (2006), has two main components. One is a predictor which knows the costs of future cache misses. The other is a way-replacement policy which considers recency information and the predicted costs. The strategy defines the MLP-based cost as the number of stall cycles that a given cache miss caused. The policy then prioritizes blocks with higher costs because they tend to impact the performance more.

Reuse and Memory Access Cost Aware (ReMAP) follows a similar strategy as LIN (ARUNKUMAR; WU, 2014). However, the cost is measured after the eviction occurs, using a linear combination. This combination uses recency information, DRAM latency, and the reuse distance after the eviction – it uses a bloom filter, as in EFA, to compute the reuse distance. Thus, the eviction policy considers the costs of each cache block before choosing the victim.

Locality-Aware Cost-Sensitive (LACS) is another cost-based policy (KHARBUTLI; SHEIKH, 2014). The cost function counts the number of instructions issued during a last-level cache miss. The claim is that this strategy reflects the processor's ability to ignore the miss penalties while doing other tasks. Thus, evicting the cache block costs high if the processor executes few instructions during the miss penalty interval. Otherwise, the cache block is low-cost; thus, it is less likely to be evicted.

Application Behavior Aware (AB-Aware) is a cache replacement policy that also concerns application behavior (PAI et al., 2018). This strategy considers memory level parallelism and data-reuse information. They are used as weights and costs in functions to determine the candidates for eviction. Results showed 23.8% IPC improvements over SRRIP and ABRIP (LATHIGARA et al., 2015) replacement policies. ABRIP is a replacement policy that takes into account streaming applications with mixed access patterns. It uses the concepts of MRU and LRU to keep cache blocks that tend to be reused as MRU and not reusable blocks from streaming applications at LRU.

3.1.3 Architecture-aware policies

We discuss another set of works that dealt with way-replacement policies in this section. These works consider different architectural aspects from caches which might impact the way-replacement strategy. Despite this fact, the taxonomy does not concern them.

For instance, cache prefetching is a technique that favors the spatial locality in the cache hierarchy. It fetches data into caches before they are requested. Therefore, changes in the prefetching mechanism might require adaptations in the way-replacement policies. For instance, consider a cache block that was cached but never used due to wrong prefetching. This situation causes cache pollution, and a way-replacement policy aware of this should prioritize this block for eviction. Thus, some past works dealt with this scenario to avoid cache pollution (ISHII et al., 2009; ISHII et al., 2012; KIM et al., 2017; SRINATH et al., 2007; SESHADRI et al., 2015; JAIN; LIN, 2018).

Likewise, prior research on caches dealt with cache partitioning. Cache partitioning is a strategy to avoid poor usage of shared caches. For instance, in a scenario with multiple threads or processes, it is possible to have many conflicts and undesirable evictions due to the concurrent use of caches. Thus, the cache can be partitioned between cores to increase fairness and reduce conflicts, at the cost of reduced space for these cores. Additionally, it is possible to adapt way-replacement policies so as they favor cache partitioning (JALEEL et al., 2008; MANIVANNAN et al., 2016; XIE; LOH, 2009; SANCHEZ; KOZYRAKIS, 2011; SUBRAMANIAN et al., 2015).

There are also past works that considered that other architectural aspects of caches could impact or be impacted by way-replacement policies (JAIN; LIN, 2019). First, the use of inclusive caches can sometimes lead to poor performance. For instance, if a given cache block is in the L1 cache, it is also in L2 and L3 caches due to the inclusive policy. However, if the way-replacement algorithm evicts the L3 block, the inclusive policy must invalidate upper-level copies. Jaleel et al. (2010) discuss in detail this scenario, showing how important it is to have a suitable replacement strategy with inclusive caches. Another architectural aspect that one can consider is cache compression. Cache compression applies when there is a need for more cache space, and it is impossible to increase the cache size due to the area and energy costs.

Consequently, compression can result in different cache block sizes. Thus, there is a need to adapt replacement policies to deal with variable-sized blocks (PEKHIMENKO et al., 2015; GAUR et al., 2016).

3.2 Related work

In this section, we present related work from three perspectives. First, we show prior research in way-replacement policies that consider cache coherence in their replacement policies. Next, we focus on works that applied Machine Learning techniques to solve general computer architecture problems. Then, we provide a report of some works that used Machine Learning to solve the way-replacement problem.

3.2.1 Coherence-based policies

In this thesis, we propose the use of cache coherence information to improve way-replacement policies. This section's primary goal is to show related work that uses coherence information to perform way-replacement too. Unfortunately, we found only two studies addressing way-replacement policies using the coherence information of a cache block.

The first one evaluated the effect of prioritizing a cache coherence state over others when choosing blocks for eviction (MOUNES-TOUSSI; LILJA, 1998). The strategy maintains an MRU state information to be used during the way selection process. Using the MESI cache coherence protocol, the authors selected which way has the block to evict statically. This selection followed the order 'Invalid', then 'Shared' and not MRU, 'Exclusive' and not MRU, 'Modified' and not MRU, and finally MRU. If more than one block in a set is a candidate for replacement, a random strategy is applied to choose among them.

Later on, Agarwal (2015) evaluated a similar strategy, but this time using dynamic priorities. This dynamic strategy stores the priority coherence state for each cache set. It updates the priority state whenever it accesses the cache block, which is the Most Recently Used State.

3.2.2 Machine learning applied to computer architecture

Machine Learning (ML) is a hot topic in several areas of research. Therefore, computer architects have been optimizing systems to enhance the efficient execution of ML algorithms or models. However, the research community had begun to consider the relationship between computer architecture and ML in a new direction. Hence, ML is seen as a way to transform computer architecture designs and their behavior.

With that in mind, Penney and Chen (2019) and Wu and Xie (2021) published two surveys related to ML applied to computer architecture design and systems. First, they presented a literature review of work done that used ML in computer architecture. Also, they highlighted

several areas in which this approach took place, for instance, in System simulation, GPUs, Memory Systems, Branch Prediction, NoCs, and System-level Optimization. It is also possible to identify ML uses for this purpose. For instance, there were online and offline ways to apply ML in computer architecture. Finally, they pointed out future perspectives from these works that optimize ML techniques to fit architectural constraints and develop generalized tools for this specific purpose.

Other works applied ML techniques in interconnection systems, more precisely, in Network-on-Chips (NoC). These techniques were used, for instance, to improve control policies in the routers of the NoC (ZHENG; LOURI, 2019). Another example is ML-based fault-tolerant mechanisms (WANG et al., 2019). Both works used Reinforcement Learning to improve NoC performance and reliability while also aiming to improve energy efficiency.

Regarding branch prediction in instruction flows, a specific past work used Convolutional Neural Networks (CNNs) to improve branch prediction accuracy (TARSA et al., 2019). The authors used global history data from existing branch predictors to train the network and used it as the new predictor in their simulated architecture. Likewise, Zouzias et al. (2021) used Reinforcement Learning to idealize the development of other predictors. They suggested the design space exploration of branch predictions through RL methods. Besides that, they exemplified this idea, presenting the concepts of a Policy-Gradient Based branch predictor. The authors concluded that RL concepts map naturally to the branch prediction principles.

Machine Learning applies to cache problems in two main scenarios. First, it is used in way-replacement policies, as we present in Section 3.2.3. Second, ML can improve data prefetching. For instance, Peled et al. (2015) exploited semantic locality using reinforcement learning to choose which blocks to prefetch. They used a two-level indexing method with online feature selection to correlate program semantics to data structures. However, this approach added some overhead to the prefetching process.

Long short-term memory (LSTM) is a type of Recurrent Neural Network (RNN) architecture. It is used in Deep Learning, more precisely, using input and output gates in the neural network to control the flow of information. Taking advantage of this, Hashemi et al. (2018) used an LSTM model to conceive a prefetch approach. Their strategy acts by learning semantics in memory access patterns and using this information to make prefetching decisions. Additionally, Braun and Litz (2019) explored a range of LSTM models with different input parameters. They found that the size of the memory access window to be evaluated and the model size itself are critical parameters to the quality of the prefetcher. Both works pointed out two main drawbacks of LSTM and neural networks: first, they suffer from long warm-up phases, and second, they require a considerable amount of bits in hardware implementations.

3.2.3 Machine learning-based policies

The state-of-the-art way-replacement policies group also comprises ML-based way-replacement strategies. For example, Vietri et al. (2018) proposed a general framework that uses the ML technique of regret minimization. Their technique, called LeCaR, uses two data structures for each cache block to track their frequency and recency of use. In other words, regret minimization works between the LRU and LFU algorithms. The proposed algorithm chooses which strategy to follow based on what it has learned from the structures in cache blocks.

ML-based way-replacement strategies also use LSTM (SHI et al., 2019). The authors trained an LSTM model to extract patterns from the program counters (PC) of load and store instructions. The model is trained offline using Béládyʼs algorithm and then used to build an online SVM-based hardware predictor to perform the way-replacement. Similarly, Liu et al. (2020) proposed an Imitation Learning approach. The main idea is to mimic some behavior from examples. More precisely, the authors build a model to learn cache access patterns when using Béládyʼs optimal algorithm.

Last but not least, Sethumurugan et al. (2021) proposed an offline tool using Reinforcement Learning to learn access patterns. The authors evaluated the learned model and implemented their way-replacement strategy focusing on specific features. The main goal was to check the reuse distance, predicted based on the distance of the cache blocks used in the past. When the predicted reuse distance was higher than the reuse distance of a cache block, the policy kept this block in the cache.

3.3 Concluding remarks

Way-replacement algorithms carry with them a decision-making process. Specifically, in the case of cache-coherent multicore processors, these algorithms significantly impact computing performance. Thus, several works on way-replacement strategies approached this topic in different ways. Only two works were aware of cache coherence in the replacement policy. These two are are closely related to the proposal of this thesis. Still, we understand that they left gaps to explore. For instance, would it be possible to use a non-random selection for more than one candidate victim? And what would be the behavior of such algorithms in processors with a higher number of cores?

For these and other open questions, we implement some of the concepts of this thesis, focusing on them. Thus, we identified that we could use the coherence information from the cache hierarchy to enhance the performance of way-replacement policies. Mounes-Toussi and Lilja (1998) and Agarwal (2015) also considered this. However, according to today's standards, the studies reported satisfactory results only with a tiny number of processors. Furthermore, none of them took into account the number of sharers a cache block has. Thus, there is an open

challenge to move forward in the State of the Art. This challenge concerns using the coherence state and sharing of cache blocks to choose the best cache block for eviction in set-associative caches.

Thus, we design and implement a "next attempt" approach that differs from the related work. For example, one of our proposals cares about the information of cache blocks related to how they are shared. In addition, we look at machine learning as a tool that can support decision-making on way-replacement policies. Several recent works have taught this way for other problems in computer architecture. Thus, there is a range of possibilities in improving the performance and efficiency of future architectural designs.

Although our RL-based approach and several works presented in the last section share the idea of using Machine Learning techniques to perform way-replacement in caches, they differ in several aspects. Table 3.1 shows characteristics of past work on ML-based way-replacement policies presented in Section 3.2.3, positioning ours against them in the State of the Art.

Table 3.1 – Comparison between related work on ML-based way-replacement

#	ML technique	Approach	Input data	Evaluation	Workload
1	MDP	online	hit/miss	Multicore simulation	Benchmark complete run
2	Regret minimization	online	reuse distance frequency	Cache simulation	Synthetic traces
3	LSTM and SVM	offline online	program counters	Multicore simulation	Benchmark sampling
4	Imitation learning	offline	memory accesses	Single core simulation	Benchmark sampling
5	RL	offline	memory accesses	Multicore simulation	Benchmark sampling
*	RL	online	cache coherence information	Multicore simulation	Benchmark complete run

1 – Maximizing cache performance under uncertainty, by Beckmann and Sanchez (2017), in HPCA
2 – Driving cache replacement with ML-based LeCaR, by Vietri et al. (2018), in USENIX
3 – Applying deep learning to the cache replacement problem, by Shi et al. (2019), in MICRO
4 – An imitation learning approach for cache replacement, by Liu et al. (2020), in PMLR
5 – Designing a cost-effective cache replacement policy using machine learning, by Sethumurugan et al. (2021), in HPCA
* The last line of the table represents our RL-based approach.

Source: Elaborated by the author

The main difference of our proposal comes from the use of Reinforcement Learning with different input data, which is cache coherence information. In this sense, our approach is complementary to the work mentioned above. In addition to that, some are different in terms of ML techniques used: our work uses Reinforcement Learning, whereas past ones used Regret Minimization, LSTM and SVM, Imitation Learning, and a single Markov Decision

Process. Furthermore, Sethumurugan et al. (2021) also used Reinforcement Learning with an offline approach, which might be a paradox considering RL purposes. Besides that, none but Beckmann and Sanchez (2017) used a complete run of workloads from a benchmark under a multicore simulation.

This overall analysis of the State of the Art and past work reinforces our contribution. Concerning the taxonomy from Jain and Lin (2019), we do not include our proposals in any group. First, because we do not devise a new policy, but we modify six Coarse-Grained policies. Second, we consider that our strategies are also architecture-aware policies, which is out of the taxonomy. Third, we also emphasize that most of the works listed here are recent to the time of this thesis. Finally, way-replacement based on coherence information and machine learning is a hot topic and deserves attention.

4 DESIGN AND EVALUATION METHODOLOGY

This chapter presents the evaluation methodology of the way-replacement policies proposed in this thesis. We start by describing our evaluation methods given the conceptual design of our proposals (Section 4.1). Then, we describe the experimental setup we use to implement and simulate these proposals (Section 4.2).

4.1 Evaluation methods

We elaborate our evaluation methodology to answer the research questions previously established. Thus, the first premise is to implement our way-replacement strategies in an environment that provides a flexible computer architecture. This environment must offer a multicore architecture with multiple cache levels and their nuances. In addition, it must allow the use of cache coherence protocols. With this scenario, we can implement our way-replacement strategies in the environment to assess their validity. Furthermore, we can execute applications on the modified architecture to understand the behavior of these strategies in different parallel workloads. We need to meet these and further requirements in the evaluation and validation of our proposals. These premises would guide their correct implementation and choose appropriate metrics to carry out the experimental analysis.

Some techniques for evaluating the performance of computer systems can be used, such as analytical modeling, simulation, and measurement. Jain (1991) lists criteria that we should consider to choose a good technique for evaluating our strategies. The first one concerns the expected innovation to what already exists. The second is related to how much time is available to conduct the research and experiments. Finally, the last criteria concern the costs to perform the technique. We addressed these criteria, deciding to use the simulation approach. Simulation allows us to identify aspects related to computer architecture that can be changed and compared to the current state. They are generally not fast; however, research time is usually sufficient for simulation batteries. In addition, simulation costs are low, mainly due to their flexibility and control of the environment in software.

In the experimental evaluation of our way-replacement strategies, we gather performance metrics from the simulation. The following sections describe our simulation setup, the workloads we use in our tests, and the metrics we collect and evaluate.

4.2 Experimental setup

To perform our experiments, we use Sniper (CARLSON et al., 2014), version 7.3, released in August 2019. It is an execution-driven multicore simulator based on the interval core model. Interval simulators model the timing individually, maintaining a window of instructions for each simulated core. Thus, their abstraction level is higher than when using

cycle-accurate simulators. Furthermore, Sniper implies cooperation between simulation and analytical modeling.

When running a single multi-threaded application, the simulator uses the Pin dynamic instrumentation framework (LUK et al., 2005). It simulates the region-of-interest (ROI) in detailed mode and fast-forwards other program regions to speed up simulation time. Thus, the application workload runs and Pin generates SIFT traces to be read by Sniper. SIFT is a technique proposed by Miranskyy et al. (2008) which filters irrelevant traces. The use of these tools in Sniper results in seamless deterministic execution of applications.

The simulator uses Python scripts to organize the execution of applications, generate traces, simulations, and provide outcome data. Furthermore, it uses hierarchical modules organization in its code, written in C/C++ language. In this sense, a wide range of computer architecture elements are already implemented, such as the cache controller and the cache itself. Therefore, most of our work is to implement or modify the way-replacement policies in this structure.

Sniper has integrated to its core the McPAT framework (Li et al., 2009), which we use to measure power and energy consumption. It uses the architecture parameters from Sniper, and simulation output to calculate these results. Thus, we do not modify the power and energy model in the framework.

4.2.1 Simulated architecture and preliminary design

We chose to simulate our proposals in a 16-core processor based on the x86 micro-architecture. Each core runs at 2.66 GHz, without Simultaneous Multithreading (SMT). We set the lithograph to 22 nanometers, both in Sniper and McPAT. Sniper models each core using the interval simulation approach. Due to this, the simulator maintains a window of instructions for these cores. This window corresponds to a Re-order Buffer (ROB) of a superscalar out-of-order pipeline. Thus, an analytical model produces the timing results for the cores instead of a fully implemented pipeline. To carry this out, this model considers the miss events which occur in branch prediction, caches, and TLBs.

The cache hierarchy has three levels. The Last Level Cache (LLC) is the L3, with a fixed 16MB size, 16-way set-associative, shared between all cores. We model the LLC as a “perfect” cache, i.e., all data requests are satisfied at this level. Further up in the hierarchy, there is the L2, which can be either private or shared. Thus, L2 can be private, one per core, with 256kB each, or a single shared L2 cache, totaling 4096kB for all 16 cores. Moreover, the first level consists of private L1 caches with 32kB for data and 32kB for instructions each.

Our experiments also evaluate how our proposals behave when the cache associativity or the size of cache blocks change. Thus, we vary the L2 cache associativity between 4, 8, and 16 ways. Furthermore, we set the size of the cache blocks to 64, 128, and 256 bits. It is important to remember that the block size must be the same throughout the cache hierarchy. Finally, we keep the associativity equal to 8 and use 64-bits cache blocks for all the other evaluations.

Table 4.1 – Cache hierarchy setup

Level	Size	Latency
L1	32 kB	4 cycles
L2	256 kB	16 cycles
	4096 kB	21 cycles
L3	16384 kB	42 cycles

Source: Elaborated by the author

Table 4.1 presents the access latency each cache has in our experiments, obtained with the aid of CACTI 6.0 (MURALIMANOHAR et al., 2009). We calculate the access latency based on the cache sizes only. We keep the values regardless of associativity or block size because the access latency is constant to our cache miss evaluation. Nonetheless, the impact would be mainly in the IPC, which we do not evaluate regarding associativity nor the size of the block.

The simulated cache hierarchy approximates those from modern conventional processors, such as in the Intel i9-10885H processor. Nonetheless, Sniper cannot reproduce some specific architectural characteristics. For instance, it can only simulate the inclusive cache model. In this model, if the cache controller can't find a cache block in the L1 but L2, the processor fetches it from the L2. However, before this process, a block copy is placed in L1. Besides that, data prefetching is disabled to reduce the possible noise that it would cause in our evaluations.

The architecture uses the "Modified–Exclusive–Shared–Invalid" (MESI) directory-based cache coherence protocol. Alternatively, one could also use the MOESI or MESIF protocols ("O" stands for "Owned" and "F" stands for "Forward"). We explain in Section 5 the impacts of changing the coherence protocol.

Concerning the way-replacement policies, we use the following well-known way-replacement policies: LFU, LRU, MRU, NRU, PLRU, and SRRIP. Our proposals, which we present in the following chapter, modify these algorithms.

Table 4.2 – Complexity comparison of way-replacement policies

The number of bits required to perform the policy is calculated as a function of the number of sets, denoted by N, and the associativity, denoted by K.

Policy	Bits required	On cache access
LFU	$N \cdot K \cdot log_2(K)$	Update frequency
LRU	$N \cdot K \cdot log_2(K)$	Update LRU order
MRU	$N \cdot log_2(K)$	Update MRU-bits
NRU	$N \cdot K$	Update NRU-bits
PLRU	$N \cdot (K - 1)$	Update tree bits
SRRIP	$N \cdot 2 \cdot K$	Update RRPV

Source: Elaborated by the author

Table 4.2 presents a modified version of the comparison made by Al-Zoubi et al. (2004). We added to this comparison the MRU, LFU, NRU, and RRIP algorithms and adapted them to

address our implementations. For each algorithm, we present their storage costs and the actions to be executed whenever cache accesses occur. The associativity is denoted by K, while the number of sets is denoted by N.

Each of our way-replacement strategies is applied to L1 and L2 simultaneously. For instance, if L1 caches use MRU, L2 caches also use MRU. We do that to avoid the local replacement hazard (ZAHRAN, 2007).

4.2.1.1 The next attempt strategy

The main goal of this thesis is not to devise an entirely new way-replacement algorithm. Instead, we claim that we can use the sharing state or the number of cores that share a cache block to enhance existing way-replacement algorithms. Furthermore, the proposal is to verify whether the choice of already known algorithms meets specific criteria.

To do so, we use an approach we call the *next attempt*. The idea is that the traditional algorithm runs unmodified to select a cache block. Next, our heuristics check the chosen block. Then, the algorithm decides to evict the block if it meets a specific condition. Otherwise, the algorithm keeps this block cached and selects the second option of the traditional algorithm.

For instance, LRU and PLRU algorithms select the least recently used block within the cache set. If this block does not match a given condition, our strategy does not evict it. Thus, the algorithm selects the second least recently used cache block. Similarly, the *next attempt* selects the second most recently used block when MRU is the way-replacement algorithm. For LFU, it chooses the cache block with the second-lowest frequency of use. The strategy costs rely on maintaining the first and second victims at each iteration.

NRU and SRRIP use the *Re-Reference Interval Prediction*; thus, they do not track a sequence of accesses or frequency. However, recall that they use a bit-wise strategy to perform the victim selection. Therefore, we tackle this problem by changing the NRU-Bit or the RRPV to a specific value. This way, the algorithm does not select the first cache block and runs another iteration. In these two algorithms, the additional cost is the need to rerun an iteration of the algorithm in the worst case.

4.2.2 Workloads

One goal of our work is to evaluate the behavior of our strategies when running parallel applications on the modified architecture. These applications must provide the workloads able to generate accesses and misses in the hierarchy, to call the way-replacement mechanism. Furthermore, they have to produce parallel workloads with communication between threads. These communications mean the use of memory, then generating coherence operations.

We could carry out our experiments using benchmarks such as SPLASH 2 (WOO et al., 1995), PARSEC (BIENIA et al., 2008), Rodinia (CHE et al., 2009), among others. These two first are available and ready to use in Sniper, which led us to use SPLASH 2 in our past

work (SOUZA et al., 2019). Thus, these benchmarks are suitable for this project; however, they may lack essential characteristics due to their age. Furthermore, the simulation method has the downside of taking a long time to be carried out, reducing our design space, considering workloads. Therefore, we opt to rely on the seven kernels from the CAP Bench suite (SOUZA et al., 2017).

Souza et al. (2017) developed the benchmark suite as a baseline to design, program, evaluate and learn about low-power manycore architectures. Nevertheless, the benchmark offers a version for use on x86 architectures, implemented in OpenMP [1]. Moreover, these kernels cover application design aspects (e.g., parallel patterns), focusing on manycore architectures even in their OpenMP version. This way, we conclude that the benchmark and the workload it produces are suitable for our simulation methodology. Next, we present an overview of each kernel.

Features from Accelerated Segment Test (FAST) is an image corner detection method implemented following the Stencil parallel pattern. Unfortunately, the implementation in CAP Bench did not follow any optimization to improve cache usage. Thus, we expect to understand how our strategies perform under this cache-constrained condition. The workload input we use is an image of 8192×8192 pixels.

Friendly Numbers (FN) computes and compares the *abundancy* of numbers in an interval, which is a mathematical property. It follows the MapReduce pattern, in which the algorithm maps the mathematical operation to each thread and applies a reduction, comparing the results in the end. FN is mainly a CPU-bound algorithm with few accesses to the memory. The interval of numbers used in our experiments is $8 \times 10^6 + 1$ to $8 \times 10^6 + 2^{14}$.

The Gaussian Filter (GF) is a filter algorithm that also deals with images using matrix convolution. It also follows the Stencil pattern; however, it performs fewer memory operations than FAST. Besides that, threads in this algorithm have equally distributed workloads. The input image has 8192×8192 pixels, and the algorithm uses masks of 11×11 pixels.

Integer Sort (IS) is an algorithm to sort a certain amount of integer numbers using the bucket sort strategy. This algorithm follows the Divide and Conquer pattern, which divides the data into several buckets. This way, the algorithm can produce irregular access patterns from threads. In our experiments, we set an amount of 2^{25} integer numbers as input size.

K-Means (KM) is a well-known clustering algorithm. CAP Bench implementation uses Lloyd's algorithm to partition points into groups, using the Euclidean distance as the measure of similarity. The parallel pattern that KM follows is Map, assigning part of the computation to each available thread. These threads irregularly access memory due to changes in which points pertain to each cluster. The input data is $2^{14}\mathbb{R}^{16}$ points and 512 centroids.

Lower-Upper Factorization (LU) is a matrix decomposition algorithm, which follows the Workpool parallel pattern. In this pattern, each thread has its independent tasks, reducing the use of shared data. However, there are more data exchanges within single threads due to matrix traversal in the algorithm. The workload in our experiments consists of a 1536×1536 matrix.

[1] Code available in <https://github.com/cart-pucminas/CAPBenchmarks>.

Table 4.3 – Characteristics of each application available in CAP Bench.

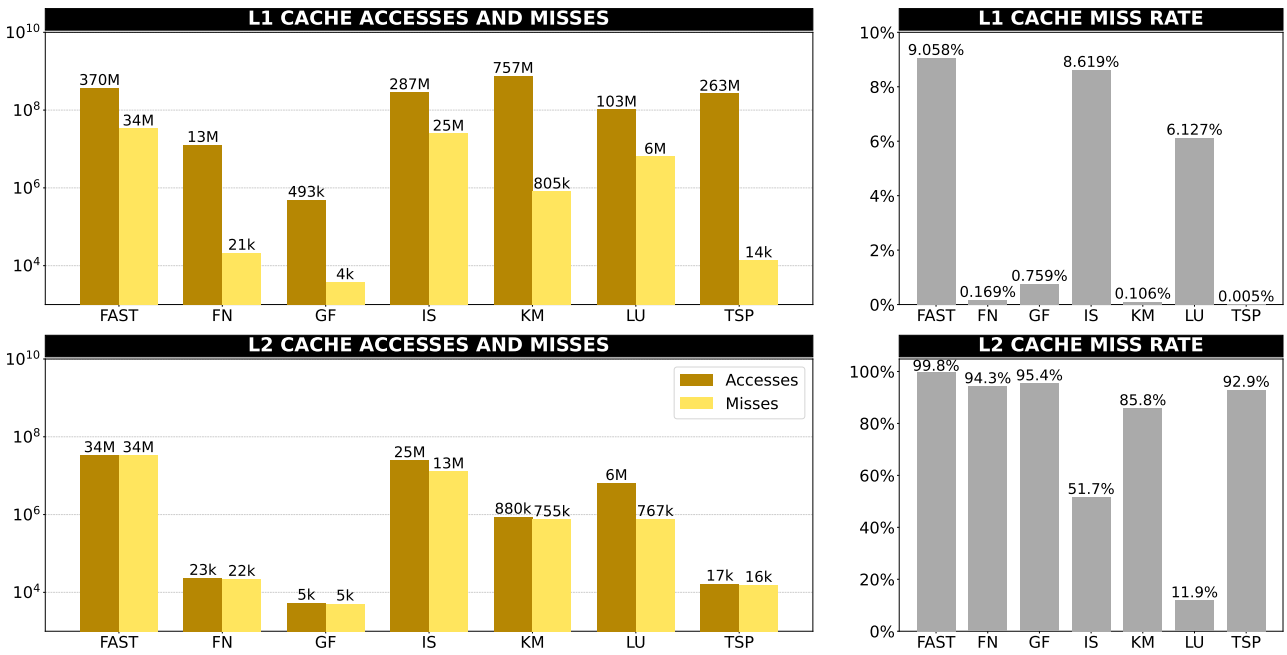
App.	Parallel patterns	Job Types	Communication intensities	Task loads
FAST	Stencil	CPU-bound Communication-bound	High	Irregular
FN	MapReduce	CPU-bound	Low	Regular
GF	Stencil	CPU-bound	Average	Regular
IS	Divide and Conquer	Communication-bound	High	Irregular
KM	Map	CPU-bound Memory-bound	High	Irregular
LU	Workpool	Communication-bound	High	Regular
TSP	Workpool	CPU-bound	Low	Irregular

Source: Souza et al. (2017)

Finally, the Traveling-Salesman Problem (TSP) solves the routing problem in a specific traveling scenario. Like LU, TSP follows the Workpool parallel pattern, based on the branch-and-bound method. However, this algorithm does not produce relevant memory access since it mainly computes traveling costs. We simulate this kernel in a scenario with 17 towns.

Table 4.3 summarizes the characteristics of all CAP Bench kernels. Furthermore, Figure 4.1 shows the number of accesses and misses of each kernel and the cache miss rate for L1 and L2 caches. We collected these values from the previously described architecture, using the unmodified LRU way-replacement policy in caches. We observe that four kernels (FAST, IS, KM, LU) present more accesses in L1 and L2 than the others. This behavior is because they

Figure 4.1 – CAP Bench kernels - cache accesses and misses



Source: Elaborated by the author

have high communication intensities between threads, which means they access shared data. It is interesting to evaluate it concerning the cache coherence protocol. We also observe high cache miss rates in the L2 caches. It can occur due to (i) lack of code optimization in the kernels to improve the use of caches; or (ii) fewer accesses to L2 than in L1, since the data request is satisfied in L1, and then, most of the misses are compulsory. In the experimental results sections, we further discuss these characteristics concerning our strategies.

4.2.3 Performance metrics and evaluation perspectives

To characterize the way-replacement approaches and their behavior, we use different metrics. First, we evaluate the **cache miss rate** from data caches – the number of accesses not satisfied by a cache (misses), divided by the total number of accesses (hits + misses). The cache miss rate is the primary metric we use to evaluate our proposals because way-replacement algorithms directly impact the presence of blocks in the cache. We do not consider the misses from instruction caches in our experiments.

Next, **coherence traffic** is measured. We do this by measuring and comparing the amount of invalidate and write-back operations between strategies. Recall that the coherence protocol can impact overall performance, i.e., the fewer the coherence operations, the better. So the goal of understanding these metrics is to assess whether or not our strategies can improve performance in two ways: reducing the drawbacks of coherence and favoring its benefits.

A more general metric is the **instructions per cycle (IPC)**, which is, as the name says, the number of instructions executed in a clock cycle. In our experiments, IPC corresponds to the throughput of instructions in the processor, thus, a measure of observed performance. Memory access instructions take more cycles to complete due to memory elements being slower than the processor. Furthermore, as we mentioned in Section 2.3, cache levels far from the processor are even slower, so computation takes more cycles to complete. Thus, the IPC will vary mainly depending on the cache miss rates presented by the applications.

Likewise, we measure the **energy consumption** – the product of the time to solution and the power consumed when a kernel runs on the architecture. If cache usage is improved, this will result in less computing time and less use of cache elements that consume more power.

To provide a broader understanding of the experiments performed, Table 4.4 presents the simulation parameters. We discuss the meaning of the *Strategy* parameter in the following chapter. The number of simulations performed in this work is up to 6.804, which may be unfeasible in time and resource-constrained research scenarios.

To surpass this challenge, we chose to evaluate our results from different perspectives. First, we verify the cache miss behavior when varying the associativity using our algorithms. Next, we re-evaluate cache miss results, varying the sharing strategy L2 cache (private and shared). Then, we compare the unmodified way-replacement algorithms (Baseline) against each of our proposals, using 8-way associative, 64-bit private L2 caches. We also used this last perspective

Table 4.4 – Simulation parameters

Sharing strategy	Associativity	Block size	Policy	Strategy	Application
Private L2	4-way	64b	LFU	Baseline	FN
Shared L2	8-way	128b	LRU	Dynamic	FAST
	16-way	256b	MRU	Modified	GF
			NRU	Exclusive	IS
			PLRU	Shared	LU
			SRRIP	Bitset	KM
				RL-Bitset	TSP
				RL-State	
				RL-Both	

Source: Elaborated by the author

for the remaining metrics. The next chapter presents our proposals, describing our design and the results obtained following the methodology mentioned above.

5 COHERENCE STATE-BASED EVICTION

In our first proposal, the decision of whether to evict or not the cache block depends on its current coherence state. This approach is subdivided into two strategies that differ in the choice of the coherence state to be checked, which can be either **dynamic** or **static**. Figure 5.1 shows an overview of the **dynamic** strategy.

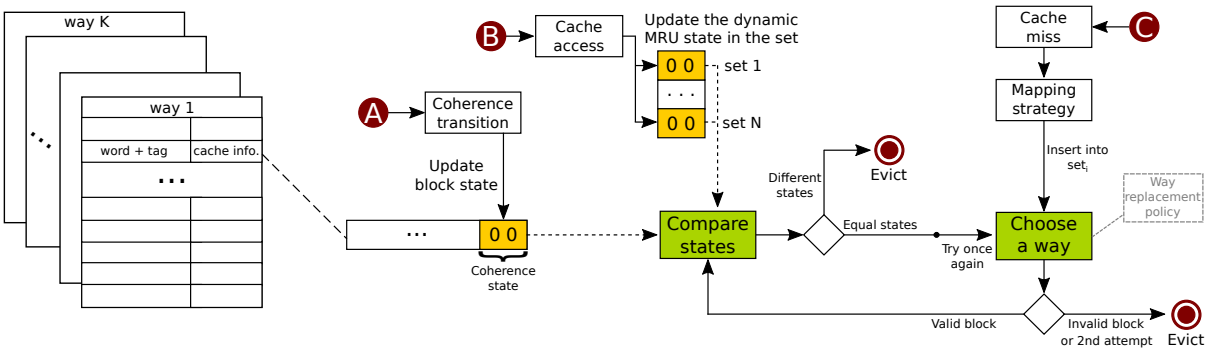
On the left side, we depict a K-way set-associative cache with N cache sets. Thus, this cache has $K \cdot N$ cache blocks. Each cache block is composed of the word, which is the data to be used; the tag for different memory addresses in the same set; and the 'cache info.', which is a group of bits used for control purposes. This group represents any information about the cache entry. For instance, the valid bit and coherence directory information.

Each cache block has its coherence state, which is updated when there is a coherence transition, as in A. Since the MESI protocol has four states, we have two bits to represent them ($log_2 4 = 2$). If the coherence protocol had up to eight states (e.g., MOESI and MESIF), the strategy would require three bits. Thus, the cache already has a total of $2 * K * N$ bits to keep the coherence state information for their cache blocks.

The core of the coherence state-based eviction is to check if the cache block is in a certain coherence state before evicting it. To carry this out dynamically, we adopt the Most Recently Used State (MRU-State) approach. In this fashion, each cache set knows the state the most recently used block is, either *Modified*, *Exclusive*, or *Shared*. Additional bits are required per set, thus adding $2 * N$ bits to the cache. When cache access occurs on any cache block in a set, its respective MRU-State is updated, as depicted in B.

Meanwhile, in C a cache miss happens, thus, the cache controller must carry out the mapping strategy to find which set will receive the copy of the block from the lower level. After this, the way-replacement algorithm must select one way to have its cache block evicted. If this old cache block is valid, its coherence state is then compared to the MRU-State from the set. The algorithm favors blocks that are not in the same state as the current MRU-State for eviction. If they are different, the cache block is evicted, otherwise, the strategy evicts the cache block from the next attempt.

Figure 5.1 – Overview of the coherence state-based eviction



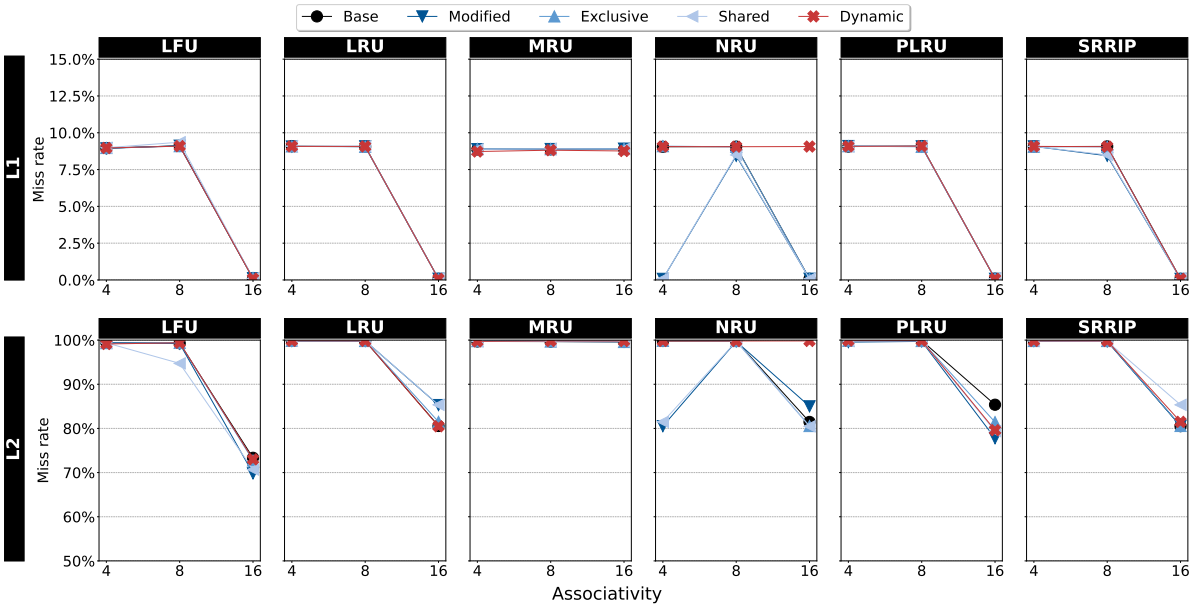
Source: Elaborated by the author

In the **static** coherence state-based strategy, we set a global state that has priority for staying in the cache. This strategy follows the same process depicted in Figure 5.1. The only difference is that in the static approach, it is not necessary to maintain and update the MRU-State in B. The global state can be *Modified*, *Exclusive*, or *Shared*. We do not set *Invalid* as an option because invalid cache blocks should be the first ones to be evicted, independently of the strategy. The static approach is less costly than dynamic because it does not require tracking the MRU-State for every cache set.

5.1 Experimental results

This section presents the experimental evaluation of the coherence state-based strategies. The first metric we want to evaluate is the cache miss rate. Thus, we carried out simulations when the caches have different associativity, cache block sizes, and sharing strategies. We also evaluate the results in terms of coherence operations, measuring invalidates and write-backs. Finally, we present instructions per cycle and energy results. Static strategies are named with the chosen state to keep (*Modified*, *Exclusive*, and *Shared*). Likewise, *Dynamic* pertains to the dynamic MRU-State strategy. The baseline (Base) is the unmodified policy. Tables with all the results presented in the charts are available in Appendix A.

Figure 5.2 – Cache miss rate vs. associativity – FAST



Source: Elaborated by the author

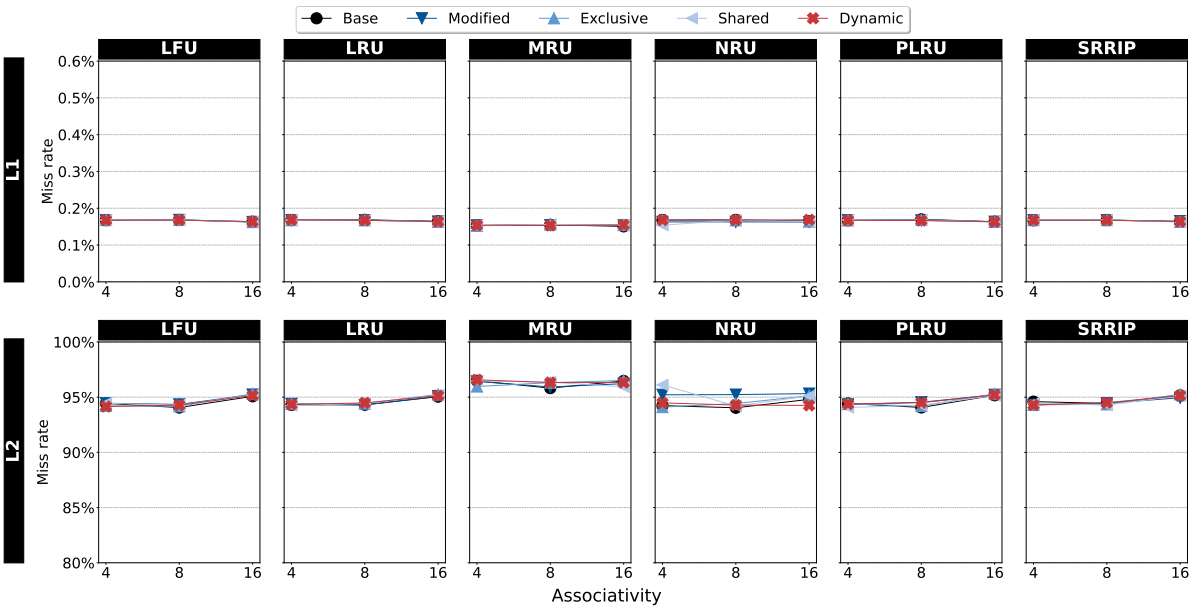
5.1.1 Cache miss rate

First, we simulated the architecture varying associativity in both private L1 and private L2 caches, with cache block sizes fixed to 64 bits. Figure 5.2 shows the L1 and L2 cache miss

results obtained from FAST with the 6 way-replacement algorithms. Most of the cases show a high decrease in cache miss when using 16-way set-associative caches. Despite using the stencil pattern, FAST implementation in CAP Benchmarks is not optimized for better use of the cache. If we consider the 2D matrix with rows contiguous in memory, horizontally related data will tend to belong to the same cache line, and vertical offset access would most likely result in cache misses. Thus, with higher associativity, we have less competition within a cache set between data from specific chunks. Nevertheless, this is not the case for MRU, which chooses to evict the most recently used block. This strategy conflicts with the idea of working on small chunks of data. Thus, varying associativity with MRU did not result in better miss rates.

For the L1 cache, the best rate is 0.07%, observed in the NRU/Shared, with 16-way caches. Most of the best results for each associativity are perceived in our strategies, except for three cases: Using 16-way L1 caches, LFU (0.09%), LRU (0.08%), and SRRIP (0.08%) show no gains when our strategies are employed. However, their cache miss rates are at least equal to the Dynamic approach results. Concerning the L2 cache, we observe from Figure 5.2 that FAST presents high cache miss rates, due to the lack of cache optimization in its implementation. Even so, we notice a behavior similar to that which occurred in L1. For NRU, we have a much lower cache miss rate in 4-way and 16-way associative caches when keeping cache blocks with Modified and Shared states. For instance, we find 80.48% as the best miss rate result for NRU running FAST (16-way with the Shared state being preserved). The best result among all four strategies is obtained with LFU using 16-way caches and the Shared approach (70.68%).

Figure 5.3 – Cache miss rate vs. associativity – FN

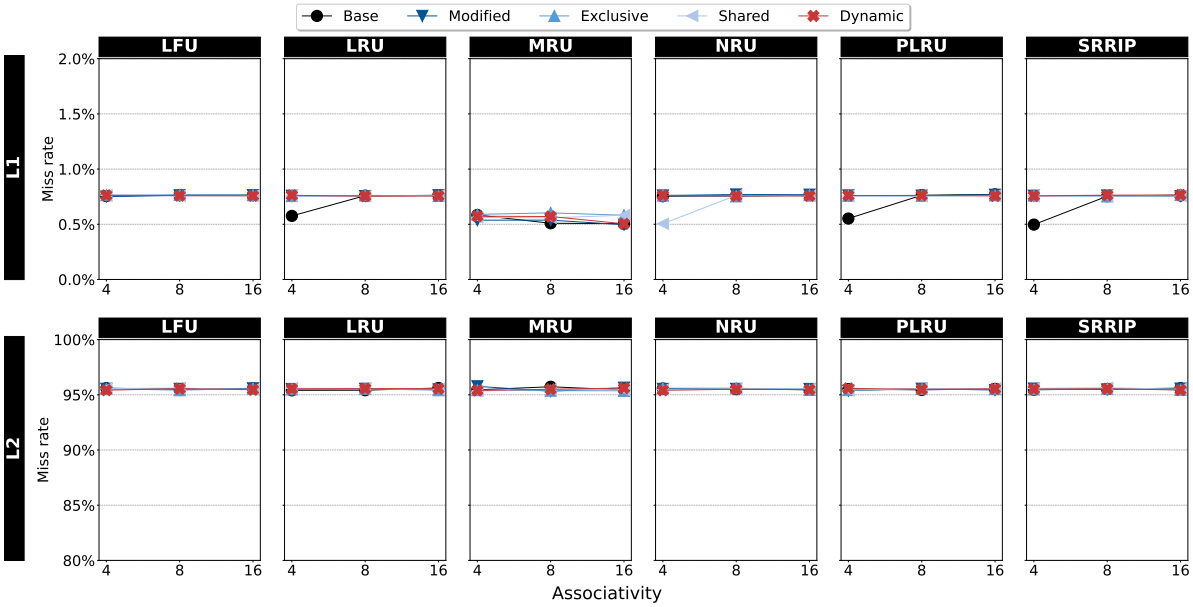


Source: Elaborated by the author

From Figure 5.3, we observe that the FN kernel presents low rates in the L1 cache since it can find most of its data at this level. Furthermore, there are few data exchanges in this algorithm. Thus, most of the misses in the L2 cache are compulsory ones, resulting in high

rates. Regarding associativity, there is no change when we vary the number of ways in the L1. The rates are very low, which inhibits the perception of any changes.

Figure 5.4 – Cache miss rate vs. associativity – GF



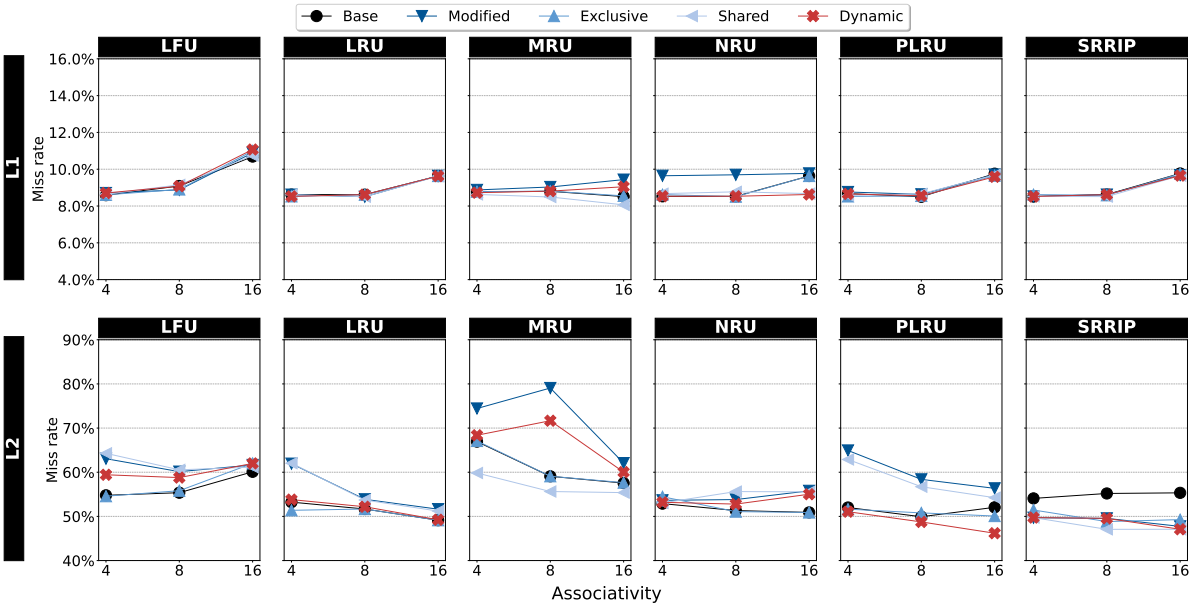
Source: Elaborated by the author

For the L2, the miss rate increases when the associativity is also increased to 16. Although increasing associativity might reduce conflict misses, FN performs fewer accesses in the L2 than some other kernels. Thus, the increase in the miss rate is due to the eviction of useful blocks within the same set. GF shows almost the same behavior as FN, which is depicted in Figure 5.4. However, our strategies are worse than the baseline for three policies using 4-way associative L1 caches. Moreover, there are no perceptible changes in the L2 miss rate when the associativity changes.

The cache miss results from IS are presented in Figure 5.5. A common behavior with this kernel is that varying from 4 to 8 ways does not have much effect in L1 misses. However, when we use 16 ways, the cache miss also increases. MRU presents a different behavior when more ways are used. IS realizes irregular memory accesses, but within each bucket sort, there are partially cached data that are sequentially read, before being exchanged if necessary. Generally, the MRU policy is defeated in this scenario. Nevertheless, using greater associativity while keeping cache blocks with Shared states in the cache could reduce miss rates. Likewise, NRU in 16-way associative caches takes advantage of the block in the Shared state, which would be read again in the sorting process.

In private L2 caches, we observe a decrease in the miss rate when associativity is increased with LRU and PLRU. We highlight the Dynamic strategy behavior with PLRU, which beats all other 4 strategies regardless of associativity. Besides that, with SRRIP, all strategies are better than baseline, also regardless of the associativity. The MRU algorithm shows inconsistent results with those in the L1, due to the irregularity in IS and to the low effectiveness of the MRU.

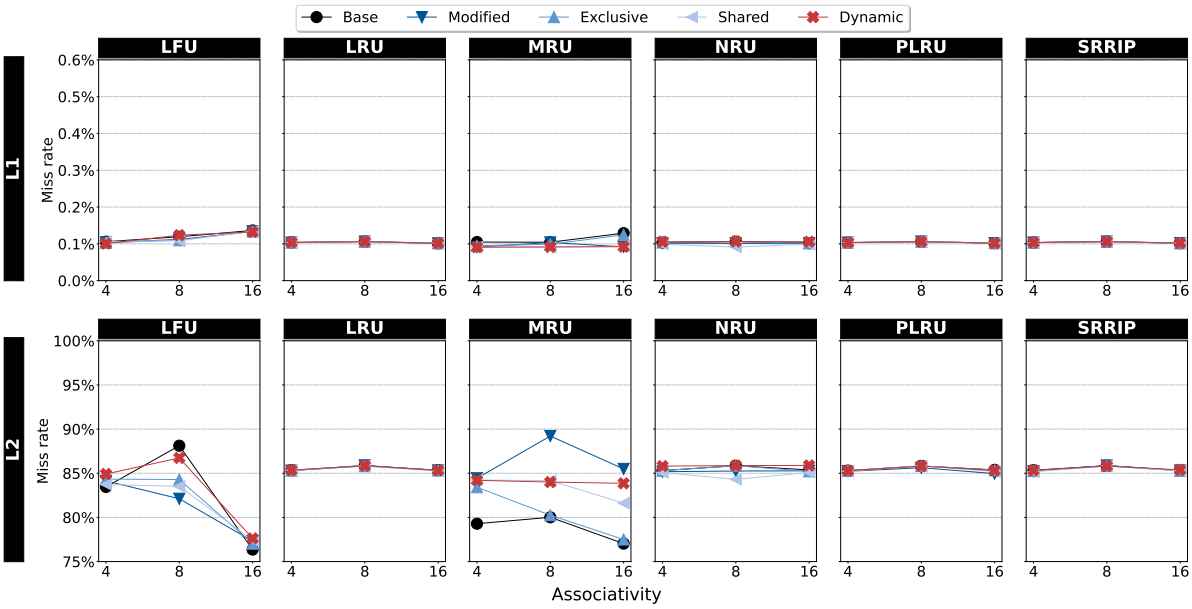
Figure 5.5 – Cache miss rate vs. associativity - IS



Source: Elaborated by the author

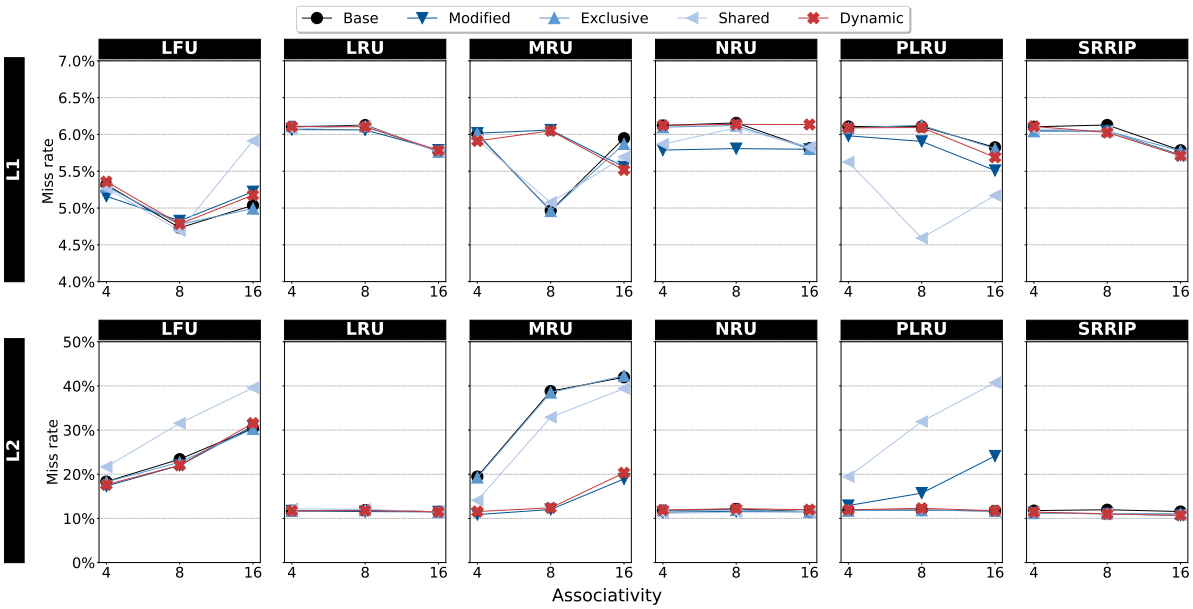
Figure 5.6 shows the miss rate results for KM. The kernel presents L1 cache miss rates below 0.2%, because the number of accesses is much higher than the number of misses at this level. This is due to the strategy used to implement the kernel, mapping a set of points to each thread, thus performing isolated but intense computation on each core. Therefore, the KM kernel does not present noteworthy changes when the associativity is increased. There is not much difference in the L2 miss rates between LRU, NRU, PLRU, and SRRIP and their variations concerning associativity. Besides performing isolated computation on each thread, KM also carries out irregular memory access patterns, mainly in the centroid update phase.

Figure 5.6 – Cache miss rate vs. associativity - KM



Source: Elaborated by the author

Figure 5.7 – Cache miss rate vs. associativity - LU



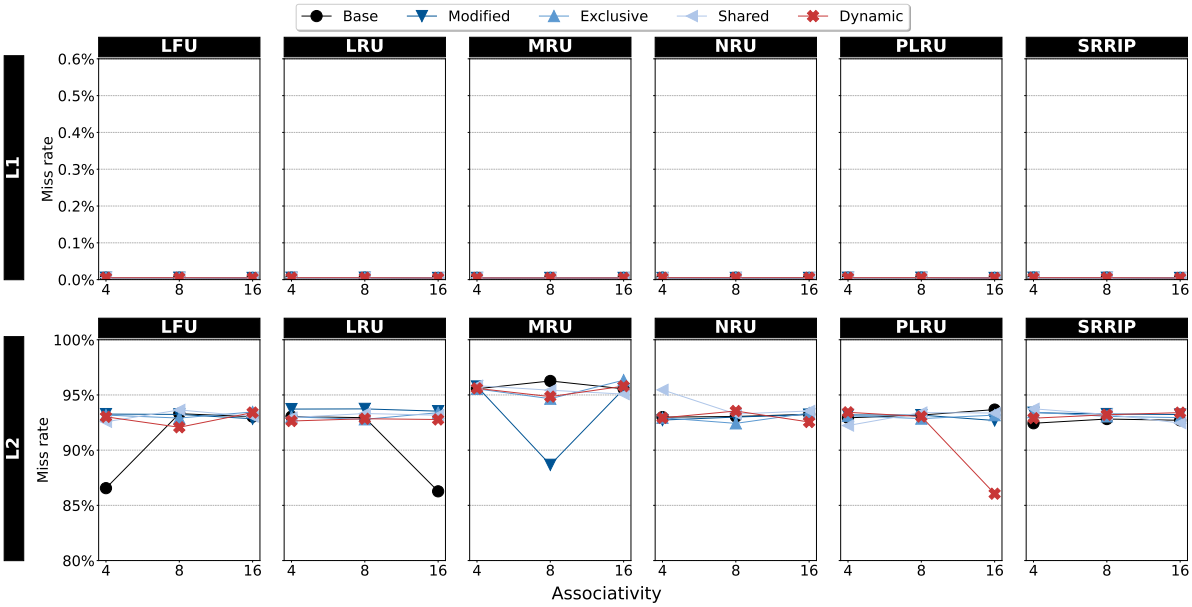
Source: Elaborated by the author

Thus, in this phase, it produces memory-intensive operations on different addresses, increasing the pressure at L2, and consequently benefiting from algorithms that consider the least or not recently used blocks. For this reason, LFU and MRU have different and non-standard behaviors, and, generally speaking, this impact is reduced when the associativity is increased.

The charts in Figure 5.7 show cache miss rates for the LU kernel. For LRU, NRU, and SRRIP, the common pattern is to have a minor increase in cache miss when associativity varies from 4 to 8, and a slight reduction when increasing it to 16. LFU and MRU, in most cases, present a different behavior, in which 8-way associative caches are the best ones. LU has a heterogeneous communication pattern, which may lead to data reuse by many threads, thus more cache blocks being in the Shared state (CRUZ et al., 2011). Thus, PLRU in the 8-way cache is more flexible than with 4 or 16 ways, and even better than the LRU itself, for the LU kernel access pattern. PLRU has a bit reset mechanism, which helps to prevent less useful data from staying in the cache.

With regard to L2 cache miss rates for the LU kernel, there are no gains from increasing cache associativity. Nevertheless, we highlight that for MRU, we achieve much better results with *Dynamic* and *Static-Modified* approaches than the baseline, regardless of the associativity. A controversial result occurs for PLRU since the *Shared state* strategy is the worst in L2, but the better in L1. If we consider that these misses in L1 lead to accesses in L2, we can say that, in this specific case, the lower the miss rates in L1, the higher the miss rates in L2.

Figure 5.8 – Cache miss rate vs. associativity – TSP



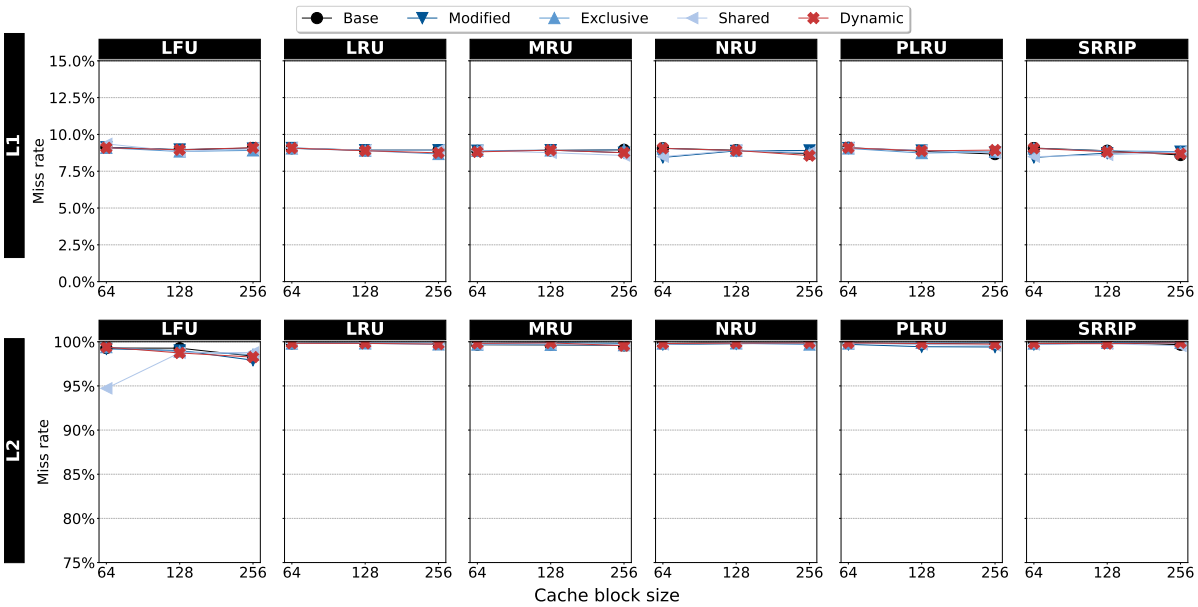
Source: Elaborated by the author

Finally, Figure 5.8 shows the miss rates observed in TSP. As for FN and GF, TSP does not perform intensive memory operations. In other words, it requests data that is usually already at the cache level closest to the processor. This leads to the low L1 miss rates, and, consequently, the low impact of varying the associativity. On the other hand, it barely requests data in the L2 cache. Despite that, our strategies do not follow the baseline in LFU/4-way and LRU/16-way configurations. However, with PLRU, the Dynamic approach reduces the rate when the associativity is increased.

Varying the cache block size is another evaluation perspective to understand the behavior of our way-replacement strategies. We carried out experiments using cache block sizes of 64, 128, and 256 bits, equally for all cache layers, while keeping the associativity fixed to 8 ways.

Figure 5.9 shows the miss rates from FAST. We do not observe considerable changes in the L1 miss rate when varying the cache block size. The exception is in the NRU and SRRIP algorithms when the Modified and Shared states are preserved. In these situations, the results are slightly better than the others, with lower miss rates. Modified and Shared are blocks that are present in other caches. We assume that preserving these blocks in cache favors the re-reference interval in NRU and SRRIP strategies. Thus, also with regard to the access pattern from FAST, they somehow better predict the future accesses of adjacent data with 64-bit blocks. On the L2 cache, the high cache miss rates provide a more accurate graphical evaluation. Still, it is possible to observe that, for most policies, little variation is achieved by changing the size of cache blocks. In the case of LFU, 256-bit blocks show better results, except for the strategy that preserves the Shared state with 64-bit blocks, which is the best result in the chart.

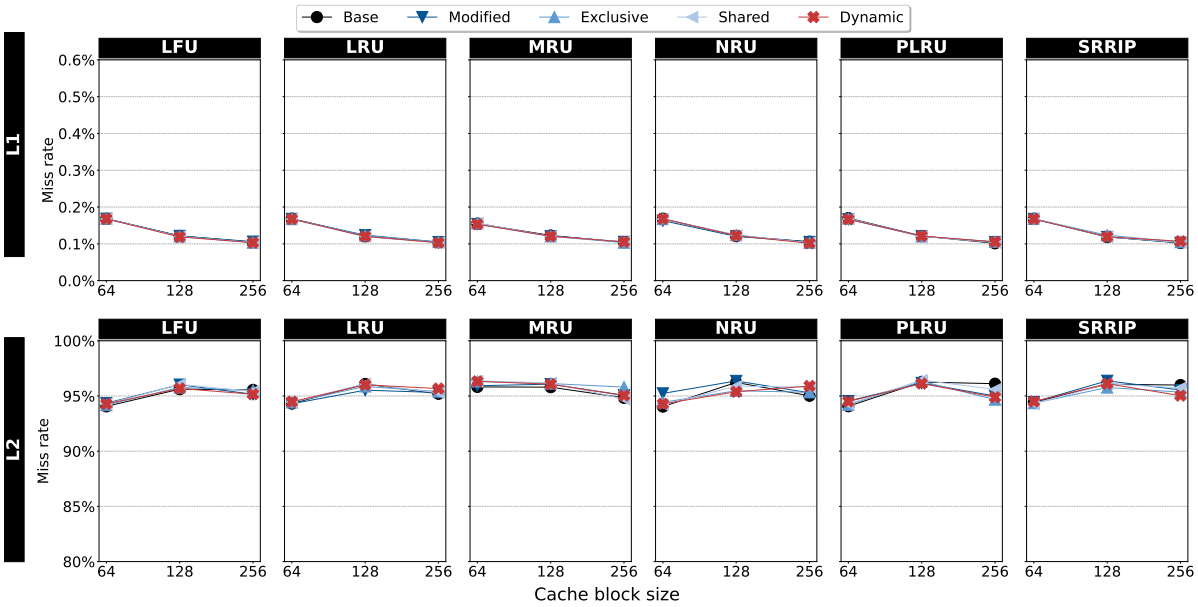
Figure 5.9 – Cache miss rate vs. block size - FAST



Source: Elaborated by the author

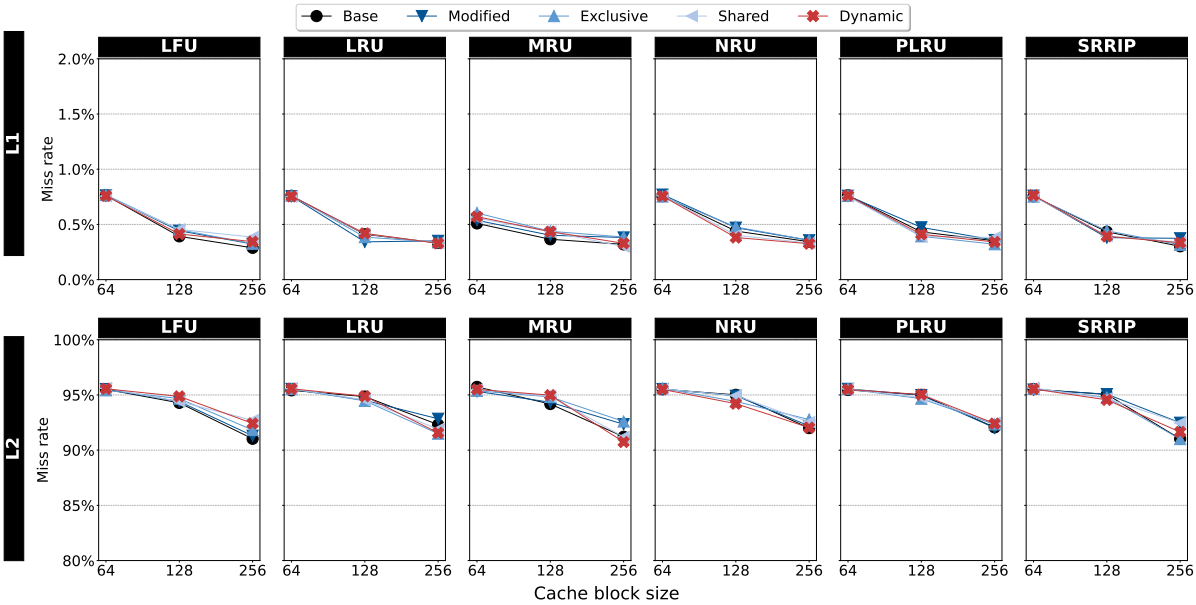
For FN, the results are depicted in Figure 5.10. Despite the low values, we observe that increasing the block size reduces the miss rate in the L1. The block increase favors the spatial locality principle since neighboring data is fetched at the same time, thus avoiding a future miss. This statement is also applied to the L2. However, the effects are less significant, due to the few accesses the algorithm performs at this level. The general behavior in the L2 is a minor increase when the block size changes from 64 to 128 bits, and a decrease if it is further increased to 256 bits. Recall that the gains in the L1 overwhelm the L2 results, thus fewer accesses at the L2 are performed, increasing the rate.

Figure 5.10 – Cache miss rate vs. block size - FN



Source: Elaborated by the author

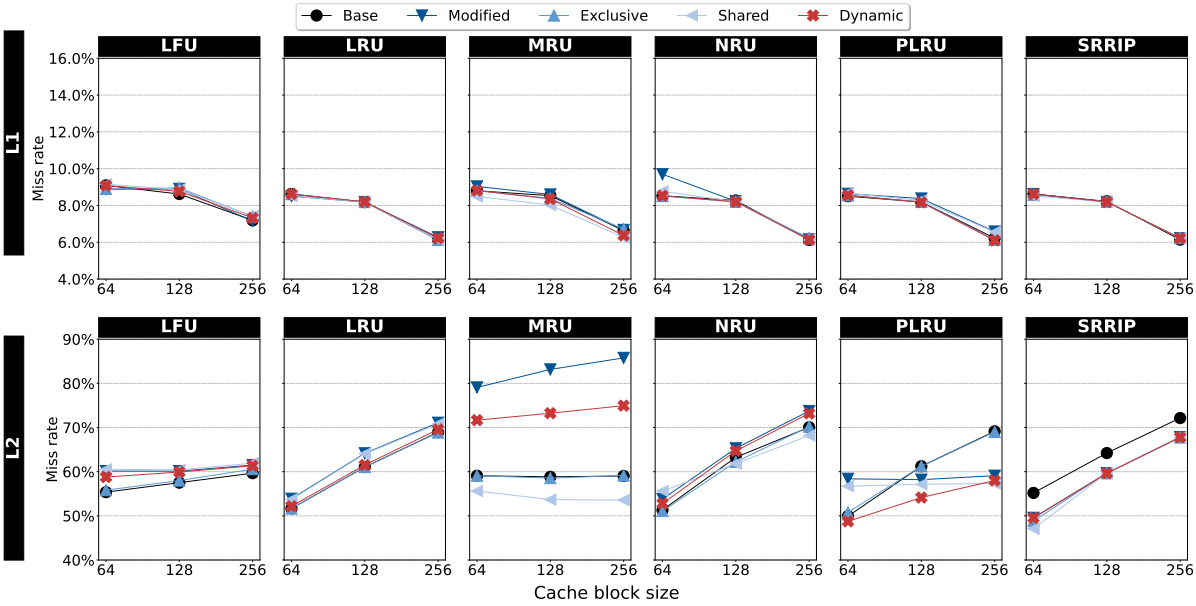
Figure 5.11 – Cache miss rate vs. block size - GF



Source: Elaborated by the author

Figure 5.11 shows GF results when we vary the cache block size. For both cache levels, the bigger the block, the lower the miss rate. GF performs less memory access than FN, however, it deals with image matrices. Thus, it requests more data exchanges, which is favored by bigger cache blocks – data that would be requested in the future is already in the cache.

Figure 5.12 – Cache miss rate vs. block size - IS

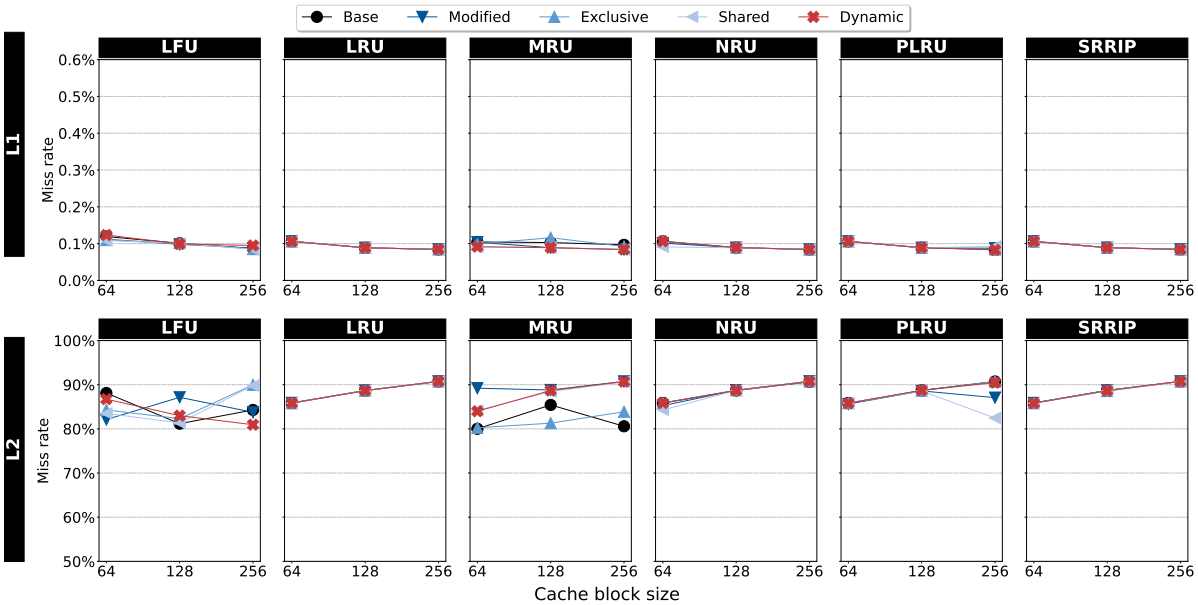


Source: Elaborated by the author

For IS, L1 cache misses are more likely to be reduced with bigger cache blocks, as depicted in Figure 5.12. We associate this to two factors: First, IS is an algorithm that deals exclusively with integers, thus, the 256-bit cache block can handle up to 8 32-bit integers. Second, a single thread performs the sorting within a small set of integers in the L1, without so many conflicts.

On the other hand, L2 cache miss rates present a different behavior. The overall behavior consists of an increase in cache misses when the cache blocks are bigger. Since the architecture could provide most of the data at the L1 level, the number of accesses in L2 reduces, thus, the cache miss rates in L2 present the reverse pattern to the L1. The strategies using the MRU policy, as well as the PLRU with Modified and Shared states preserved, do not follow the overall behavior. We made the same assumption from IS irregular memory accesses when we changed the number of ways here. A specific highlight for SRRIP, which presents better cache miss results than the baseline regardless of the cache block size.

Figure 5.13 – Cache miss rate vs. block size - KM

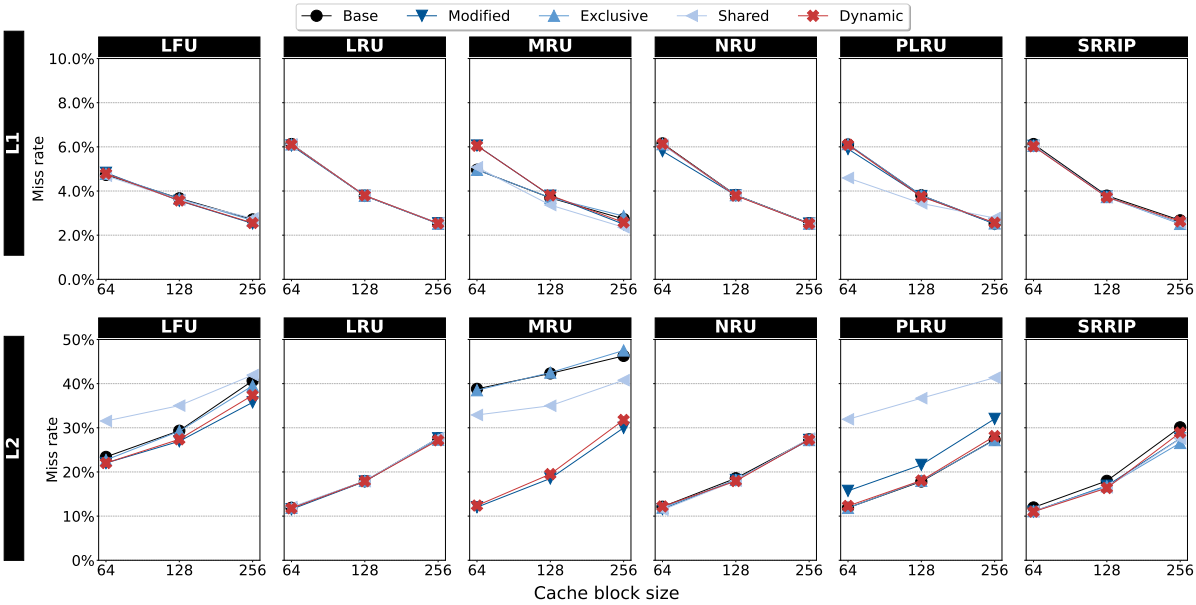


Source: Elaborated by the author

Figure 5.13 shows KM cache miss rates when we vary the cache block sizes. Despite the L1 miss rates being too small to be graphically evaluated, we observe a cache miss reduction with higher cache blocks, as well as in FAST and IS. The reverse pattern appears again for LRU, NRU, PLRU, and SRRIP, when we move from the L1 evaluation to the L2 one. For LFU with 64-bit blocks, all strategies are better than the baseline. However, when we increase the cache block size to 256 bits, the baseline can be only beaten by the Dynamic state approach. With the MRU policy, the only approach that surpasses the baseline is the one that maintains the Exclusive state, for the 128-bit configuration.

The cache miss rates from LU are presented in Figure 5.14. The L1 rates from LU are similar to the ones from IS, with a more accentuated decrease than in other kernels, when the cache block is increased. Since LU is a matrix factorization algorithm, larger blocks might favor the spatial locality for future data reuse. Like in the other kernels, the opposite occurs in the L2 level, i.e., larger cache blocks present the worst results. We observe in MRU that the baseline and two of our approaches (the ones which keep blocks in Exclusive and Shared states) are worse than most of the LRU-based and RRIP-based approaches. However, we highlight an

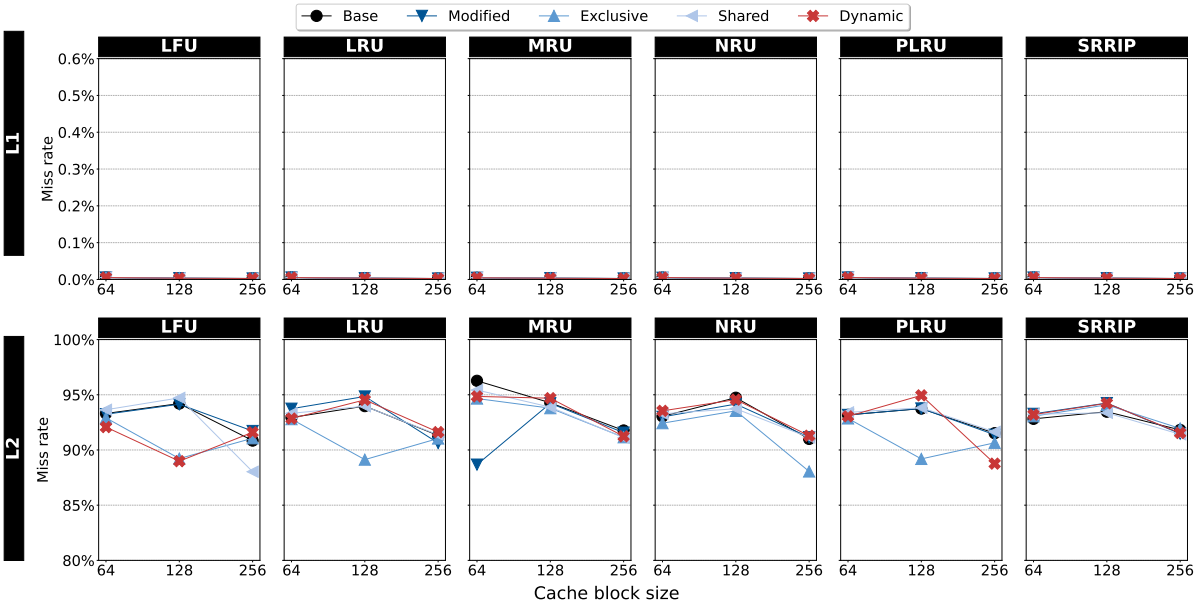
Figure 5.14 – Cache miss rate vs. block size - LU



Source: Elaborated by the author

interesting behavior with MRU: using Dynamic MRU-State strategy and the Modified-based strategy can compete with the others. Recall that the MRU is a policy with less computation and memory complexity than the others.

Figure 5.15 – Cache miss rate vs. block size - TSP

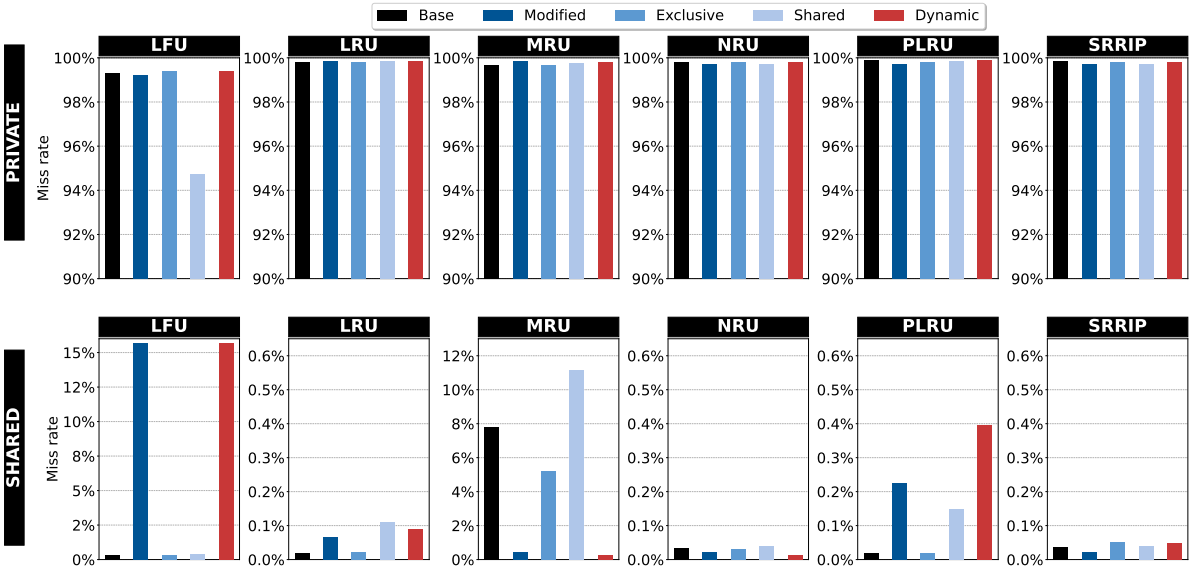


Source: Elaborated by the author

TSP miss rate results are shown in Figure 5.15. When we vary the block size of the L1 cache, we observe the same behavior as when we vary the associativity. The algorithm accesses blocks that do not change while computing occurs, increasing the hit rate. Thus, varying cache block size has little impact on L1 miss rates. In the L2 cache, we found that bigger block sizes produce better results in general. We highlight the Dynamic and Static/Modified approaches, which reduce the miss rate for LFU, NRU, and PLRU with bigger cache block sizes.

Another aspect we use to evaluate our way-replacement approaches is varying the sharing strategy. Thus, we simulate our strategies with the L2 cache being either private or shared.

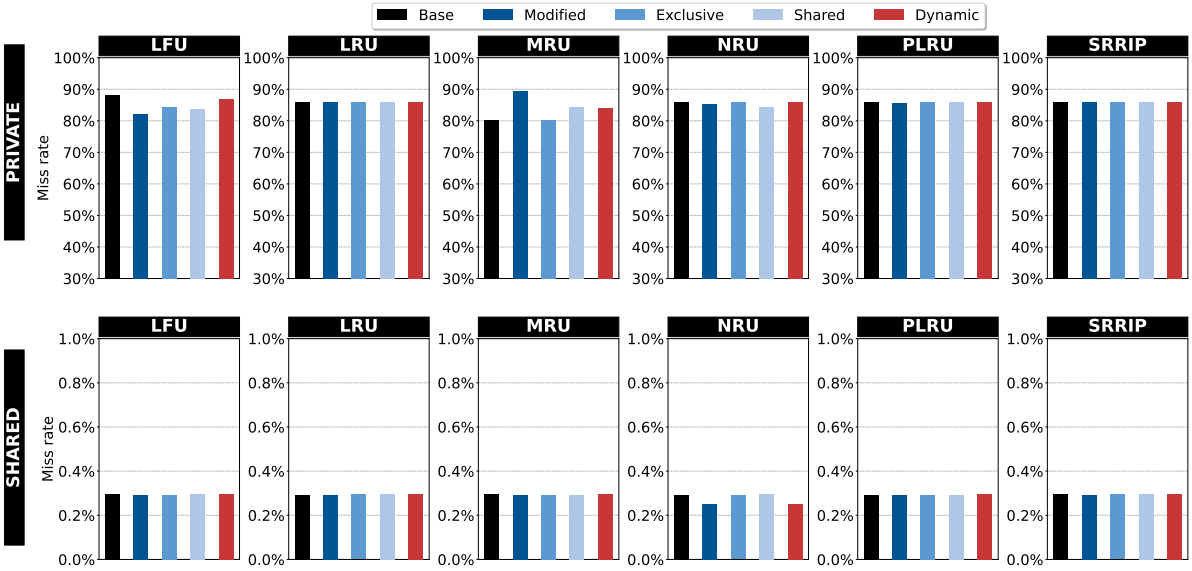
Figure 5.16 – L2 cache miss rate vs. sharing strategy - FAST



Source: Elaborated by the author

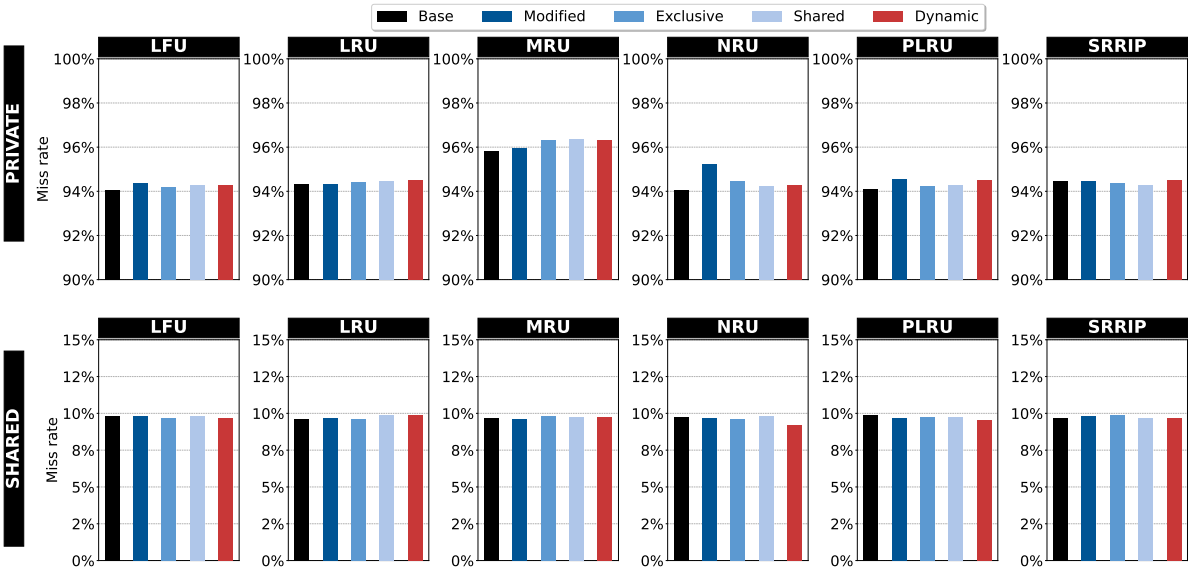
Figures 5.16 and 5.17 present the L2 cache miss rates from FAST and KM respectively. We observe shared L2 cache miss rates much lower than in private ones. Most of the kernels from the CAP Bench were developed thinking of fine-grained tasks. Thus, the data the threads from such applications use can mostly be provided in L1. However, they are still parallel kernels that share the same data. Thus, the shared L2 cache in the 16-core architecture presents lower cache miss rates. Notwithstanding, FAST presents different values in LFU, in which the Modified strategies are worse than the others. We assume here that Modified and less frequently used blocks are not the best to keep for FAST while traversing the image to detect corners.

Figure 5.17 – L2 cache miss rate vs. sharing strategy - KM



Source: Elaborated by the author

Figure 5.18 – L2 cache miss rate vs. sharing strategy - FN

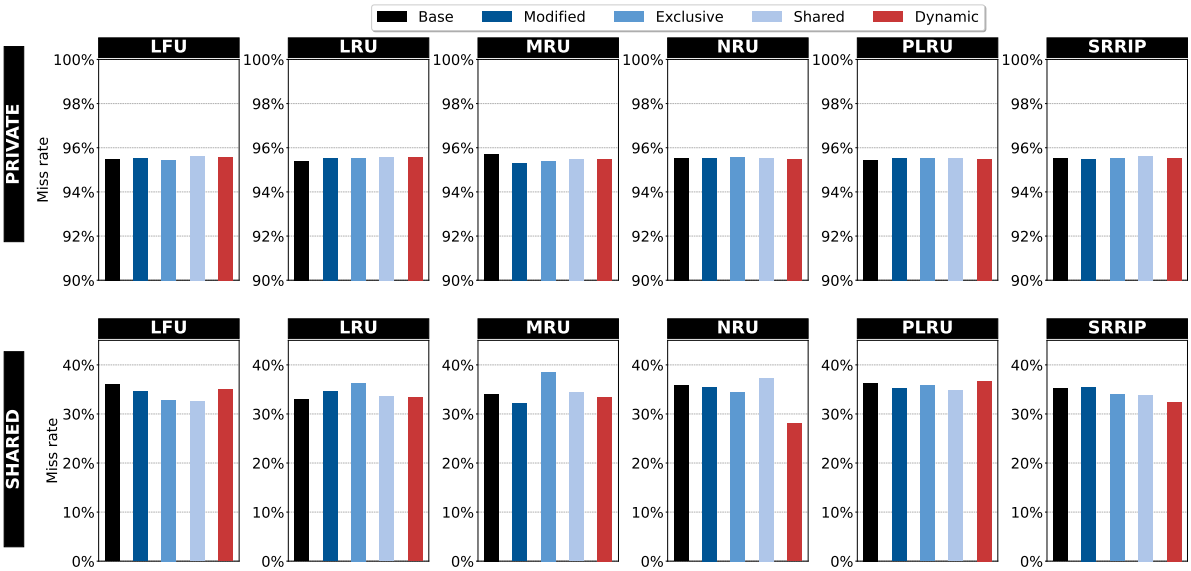


Source: Elaborated by the author

Since other Modified blocks are the most frequently used, the Dynamic MRU-state approach presented results similar to the Modified strategy. Contrarily for MRU, the most recently used blocks which are in Exclusive and Shared state are not good candidates to be cached.

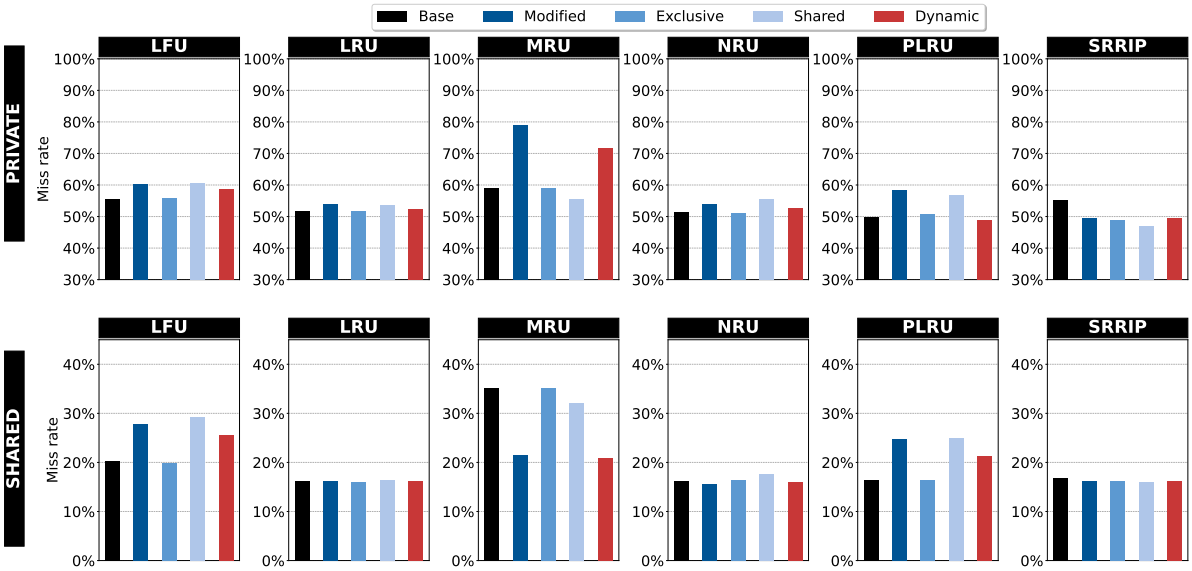
Figures 5.18 and 5.19 show the L2 cache miss rates gathered from FN and GF simulations. We note once again the same general behavior of other applications: higher miss rates at the L1, and reduced miss rates in the L2 cache. GF has higher rates than FN, due to the fact that it deals with chunks of data from image matrices. Thus, although more cache is available, there are still conflicts for the same cache set.

Figure 5.19 – L2 cache miss rate vs. sharing strategy - GF



Source: Elaborated by the author

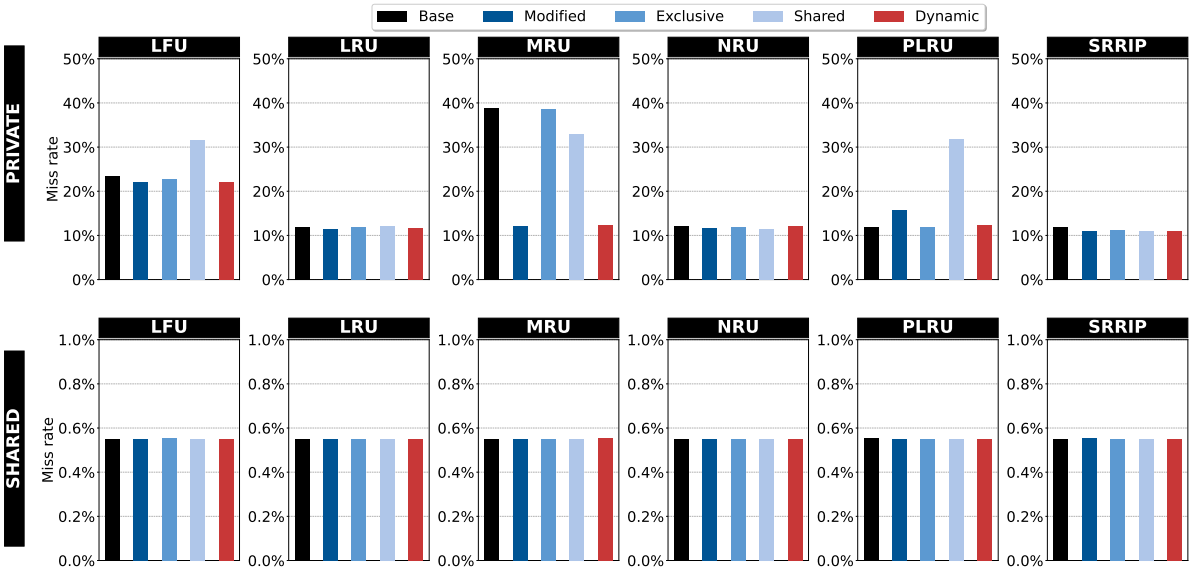
Figure 5.20 – L2 cache miss rate vs. sharing strategy - IS



Source: Elaborated by the author

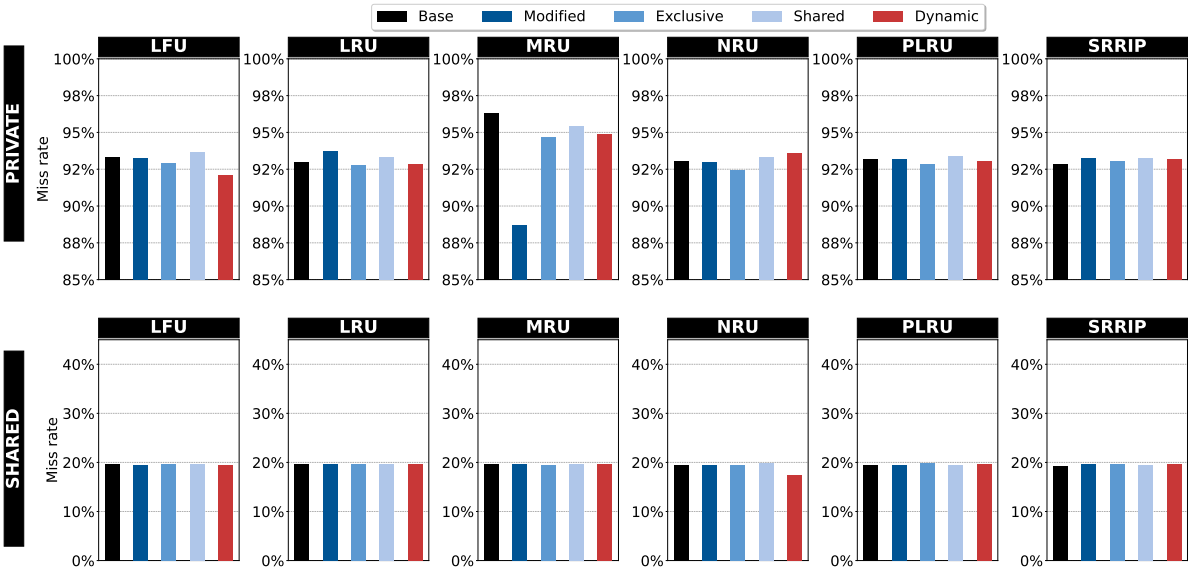
We observe from Figure 5.20 that IS follows the overall behavior. However, shared L2 caches in this kernel presented higher values than in the other kernels. We attribute this to the irregular access pattern from the integer sort implementation, which leads to fewer gains when sharing the cache within all cores. We also observe the same behavior from FAST and IS kernels, with the LFU and MRU approaches. In the former, the gap between the cache miss rates for private and shared strategies for Modified and Dynamic approaches is smaller than in the other ones. In the latter, the small gap is observed in Shared and Exclusive approaches. From LU, we still observe from Figure 5.21 the reduction in cache miss when the L2 cache is shared among all cores, regardless of the way-replacement strategy.

Figure 5.21 – L2 cache miss rate vs. sharing strategy - LU



Source: Elaborated by the author

Figure 5.22 – L2 cache miss rate vs. sharing strategy - TSP

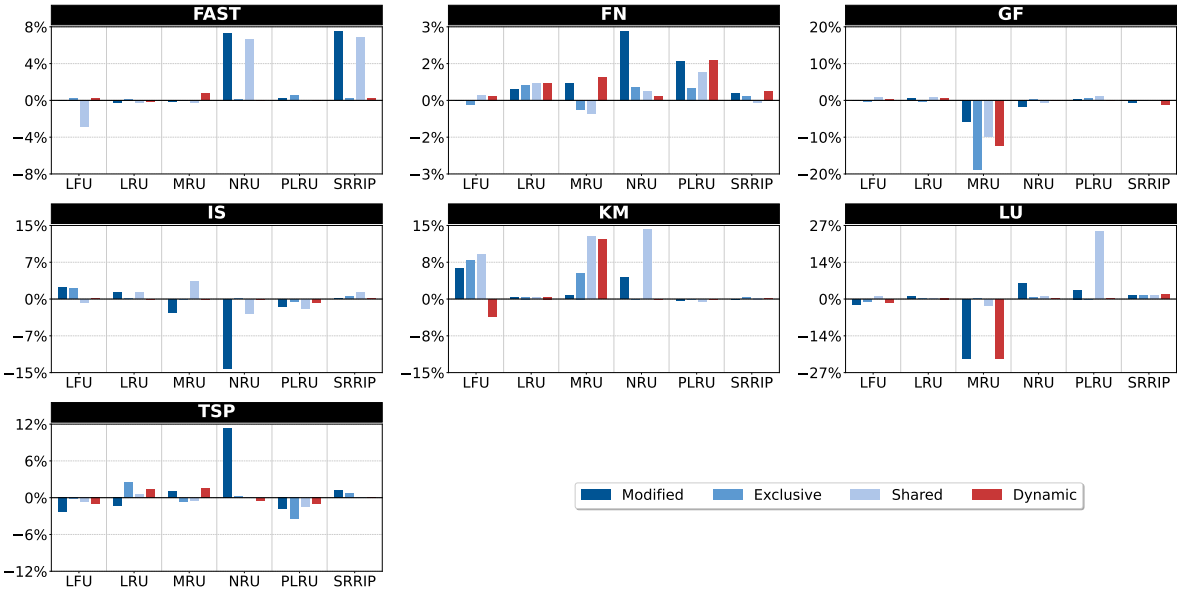


Source: Elaborated by the author

Finally, Figure 5.22 shows L2 miss rates for TSP. Besides the aforementioned overall behavior, we observe specific behavior with the Dynamic approach and NRU policy. This strategy is better employed in the Shared cache, showing the lowest miss rate values for this policy. Conversely, it was the worst strategy to be used in private caches with NRU.

In order to allow an observation with a broader look at the cache miss rates, we created overview charts. The architecture used to produce these results has 8-way set-associative private L1 and L2 caches, and 64-bit block sizes. Figure 5.23 below presents the L1 cache miss of all strategies and kernels.

Figure 5.23 – L1 cache miss rate decrease



Source: Elaborated by the author

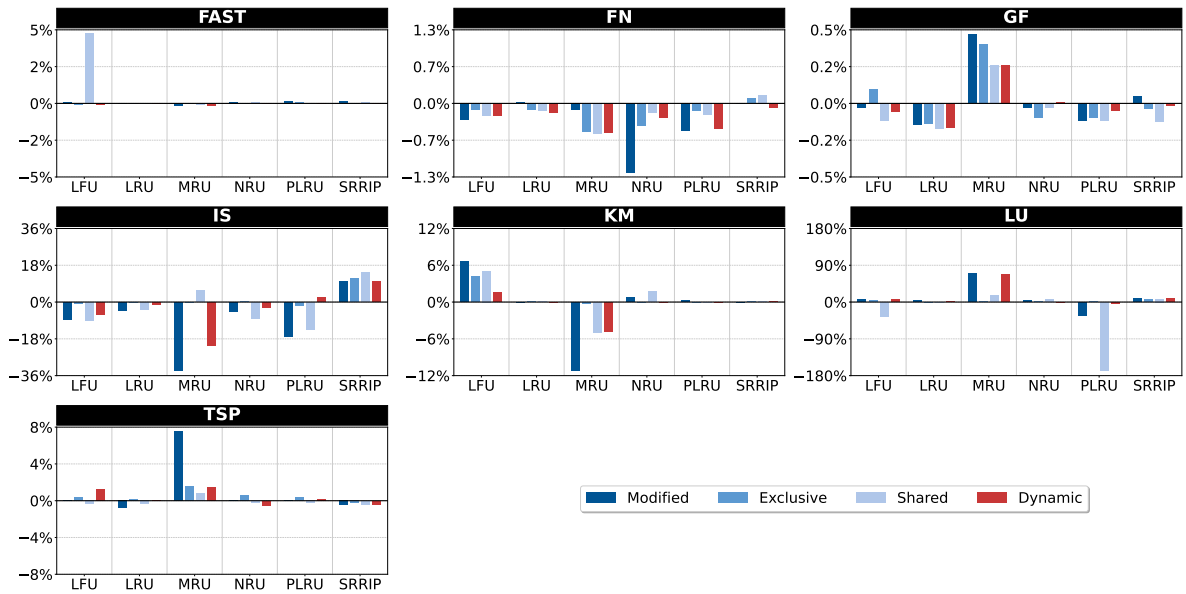
For FAST, the best results in this perspective are from RRIP-based approaches: NRU and SRRIP. They present a decrease in the miss rate which is higher than 6.2% for Modified and Shared approaches, in comparison with their unmodified versions. Despite being less than 3%, our strategies can improve FN results a bit in the L1 cache for most of the cases. For GF, the most visible result is with the MRU, which is worse than the baseline with our techniques.

IS does not present great improvements in L1 cache miss rates either. The maximum decrease in the miss rate at this level was observed with MRU using the Shared strategy (3.5%). We highlight that this result makes the way-replacement better than the LRU and PLRU baselines. Thus, MRU can compete with more complex policies if our strategies are employed.

The L1 cache miss rates from KM appear sharper for some cases in the algorithm because of the low values and, consequently, the scale. In spite of that, we can observe a small improvement with LFU, MRU, and NRU policies in some cases. The best decrease is 14.2% less miss rate with the NRU/Shared strategy, followed by a 12.78% decrease in the miss rate with the MRU/Shared one.

From LU, we observe that the highest decrease in the miss rate is around 25%. This is the case for the PLRU policy using the static Shared strategy. There is also an impressive decrease for TSP, with the NRU/Modified strategy – a 10.82% decrease in the L1 cache miss rate.

Figure 5.24 – L2 cache miss rate decrease



Source: Elaborated by the author

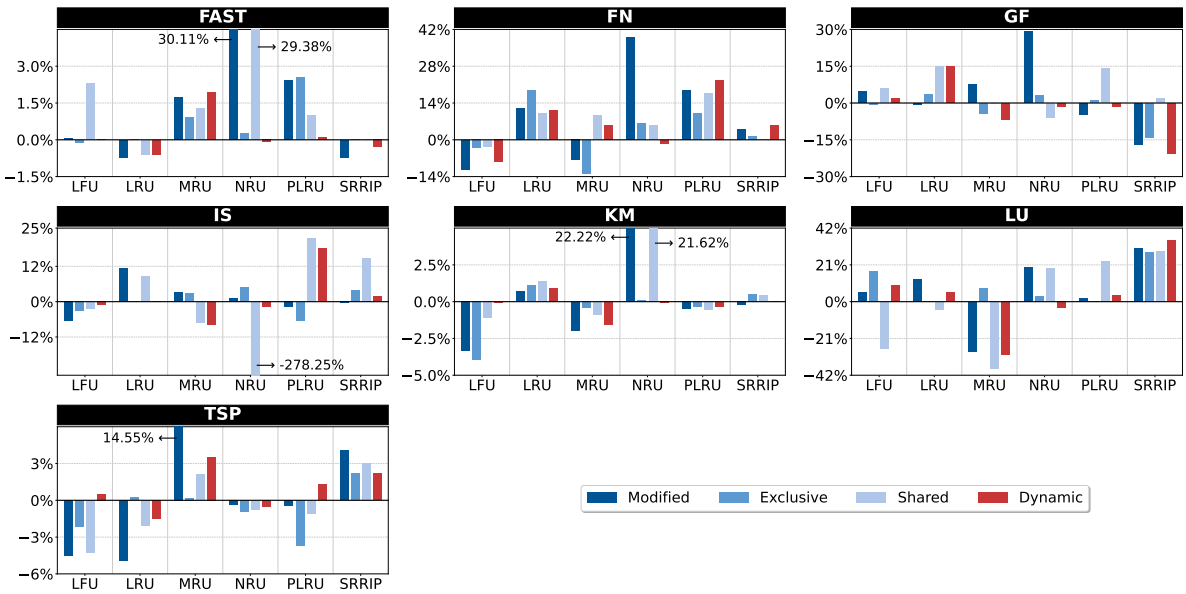
The same overall analysis is presented in Figure 5.24 for L2 cache miss rates. Initially, we find that the behavior of L2 is directly influenced by what happens in L1 in some kernels and policies. For instance, for FAST, LFU/Shared presents the most relevant decrease in the miss rate. However, it is the worst LFU case in the L1 cache. Thus, we assume that data not provided in L1 is then found in L2, which reduces the miss rate at this level. Likewise, FN, GF, and LU exhibit the same behavior. Anyway, we highlight for LU that we can improve

MRU with Modified (69.04% decrease) and Dynamic (68.07% decrease) approaches, so these policies compete with the unmodified LRU policy. The best result for the IS kernel is perceived with SRRIP/Shared, which shows a 14.71% decrease in the miss rate. For KM, we observe good improvements in LFU, which is up to a 6.82% decrease in the L2 miss rate. Finally, for TSP, the best case is also with the MRU using the Modified strategy (10.82% less miss rate).

5.1.2 Coherence operations

To evaluate the impact of our way-replacement strategies in the cache coherence protocol, we gathered the number of invalidates and write-backs from Sniper. It is worth remembering that an invalidate operation occurs when there is an upgrade in some cache block that has other copies in other caches, which must be discarded. Also, we recall that coherence write-backs result from memory access in cache blocks that have copies in the Modified state in other levels (they need to be updated before they can be used).

Figure 5.25 – Coherence invalidates decrease

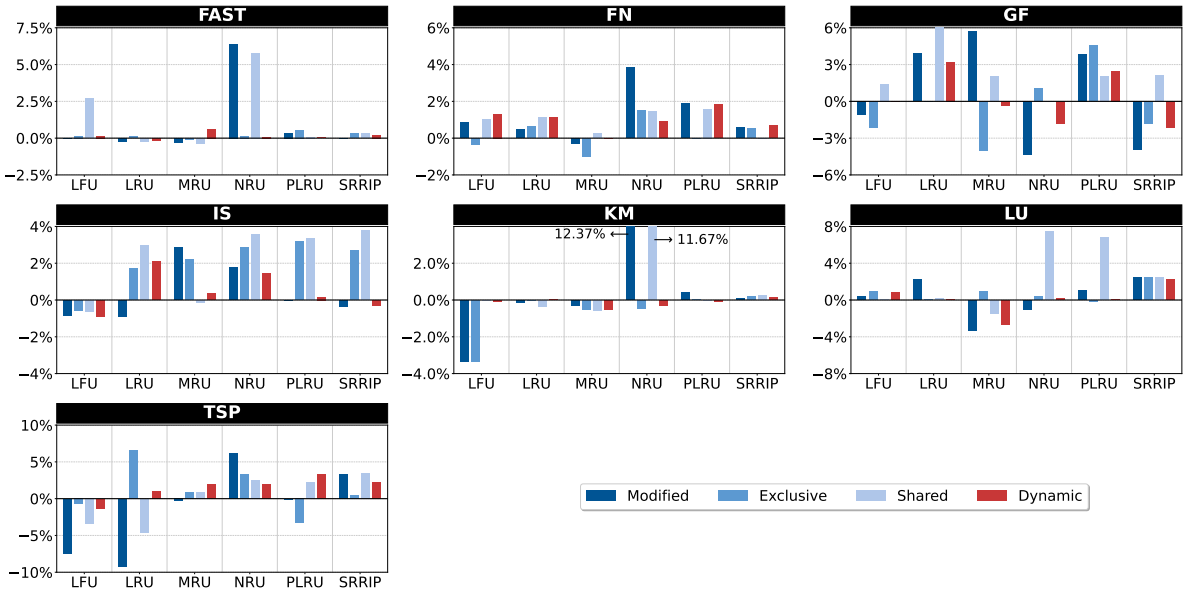


Source: Elaborated by the author

Figure 5.25 shows the decrease in invalidate operations that occur during the execution of the kernels. FAST performs the highest amount of coherence operations because it performs more memory accesses than other kernels. The most significant results for this kernel are presented with NRU/Modified and NRU/Shared. These configurations show 33.11% and 29.38% reductions in invalidates. Likewise, these strategies present improvements in the number of invalidates over the baseline with KM (22.22% and 21.62%) and LU (19.77% and 19.00%). This is due to most of the blocks in the Modified state not being updated in other threads in a short time, thus they do not cause invalidation. The same behavior applies to blocks in the Shared state; however, invalidates are caused when copies are updated in other caches.

For LU, we highlight the results observed with SRRIP, with more than 28% decrease in the number of invalidates. More precisely, the best scenario is with the SRRIP/Dynamic, which shows 34.69% fewer invalidates than the unmodified policy. Generally speaking, the best result for IS is with the Shared state strategy using the PLRU policy (approximately 21.7% decrease in the number of invalidates). For TSP, the best case is with the MRU policy using the Modified state as the one to preserve, showing 14.55% fewer invalidates.

Figure 5.26 – Coherence write-backs decrease



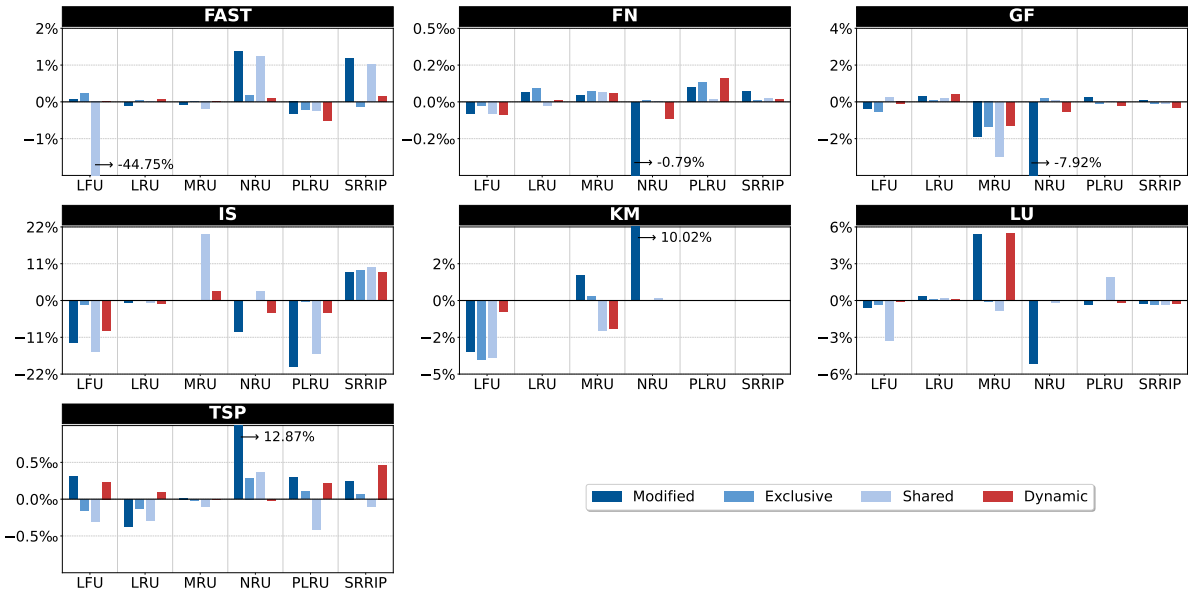
Source: Elaborated by the author

For write-backs, Figure 5.26 is presented. We observe fewer variations in the number of write-backs than invalidates. Notwithstanding, NRU/Modified and NRU/Shared, again, show improvements for FAST and KM, which are also the best results for these kernels. Although FN and IS show some improvements with our strategies, they are under 4%. Besides that, GF also shows some improvements, with the best case being 6.03% fewer coherence write-backs with the LRU/Shared strategy. The highest decreases when running the LU kernel are perceived with NRU (7.66%) and PLRU (6.95%), both using the Shared state approach. Finally, for TSP we highlight that with the LRU policy, the static Modified and Shared approaches are the worst. However, for LRU, the static Exclusive shows the highest decrease in write-back operations among all policies, which is 6.91%.

5.1.3 Instructions per cycle and energy consumption

In our experiments, we use the committed instructions per cycle (IPC) to quantify the performance of the whole system. Our analysis focuses on the same private cache architecture using 8 ways and 64-bit cache blocks. Figure 5.27 on the next page shows the IPC increase for all kernels and policies and the strategies discussed in this section.

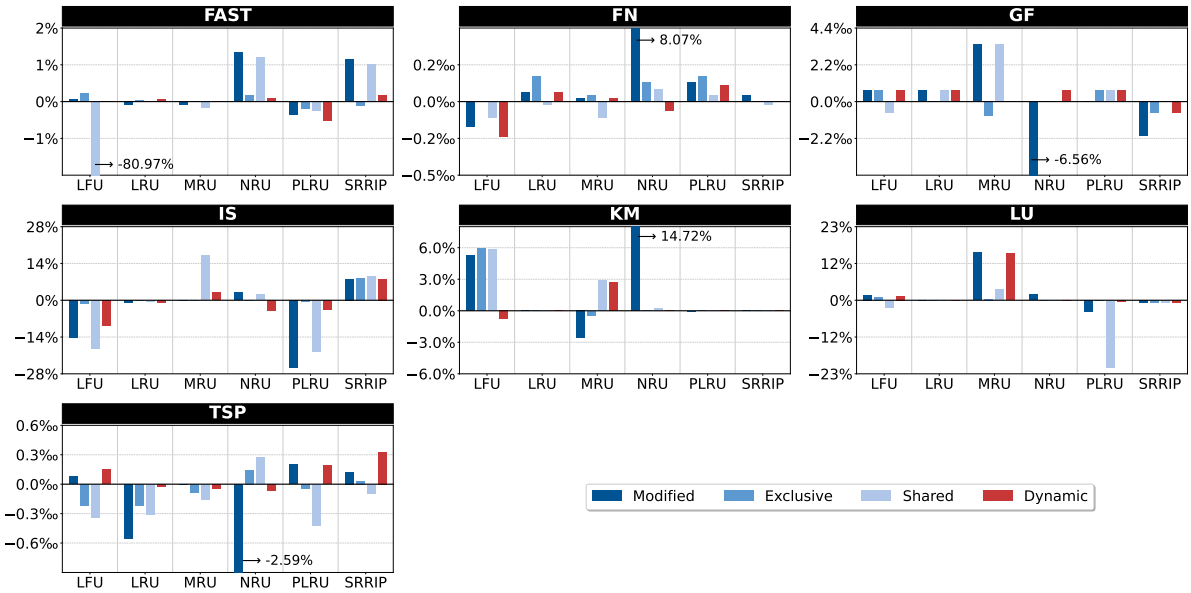
Figure 5.27 – IPC increase



Source: Elaborated by the author

For FAST, the improvements are under 1.37%, which is the result observed in NRU with the Modified state being chosen. FAST can perform many instructions in parallel, overlapping the impact of the high cache miss rates. However, a minor change in the number of misses in the L1 cache impacts performance drastically. For instance, the L1 cache miss with the LFU/Shared strategy is higher than the baseline. This difference is sufficient to make FAST suffer from the high access latency to access the L2. Thus, for this case, the result is a high decrease (44.75%) in the IPC.

Figure 5.28 – Cycle decrease



Source: Elaborated by the author

Figure 5.28 shows the relative decrease in the number of cycles for all kernels. We can see that the example mentioned for FAST is confirmed since the number of cycles in the LFU/Shared strategy is increased significantly (80.97%).

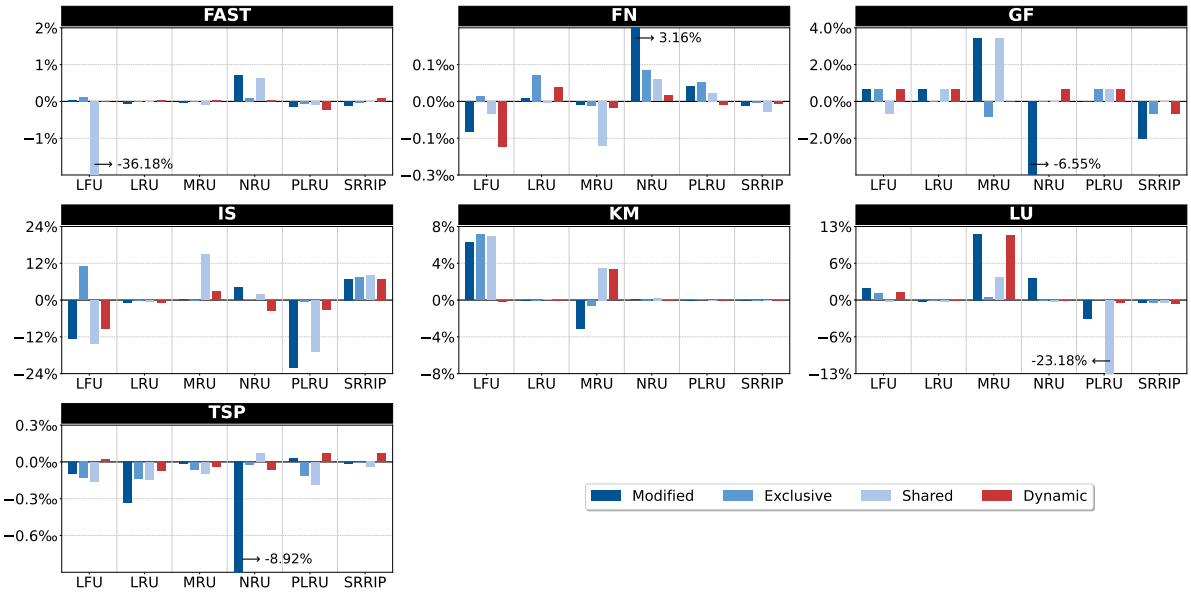
The IPC results for FN, GF, and TSP show very low improvements over the baseline. An exception is TSP, in which we observe 12.87% more IPC with the NRU/Modified than the baseline. We associate this to the reduction in the coherence write-backs for this kernel and strategy. However, the number of cycles has increased, but at a lesser rate than in the last example. Reducing the coherence write-backs favors the instructions throughput since the dirty data can be updated by many instructions within a single time window, before eviction. Similar behavior occurs for FN, also with the NRU/Modified strategy, however, in the opposite direction – the IPC was reduced by 0.79%.

In addition, we observe some performance variations in the IS kernel. For instance, all the modified SRRIP policies surpass the baseline up to 10.26%. The policy which shows the highest improvement is MRU with the Shared state strategy – 20.31% IPC increase over the baseline. Generally speaking, the increase in the IPC for IS follows the decrease in cycles.

For KM, the best result is with the NRU/Modified strategy (10.02% improvement). As well as FAST, KM has many independent but intensive threads running, which favors parallel computation. The small reduction in cache misses and in the number of write-backs raises the performance. For the LFU policy, we observe that the IPC using our strategies reduces in comparison to the baseline, as the number of cycles. With this policy, we see from the coherence operations analysis that both invalidates and write-backs increased. These changes in cache blocks result in a relatively lower number of instructions being executed in an equal number of cycles, increasing the IPC.

Last, but not least, we measured the energy consumption of the architecture while employing our proposals. These measurements were obtained from McPAT after each run in Sniper. McPAT considers the architecture configuration and the statistical results after the simulation. Figure 5.29 shows the decrease in energy consumption for the proposals in this section.

Figure 5.29 – Energy consumption decrease



Source: Elaborated by the author

At a first glance, we observe FAST with only LFU/Shared presenting an outlier result, which is a 36.18% increase in energy consumption. This increase in the energy consumption is a consequence of the reduction in the IPC along with the increase in the power consumption due to the use of L2 caches when L1 had the misses. The best case in this kernel is with NRU/Modified, which is 0.17% less energy consumed than the baseline.

On the other hand, FN presents a different behavior in energy consumption with regard to IPC results. For instance, the NRU/modified approach reduces the energy consumption by 3.16%, however, this configuration increased the IPC a bit (0.79%). The fact is that, although taking more time to solve, FN is a CPU-bound kernel, thus the power consumption from memory operations is lower. Furthermore, this strategy can reduce the coherence operations (invalidates and write-backs), which results in improvements in the overall energy consumption. Likewise, TSP shows almost no improvements in energy consumption, regardless of the improvements in the IPC. The IS kernel also presents the energy consumption inversely proportional to the IPC – when the IPC increases, the energy decreases. Thus, we can observe a reduction in energy consumption with at least three policies in this kernel: MRU, NRU, and SRRIP. MRU/Shared presents 14.87% less energy consumption than the unmodified MRU version, which is the best case for the kernel.

The LRU, PLRU, and SRRIP policies show almost no variations when we run the KM kernel using them. On the other hand, our strategies result in less power consumption in three cases with LFU, two with MRU, and one with NRU. The best strategy with LFU is the one that favors blocks in the Exclusive state, which shows 7.1% energy savings. For MRU, the best is a 3.49% decrease, in the strategy which favors the Shared state. MRU is also the policy that can reduce more energy for the LU kernel. It presents 11.67% and 11.37% of energy consumption reductions for Modified and Dynamic approaches, respectively.

6 SHARER COUNT-BASED EVICTION

Another proposal of this thesis consists of counting how many sharers a single cache block has. We then consider this information when deciding whether this cache block should be evicted or not in the replacement strategy. This heuristic is not directly related to cache coherence. However, the structure needed for the cache coherence mechanism supports this strategy. We use a directory-based MESI cache coherence protocol in our simulated architecture. In directory-based protocols, additional meta-data is needed to keep track of the sharing set, *i.e.* the set of caches that cache a given block. Figure 6.1 presents an overview of our strategy and also depicts a single cache block and its sharers bit-vector.

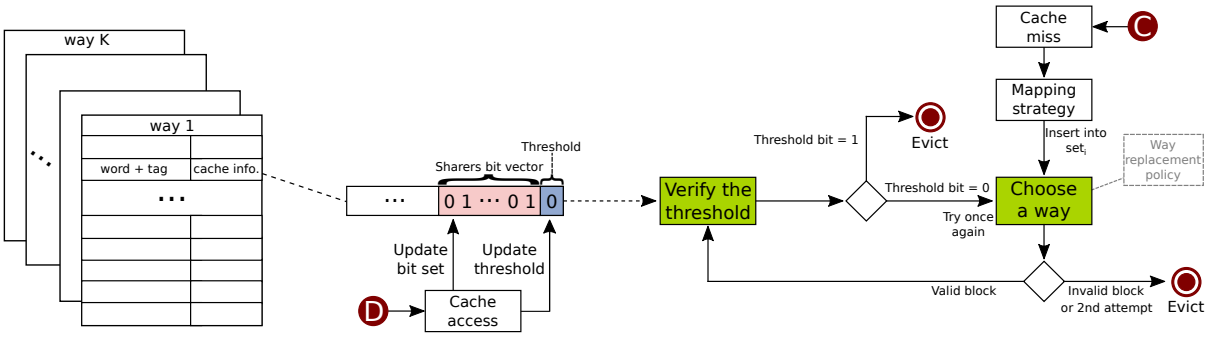
In this example, there are also $K * N$ cache blocks in the cache. Besides the cache coherence state, each cache block has its own sharing set that is a simple bit-vector. A 1 in position j in this bit-vector indicates that cache j caches this block. The number of bits in a set is the same as the number of cores within the architecture. Thus, if C is the number of cores, we need C bits per cache block to use this strategy. We consider that this is not an increase in the cache size, because the directory already uses these bits.

With that in mind, our approach simply tracks the bit-vector, counting how many cores share each cache block. We added an additional bit to each block, which is set to 1 when the number of cores sharing a block is lower than a threshold, otherwise 0. The bit-vector and the threshold bits are updated when cache access occurs, as in D in Figure 6.1. Thus, despite the 1-bit for each cache block, this approach adds new computational complexity to update the threshold bit on every cache block access. However, the strategy occurs without blocking the cache access.

We want to verify if a cache block is shared by many or few cores. Thus, for simplicity reasons, we set the threshold at 50%. In spite of this, the purpose is that architects can modify it in their projects, or even have this information stored, for instance, in a hardware register.

The replacement process is triggered after the mapping strategy in C. A candidate way is chosen, and if the block is valid, the algorithm checks the threshold bit. If set 1, the block is chosen and the data is evicted, otherwise, the next attempt is performed, with a new victim.

Figure 6.1 – Overview of the sharer count-based eviction strategy



Source: Elaborated by the author

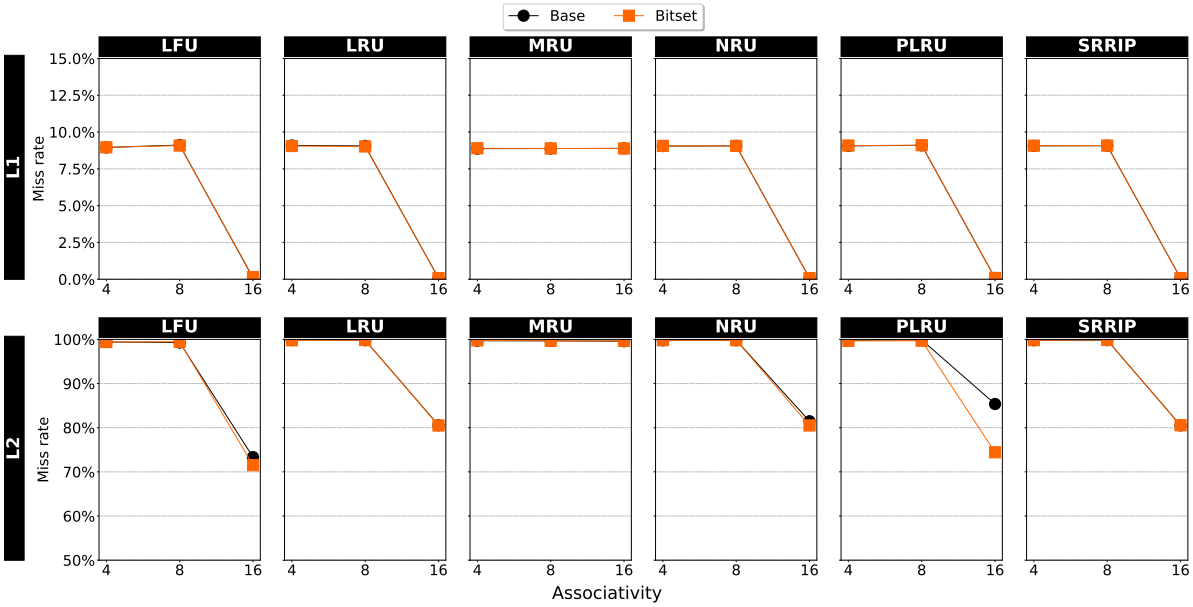
6.1 Experimental results

This section presents the results we gathered from the sharer count-based strategy in comparison with unmodified policies.

6.1.1 Cache miss rate

We now evaluate this proposal in terms of cache miss rates when varying the cache associativity, the block size, and the sharing strategy.

Figure 6.2 – Cache miss rate vs. associativity - FAST

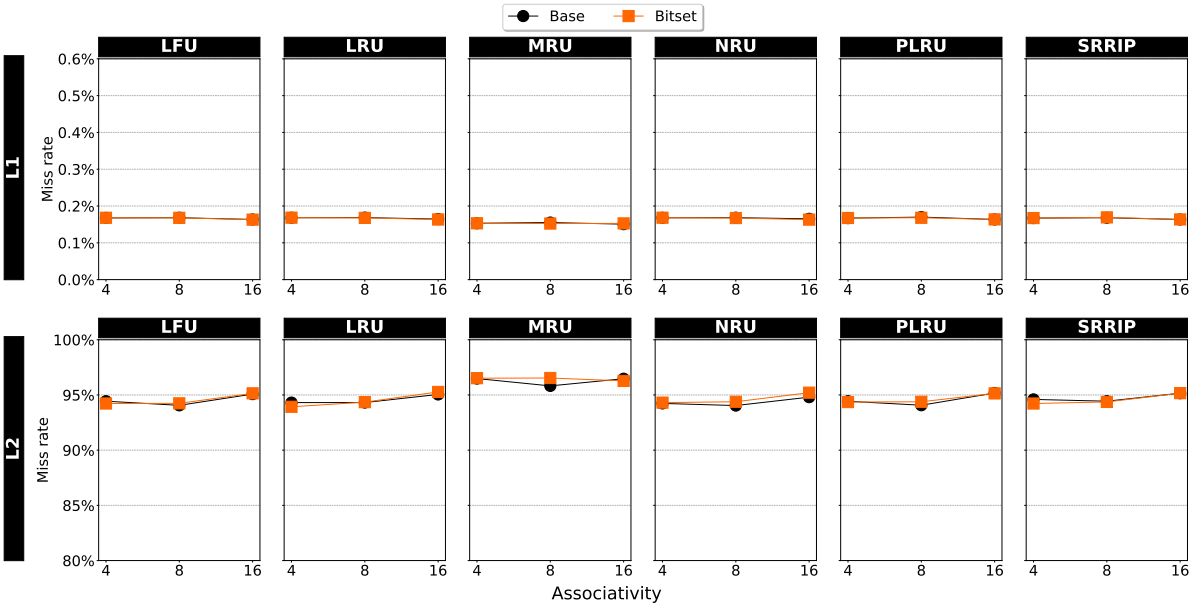


Source: Elaborated by the author

Figure 6.2 shows the cache miss rates perceived from FAST when we vary the associativity. In the L1, we observe the same behavior as in coherence-based approaches: a high decrease in cache miss when using 16-way set-associative caches, except for MRU. This is the effect of the unoptimized implementation. More specifically, for MRU, we recall the conflict between choosing the most recently used block to be evicted while working on small chunks of data. We also observe almost no different behavior between the unmodified policy and ours. The lines that are on the highest plane of the charts represent cache miss rates between 8.87% and 9.11%. The ones on the lowest plane have values between 0.08% and 0.14%.

For L2 miss rates, except for MRU, we perceive the decrease in the cache miss rate when using 16-way set-associative caches. We observe a minor improvement using LFU, in which our strategy results in 71.51% against 73.35% from the unmodified LFU. Likewise, the NRU policy that is aware of the sharers bit-vector presents a miss rate of 80.50%, while its baseline presents 81.47%. We observe the highest decrease in miss rate for FAST with the PLRU policy and 16-way caches. It shows a 74.44% miss rate, while the unmodified PLRU presents 85.37%.

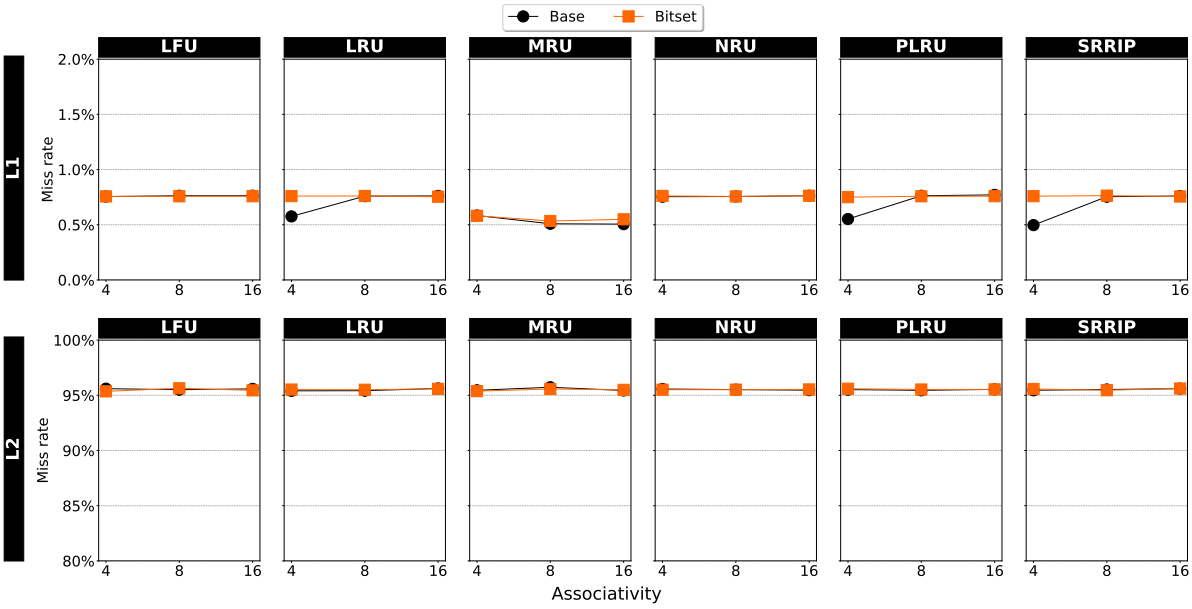
Figure 6.3 – Cache miss rate vs. associativity - FN



Source: Elaborated by the author

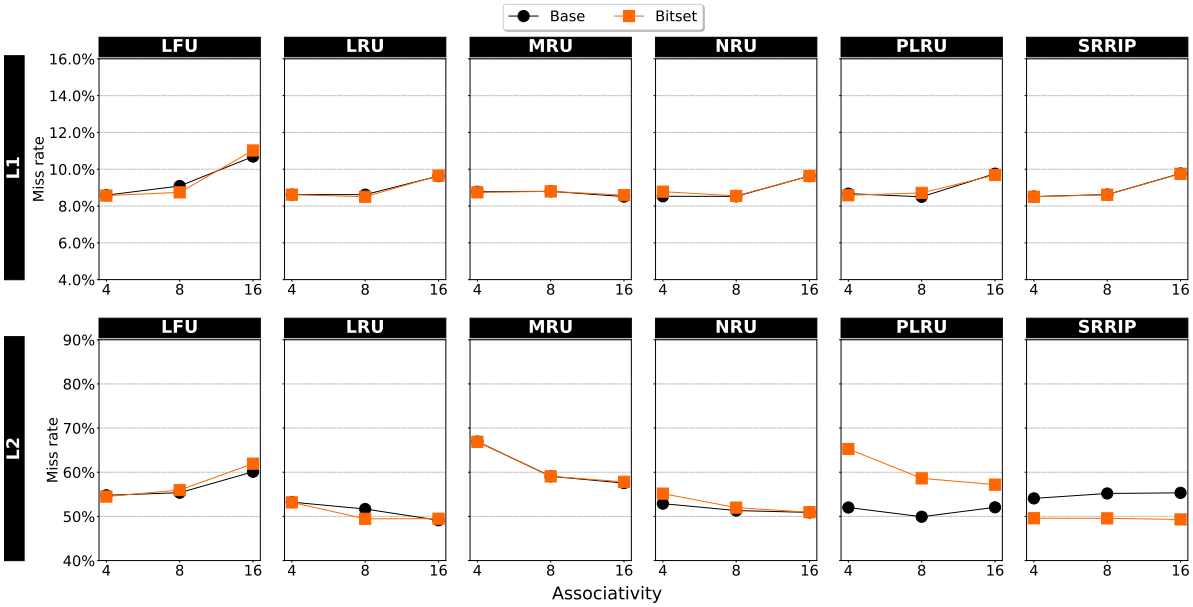
Figures 6.3 and 6.4 show, respectively, the miss rates for FN and GF when we vary the associativity. These results are almost the same we found in Section 5, with our first set of way-replacement strategies. For both kernels, the Bitset strategy does not present a different behavior for most of the policies. Even so, we notice that it is also worse than the baseline for LRU, PLRU, and SRRIP with 4-way caches running GF.

Figure 6.4 – Cache miss rate vs. associativity - GF



Source: Elaborated by the author

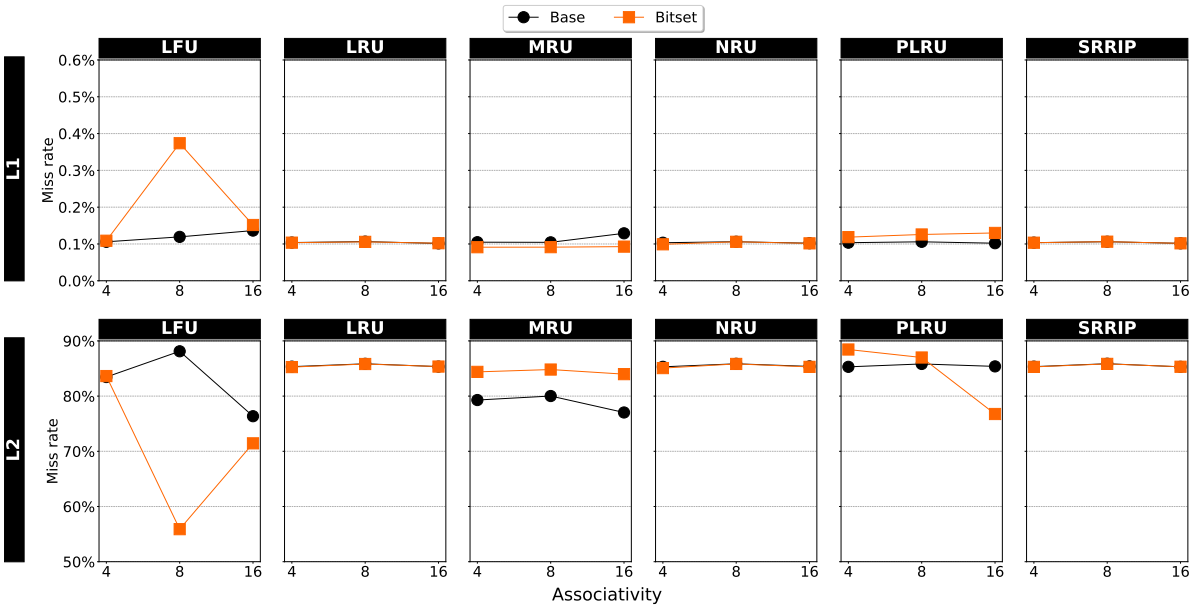
Figure 6.5 – Cache miss rate vs. associativity - IS



Source: Elaborated by the author

The results from the IS kernel, presented in Figure 6.5, follow a pattern similar to the one from the coherence-based approach, mainly when the Shared state is selected. Indeed, if we consider the Bitset with cores sharing the same block, these blocks will be in the Shared state at some moment. The best result we observe in L1 is 8.50%, with the unmodified PLRU in the 8-way associative cache. This value is perceived also with our approach, with LRU and SRRIP policies, using 8-way and 4-way associative caches, respectively.

Figure 6.6 – Cache miss rate vs. associativity - KM

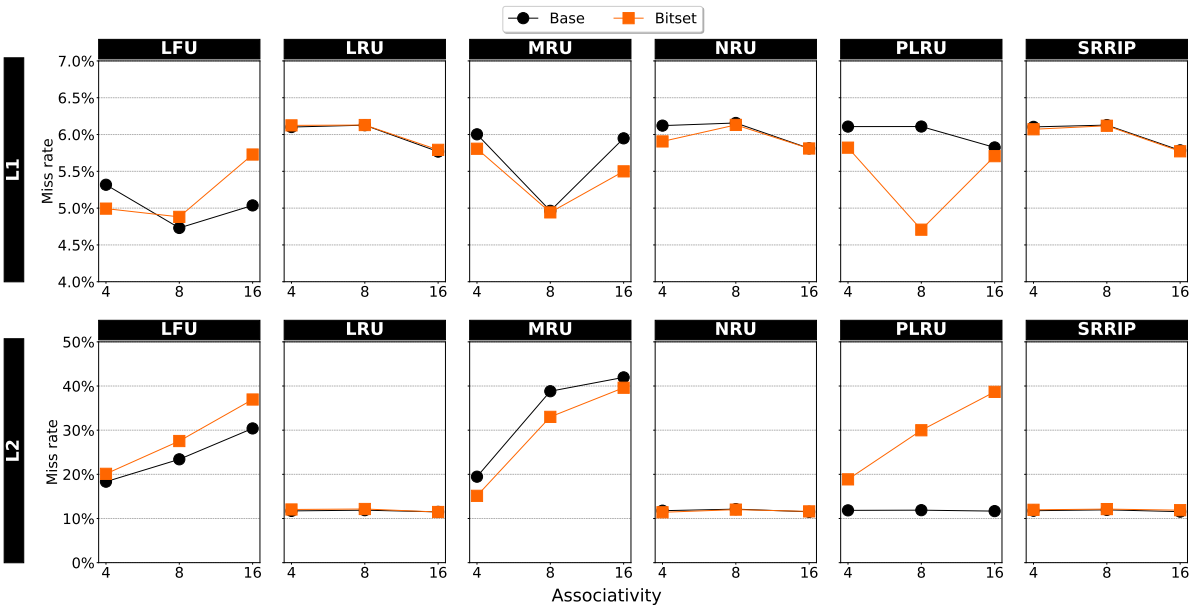


Source: Elaborated by the author

Figure 6.6 shows the cache miss rates for the KM kernel. First, we look at the abnormal LFU result when we employ our strategy on the 8-way L1 cache. It is equal to a 0.37% miss rate, against 0.12% from the baseline in the same configuration. Additionally, PLRU using our approach presents the results all worse than the baseline. Differently, with the MRU policy, our approach achieves slightly better results than the unmodified version.

Furthermore, as noted in the past analysis, we observe that most L2 miss rates are directly impacted by the misses in the L1. Those misses lead to new L2 requests, which might be successfully answered, thus reducing the L2 miss rates. For instance, the 8-way configuration with our Bitset approach is the best result (55.89%), unlike what happened in the L1. Likewise, our approach with the MRU policy is the worst regardless of associativity. We also highlight the improvement when using the adapted PLRU policy with 16-way caches. In this case, the policy results in a 76.76% miss rate, while its unmodified version shows 85.38%.

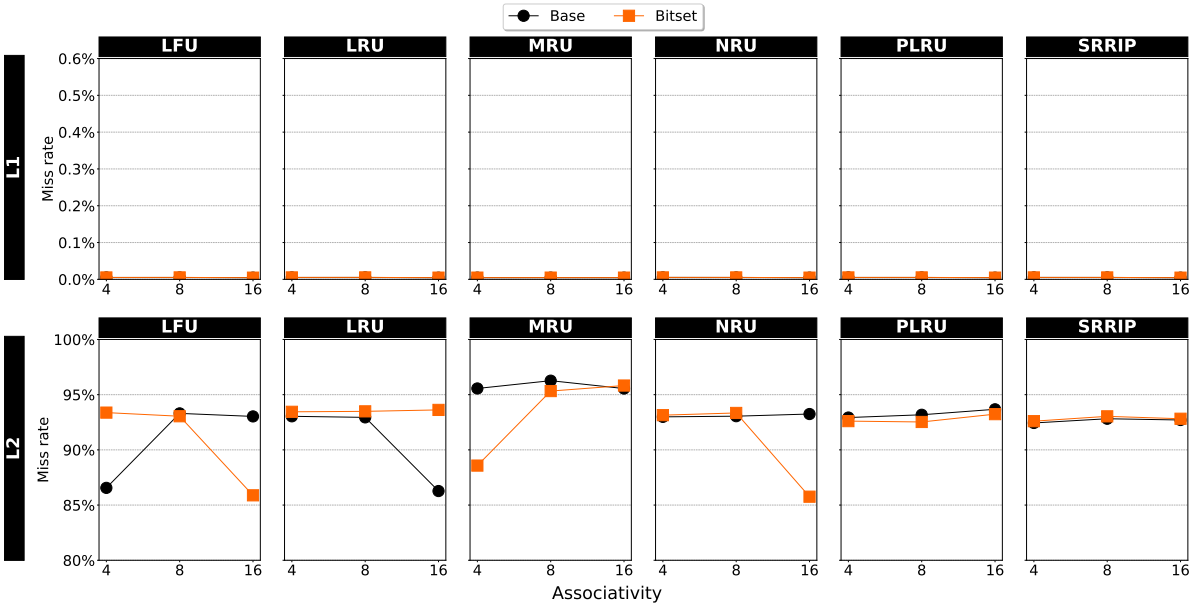
Figure 6.7 – Cache miss rate vs. associativity - LU



Source: Elaborated by the author

With regard to LU, we observe that it follows the same pattern in cache miss rates as in the coherence-based approach that favors the Shared state. Figure 6.7 shows these results. We noticed that in L1, using LRU, NRU, and SRRIP policies, the cache miss rates decrease a bit when the associativity is set to 16. At the same level, we observe our strategy being better than the baseline for all configurations using MRU, NRU, PLRU, and SRRIP policies. The lowest cache miss is 4.71%, using our approach applied to PLRU on the 8-way associative cache. At the L2 cache, using our approach for LU shows fewer improvements in cache miss rates. For instance, in this case, the modified PLRU policy is worse than the baseline, which is more noticeable when the associativity increases (up to 38.68% miss rate). Nevertheless, MRU shows the opposite since our strategy is better than the baseline regardless of associativity.

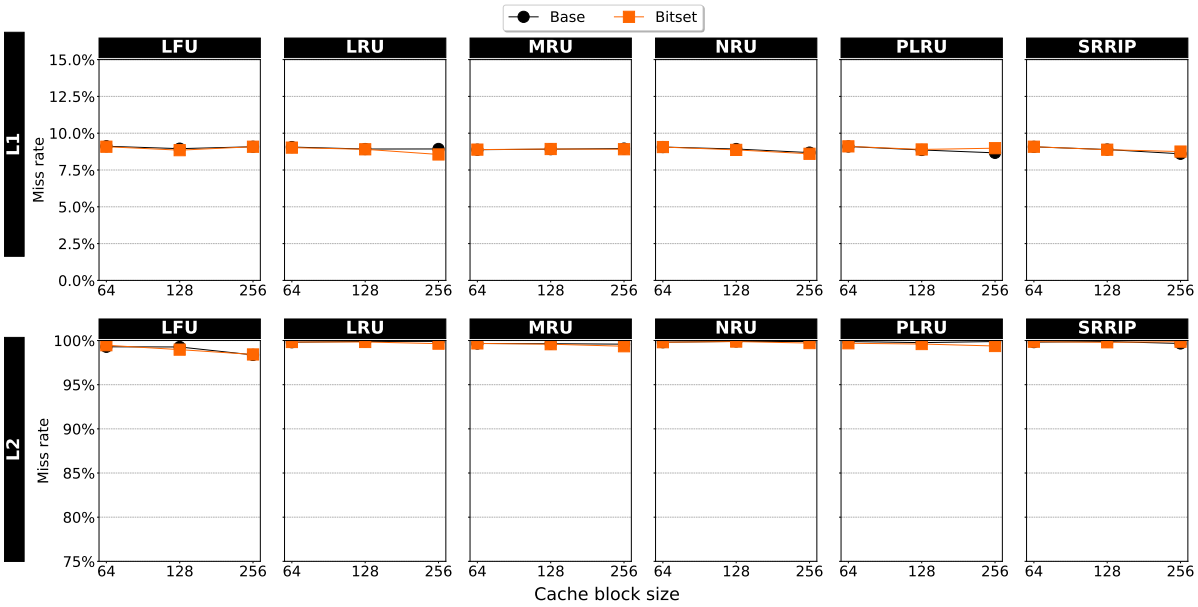
Figure 6.8 – Cache miss rate vs. associativity - TSP



Source: Elaborated by the author

Figure 6.8 shows the miss rate results for TSP with different cache associativity. For L1, we repeat the same analysis as in our state-based approaches, due to the low miss rates. With respect to the L2 cache miss results in this kernel, we highlight three results. First, with 16-way caches, our approach performs better than the baseline when it is employed in the LFU and NRU policies. Moreover, the miss rate is also lower when we apply the Bitset approach to MRU in a 4-way associative cache.

Figure 6.9 – Cache miss rate vs. block size - FAST

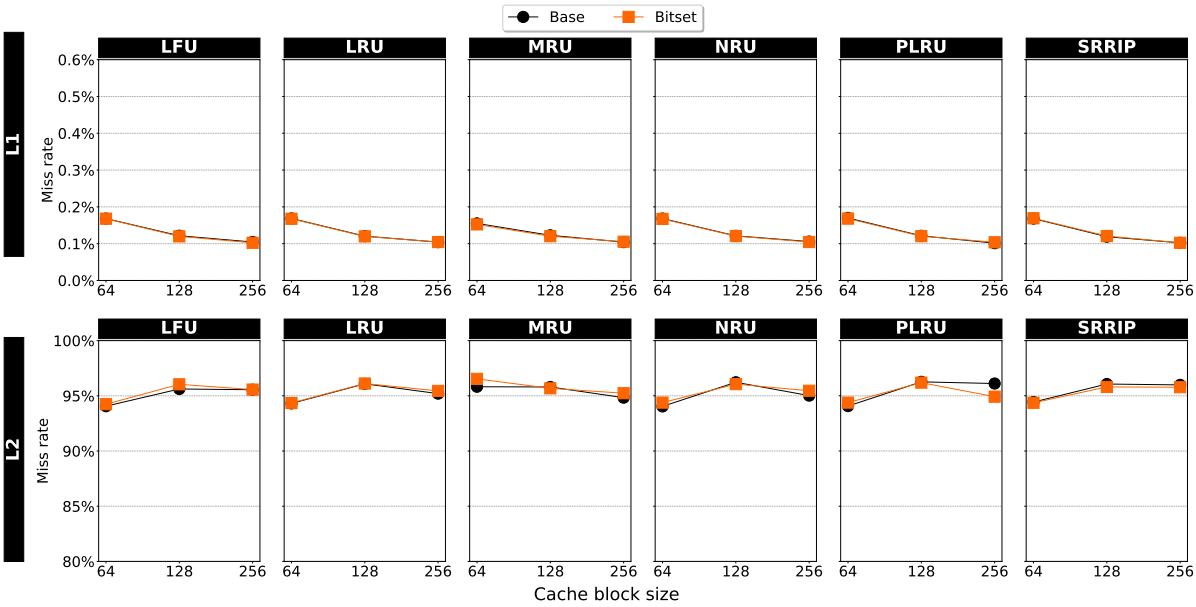


Source: Elaborated by the author

Continuing the evaluation on the perspective of changing block sizes, Figure 6.9 shows the miss rates for FAST. We still do not observe notable changes in the L1 miss rate when varying the cache block size. Even so, our strategy can achieve the best result for three of the policies.

For instance, LFU with 128-bit blocks presents an 8.85% miss rate, while its baseline shows 0.10% more. Next, the 256-bit configuration in the LRU policy shows 8.56% – the miss rate in the unmodified version with the same block size is 8.93%. Finally, the NRU miss rate with 256-bit blocks is 8.60% against 8.68% without employing our strategy. We noticed the same small changes for FAST in the L2 when varying the block size. The policies in which we can reduce the cache miss are MRU, NRU, and PLRU, all using 256-bit blocks.

Figure 6.10 – Cache miss rate vs. block size - FN

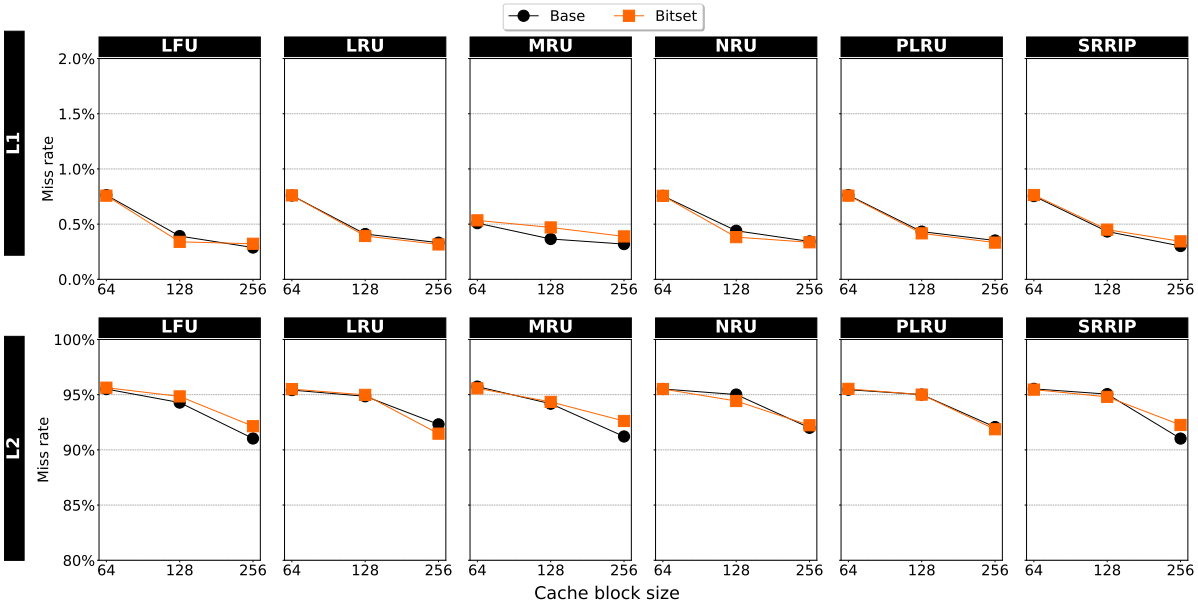


Source: Elaborated by the author

Figure 6.10 above shows the results varying the cache block size for FN. The Bitset approach does not impact L1 cache miss rates. For L2, the behavior is almost the same as the baseline when the block size is changed. We perceive an improvement of our strategy over the base in the PLRU policy, with 256-bit L2 caches. However, this result is not better than the baseline with 64-bit blocks. For GF, whose results are depicted in Figure 6.11 on the next page, we find our strategy worse than the baseline when bigger blocks are used. However, we highlight three exceptions with this kernel. First, in the L1 cache with LFU and, second, with NRU, our strategy is better than the unmodified version when 128-bit blocks are used. Third, it is better to employ the Bitset strategy with the LRU policy with 256-bit blocks.

Figure 6.12, on the next page, shows the cache miss rates for each block size we gathered from IS executions. Overall, when the block has bigger sizes, the L1 cache miss reduces, due to the number of integers the blocks can handle at each load instruction. Our strategy can beat the baseline in terms of L1 cache miss rate in the following configurations: LFU using blocks of 256 bits, from 7.19% to 7.00%; LRU using the 64-bit configuration, from 8.62% to 8.50%; MRU, 256-bit, from 6.63% to 6.56%; and NRU with 128-bit blocks, from 8.27% to 8.19%.

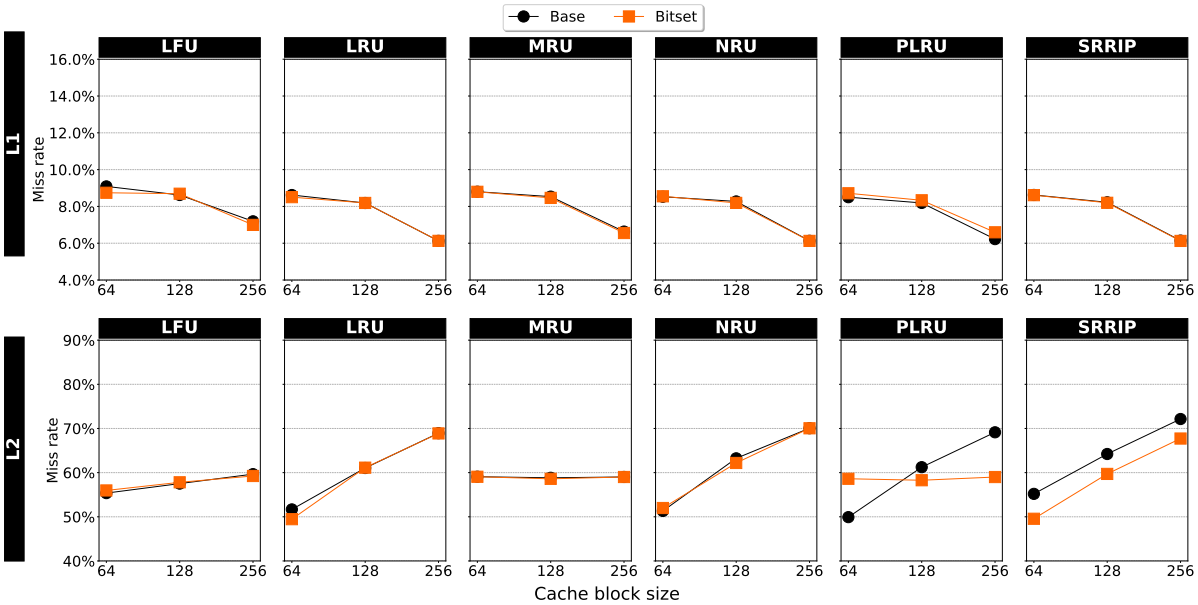
Figure 6.11 – Cache miss rate vs. block size - GF



Source: Elaborated by the author

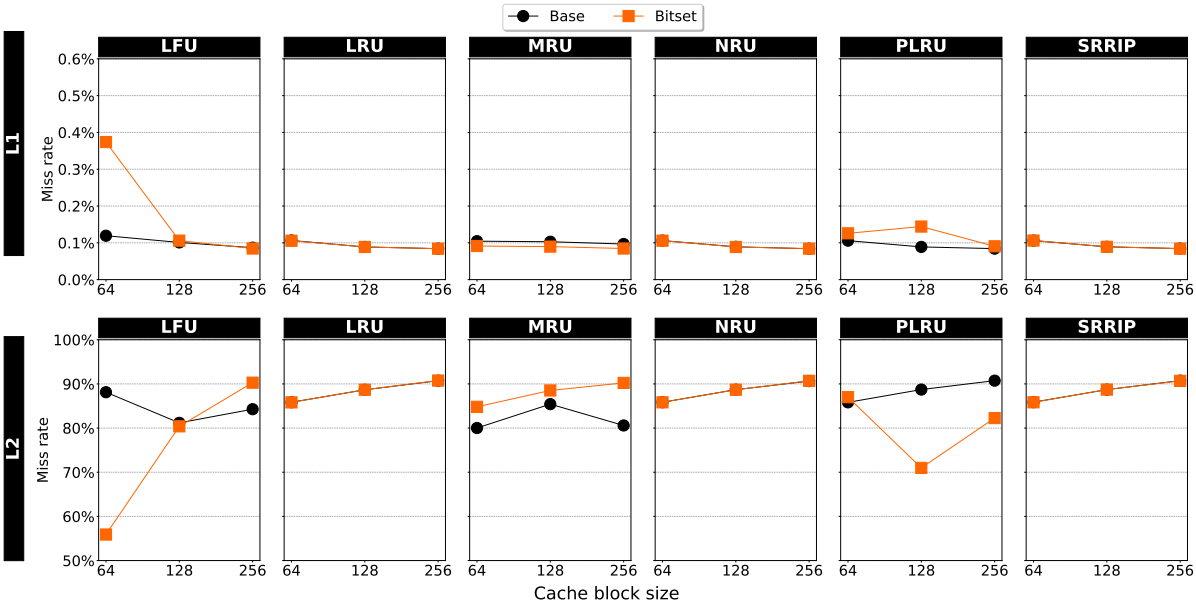
We see in the results from the L2 cache a pattern that is more significant than in the L1. Generally speaking, when we increase the block size, the cache miss rate also increases. A first exception is observed with MRU, which presents almost no changes when we modify the cache block size. MRU results are in the range of 58.06% to 59.08%. The sharer count approach could improve PLRU in this cache level when we increase the block size. Thus, the cache miss rate does not vary too much, unlike with the unmodified policy. Furthermore, with the SRRIP policy, our approach is better than the baseline in all cases for L1 observations.

Figure 6.12 – Cache miss rate vs. block size - IS



Source: Elaborated by the author

Figure 6.13 – Cache miss rate vs. block size - KM



Source: Elaborated by the author

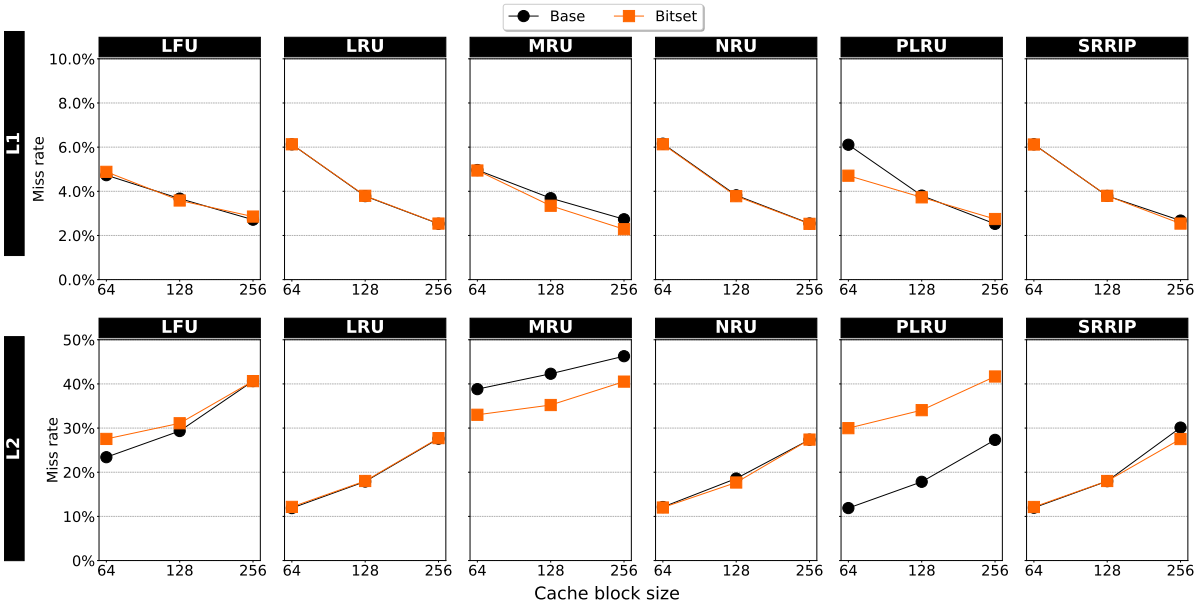
Figure 6.13 shows the L1 cache miss rates from KM when we vary the cache block sizes. The behavior is almost the same as the coherence-based approaches at this level. However, we highlight two different behaviors in LFU with 64-bit (0.37%) and PLRU with 128-bit (0.14%) block sizes. In these cases, the cache miss rates do not follow the decreasing behavior when the cache block size is higher. Furthermore, these results are worse than in other configurations within the same policies. MRU is the only policy that shows at least 0.01% less cache miss rate than the baseline, regardless of the cache block size.

These values result in the L2 miss rate behavior. We observe that LFU with 64-bit blocks shows miss rates much lower than 128 or 256-bit configurations (55.89%, 80.40%, and 90,82%, respectively). Our approach using the sharers bit-vector with the MRU policy, unlike in the L1, is worse than the baseline in all cases. The second-best miss rate among all policies and block sizes is presented with our approach in PLRU with 128-bit blocks. This policy/block size pair shows a 70.99% miss rate, against 88.72% from PLRU unmodified.

The behaviors of miss rates from LU simulations using the cache block size perspective are depicted in Figure 6.14. We recall that larger blocks might favor the spatial locality for future data reuse in LU. Thus, the miss rate is reduced when the block size increases in the L1. The highest improvement over the baseline is perceived in PLRU with 64-bit blocks – it shows 4.71% against 6.11%. In MRU we observe an increase in the improvement of our strategy when we increase the cache block size. The miss rates are 0.02, 0.34, and 0.45 percentage points higher than the baseline, respectively for 64, 128, and 256 bits. For NRU, our approach is also the best for all cases, however, the improvements are under 0.04 percentage points.

The L1 behavior impacts the L2 miss rates, in which the opposite occurs. As we already explained before, LU also follows almost the same behavior from the coherence-based strategy,

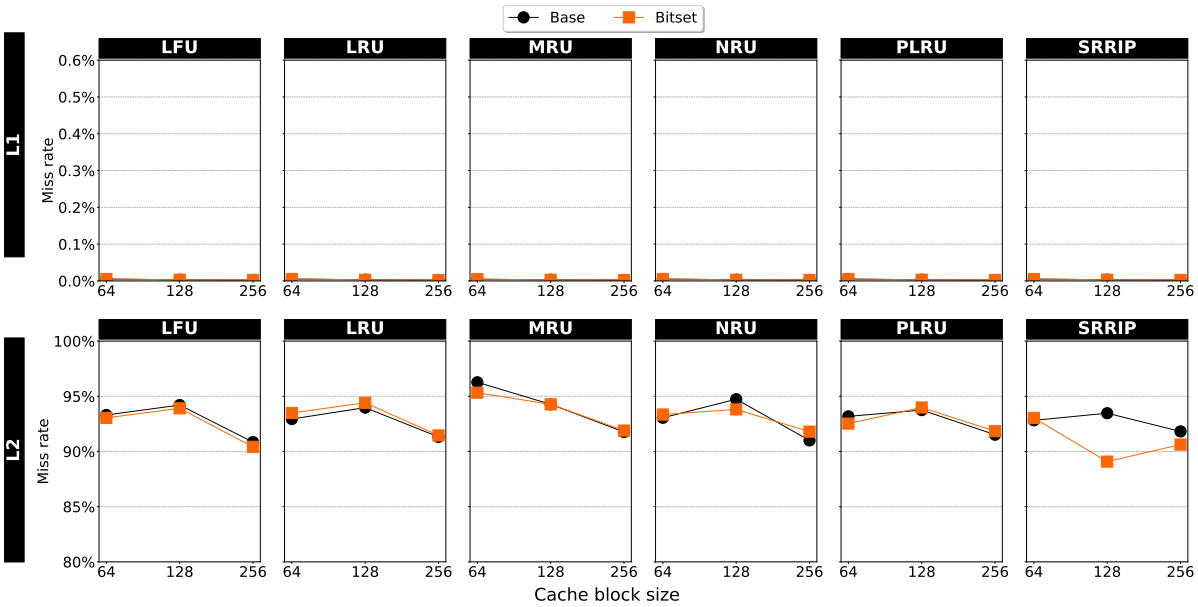
Figure 6.14 – Cache miss rate vs. block size - LU



Source: Elaborated by the author

selecting the Shared state as preference. Even so, in the L2 level, we find that for LFU and PLRU policies the Bitset strategy is better than prioritizing the Shared state statically. On the other hand, these results are still worse than the baseline ones.

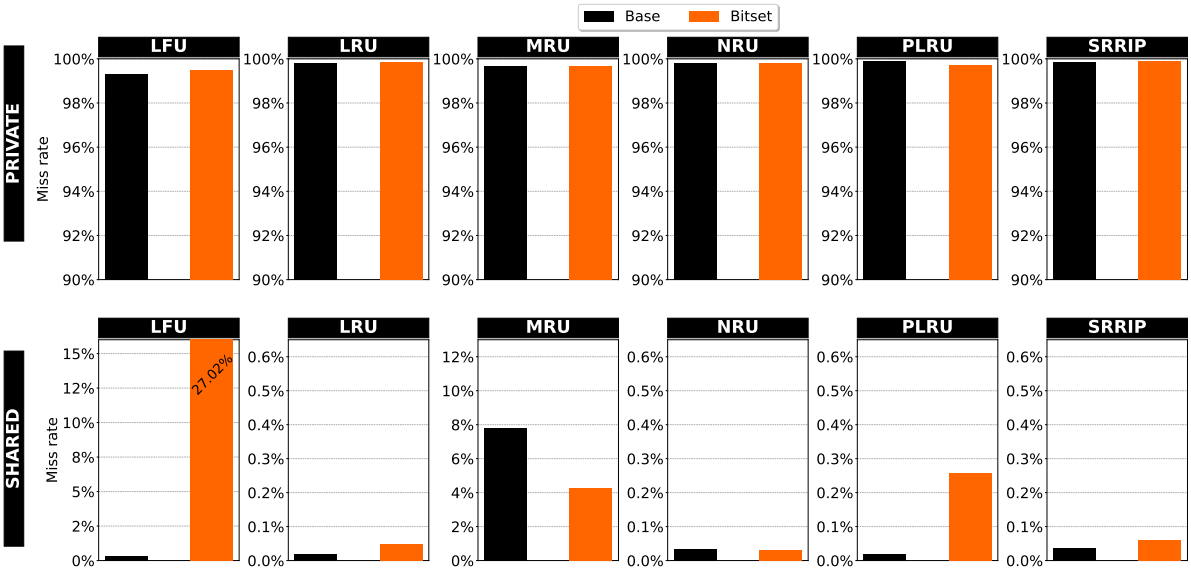
Figure 6.15 – Cache miss rate vs. block size - TSP



Source: Elaborated by the author

For TSP, we observe from Figure 6.15 two situations in which the behavior of our strategy differs from the baseline. Both cases are observed in the L2 cache. First, with the NRU policy, the Bitset approach is better than the baseline with 128-bit blocks, contrary to what is observed with 64 or 256 bits. The second case is with SRRIP the policy, which shows our strategy is better than the unmodified policy for 128- and 256-bit blocks.

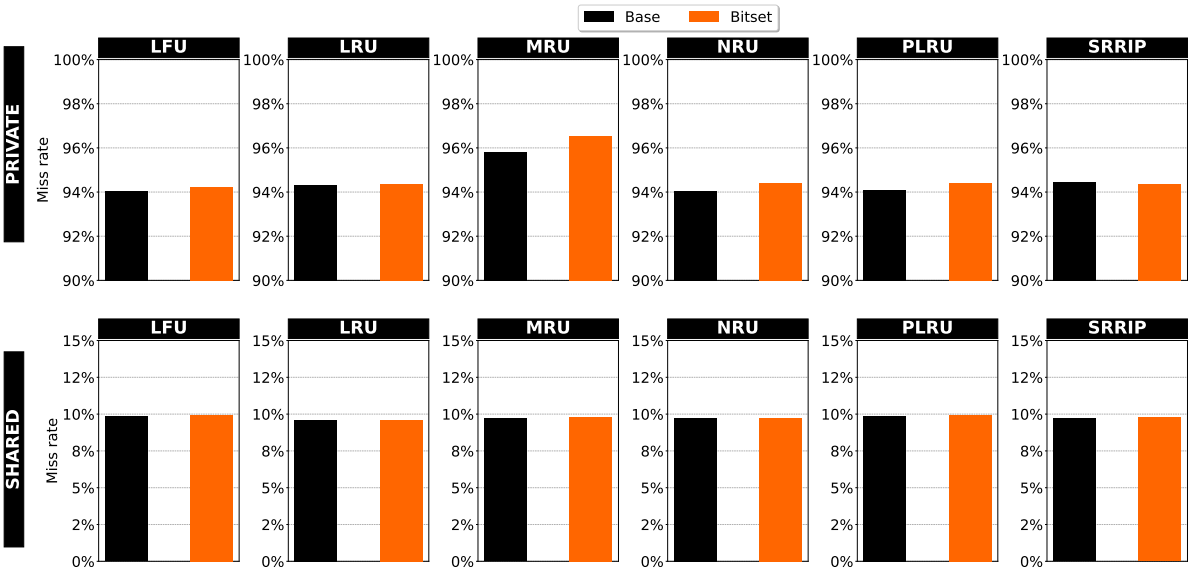
Figure 6.16 – L2 cache miss rate vs. sharing strategy - FAST



Source: Elaborated by the author

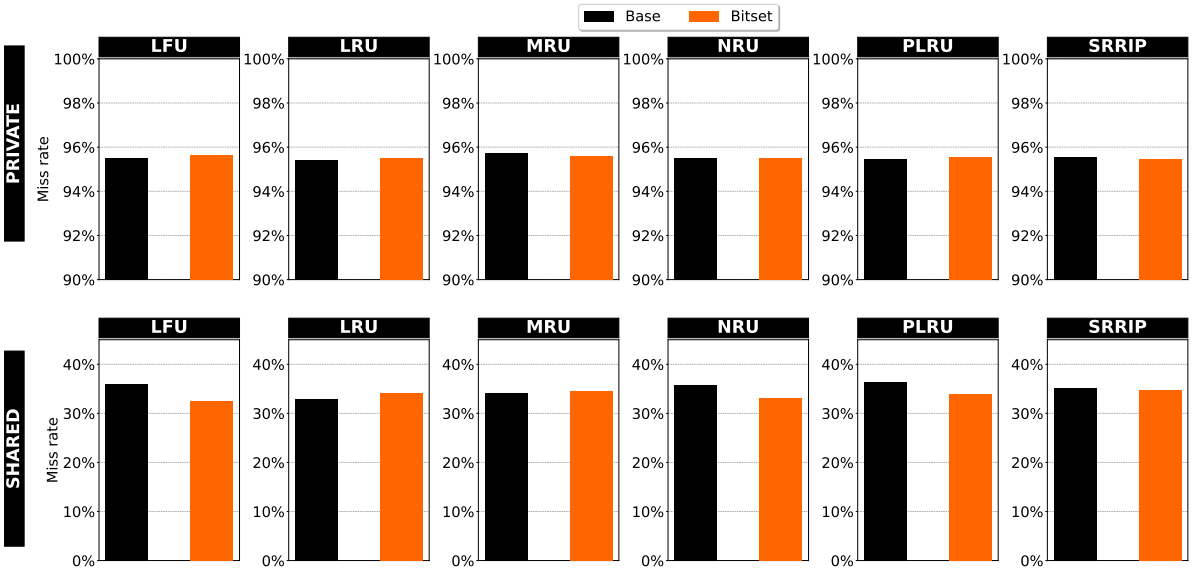
We also vary the sharing strategy between private and shared for the L2 cache using the Bitset approach. The overall observed behavior is the reduction in cache miss rates when using shared caches. This is due to the biggest cache size and number of available sets, thus reducing conflict misses. Figure 6.16 shows the results in this perspective for FAST. The miss rates from the shared cache, when FAST is simulated using the policies LRU, NRU, PLRU, and SRRIP, are too low, varying from a small range between 0.03% and 0.06%. With shared caches, we can also observe that our strategy using LFU is much worse than the baseline. Contrarily, MRU shows a 4.24% miss rate, less than with the unmodified version. Following, Figure 6.17 shows that the sharing strategy does not impact the Bitset approach for the FN kernel.

Figure 6.17 – L2 cache miss rate vs. sharing strategy - FN



Source: Elaborated by the author

Figure 6.18 – L2 cache miss rate vs. sharing strategy - GF

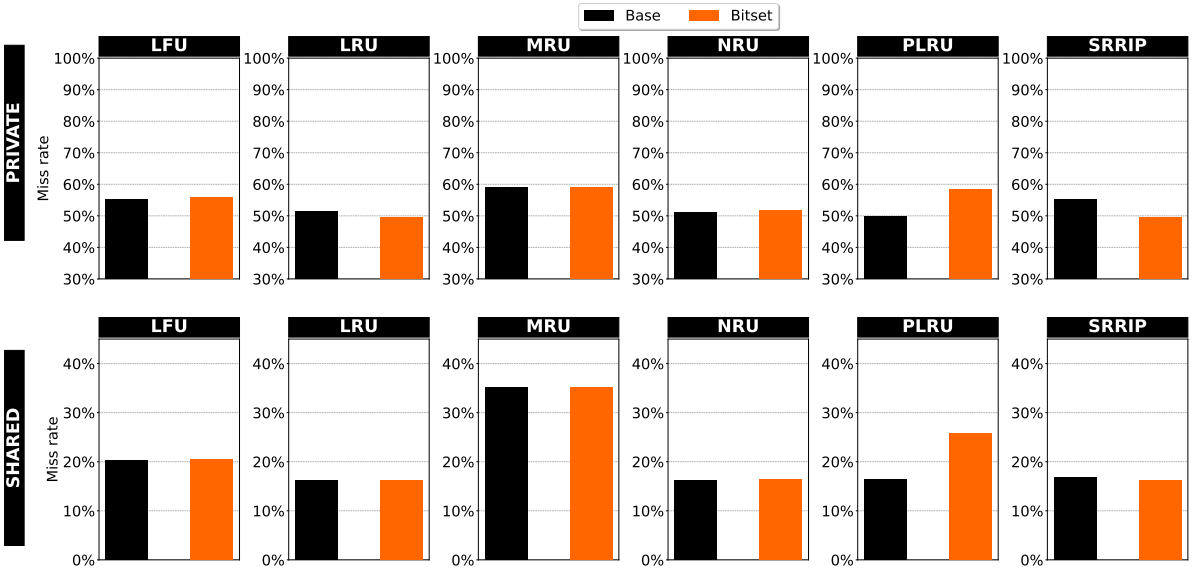


Source: Elaborated by the author

Figure 6.18 shows the comparison between the shared and private L2 caches for GF. Besides the expected reduction in the miss rate for shared caches, the charts show that our strategy is better than the baseline for three policies: LFU, NRU, and PLRU.

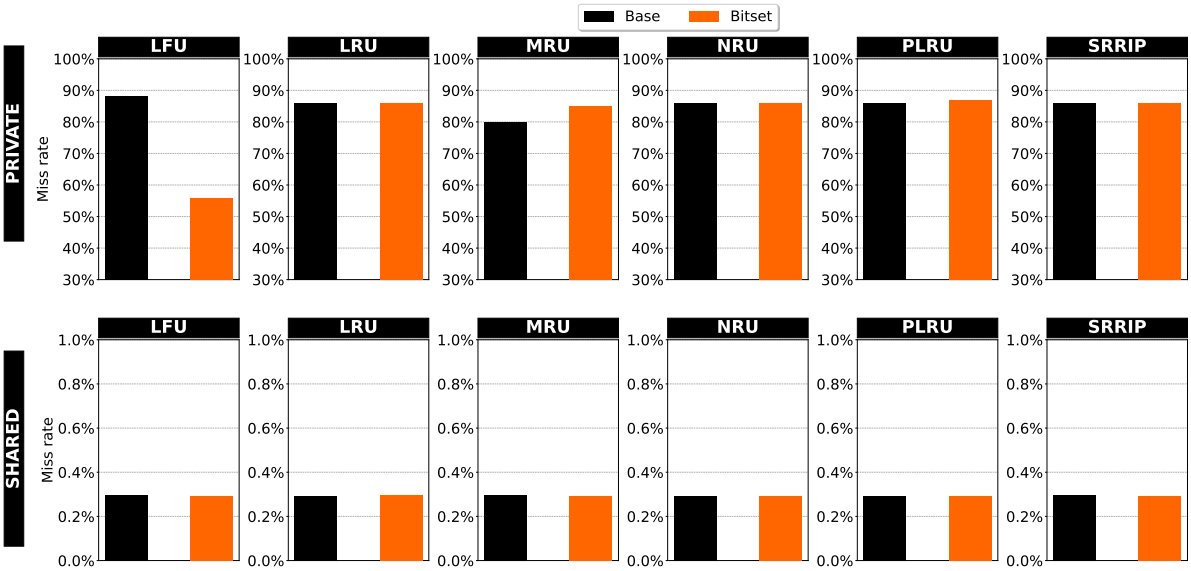
On the other hand, the results from the IS kernel show that our strategy improves LRU and SRRIP mainly with private caches. These results are depicted in Figure 6.19 below. For other policies, the behavior is pretty much the same.

Figure 6.19 – L2 cache miss rate vs. sharing strategy - IS



Source: Elaborated by the author

Figure 6.20 – L2 cache miss rate vs. sharing strategy - KM

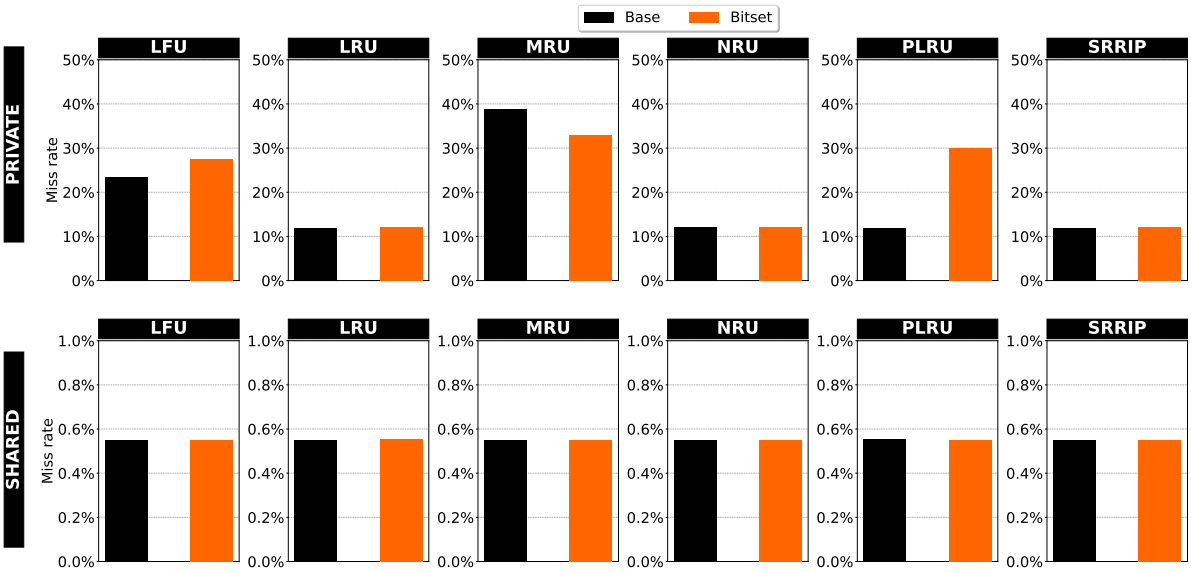


Source: Elaborated by the author

With regard to KM, we see changes in the behavior of some policies. Figure 6.20 shows this case, in which we first observe a more pronounced change using LFU. The improvement that our strategy causes is higher in private caches. Although this occurs, we recall that the miss rates in shared L2 caches are much lower than in private ones.

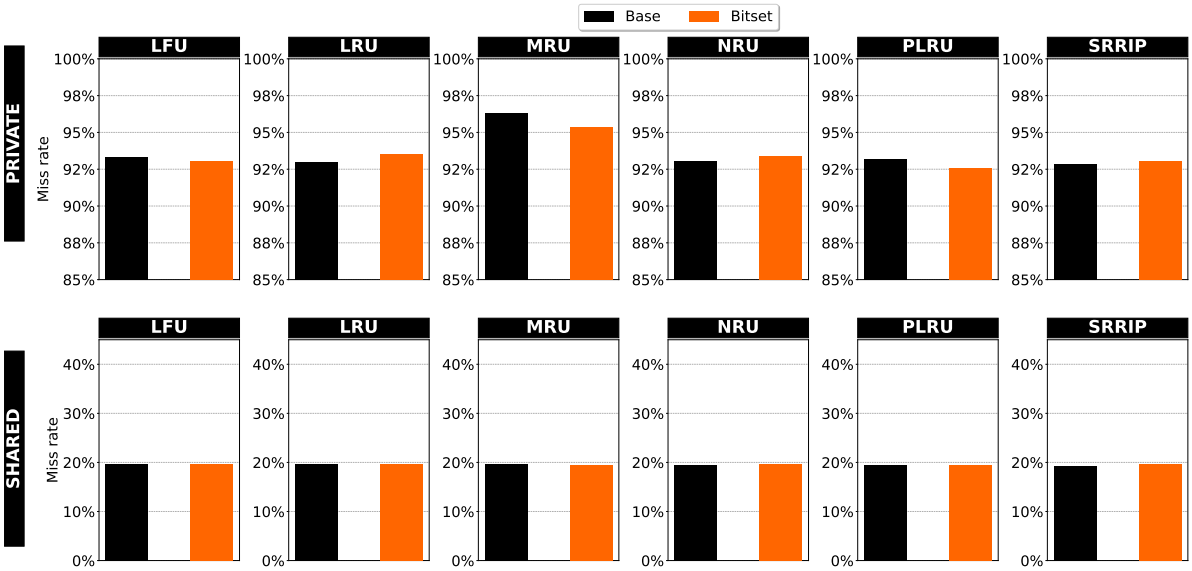
The same analysis takes place for LU, whose results are in Figure 6.21 below. We observe that in private caches, our strategy improved the MRU policy, but it is worse than the baseline for LFU and PLRU. For shared caches, due to the very low values this sharing strategy presents, the gains or losses are imperceptible.

Figure 6.21 – L2 cache miss rate vs. sharing strategy - LU



Source: Elaborated by the author

Figure 6.22 – L2 cache miss rate vs. sharing strategy - TSP

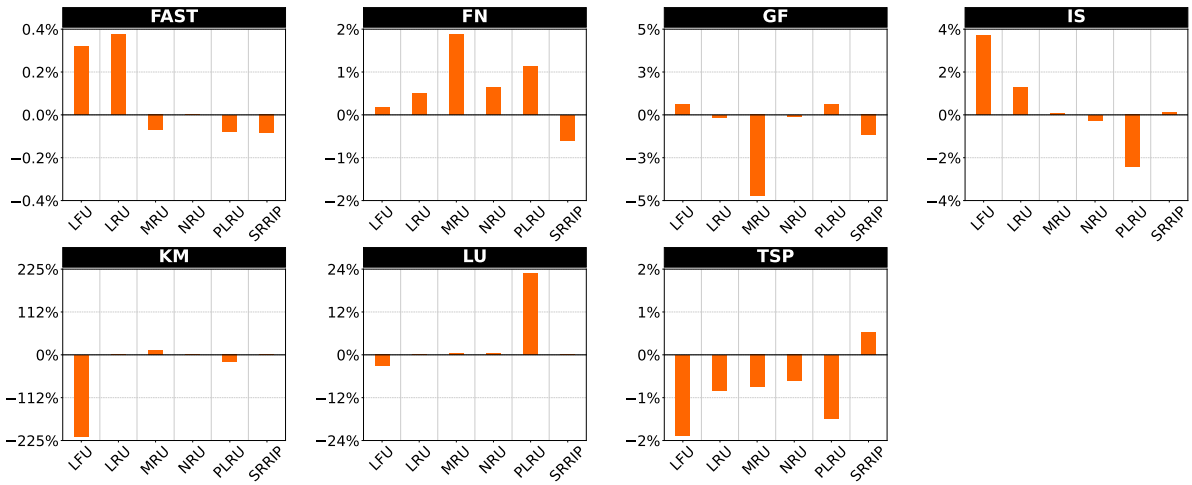


Source: Elaborated by the author

Finally, Figure 6.22 shows the results for TSP, in the context of the sharing strategy perspective. For private caches, the charts show that the Bitset approach is worse than the baseline for LRU, NRU, and SRRIP. On the other hand, they also show that it is better for the other three policies (LFU, MRU, and PLRU). In shared caches, due to the reduction in the miss rates, the behavior we mentioned is not noticeable.

For an overview of the strategy described in this section in terms of miss rates, Figure 6.23 below shows these values for the L1 cache. These comparisons, as well as others to come in this section, are made using 8-way set associative caches with 64-bit block sizes.

Figure 6.23 – L1 cache miss rate decrease



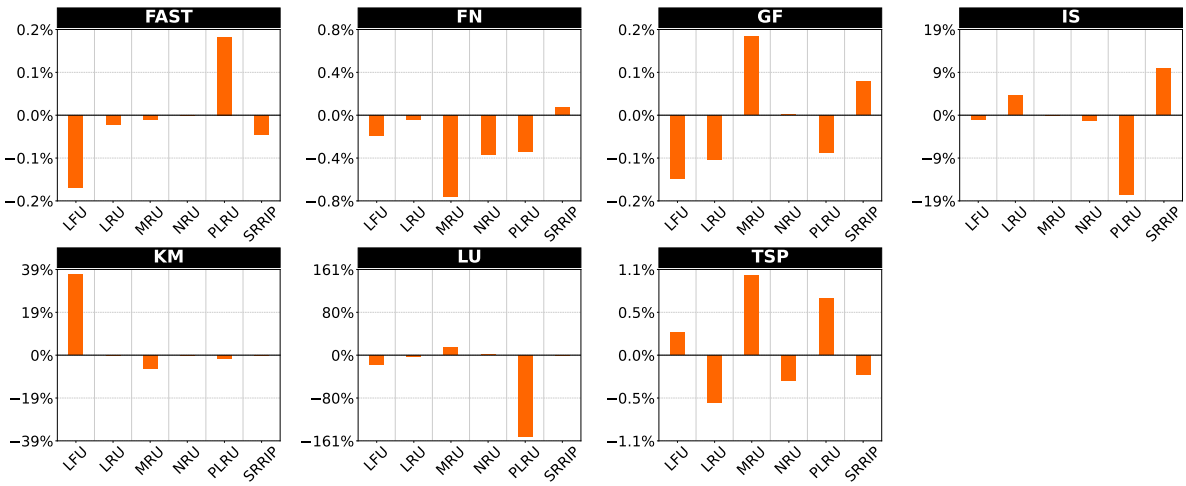
Source: Elaborated by the author

For FAST, although there are no significant variations, it is possible to obtain some improvement with LFU and LRU policies, using our Bitset alternative. In contrast, for FN, the only policy that failed to break its unmodified version with our approach is the SRRIP. In

this kernel, the best reduction in the L1 miss rate is observed in MRU (1.71%). On the other hand, the MRU/Bitset strategy is the worst for GF.

With the IS kernel, LFU and LRU are slightly improved by the Bitset strategy, achieving rate decreases of 3.8% and 1.34%, respectively. Positive results from KM are imperceptible. Even so, there is a significant increase in the miss rate – more than 2 times greater – for LFU when our approach is employed. For LU, our approach improved PLRU by a good margin, which is 24.84% fewer miss rates in the L1. TSP shows only one small improvement in the L1 cache miss rate, which is in the SRRIP policy (0.63% less miss rate).

Figure 6.24 – L2 cache miss rate decrease



Source: Elaborated by the author

Figure 6.24 shows the overall comparison between L2 cache miss rates perceived in the modified and unmodified policies. We do not find significant improvements with FAST (the variations are under 0.2%). Indeed, the only improvement with the Bitset approach is observed in the PLRU policy, which is 0.19% better. For GF, the strategy can improve MRU and SRRIP by a small margin too. Moreover, FN shows almost no improvements using the Bitset strategy.

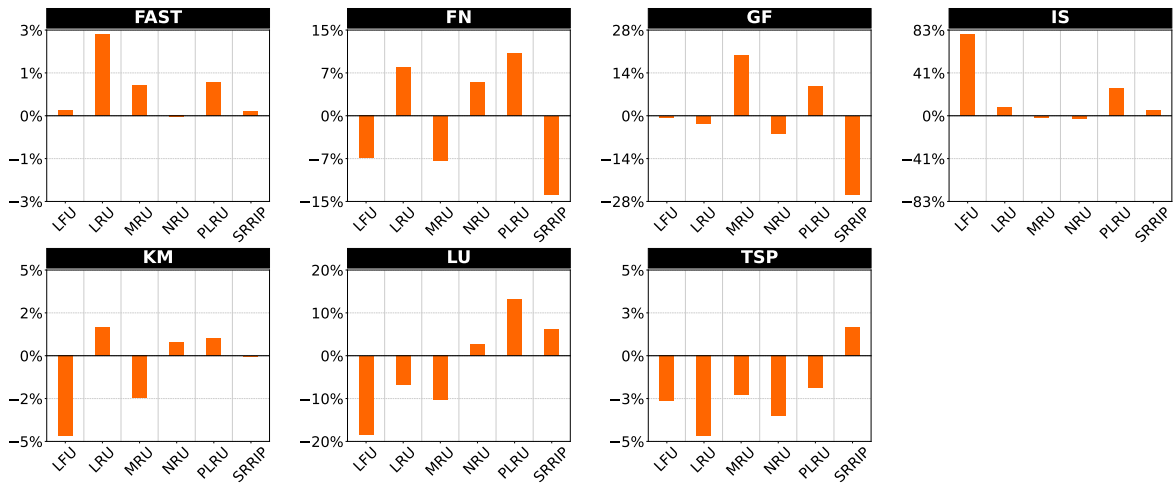
For IS, the best L2 miss rate reduction is perceived with the SRRIP and our strategy, which is 10.21% less miss rate. The strategy can also improve SRRIP, reducing the cache miss in L2 by 4.28%. On the other hand, the Bitset approach makes PLRU worse for this kernel, with an increase of 18.82% in the miss rate.

Although most policies could not be improved by our strategy for KM, we observe a good improvement with the LFU policy. The strategy can reduce the rate by 36.57%. The LU kernel presents the best result with MRU, which is 20.39%. Conversely, it increases the cache miss rate with PLRU by approximately $1.5\times$. Last, for TSP, our strategy achieves a 0.98% miss rate reduction, at most, when used with the MRU policy.

6.1.2 Coherence operations

The use of coherence protocols in the memory hierarchy brings some cost, as a result of operations that must be performed to ensure the correct use of data. Figure 6.25 shows the invalidate operations perceived when we simulate the approaches described in this section. We observe from FAST that our approach can reduce the number of invalidates, regardless of the policy. The best result in this kernel is with the LRU policy, in which we observe a reduction of 2.78%. For IS, we perceive improvements for most of the policies, except for NRU and MRU ones. For instance, with LFU, our technique reduces the number of invalidates by 78.67%. With the KM kernel using the Bitset approach, we observe small improvements with LRU, NRU, and PLRU. However, a more pronounced increase in the number of operations is perceived with LFU for this kernel – 4.37% more invalidates. Likewise, for LU, the Bitset technique employed in the LFU, LRU, and MRU policies causes the number of invalidates to increase. Notwithstanding, the technique reduces these operations by 13.2% when used with PLRU.

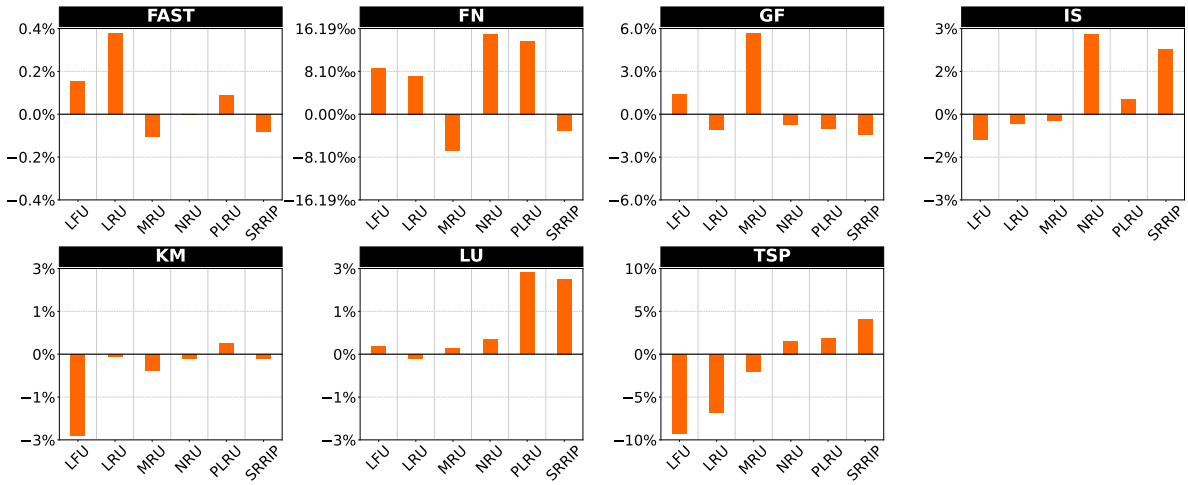
Figure 6.25 – Coherence invalidates decrease



Source: Elaborated by the author

With respect to write-backs, which occur when the data is forced to be written in the lower levels, we show Figure 6.26 on the next page. For FAST, FN, IS, KM and LU, we observe relatively small changes (under 3%) in relation to the number of write-backs the kernels performed due to coherence. On the other hand, GF shows higher reductions in write-back operations. More precisely, when we modify the MRU policy, this results in 5.69% fewer coherence write-backs for this kernel. In the case of TSP, we observe LFU, LRU, and MRU getting worse with the use of our strategy, while NRU, PLRU, and SRRIP exhibit the opposite behavior.

Figure 6.26 – Coherence write-backs decrease

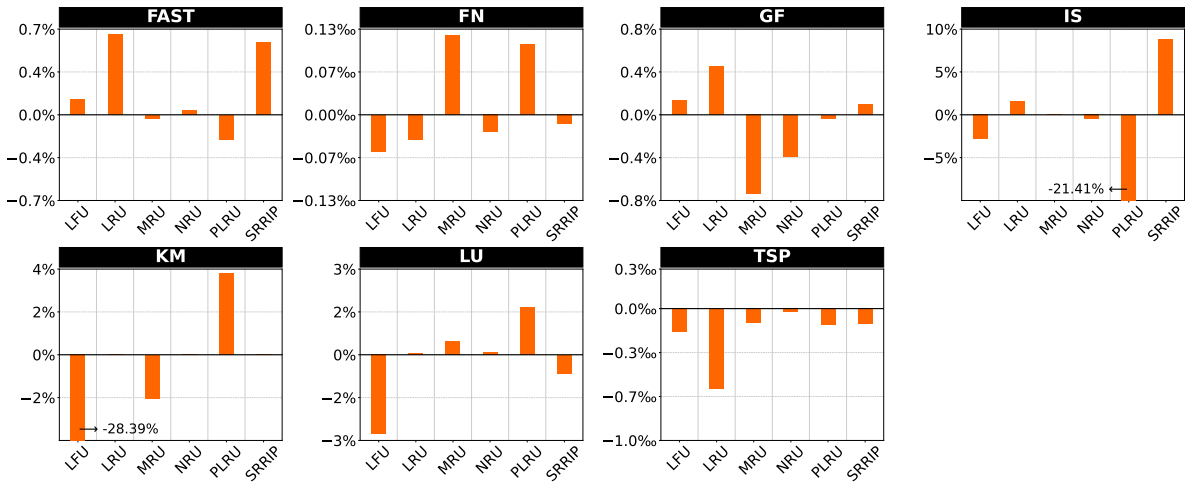


Source: Elaborated by the author

6.1.3 Instructions per cycle and energy consumption

Moving on to the performance discussion, Figure 6.27 below shows the IPC of the Bitset approach, in an overall perspective.

Figure 6.27 – IPC increase

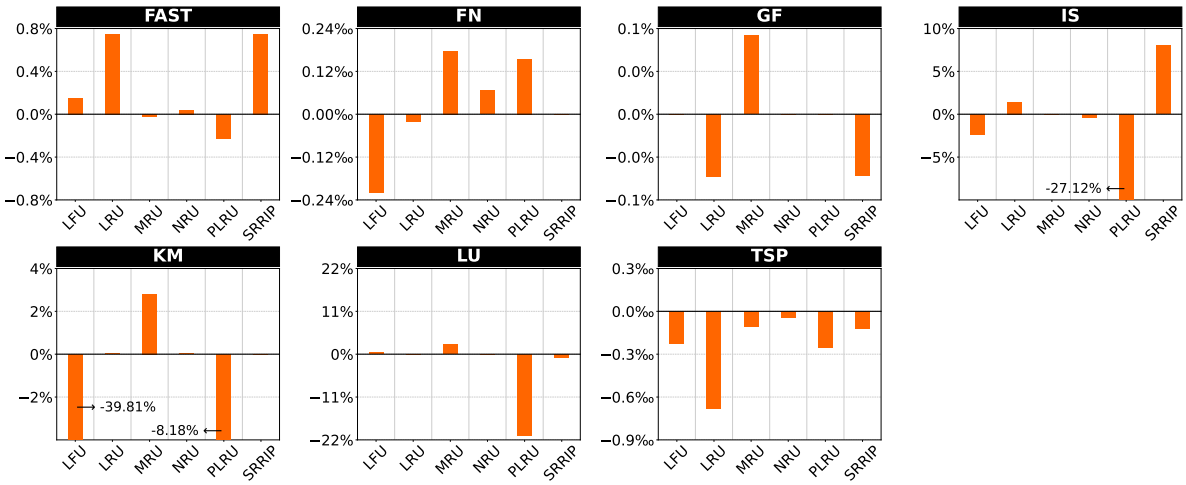


Source: Elaborated by the author

Four kernels change their IPC by less than 1% when we modify the policies. For instance, for FAST, the LRU policy with our strategy is capable of increasing the IPC by 0.68%. The CPU-bound kernels FN and TSP also do not show high variations, so much so that their rates are on the scale of per thousand (‰). In addition, TSP presents no improvements in the IPC when using the Bitset approach. TSP presents more cache miss rates in the L2 and L1 caches, respectively. Moreover, it shows bad results in terms of coherence operations. Thus, as we can see from Figure 6.28 on the next page, the cycles spent during the execution increase, and consequently, the IPC decreased.

For GF we observe LRU with some improvement when using our strategy (0.43%), while

Figure 6.28 – Cycle decrease



Source: Elaborated by the author

MRU, the antagonist, shows a decrease in IPC of 0.69%. The MRU result is the worst for GF, also due to the increase in the cycles spent. The gains in coherence operations are not enough to pay for the penalty suffered as a result of the 4.96% miss rate increase in the L1 cache.

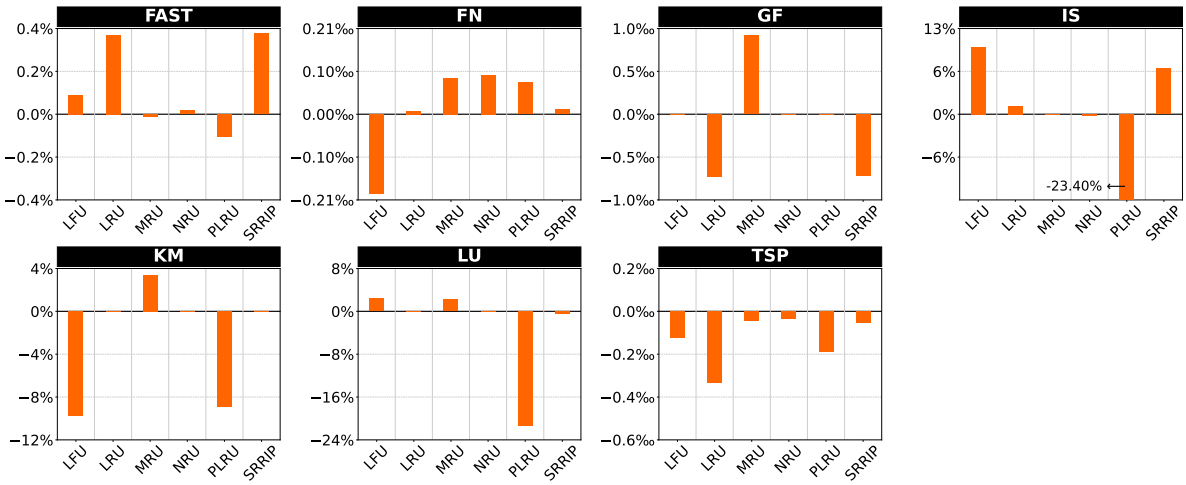
Furthermore, we observe that IS has its IPC lowered with the LFU and PLRU policies using our strategy. The PLRU/Bitset strategy shows 21.41% less IPC than the baseline. On the other hand, there is a minor increase in the kernel IPC with LRU and a more significant one with SRRIP. More precisely, for SRRIP, our strategy improved the IPC by 8.75%. IPC results from IS exactly follow the inverse behavior of the decrease in cycles.

For KM, the LFU policy shows a high decrease, which is 28.39% less IPC than its original version. More precisely, this value is a result of L1 cache misses, which results in higher memory access times. The Bitset strategy also shows a minor drawback with MRU (2.05% decrease) and an improvement of 3.79% with the PLRU policy. These results are inversely proportional to the decrease in cycles, as observed in Figure 6.28. Although KM spent fewer cycles with MRU and our approach, it also shows more invalidates and write-backs due to coherence transitions. Thus, we assume that the kernel executes more instructions also due to these transitions. The same analysis applies to PLRU in this kernel, and for other policies in LU. For instance, PLRU with the Bitset technique shows 1.73% more IPC than the baseline for LU, despite the increase in the number of cycles spent.

Figure 6.29 shows the energy consumption decrease of our strategy in comparison to unmodified policies. We observe that FAST follows a behavior directly proportional to what happens in the IPC analysis. That is, the faster the computation, the higher the decrease in the energy consumed. For this kernel, LRU and SRRIP are improved by our techniques, achieving around 0.35% less energy consumed. FN, GF, and TSP show very low variations under 0.1%.

The PLRU policy along with our technique shows bad results for IS, KM, and LU. For these kernels, it increased the energy consumption by 23.4%, 8.88%, and 21.36%, respectively. For IS, the most relevant factor to increase energy consumption is the IPC, which is worse for

Figure 6.29 – Energy consumption decrease



Source: Elaborated by the author

this policy with the Bitset strategy. For KM, the IPC increases a bit, but this cannot pay for the power consumed when the L2 cache is used due to L1 misses. This is also the case for LU, however, the misses occur at the L2, which is even worse. The IPC, as well as the miss rates, are the metrics that cause the most impact on the energy consumption, which is modeled mathematically in McPAT.

7 REINFORCEMENT LEARNING-BASED EVICTION

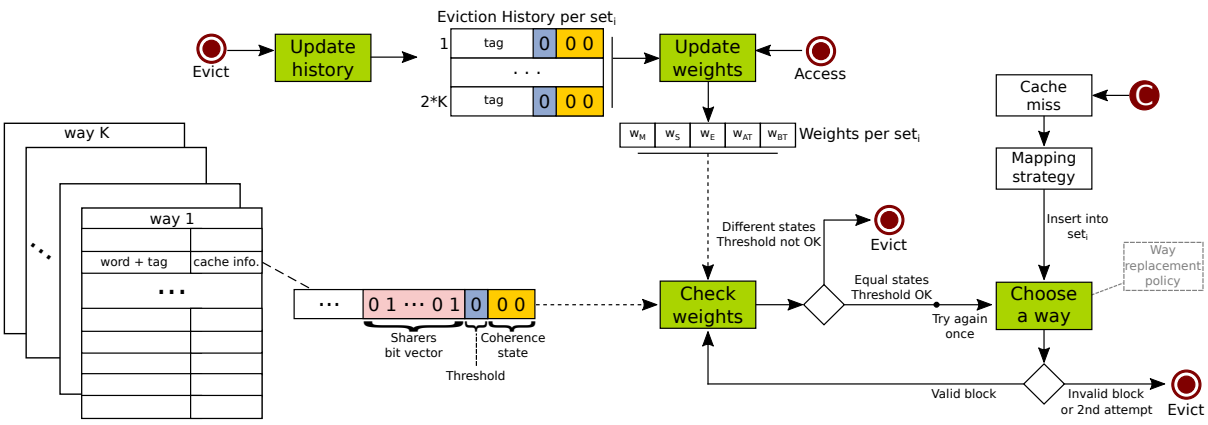
Finally, our last approach is to use Reinforcement Learning to decide if a chosen cache block should or not be evicted. While it may sound complex, this strategy is simple. We use the same premises as the other two approaches presented in the previous sections. However, the choice of the dynamic state involves learning which state to use, as well as observing the access to decide how to use the threshold.

As we discussed in Section 2.4, an RL system is composed of an agent, an environment, a policy, a reward signal, and a value function. Thus, first, we show our way-replacement strategy formulated as an RL problem. It is noteworthy that this formulation of way-replacement in RL can be done in several ways, as they have compatible theoretical principles.

In particular, our agent is the entity that decides whether or not to evict a cache block. The information from the cache – the eviction history containing coherence and sharing information from evicted cache blocks – corresponds to the environment. The environment communicates its state to the way-replacement agent so that the latter can decide which action a from the action space $A = E, DNE$ will be followed. The action space contains E, which stands for *evict*, and DNE for *do-not-evict*.

Depending on the chosen action, the environment responds with the reward signal $R \in \mathbb{R}$ and the updated state $s \in S$, which we only process in the future. This is a tricky decision for our model because we would only have a way to decide whether or not a block should be evicted when we need it again in the future. Thus, our policy $\pi(s)$ delays the use of states by at least one unit of time t. The state content in our approach consists of information from cache coherence and the sharing state of cache blocks. Then, we formulate a vector of weights $w_M, w_E, w_S, w_{AT}, w_{BT} \in \mathbb{R}$, which forms our value function $V^{\pi}(s)$. Although the weights are real numbers ($\mathbb{R}$), further improvements can consider the use of natural numbers ($\mathbb{N}$) for better performance. The vector of weights is updated with regard to an eviction history that is part of the policy in the RL system. Figure 7.1 presents this strategy out of the RL box.

Figure 7.1 – Overview of the RL-based eviction strategy



Source: Elaborated by the author

Every time an eviction occurs, information from the evicted block is put into the eviction history of each set. Besides that, if the new cache block has the corresponding tag in the history, this entry is removed from it. Each entry contains the tag from the cache block; a bit which indicates if it was above (0) or below (1) the threshold of sharers; and two bits to represent the coherence state it was before being evicted. The size of the eviction history was arbitrarily set to two times the associativity. Thus, we need $2 \cdot K \cdot N$ entries in the whole cache to carry the strategy out. Although this setting may impact the quality of the strategy, we decided to evaluate changes in the history size in future work.

The tag is stored to ensure the uniqueness of the entries in the history. If the history is full, we remove the entry which was least recently updated. Cache accesses trigger the process of updating weights. If the requested block is not in the table, we consider that there was a positive decision, otherwise, a negative one. Hence, reward values are updated with regard to this approach, in the form of the vector of weights. The vector contains two probability distributions. The first one pertains to which coherence state has the probability of being a good decision of eviction (w_M, w_E, w_S). The other one indicates which direction to observe, above or below, in relation to the given threshold for the number of sharers (w_{AT}, w_{BT}).

It is necessary to give the algorithm some directions in the first moments of the computation when there is no probability distribution yet. Thus, we choose to set the Exclusive coherence state as the most likely to be evicted, since this will cause less coherence traffic. We applied the same principle of less traffic when we decided to start the algorithm evicting blocks with the sharers bit-vector with a count below the threshold. The exploration-exploitation trade-off is naturally solved during computation because this information changes regardless of the reward.

Finally, as in other proposals, the replacement process starts in Ⓒ, a candidate way is chosen, and we decide to evict or not by comparing the vector of weights against the cache block. If the block information matches the ones with high probability in the vector, the block is evicted; otherwise, the next attempt is carried out.

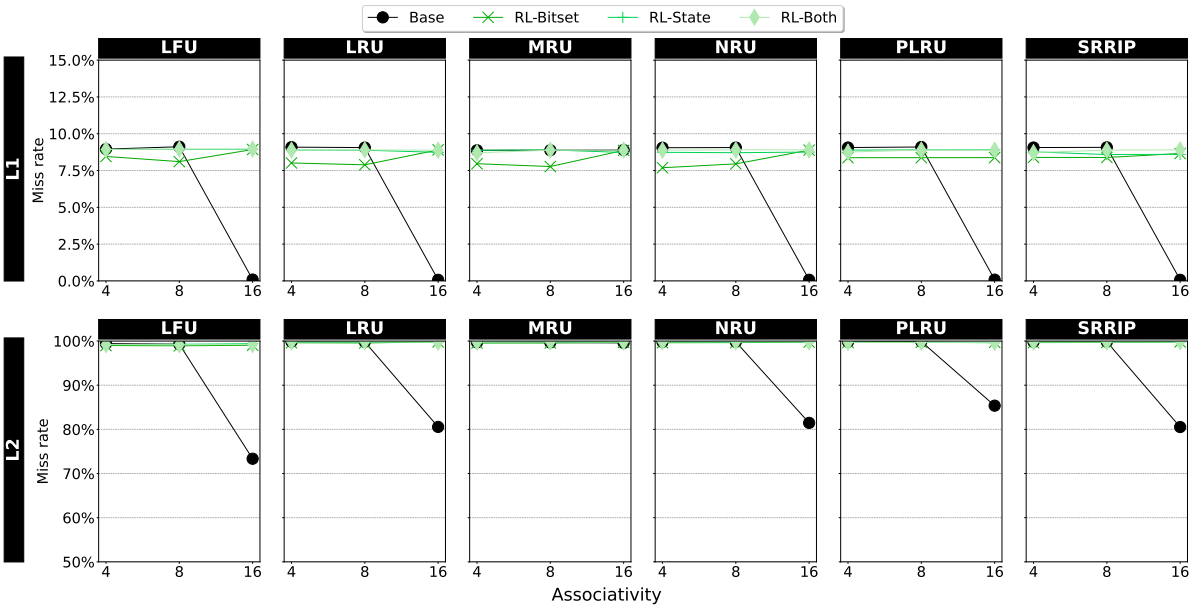
7.1 Experimental results

We decided to evaluate this strategy in two steps. First, we separately use the coherence state and the sharer's count as information for the RL algorithm. From now on, we call these RL-State and RL-Bitset strategies, respectively. Next, we evaluate the use of both information in the same run (RL-Both).

7.1.1 Cache miss rate

We use the same perspectives to present cache miss rate results from the way-replacement policies adapted with our RL-based strategies. We vary the associativity and the cache block size. Then, we change the sharing strategy, and present an overall analysis of the miss rate.

Figure 7.2 – Cache miss rate vs. associativity - FAST

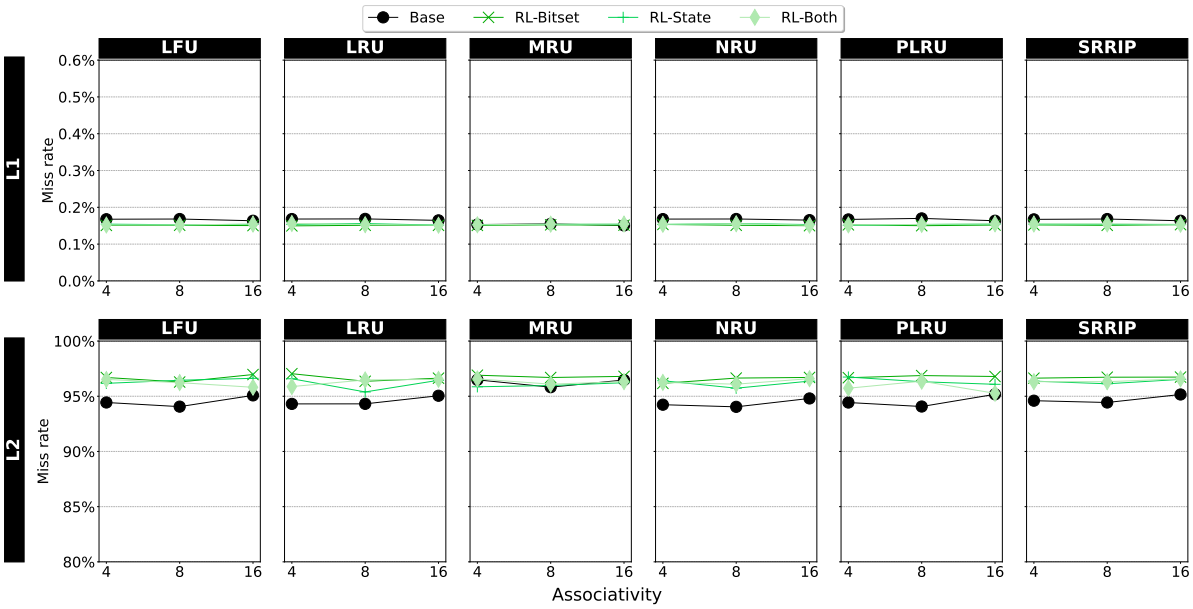


Source: Elaborated by the author

Figure 7.2 shows the miss rates for FAST when we vary the associativity. First, we observe in the L1 that our strategies do not follow the behavior perceived so far for this kernel when using 16 ways. The RL-based strategy has lost the ability to favor FAST when there are more ways available in a single set. However, when fewer ways are employed, the RL approach shows better results than the ones from unmodified policies. In the 4-way associative cache, NRU with the RL-Bitset approach achieves a 7.69% miss rate, which means 2.36 fewer percentage points than with the unmodified NRU policy. The best results with our techniques for other policies are observed with 8-way caches. These results are better than the original policy by a range of 0.68 to 1.17 percentage points. We observe from the L2 miss rates from FAST that, again, our RL strategy does not show gains when the associativity is set to 16. The best result we find with our strategies is 98.93%, more precisely, using the bitset as input data, along with LFU and 8-way caches.

The cache miss rates for FN are shown in Figure 7.3 on the next page. Unlike what happens with the other proposals, the RL-based policies show lower L1 miss rates than most unmodified policies. Furthermore, this is regardless of associativity. In the L2 cache, we observe two cases that present patterns which are different from the baseline. First, when the associativity is increased from 8 to 16, the miss rate also increases. Meanwhile, the RL-Both strategy shows a decrease when the associativity changes in this same way. Second, with the PLRU policy, the baseline presented the best case with 8-way associative caches. Contrariwise, the RL-Both approach shows the worst result in this configuration.

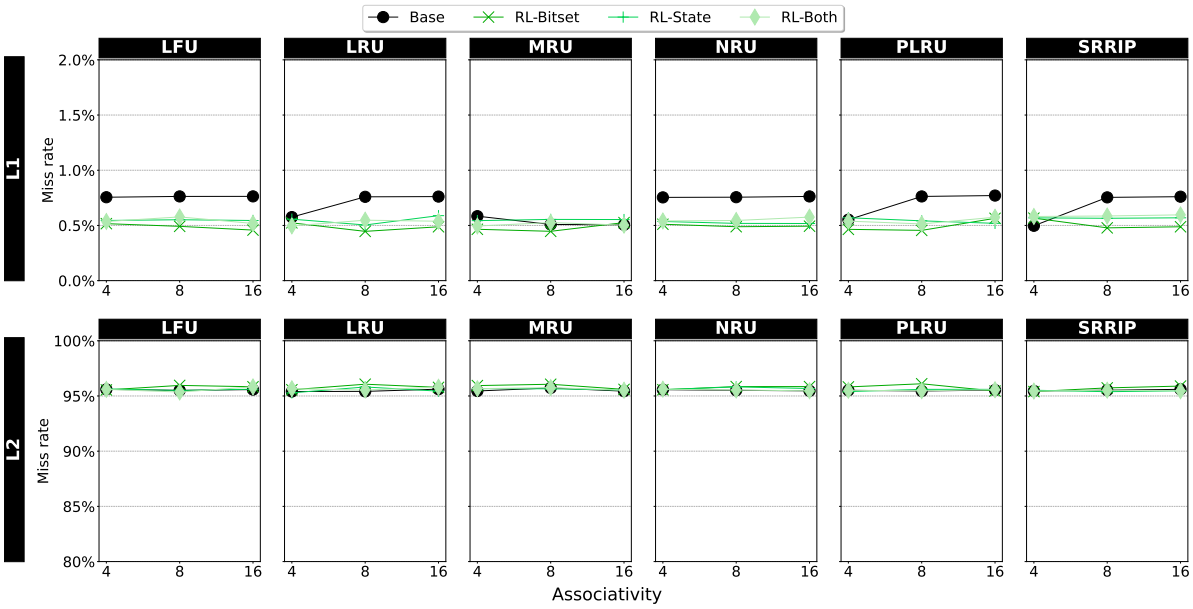
Figure 7.3 – Cache miss rate vs. associativity - FN



Source: Elaborated by the author

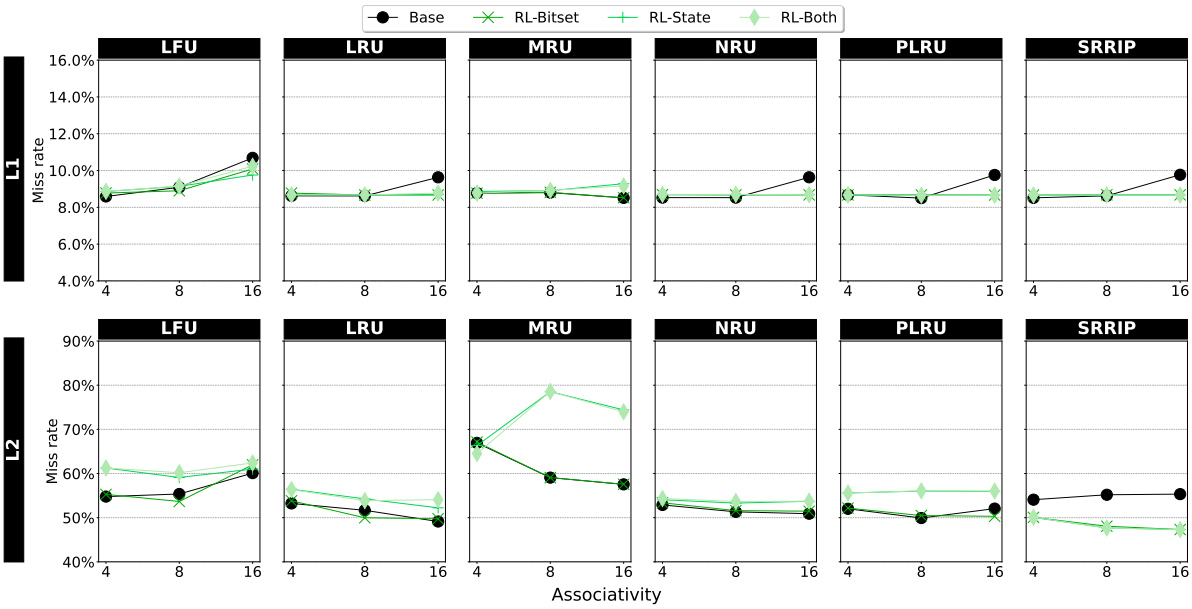
Figure 7.4 presents the results for GF when we modify the associativity. We observe that our strategies show more gains than the baseline when the associativity is increased. For instance, the unmodified LFU presents almost the same miss rates regardless of the number of ways. Another more accurate example is observed with SRRIP when the associativity increases from 4 to 8. With the unmodified SRRIP, the rate also increases. On the other hand, the RL strategies show lower miss rates with 8-way caches than with 4-way ones, mainly the RL-Bitset approach. The low miss rates in the L1 led to the high rates in L2, thus, we perceive few changes in the behaviors when the associativity is changed.

Figure 7.4 – Cache miss rate vs. associativity - GF



Source: Elaborated by the author

Figure 7.5 – Cache miss rate vs. associativity - IS

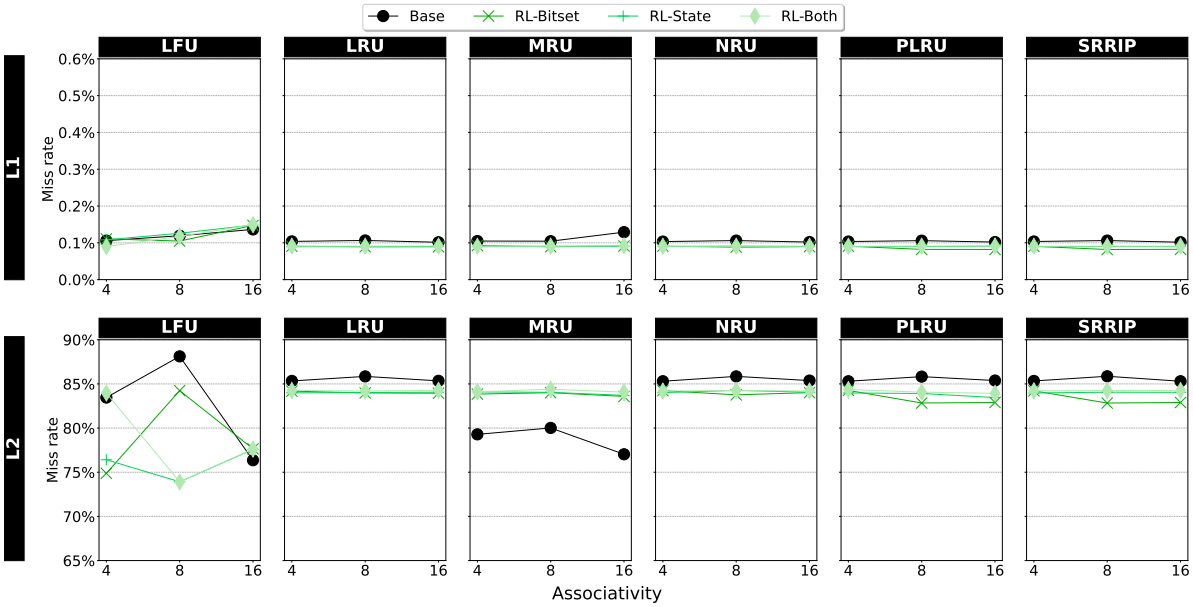


Source: Elaborated by the author

With regard to the IS kernel, Figure 7.5 shows its miss rates. We detect that the RL-based strategies are better than the baseline mainly with 16-way set associative L1 caches. The policies LRU, NRU, PLRU, and SRRIP present similar line patterns. The first two show around 0.95 more percentage points than the baseline with the RL-Bitset and 16-way caches. The last two are, with the RL-State strategy, better than the unmodified policy by approximately 1.10 percentage points. We observe from the L2 miss rates that, overall, our RL-based approaches lose to the original policies. Even so, the RL-Bitset strategy is better than the baseline in some cases. For instance, with LFU in the 8-way L2 cache, the miss rate is 53.68%, while being 55.36% with the unmodified policy. Furthermore, using the same associativity configuration, LRU and SRRIP have rates of 49.99% and 48.07% respectively – against 54.67% and 55.19%. We emphasize that SRRIP with our approaches is always better than its original version, regardless of associativity. The best result, in this case, is a rate of 47.31%, perceived with RL-State and RL-Both strategies.

Figure 7.6 on the next page shows the miss rates for the KM kernel. The lines which represent our strategies are under the one which represents the unmodified policy. However, the rates are too small to conclude that these differences represent some improvement. The other way around, we observe from L2 cache miss rates that our approach can produce some better results than the original policies. Before exemplifying these cases, it is possible to note the negative results of the MRU. With this policy, our strategies are worse than the baseline regardless of associativity. On the other hand, we observe that for 4 and 8-way associative caches, our approaches were better than the baseline. The best case for this policy is a 73.90% miss rate, versus 88.12% from the unmodified LFU. The use of the RL-Bitset approach results in even better results for PLRU and SRRIP, mainly with 8 and 16-way caches (around 82.80%).

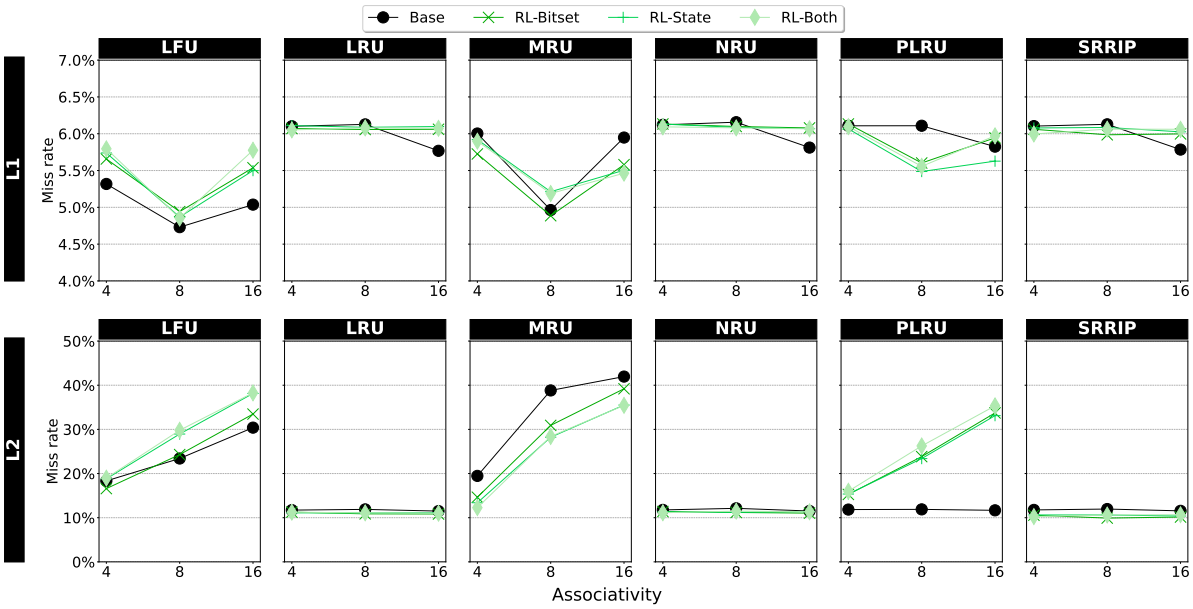
Figure 7.6 – Cache miss rate vs. associativity - KM



Source: Elaborated by the author

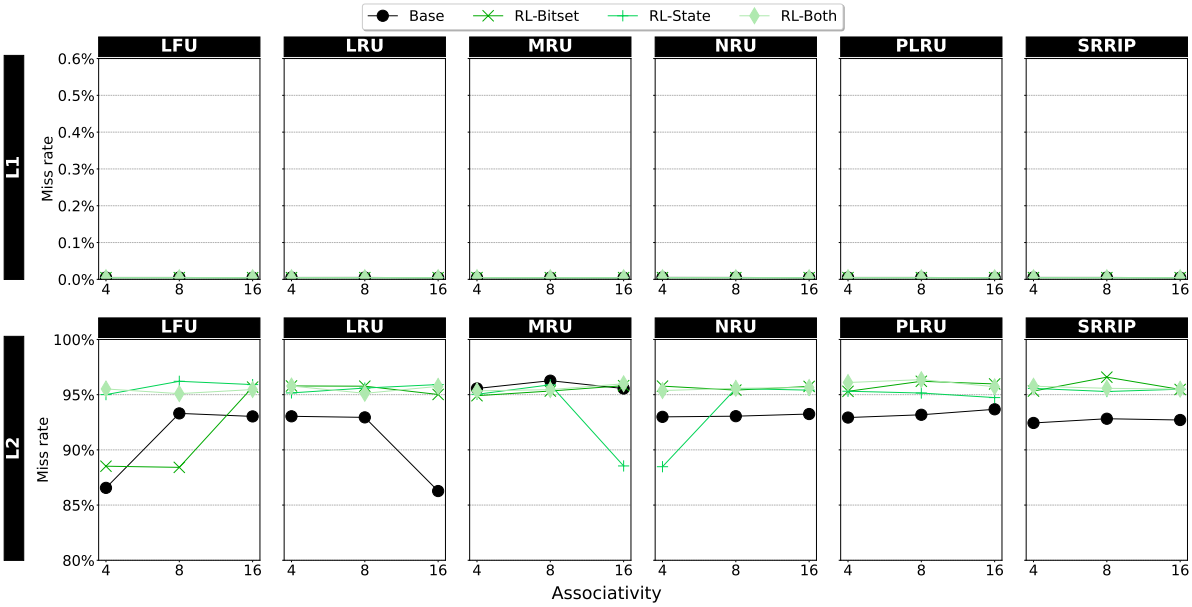
Figure 7.7 shows the cache miss rates for the LU kernel. Our RL-based strategy fails to reduce L1 miss rates for LFU, although the behavior when associativity is varied is similar to the baseline. We also observe that using 16-way caches is worse for LRU, NRU, and SRRIP at this cache level. Fortunately, when we modify PLRU, it can beat the original policy. The RL-State approach results in a 5.49% miss rate with PLRU when 8-way caches are used – the original one shows a 6.11% miss rate. Besides that, with MRU in 8-way caches, our approach can yield results close to the original policy. In the L2 cache, we observe that the rates for LRU, NRU, and SRRIP do not vary much. In a different way, the behavior of LFU, MRU, and PLRU policies show that the greater the associativity, the worse the rates.

Figure 7.7 – Cache miss rate vs. associativity - LU



Source: Elaborated by the author

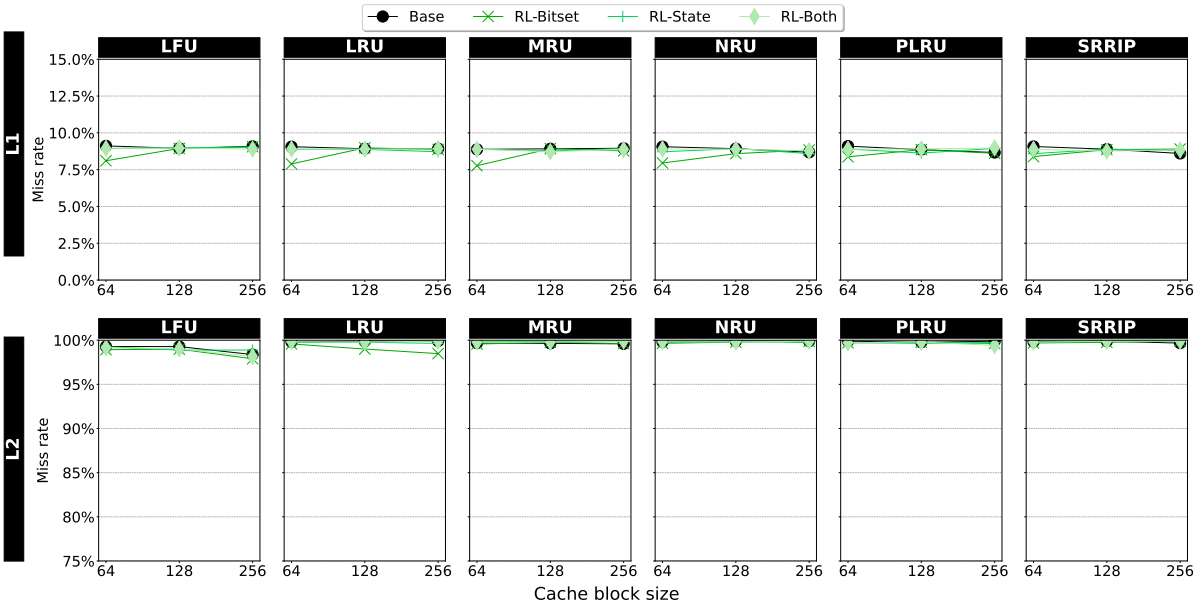
Figure 7.8 – Cache miss rate vs. associativity - TSP



Source: Elaborated by the author

The L1 cache miss rates for TSP, depicted in Figure 7.8, are too low. Thus, the behavior using our RL-based strategies and the baseline are visually the same. With regard to the L2 cache, we observe different behaviors with LFU. Baseline results are worse with higher associativity and better with 4-ways. On the other hand, the RL-Bitset in this policy kept similar rates with 4 and 8 ways. Besides that, the RL-State is the best for MRU in 16-way caches, and for NRU with 4-way caches. Next, we discuss the results with respect to cache block sizes.

Figure 7.9 – Cache miss rate vs. block size - FAST

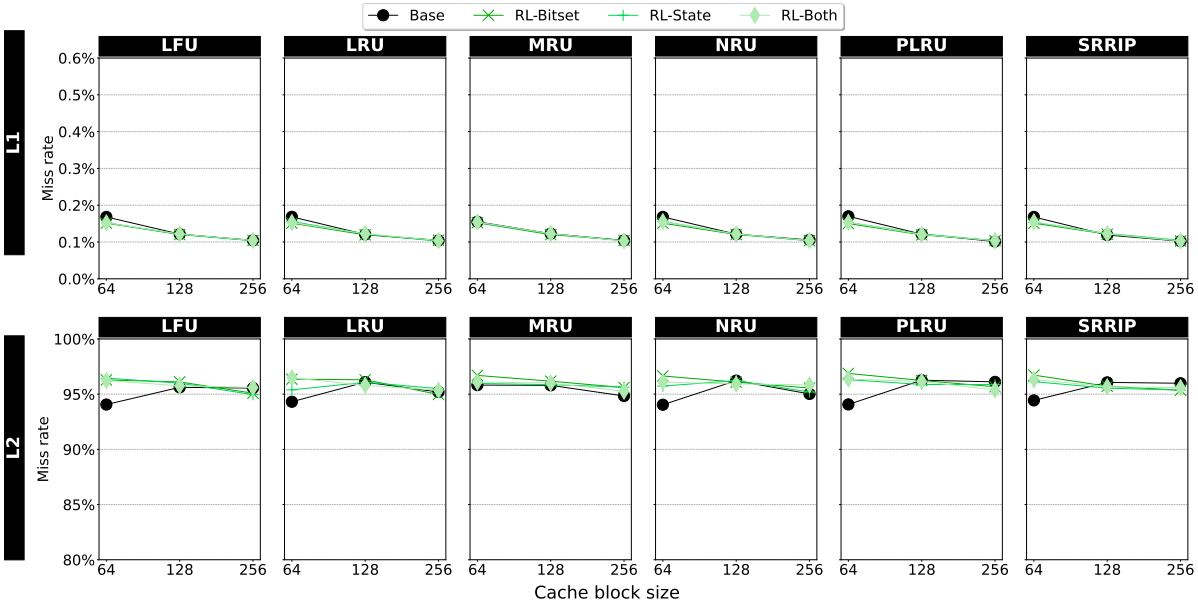


Source: Elaborated by the author

Figure 7.9 shows what happens to miss rates when we vary the block size. The main result we can observe pertains to the RL-Bitset approach in the L1. It presents the best results for all policies we chose to modify when the caches have 64-bit blocks. The lowest rate occurs with

MRU and the RL-Bitset strategy, which is 7.77%. If we increase the block size, the L1 miss rate also increases a bit. There is not much to be discussed about the cache miss rate at L2 gathered from FAST, due to the high values it presents. Even so, we see in the LRU chart that our RL-Bitset strategy can somehow reduce the miss rate, mainly with higher block sizes.

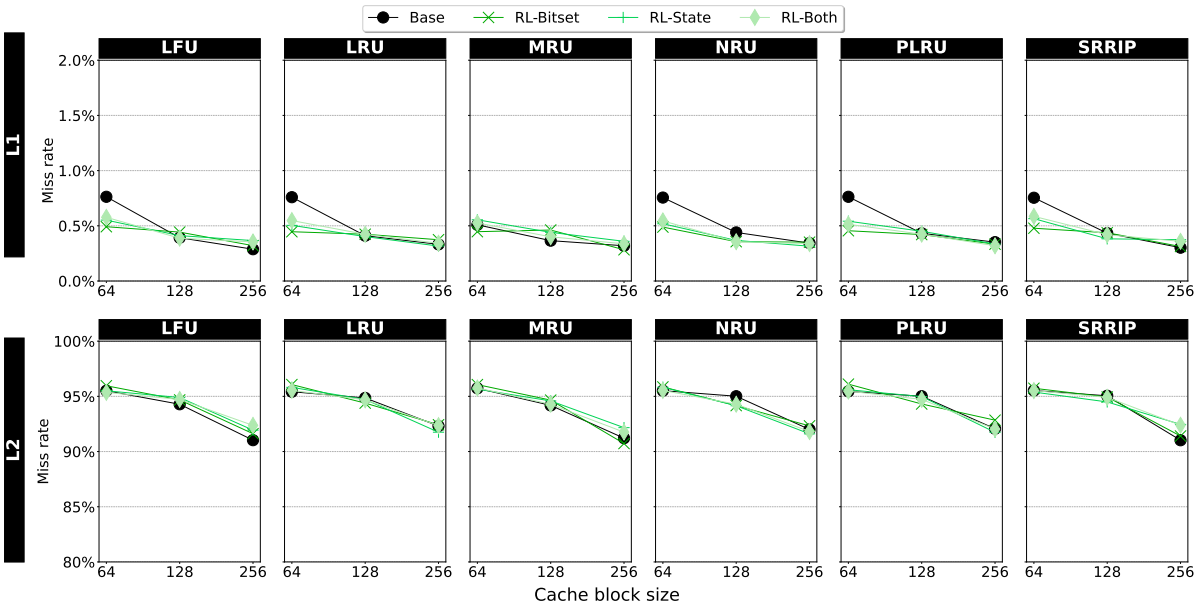
Figure 7.10 – Cache miss rate vs. block size - FN



Source: Elaborated by the author

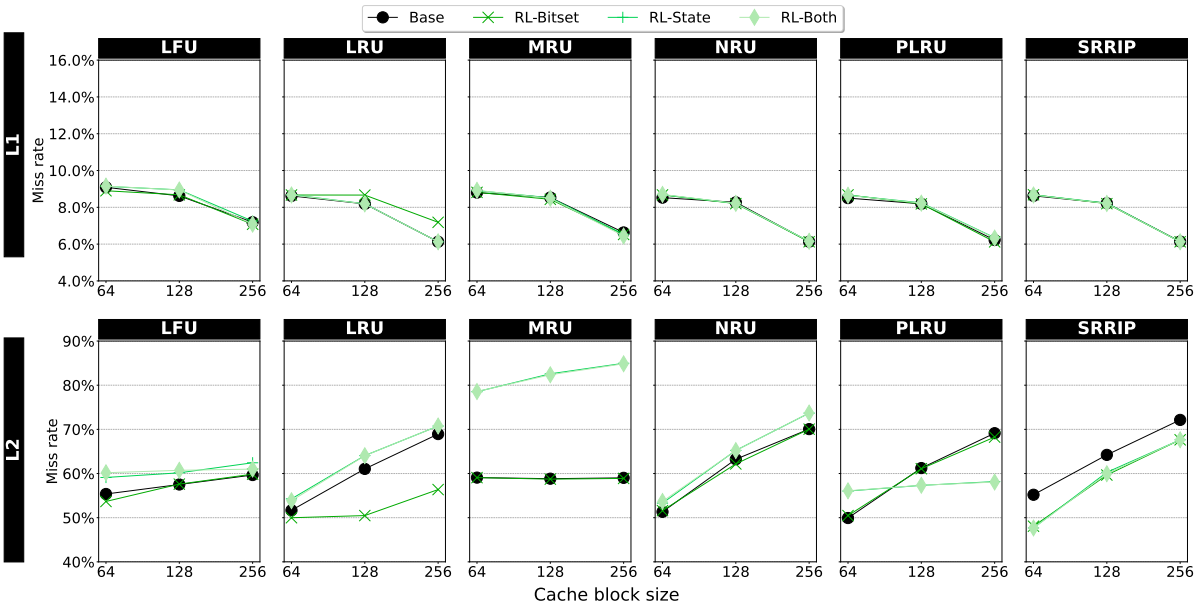
FN and GF results, presented in Figures 7.10 and 7.11 are very similar. They show a decrease in the miss rates in both cache levels when the block size is increased. Overall, with 64-bit blocks, the RL-based approaches are better than the baseline.

Figure 7.11 – Cache miss rate vs. block size - GF



Source: Elaborated by the author

Figure 7.12 – L1 cache miss rate vs. block size - IS



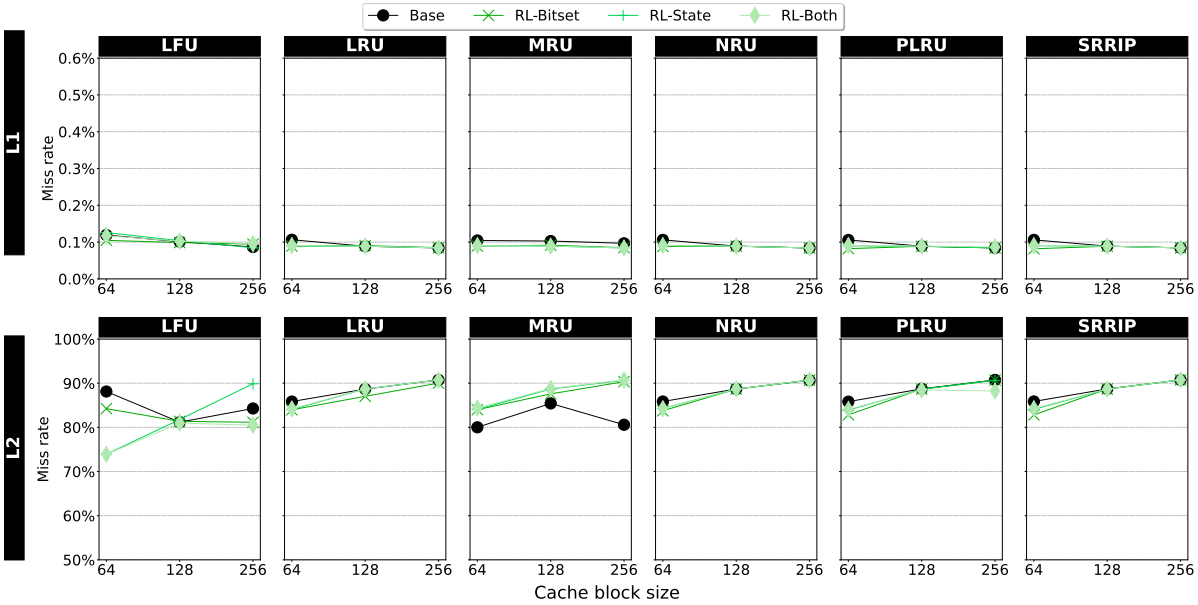
Source: Elaborated by the author

Figure 7.12 shows the miss rates gathered with IS. Unlike when we increase associativity, if we increase the block size, the miss rate gets better. However, we see no substantial improvements in this level over the baseline policies when our strategies are employed, regardless of the block size. The L2 results show that all RL-based eviction strategies with SRRIP are better than the unmodified policy. This is the policy that presents the lowest miss rate for IS, which is 47.67% with the RL-Both approach. The RL-Both approach can also improve PLRU when 128 or 256-bit blocks are set, but these results cannot beat the unmodified PLRU policy in caches with blocks of 64 bits. Last, we observe that LRU with the RL-Bitset approach shows better results than other versions for all block sizes. Furthermore, for its opposite (MRU), only the RL-Bitset can compete with the baseline policy, showing almost equal results.

On the next page, Figure 7.13 shows the miss rates gathered from KM simulations. The strategies we employed do not result in significant gains or losses, due to the low rates, in the L1 cache. Nonetheless, the L2 cache miss results are worthy of discussion. Generally speaking, the bigger the block size, the worse the miss rate. An exception is LFU, in which the RL-Bitset miss rates fall when the block size is increased (from 84.24% to 81.15%). Additionally, we find no improvement over the baseline using the MRU policy. In spite of this, the RL-based approaches present better results than the baseline when the block size is set to 64 bits. For instance, PLRU and SRRIP with the RL-Bitset are around 3.0 percentage points better than their unmodified versions. Another example, LFU is improved by RL-Both, achieving the rate of 73.90%, compared to the 88.12% from the original policy.

We show, also on the next page, the miss rates from the LU kernel in Figure 7.14. With our strategies, the diminishing rate behavior is kept in L1 when the block size is increased. The biggest difference we observe with our techniques is with PLRU and 64-bit blocks. The

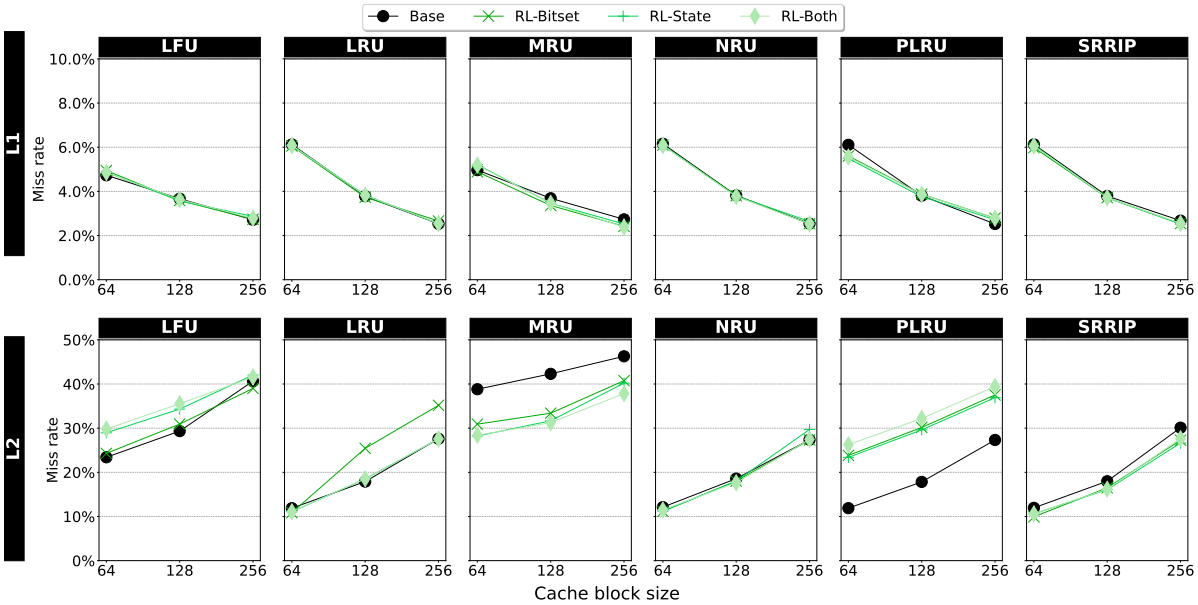
Figure 7.13 – L1 cache miss rate vs. block size - KM



Source: Elaborated by the author

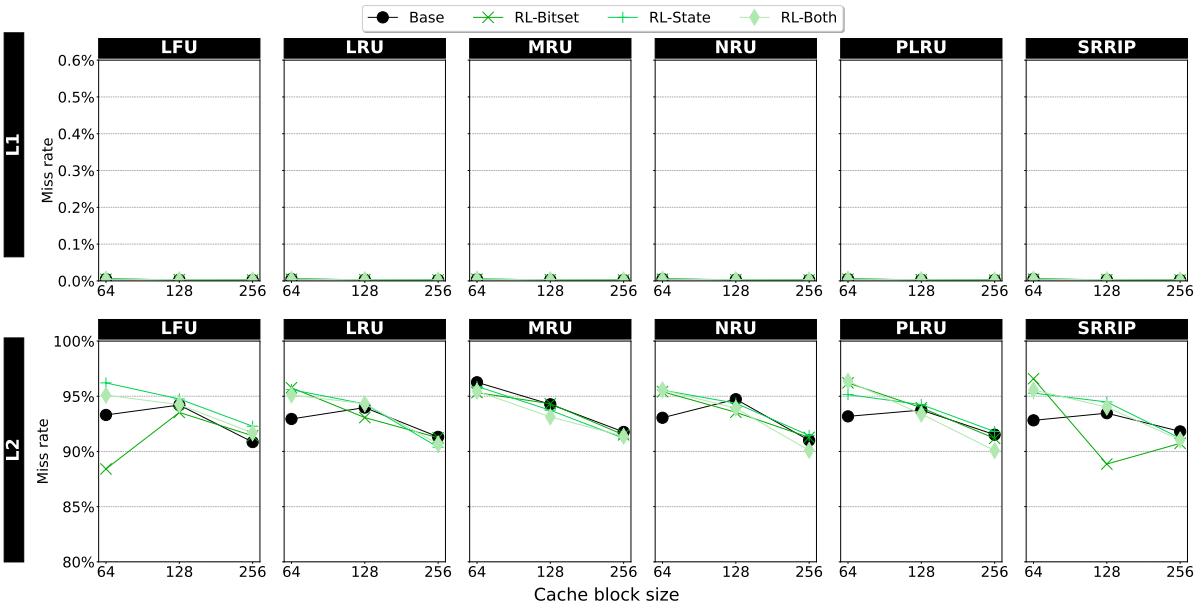
RL-Both strategy with this policy shows a rate of 5.56%, which is 0.55 percentage points less than the baseline. For the L2 cache, unlike in L1, the miss rate increases when we employ bigger cache blocks with our strategies. We observe that our strategies can improve MRU, regardless of the block size. This is also the case for SRRIP, however, in a less significant manner.

Figure 7.14 – L1 cache miss rate vs. block size - LU



Source: Elaborated by the author

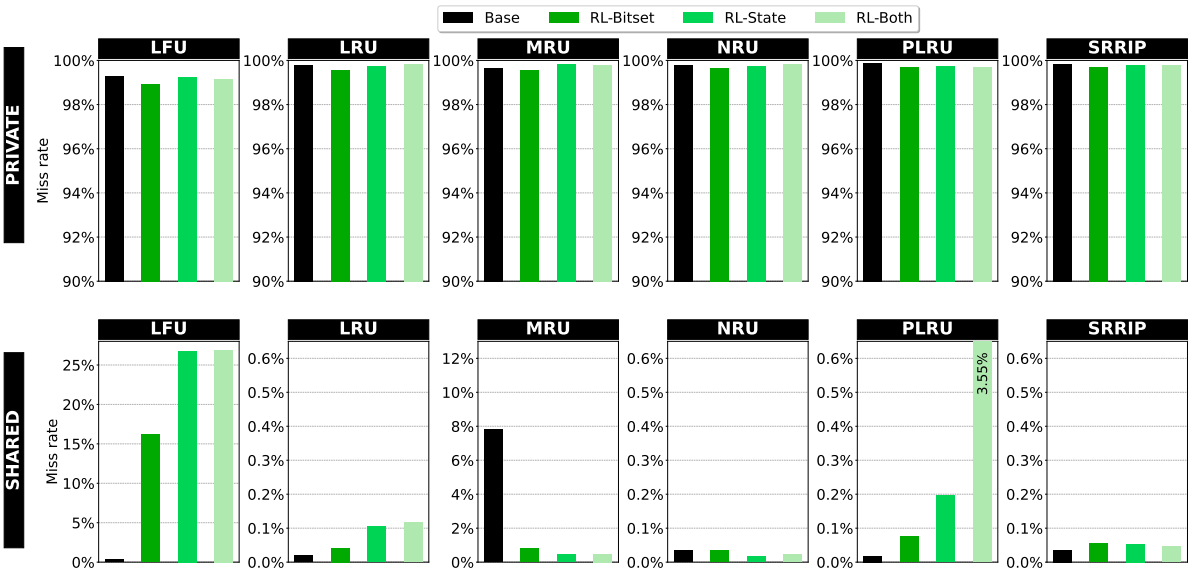
Figure 7.15 – L1 cache miss rate vs. block size - TSP



Source: Elaborated by the author

Figure 7.15 shows results from the TSP kernel. Directly to the L2 cache, we observe that generally speaking, the bigger the block size, the lower the miss rate for our approaches. Exceptions are the LFU and SRRIP policies with the RL-Bitset approach, for blocks with 64 and 128 bits, respectively.

Figure 7.16 – L2 cache miss rate vs. sharing strategy - FAST

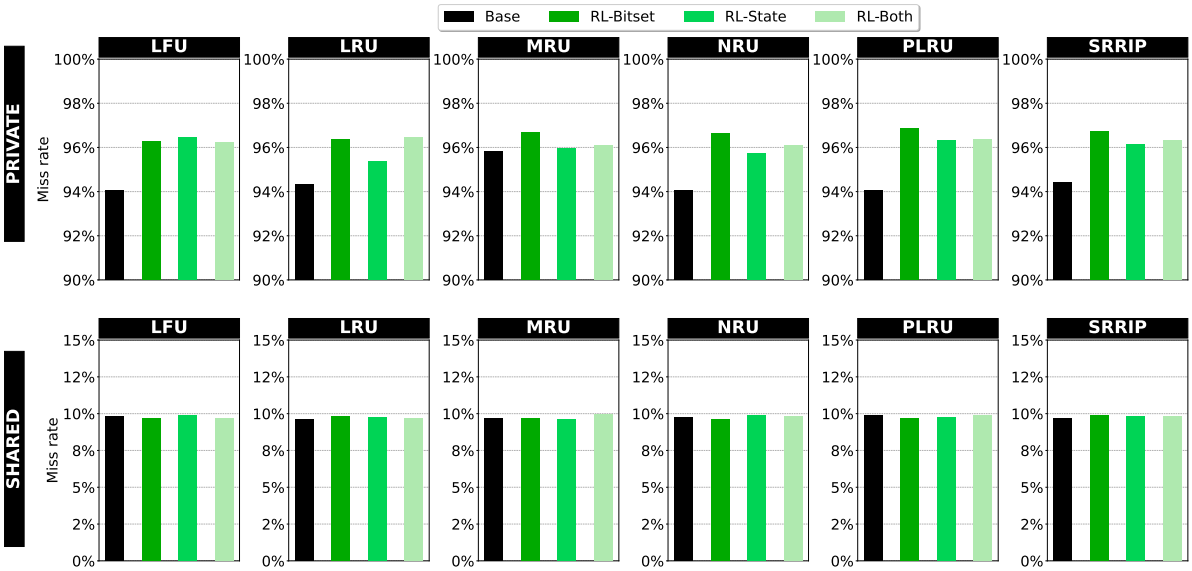


Source: Elaborated by the author

The last evaluation to be carried out concerns how the cache miss rate behaves when we change the sharing strategy. Thus, Figure 7.16 shows the results for FAST. We observe a common pattern, in which the cache miss rates have dropped significantly for shared strategy. Besides that, there are two behaviors we find interesting. With LFU, we observe the RL-based

approaches showing higher miss rates than the baseline, with shared caches. This is also the behavior of PLRU, however, its rates are much lower than the ones perceived in LFU. Conversely, the RL-based strategies with MRU are better than the baseline.

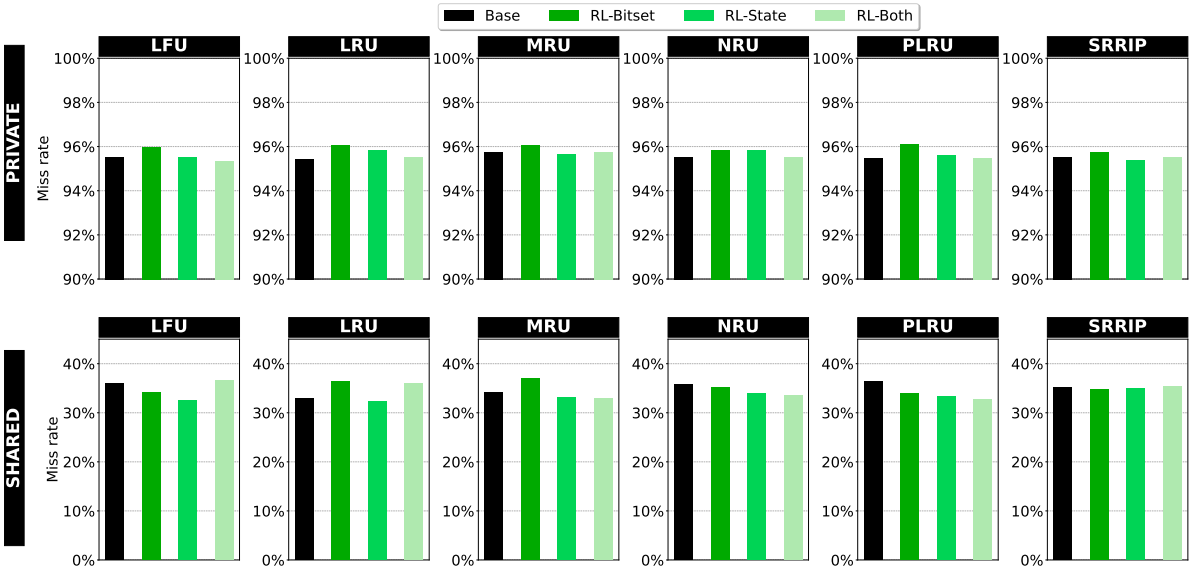
Figure 7.17 – L2 cache miss rate vs. sharing strategy - FN



Source: Elaborated by the author

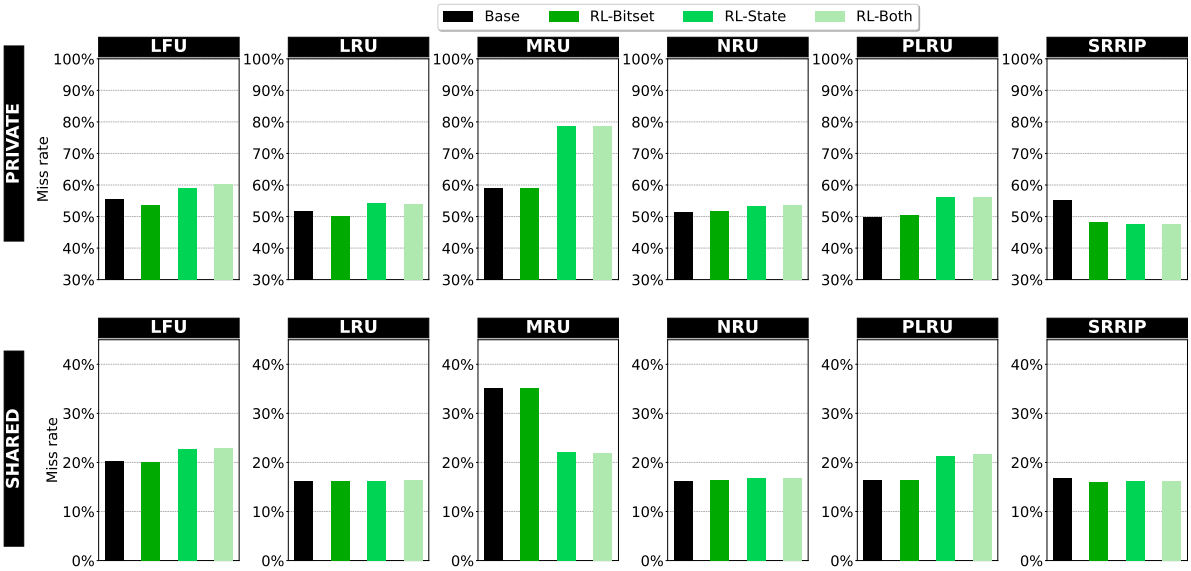
The results for FN show different behaviors between private and shared caches. We perceived from Figure 7.17 that in private caches, our RL-based strategies are worse than the baseline in all cases. However, they can at least compete with the respective unmodified policies in shared caches. For GF, whose results are below in Figure 7.18, there are more different patterns. For instance, with LFU, the pattern in the private is the opposite of the shared cache.

Figure 7.18 – L2 cache miss rate vs. sharing strategy - GF



Source: Elaborated by the author

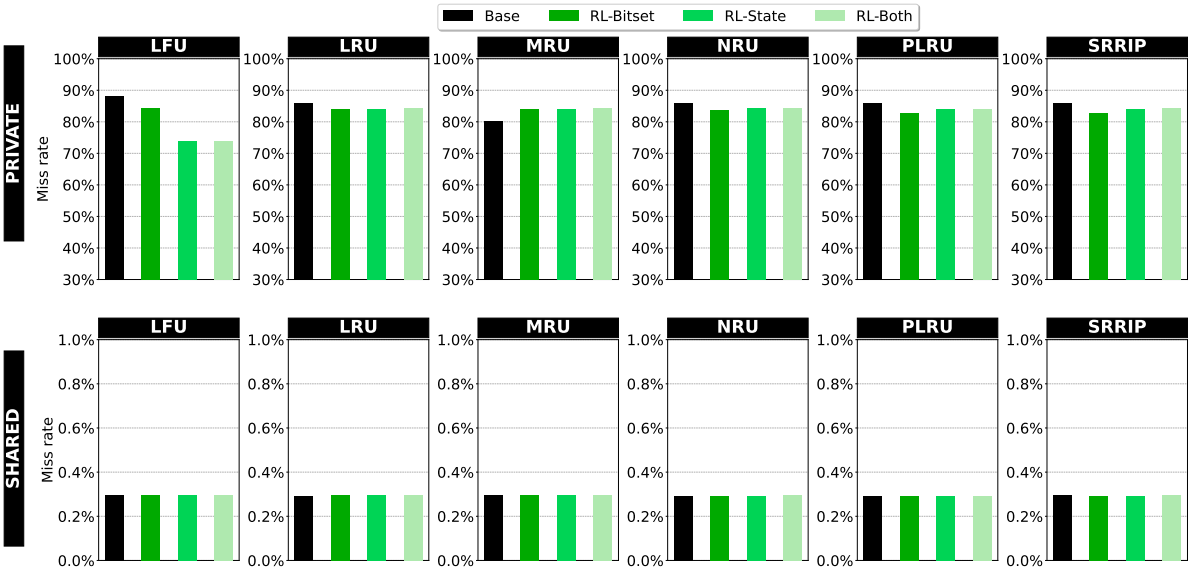
Figure 7.19 – L2 cache miss rate vs. sharing strategy - IS



Source: Elaborated by the author

Figure 7.19 shows IS results from the sharing strategy perspective. We observe the opposite behavior between shared and private caches with the MRU policies. That is, for this kernel, the unmodified policy and the one with the RL-Bitset are the best for shared caches, and, conversely, are the worst for private ones. The other policies follow pretty much the same behavior regardless of the sharing strategy.

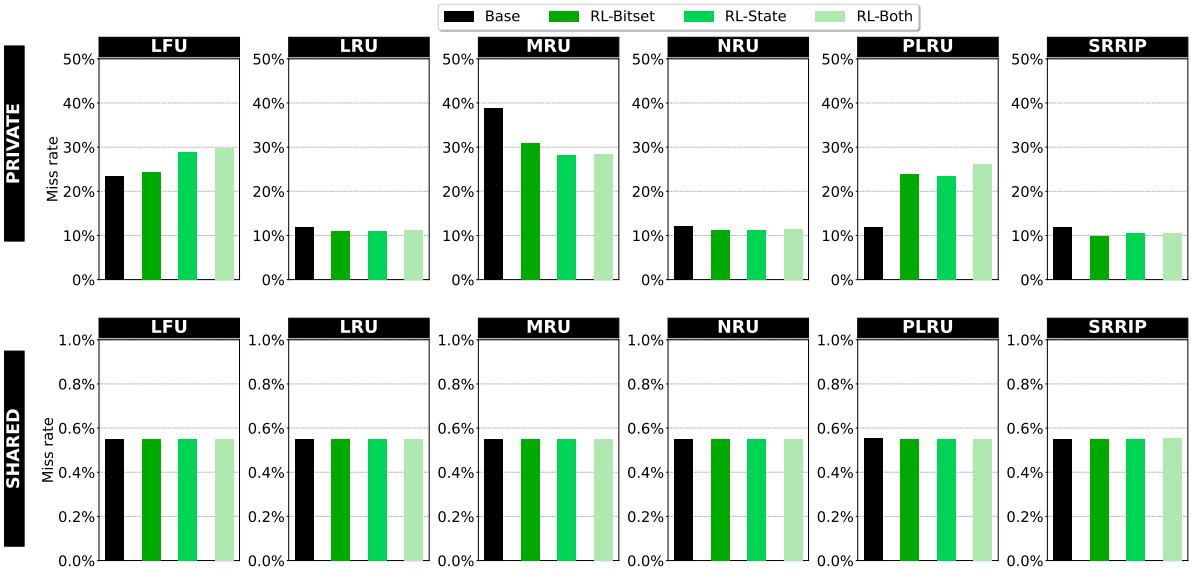
Figure 7.20 – L2 cache miss rate vs. sharing strategy - KM



Source: Elaborated by the author

The L2 cache miss rates for KM with shared caches are much lower than in the private ones. This is the most relevant behavior in this kernel, observable from Figure 7.20. Nonetheless, there is some space for improvements with private caches. For instance, our strategies show much lower miss rates than the baseline for LFU.

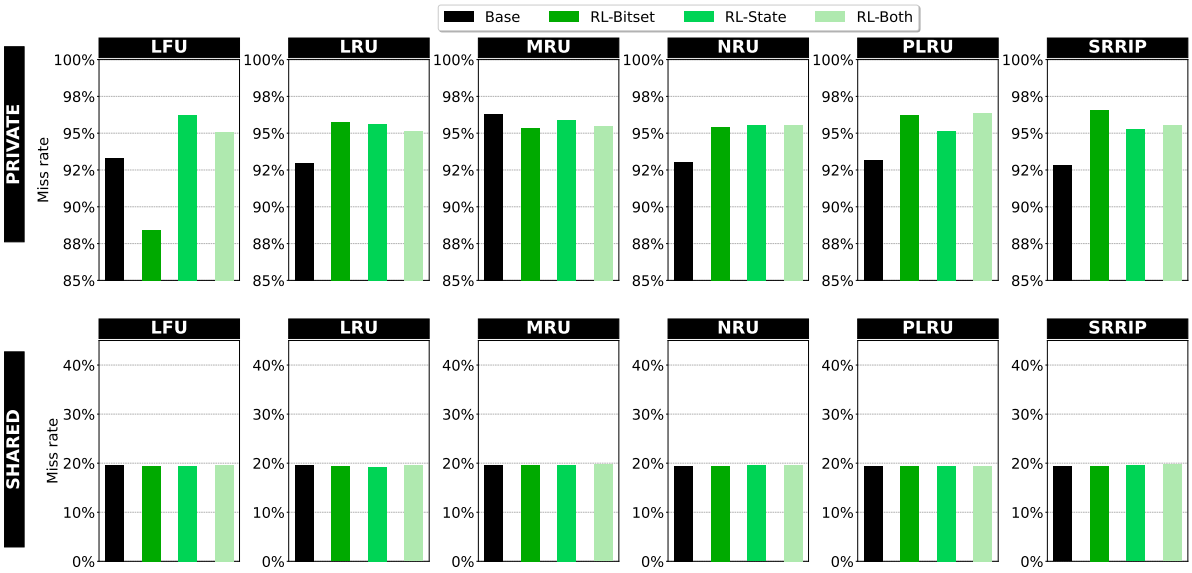
Figure 7.21 – L2 cache miss rate vs. sharing strategy - LU



Source: Elaborated by the author

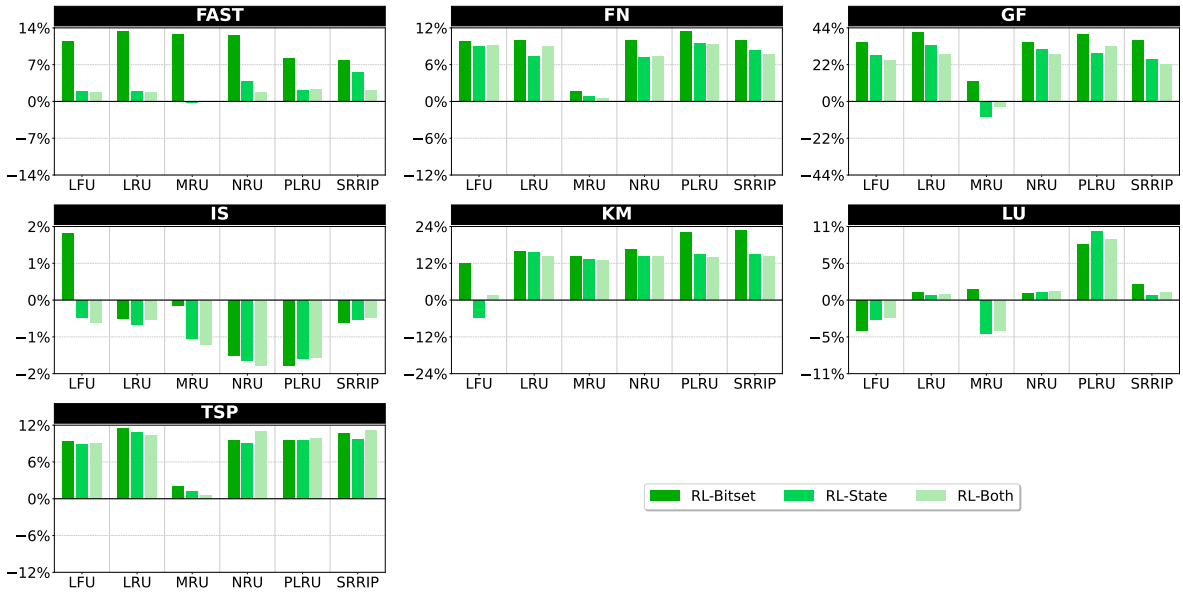
Figure 7.21 and Figure 7.22 show the results for LU and TSP, respectively. For both kernels, the reduction in the miss rate when using shared caches led to a standardization of rates, when comparing the strategies employed. Thus, we assume that for these two kernels, our strategies are better employed in the MRU policy for private caches. Furthermore, the same applies to LFU in TSP, specifically for the RL-Bitset approach, which is the best in private caches.

Figure 7.22 – L2 cache miss rate vs. sharing strategy - TSP



Source: Elaborated by the author

Figure 7.23 – L1 cache miss rate decrease



Source: Elaborated by the author

Figure 7.23 shows the overall comparison between the strategies with regard to the decreases in L1 cache miss rate. We first observe that our RL-based strategies are better than the unmodified policies in most cases. More precisely, with FAST, the RL-Bitset shows up to a 12.91% decrease in the miss rate with the LRU policy.

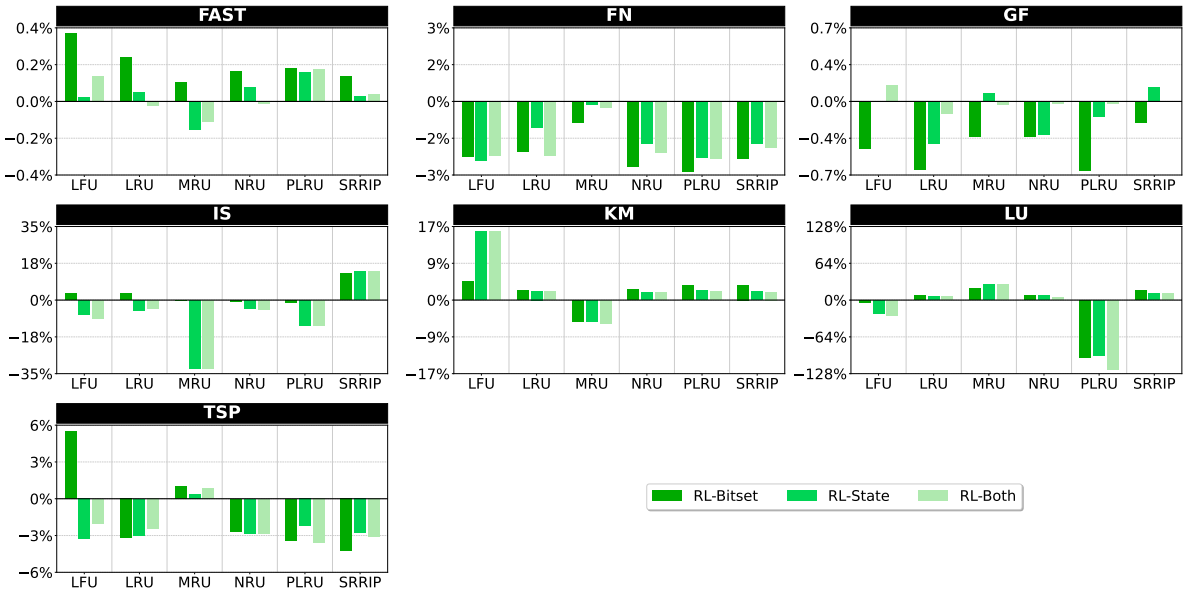
For FN, GF and TSP, the MRU policy is the one in which our approaches present fewer improvements, sometimes presenting higher miss rates. The highest improvement in the L1 miss rate for FN is 11.69% with the PLRU/RL-Bitset strategy. For GF, the best is 41.2%, also with the RL-Bitset approach, however in the LRU policy. Finally, an 11.73% reduction in the miss rate is perceived in TSP with the LRU/RL-Bitset strategy.

Despite the short differences, we also observe that our strategies reduce the miss rates for KM, LFU excluded with the RL-State. The best L1 miss rate improvement for this kernel is 22.88% with the SRRIP policy using the RL-Bitset technique. For the LU kernel, we highlight the PLRU policy, in which our strategies reduce the miss rates by a more significant margin than in the other policies. For instance, for the PLRU/RL-State approach in this kernel, there is a reduction of 10.18% in the miss rate.

When compared to other kernels, IS presents different behaviors. For this kernel, only the RL-Bitset strategy presents some improvement – the miss rate with LFU and the RL-Bitset is reduced by 2.07%. The other policies cannot be improved by our RL-based strategies.

An overview of the decrease in the L2 cache miss rates is presented in Figure 7.24, on the next page. The rate variations for FAST at this cache level are, again, too small in the L2 cache, as well as for GF. In spite of this, the best miss reduction for FAST is 0.37% with the RL-Bitset strategy and the LFU policy. For GF, we see negative results for LRU and PLRU with the same strategy, which is 0.68% worse than the baseline.

Figure 7.24 – L2 cache miss rate decrease



Source: Elaborated by the author

Still regarding Figure 7.24, we observe that FN, GF, and TSP are the kernels that show almost no miss rate reductions in the L2 cache, regardless of which of our strategies are employed. This follows the pattern we already discussed before, in which the improvements in L1 cache miss rates lead to fewer accesses in the L2 cache, thus increasing the rate at this latter level. Even so, the RL-State approach shows the best L2 miss rate reductions for the SRRIP policy when GF runs (0.15%). Moreover, with TSP, the RL-Bitset can improve the LFU policy by reducing 5.24% from the perceived L2 miss rate.

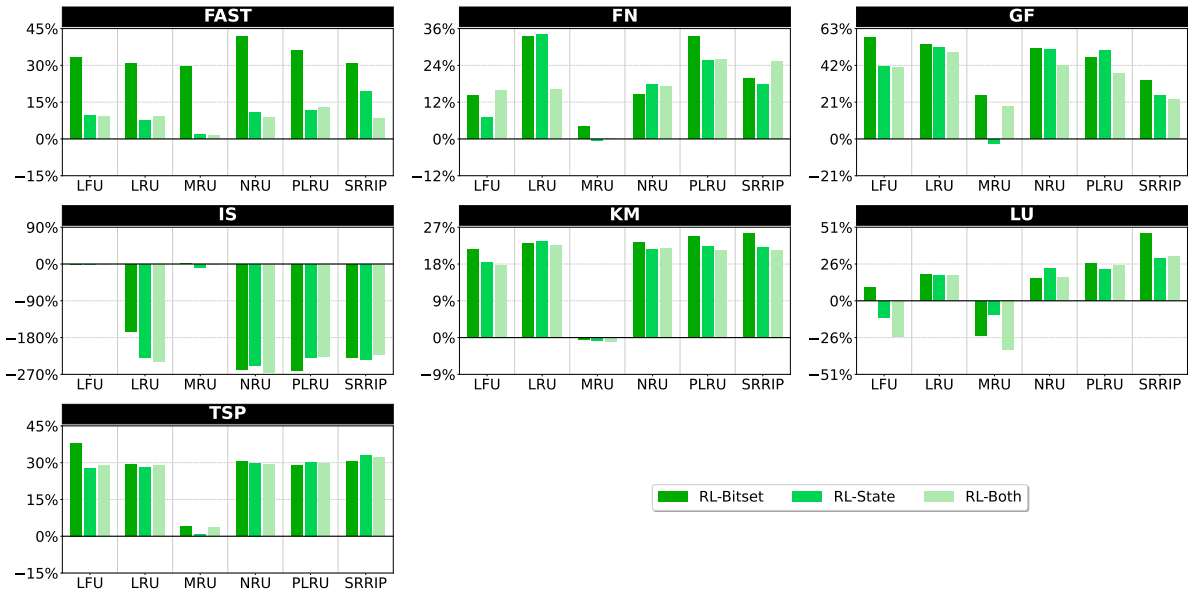
For IS, we highlight the LFU, LRU, and SRRIP policies, in which our RL-Bitset approach achieves better results than the baseline. More specifically, for SRRIP, all the RL-based approaches are better than the original SRRIP. RL-Both shows the best result for it, which is a reduction of 13.63% in the cache miss rate.

With the KM kernel, the most prone variations are perceived with LFU. RL-State and RL-Both show around 16.2% of miss rate reductions over the baseline with this policy. MRU is the policy in which we could not achieve any improvement for this kernel. On the other hand, our techniques reduced the L2 miss rates with MRU for the LU kernel. For instance, the MRU/RL-State approach reduced this miss rate by 27.30% in comparison to the unmodified MRU version.

7.1.2 Coherence operations

We gathered the number of invalidates from the RL-based approaches, and depicted these values in Figure 7.25, on the next page. FAST, FN, GF, KM, and TSP present substantial improvements when our RL-based techniques are employed. It means that our strategies chose

Figure 7.25 – Coherence invalidates decrease



Source: Elaborated by the author

better which blocks to keep in the cache, thus reducing the need to invalidate blocks that would be in other caches. Nonetheless, with the MRU policy, we cannot find such improvements.

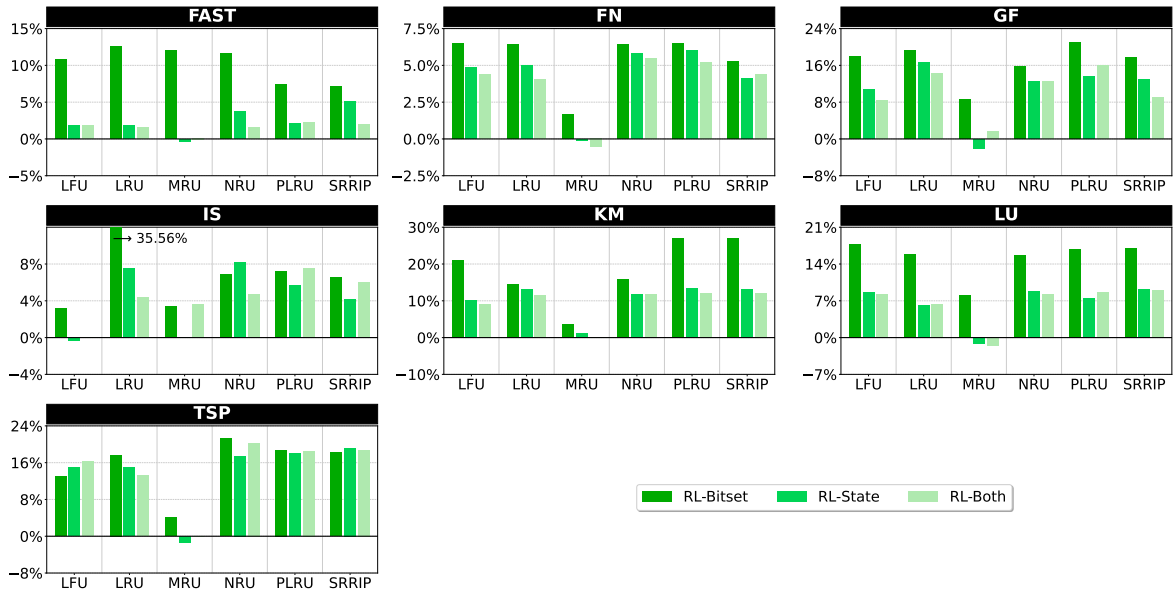
More precisely, RL-Bitset is the best for FAST regardless of the policy it is employed in. In this kernel, this approach can improve the NRU policy so as it performs 41.85% less invalidates. For FN, the greatest result is a decrease of 34.01% with the LRU/RL-State. GF is the kernel with the highest decreases, achieving up to 57.77% less invalidates than the baseline, with the LRU policy using the RL-Bitset strategy. The RL-Bitset strategy is also the best for KM and TSP. We find 27.16% fewer invalidates with SRRIP and 37.96% with LFU for this approach and these kernels, respectively. We also observe some improvements in LU. There are up to 47.03% fewer invalidates in this kernel, when the SRRIP is used with the RL-Bitset strategy.

Next, Figure 7.26 presents the improvements we find with regard to coherence write-backs.

Most of the kernels present good results, although they are relatively smaller than the ones from invalidates. The best results for each kernel, which we detail next, are observed with the RL-Bitset approach. First, for FAST with the LRU policy, we obtain 12.57% fewer write-backs than the unmodified policy. The best case for FN is 6.52% with the PLRU, as well as for GF, which shows 20.98% fewer operations than the unmodified PLRU. IS presents the highest decrease among all kernels, which is 35.56% using the LRU with the RL-Bitset approach.

Still using the RL-Bitset approach, the coherence write-backs are reduced by 27.16% using the SRRIP policy and the KM kernel. For LU, the LFU policy shows the highest decrease, which is 17.87% over the baseline. Finally, the TSP kernel, when run in the architecture with the NRU policy, shows 21.24% fewer write-backs than the unmodified version.

Figure 7.26 – Coherence write-backs decrease

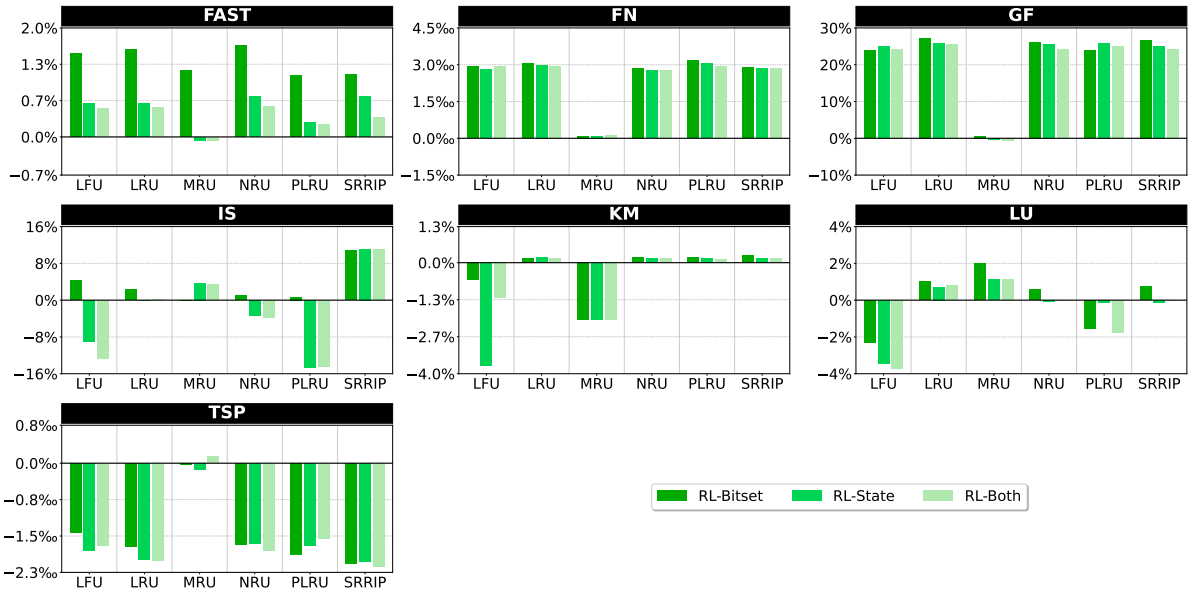


Source: Elaborated by the author

7.1.3 Instructions per cycle and energy consumption

To assess the performance of our RL-based approaches, we depict their perceived IPC next, in Figure 7.27.

Figure 7.27 – IPC increase

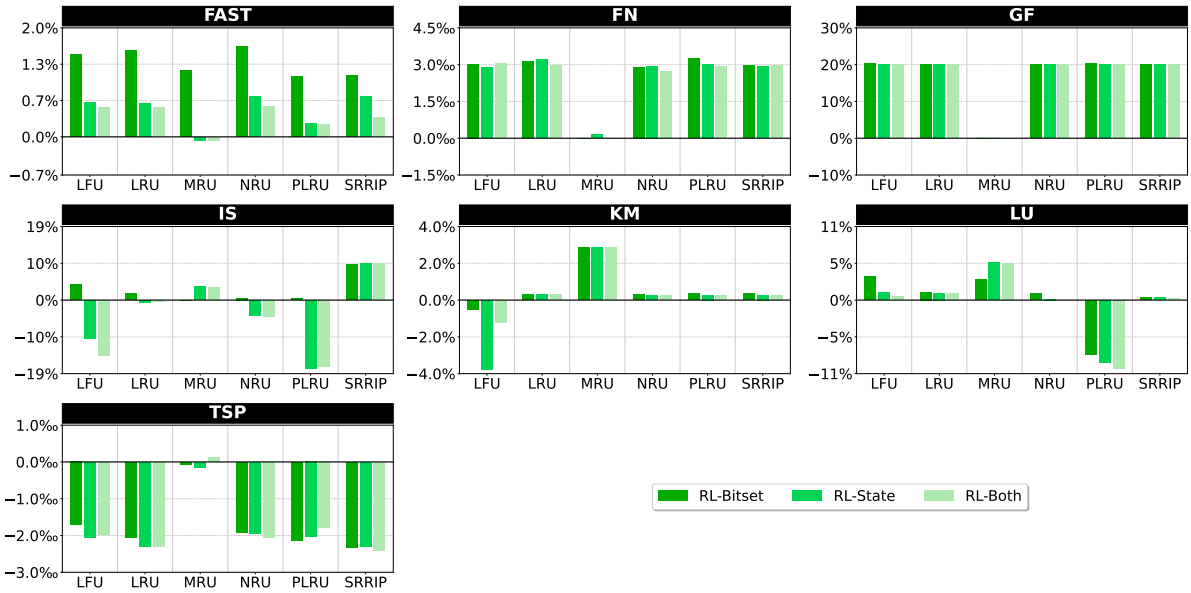


Source: Elaborated by the author

Again, FN and TSP show less significant variations in IPC than other kernels when our strategies are employed. Nevertheless, the IPC for these kernels follows the cycles spent behavior, depicted in Figure 7.28.

FAST and GF kernels show good improvements over the baseline with all RL-based

Figure 7.28 – Cycle decrease



Source: Elaborated by the author

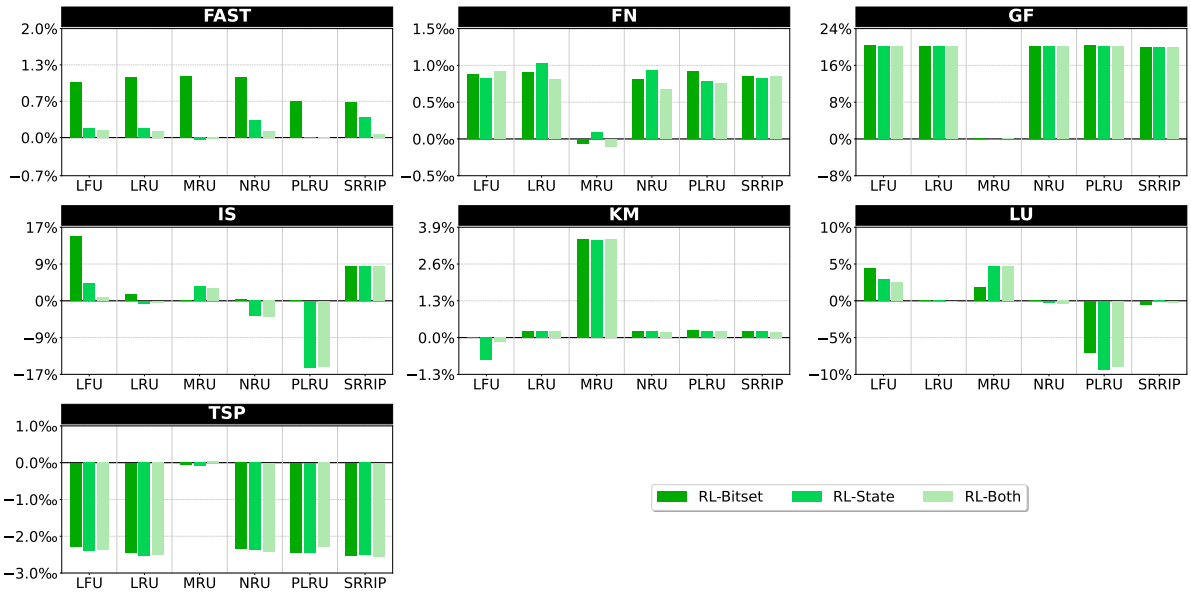
strategies, MRU policy excluded. The best result for FAST is perceived with the NRU policy using the RL-Bitset approach, which is 1.68% more IPC than the baseline. For GF, the LRU policy is the one in which our RL-Bitset strategy can improve the IPC more, achieving up to a 27.33% increase. These IPC improvements also follow the decrease in the number of cycles. Likewise, the fewer cycles spent comes from the improvements in the miss rates and coherence operations for these kernels. In addition, FAST and GF deal with the same input type and follow the same parallel pattern, however, they are different though. GF performs fewer memory accesses and takes more time to execute than FAST (SOUZA et al., 2017). Thus, the impact of our strategies in GF is greater, which results in higher IPC improvements.

With respect to the IS kernel, the best results are obtained with SRRIP, due to the cycles spent with each strategy. This policy shows up to 11.22% more IPC than the baseline when the RL-Both strategy is employed. However, we also find some IPC decreases in this kernel, using the PLRU policy, which is around 14.8%.

As in TSP, KM shows more pronounced decreases than increases. For instance, the IPC when using LFU is decreased by 3.7% with the RL-State technique, in relation to the baseline. And differently from other kernels, with LU, the MRU is the one that shows the best improvements in IPC. We find 1.9% more IPC with this policy using the RL-Bitset approach. However, as in KM, we cannot improve the LFU policy using our RL-based strategies. Moreover, we observe the inverse behavior using the MRU policy, as in the Coherence- and Bitset-based approaches.

Figure 7.29 shows the decrease in energy consumption that our strategies can achieve. Repeatedly, FAST, FN and TSP do not show substantial changes in energy consumed, as its IPC and cache miss results also do not vary much when we vary the way-replacement strategy.

Figure 7.29 – Energy consumption decrease



Source: Elaborated by the author

We already mentioned that the greater the IPC, the lower the energy consumption. For these kernels, as well as for GF and IS, the energy results show a behavior similar to the IPC.

FAST shows up to 1.12% less energy consumed than the baseline, using the RL-Bitset approach. For FN, the best result is with RL-State applied to LRU, which is 0.1% lower than the unmodified policy. GF shows almost equal results for all policies – MRU excluded – and strategies, which is around 20.1%. Recall that the MRU energy consumption does not vary much due to the IPC not varying too. The best result for IS is observed in the LFU/RL-Bitset configuration, which is 15.09% less energy consumed. We also highlight for IS the SRRIP policy, which can be improved by all of our RL-based strategies. Finally, TSP shows almost no improvements in energy consumption, being worse by a margin of approximately 0.23% than the baseline.

With regard to KM and LU, the best results are observed with the MRU policy. We find that, for the former kernel and this policy, the IPC decrease is not sufficient to impact energy consumption negatively. With the LU kernel, this same analysis applies to LFU. Generally speaking the best improvement for KM using MRU is 3.48%, using the RL-Bitset approach. For LU, the best result is 4.65% less energy consumed with the MRU/RL-State strategy.

7.2 Hardware overhead

Later in Section 4.2.1, we presented the Table 4.2 that shows the complexity comparison of traditional way-replacement algorithms. Furthermore, in this chapter, as well as in Chapters 5 and 6, we briefly discussed the overheads that each of our proposals brings to a single cache. In this section, we propose to summarize that information, comparing our three approaches in terms of storage complexity, and showing by example these overheads. Thus, Table 7.1 shows the additional bits required to implement our strategies. The number of sets and the associativity of the cache is denoted by N and K, respectively. T is the number of bits required to the *tag*, present in each cache block.

Table 7.1 – Additional bits required by the strategies

Static	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
2	$2N$	0	$2KN(T+1)$	$2KN(T+2)$	$2KN(T+3)$

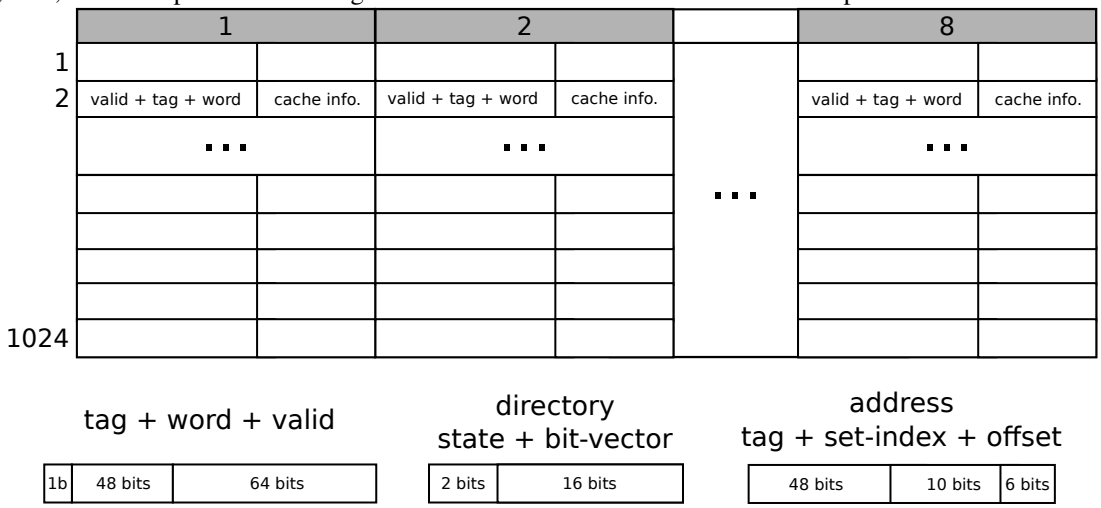
Source: Elaborated by the author

Before explaining these values, recall that we use the directory-based protocol in our experiments. Each directory entry contains the bit-vector with a size equal to the total number of cores – 16 bits in our simulated architecture. Besides that, it has two bits that represent the coherence state of the respective cache block. Thus, in our experiments, each directory entry has at least 18 bits.

Back to Table 7.1, we see that the RL-based strategies have higher costs than the others. The static approach requires only two additional bits in the cache to store the state that the strategy uses in every set. The dynamic strategy also needs to know the MRU-State to compare, which is dynamically provided per set; thus, two bits per set. The RL-based strategies use the eviction history table. There is one table for each cache set and each table contains a number of entries equal to two times the associativity. Thus, the strategy requires a total of $2KN$ entries in the cache. Each entry contains the tag T, which we use to make them unique. Besides that, the entries also contain bits to track the direction of the threshold (1 bit) and the state a cache block was before being evicted (2 bits). The RL-Both strategy, which uses both information, has then 3 bits per entry, besides the tag.

Figure 7.30 – Simple cache example

A simple 64kB cache considering 64-bit words. This example is a 8-way set-associative cache, with 64-bit block size; thus, one word per block. The figure also details the cache entries and the composition of the cache address.



Source: Elaborated by the author

For a broader understanding, consider Figure 7.30. It shows a 64kB, 8-way set-associative cache, with 64-bit blocks. This cache supports 64kB $\div$ 64b $=$ 8192 words; thus, since the associativity is 8, it has 8192 $\div$ 8 $=$ 1024 sets. Each cache entry contains the valid bit (1b), the word (64b), and the tag. To find the tag size, we first verify the composition of the address.

Sniper's addressing mechanism adopts addresses with four parts. First, a bit/byte offset, whose size is given by $bi = log_2(word_size)$; thus, in our example, this offset occupies 6 bits. Second, it uses a block offset for cache architectures with block size greater than the word size. The example shows a cache with single word blocks, thus there is no need to have the block offset. If the architecture has, for instance, 256-bit blocks, the addressing mechanism would require 2 bits, which is the result from $bo = log_2(block_size \div word_size)$. Third, the cache mapping algorithm needs to use the set index. The example cache requires $i = log_2(number_of_sets)$, which is equal to 10 bits. Finally, the tag uses the rest of the address; thus it size follows the equation $T = adddress_size - (bi + bo + i)$. Since Sniper uses 64-bit addresses, the tag size in the example cache is 48 bits.

With that in mind, we can calculate the total theoretical size of the example cache. Each entry has 113 bits (valid bit, word, and tag), besides 18 bits for directory-based coherence purposes, totaling 131 bits. With 8192 entries, the total size of the example 64kB cache is 1 073 152 bits, or **131kB**. That is, the cache requires additional **67kB**.

Concerning our strategies, we observe the RL-Bitset showing most of the best results. Thus, we evaluate the total theoretical size of the example cache, but now using this approach. This approach adds the eviction history table for each cache set. We already know that each eviction history table has $2K$ entries, which is equal to 16 in our example. Each table entry has $T + 1$ bits for this way-replacement strategy, resulting in 49 bits per entry. Thus, each table uses $16 \times 49 = 784$ bits. The example cache has 1024 sets (the value of N), which results in 802 816 bits or **98kB**, for the RL-Bitset strategy.

To summarizing, the example cache in Figure 7.30 already requires an additional **67kB**. Additionally, it requires **98kB** to implement the RL-Bitset strategy, as we demonstrated. Thus, summing the data portion of the cache, the total size is **229kB**. The RL-Bitset strategy causes an increase of 74.81% in the total size of the cache. In other words, this strategy represents 42.79% of the cache.

Table 7.2 – Total size of the cache using the strategies

The values are calculated considering a 64kB cache with 64-bit words. The cache is 8-way set-associative. The initial overhead of this cache varies according to the block size.

	Block size	Static	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
Init. Overhead	64 bits	$67kB$					
Add. Overhead		$\approx 0kB(2b)$	$0.25kB$	$0kB$	$98kB$	$100kB$	$102kB$
Total size		$\approx 131kB$	$131.25kB$	$131kB$	$229kB$	$231kB$	$233kB$
% increase		$\approx 0.00\%$	0.19%	0.00%	74.81%	76.34%	77.86%
% of the cache		$\approx 0.00\%$	0.19%	0.00%	42.79%	43.29%	43.78%
Init. Overhead	128 bits	$34kB$					
Add. Overhead		$\approx 0kB(2b)$	$0.13kB$	$0kB$	$49kB$	$50kB$	$51kB$
Total size		$\approx 97.5kB$	$97.63kB$	$98kB$	$146.5kB$	$147.5kB$	$148.5kB$
% increase		$\approx 0.00\%$	0.13%	0.00%	50.26%	51.28%	52.31%
% of the cache		$\approx 0.00\%$	0.13%	0.00%	33.45%	33.90%	34.34%
Init. Overhead	256 bits	$17kB$					
Add. Overhead		$\approx 0kB(2b)$	$0.06kB$	$0kB$	$24.5kB$	$25kB$	$25.5kB$
Total size		$\approx 80.75kB$	$80.81kB$	$80.75kB$	$105.25kB$	$105.75kB$	$106.25kB$
% increase		$\approx 0.00\%$	0.08%	0.00%	30.34%	30.96%	31.58%
% of the cache		$\approx 0.00\%$	0.08%	0.00%	23.28%	23.64%	24.00%

Source: Elaborated by the author

Table 7.2 shows the total sizes of the caches for each of our strategies. We highlight that this value can be reduced accordingly to the cache configuration. For instance, if we increase the size of the block, the cache will have fewer sets, reducing the number of eviction history tables. Besides that, it would also reduce the tag size, because, with higher block sizes, the address has fewer bits to spend with the tag. With that in mind, we can use the RL-Bitset with 256-bit blocks at the cost of **24.5kB**, or 30.34% additional storage. It represents 23.28% of the total size of the cache. Notwithstanding, our results show some improvements in the metrics when the block size is increased. Finally, the strategies which do not use Reinforcement Learning require much less hardware overhead than the RL-based ones, and should also be considered.

7.3 Concluding remarks

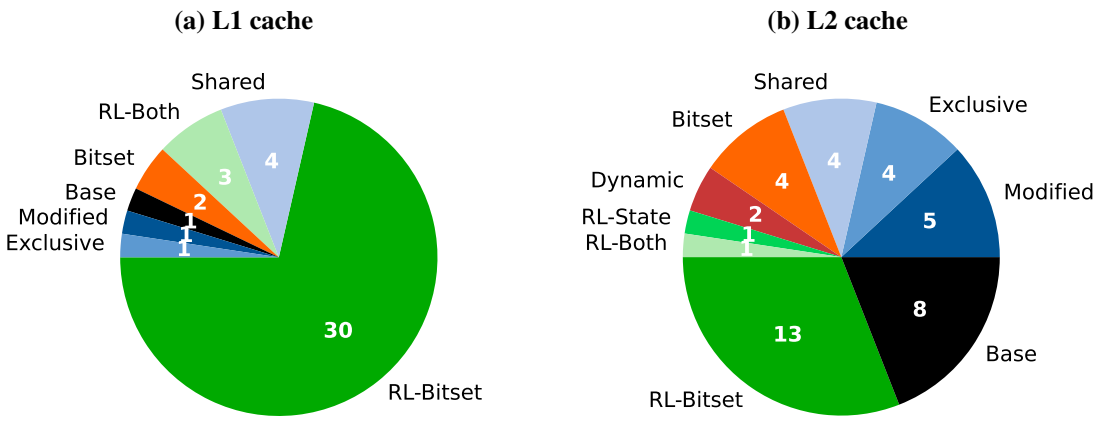
In this chapter, we presented our way-replacement strategy using the Reinforcement Learning support and discussed the experimental results that we obtained with it. We employed the Reinforcement Learning by letting the choice of the coherence state or thresholds follow the hits and misses perceived during program execution. We recall that past work also used some Machine Learning techniques to improve way-replacement policies. However, Table 3.1 shows that none of them took cache coherence into account.

To provide a more accurate understanding of our contributions, we summarize in this section the most important findings that we reached out in our analysis, concerning also the non-RL proposals from Chapters 5 and 6. First, regarding the cache miss rates, we observed that the associativity and cache block sizes are more related to the kernel characteristics than our approaches. Notwithstanding, the kernel behavior also impacts the coherence protocol, e.g., changing the predominant coherence state in the cache.

Concerning the sharing state, we saw that shared caches presented much better miss rates than private ones, which we expected. However, the downside is that, in theory, larger caches have higher latency and consume more power. Thus, we observe a negative compensation in terms of IPC and energy consumption. Furthermore, we assume that we must adapt CAP Bench kernels to optimize the cache usage for a fairer comparison.

Generally speaking, we saw fit to perform the overall analysis using 64-bit blocks, and 8-way set associative caches, with the L2 cache being private. It is worth noting that these are commonly used configurations in processors of the market.

Figure 7.31 – Distribution of the best cache miss results



Source: Elaborated by the author

Figure 7.31 shows the distribution of the best cache miss results we gathered. This distribution counts the strategy which presents the lowest miss rates for each kernel and policy pair. The strategy which exhibited the best results in cache miss rates was the RL-Bitset, by a good margin, regardless of the cache level. Preserving the cache blocks that are most shared was also a good option for some kernels and policies. Remember that a block with more than one core that shares it will at some point be in the Shared state. Thus, the approach that maintains blocks with this coherence state also was the best in some cases.

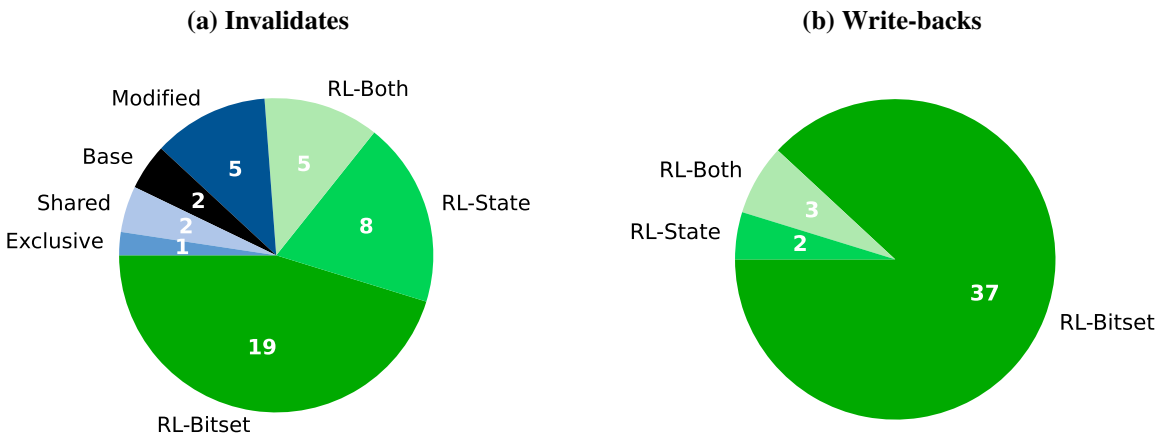
Table 7.3 – Summary of cache miss rate results

L1 cache miss rate								
	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
MIN	-22.10%	-18.57%	-9.68%	-21.88%	-213.35%	-4.47%	-8.98%	-4.53%
MAX	10.82%	7.97%	24.84%	12.27%	22.94%	41.20%	33.58%	32.64%
AVG	0.12%	0.01%	1.55%	-0.44%	-4.84%	11.55%	7.43%	7.35%
L2 cache miss rate								
	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
MIN	-33.84%	-1.74%	-168.57%	-21.31%	-152.22%	-100.50%	-96.76%	-120.83%
MAX	69.04%	11.53%	15.23%	68.07%	36.57%	20.39%	27.30%	26.86%
AVG	0.02%	0.68%	-4.44%	1.36%	-3.20%	-1.04%	-2.83%	-3.64%

Source: Elaborated by the author

More precisely, Table 7.3 shows the maximum and minimum cache miss rates that we gathered in our experiments. Additionally, it also presents an average value of all policies and applications. We highlight the maximum values in the L1 cache for RL-based approaches, which achieve up to 41.20% of cache miss rate reduction. Despite that, our strategies must be wisely used since they can also result in very high miss rates. Nevertheless, the L2 cache was used less in our experiments than the L1. Thus, our strategies may scale well with higher workloads that require more access to the L2 cache.

Figure 7.32 – Distribution of the best coherence operations results



Source: Elaborated by the author

Concerning coherence operations, Figure 7.32 shows the distribution of the best results. The Reinforcement Learning approach using the sharer's count bitset as input data also appears as the best. It presented the lowest number of invalidates for 19 cases. About the write-backs, it appears in almost all the best results for kernels and policies (37 of 42). Furthermore, as we observed from cache miss results, unmodified policies were the best only once in coherence operations.

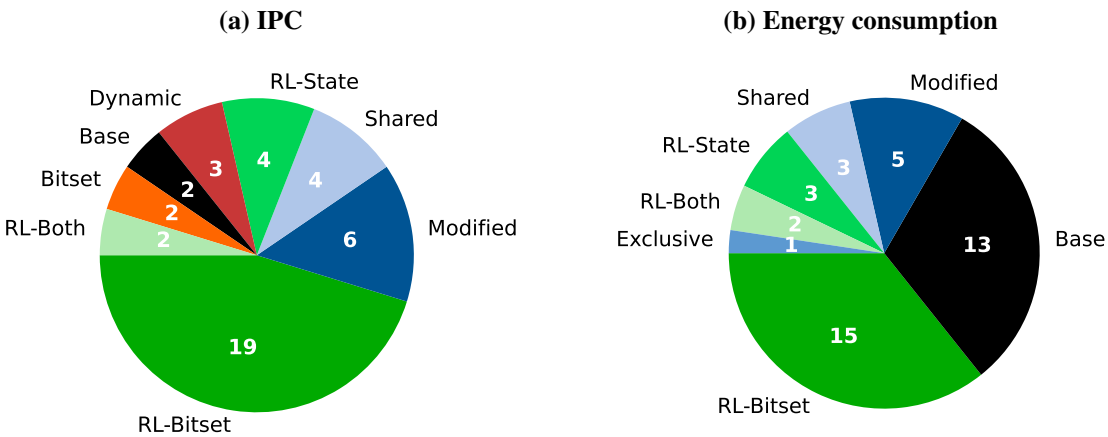
Table 7.4 – Summary of coherence operations results

Coherence operations – Invalidates								
	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
MIN	-28.30%	-13.94%	-278.25%	-30.03%	-25.46%	-261.55%	-247.50%	-265.95%
MAX	38.84%	28.01%	29.38%	34.69%	78.67%	57.77%	52.06%	49.49%
AVG	4.31%	1.54%	-2.82%	1.31%	1.88%	1.12%	-5.72%	-7.34%
Coherence operations – Write-backs								
	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
MIN	-9.66%	-4.07%	-4.83%	-2.74%	-9.50%	1.64%	-2.03%	-1.52%
MAX	12.37%	6.91%	11.67%	3.51%	5.69%	35.56%	19.23%	20.23%
AVG	0.55%	0.48%	1.61%	0.49%	0.06%	13.12%	7.51%	7.21%

Source: Elaborated by the author

From Table 7.4 we observe the maximum and minimum gains we perceived and the average results. Although the RL-Bitset shows high reductions for both invalidates and write-backs, the best way to reduce invalidates is using the more straightforward Bitset approach. The average decrease in write-backs for RL-Bitset is 13.12%. This number means that many write operations in lower levels of memory would occur if our strategy were not employed, reducing the performance. Furthermore, all strategies present positive average values, which means that they reduced undesirable write-backs.

Figure 7.33 – Distribution of the best IPC and energy consumption results



Source: Elaborated by the author

Figure 7.33 depicts the distribution of IPC and Energy consumption results. The good results we perceived in other metrics lead to similar results for IPC and Energy consumption. The RL-Bitset strategy was the one that appeared more times as the one which could improve performance and reduce the energy consumed.

Table 7.5 – Summary of performance results

Instructions per cycle								
	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
MIN	-20.43%	-4.06%	-44.75%	-9.36%	-28.39%	-2.18%	-14.85%	-14.53%
MAX	12.87%	9.19%	20.31%	8.73%	8.75%	27.33%	25.83%	25.65%
AVG	-0.53%	0.03%	-1.27%	-0.16%	-0.99%	3.70%	2.64%	2.45%
Energy consumption								
	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
MIN	-22.11%	-0.56%	-36.18%	-9.22%	-23.40%	-7.00%	-15.66%	-15.35%
MAX	11.67%	10.96%	14.87%	11.37%	10.17%	20.36%	20.12%	20.15%
AVG	-0.46%	0.61%	-1.25%	0.18%	-0.89%	3.19%	2.33%	2.24%

Source: Elaborated by the author

Table 7.5 shows the summary of energy consumption reductions and IPC gains. Once again, we observe the RL-based techniques with superior results than the others.

As a final remark, our intuitions pointed out in our thesis statement (Section 1.1) were confirmed. Although it cannot be generalized for any scenario and application, this thesis presented an interesting first set of results. For example, we could use the coherence sharing state of cache blocks and the number of cores sharing data at a cache level to carry out better eviction decisions. Furthermore, we modified six different way-replacement policies, and we employed Reinforcement Learning successfully on them. Thus, we achieve our goals, culminating in the contributions of the thesis.

8 CONCLUSIONS

Caches are critical components for the performance of modern processors. The way-replacement policy it implements is one of the critical factors for cache effectiveness. For instance, a policy that can predict the future and evict the right blocks will effectively reduce the memory access latency and improve overall performance. Unfortunately, it is impossible to implement Bélády's algorithm in current caches; thus, prior work has proposed several heuristics. Meanwhile, most modern processors are multicore, which leads to the cache coherence problem. Thus, these processors need a mechanism to ensure that data shared among cores are coherent.

With that in mind, among the extensive corpus of research, we could not find any replacement policy that considers information relative to the sharing state of a cache block. Thus, in this thesis, we have shown how the information from cache coherence protocols can help to improve the replacement problem. We used the sharing state or the number of cores that share a cache block to make better eviction decisions despite devising new temporal-based algorithms. To perform this, we selected six well-known replacement policies: LFU, LRU, MRU, PLRU, NRU, and SRRIP.

Using the Sniper simulator, we modified the replacement policies in different ways. First, we design a strategy considering the number of cores that share a cache block or the coherence state. Next, we extend this design to carry it out statically and dynamically. Then, we employed Reinforcement Learning on these techniques to improve previous strategies. Finally, we simulated parallel kernels from the CAP Benchmarks suite in Sniper, where we built a 16-core architecture, gathering some metrics.

Our experimental evaluations resulted in the following contributions:

Coherence state-based eviction strategy: the eviction decision is made based on the state of coherence. We found that this approach was good for some kernels, but not all. The application behavior impacts the coherence protocol. Thus, arbitrarily selecting a state might not produce the best results, however, this improved our metrics in some cases.

Sharer count-based eviction strategy: the cache keeps tracking how many cores use the data and use this information to decide whether to evict the block or not. This one uses a more direct strategy. If there are copies of the block on other caches (according to a threshold), we keep it in the cache. This approach was not as good as the state-based eviction.

Reinforcement learning-based eviction strategy: Reinforcement Learning observes a history, which contains evictions from the short past. The strategy learns from this history and decides whether or not to evict a cache block based on what it has learned. More precisely, the RL-Bitset approach considers the number of cores that share a cache block. Then, it learns and decides whether it wants to keep more or less shared blocks in the cache. This technique was the best in many of our results.

Overall, the proposed way-replacement policies could outperform their unmodified version in many cases or achieve comparable performance to those. In our analysis, we identified a correlation between the cache coherence information and the way-replacement strategy. Furthermore, we found that the sharing state and the number of cores that share a cache block are information that can be used to enhance the performance of cache way-replacement policies. For instance, the RL-Bitset approach reduced the L1 cache miss rate by an average of 11.50%, reaching up to 41.20% over the baseline. Likewise, Static/Modified and Dynamic approaches reached up to 69.04% and 68.07% less cache miss rate than the baseline in the L2.

These results also led to improvements in performance and energy consumption results. We could achieve up to 27.33% IPC gain over the baseline with the RL-Bitset approach. On average, this strategy improved IPC by 3.70%. This same approach achieved the most significant reduction concerning energy consumption, which was 20.36% over the baseline (3.19% on average).

As a final remark, our findings confirmed the hypothesis we state in Section 1.1. First, the coherence sharing state of cache blocks can be used as an input to the eviction decision in the way-replacement problem. Next, the number of cores sharing data in a cache level impacts the eviction decision. Finally, the well-known algorithms we used could be improved by our strategies and presented better eviction decisions than the original policies.

8.1 Future work

Research topics in the field of Computer Architecture have a vast design space to be explored. Therefore, one must narrow the goals and methodology so that the level of focus of the study increases. Thus, in this thesis, some decisions were taken to increase the focus, but they also pointed out methodological restrictions of the work. For instance, future research can explore several other architectural parameters of multicore processors. Likewise, they can go beyond our proposed scope, testing applications with different characteristics and workloads.

Even so, we believe that this thesis produced good results and opened up a wide range of insights and further research. It can be extended in several directions that concern way-replacement algorithms for multicore processors. Next, we highlight the possibilities that we find the most relevant:

- **Way-replacement strategies that are aware of global memory information.** Our solutions used information from coherence and the sharer's bitset locally, i.e., within cache sets. However, one future direction would be to use this information globally for all the cache. Thus, it would answer if the policy in a cache set can use the information from other sets.
- **Modify the size of the eviction history table.** As in many machine learning problems, there is the challenge of improving the space to store information. Our RL-based

approaches use the eviction history table with size $2\times$ the associativity. We did not test smaller sizes, nor bigger ones. Thus, we believe that future work should carry out a wide range of tests considering the size of the eviction history table and further optimizations.

- **Explore the use of our strategies in other memory environments.** Although this thesis focused on way-replacement algorithms for caches in multicore processors, other environments and architectures can also apply our proposals. For instance, the Operating System deals with a bigger memory address space than the physically available in a virtual memory scenario. Thus, a mechanism to deal with page replacement is necessary, and a policy to carry this out is employed. In future work, one can adapt our strategies to modify page replacement policies in this scenario. Likewise, Web caching is an even larger scenario, which follows the same principles.
- **Modify other architectural parameters.** Simulation is a technique that enables the configuration of different types of computer architecture with several architectural features. Thus, one can test the way-replacement strategies presented in this thesis in many scenarios. For instance, we could modify the number of cores the architecture has, or perhaps modify the size of the cache memories, and even configure and simulate a more complex interconnection system. Thus, it would be possible to assess the impacts of these new parameters on the results when used in conjunction with our proposals.
- **Conduct tests using other workloads.** In this work, we used kernels from the CAP Bench suite that covered a set of characteristics suitable for our experiments. Still, it is interesting to explore a different methodological scope, including applications that consider other characteristics. Such experiments can use other benchmark suites or other environments, such as Web servers and database systems.
- **Hardware implementation.** Simulation is an excellent technique to implement the proposals and carry out the experiments. Even so, our proposals have the vision to be implemented in a real processor. Using techniques that allow a closer approach to this reality would be ideal at the expense of flexibility. Thus, we understand that it would be essential to implementing our strategies in Field Programmable Gate Arrays for the future. With that, new challenges would arise. For instance, it would be necessary to optimize the algorithms, paying particular attention to the area and power they could consume in such environments.
- **Explore further opportunities with ML-based algorithms.** In this thesis, we used ML to modify existing way-replacement policies. Another approach would be to deliver an utterly ML-based policy. Therefore, it would be necessary to consider which aspects of cache accesses would result in learning favoring locality principles. Furthermore, one can further explore our approaches, concerning applications such as web servers or big data, which run the same code repeatedly on different data sets. We wonder what would happen if the knowledge learned was sustained between repeated runs.

REFERENCES

AGARWAL, A. et al. An evaluation of directory schemes for cache coherence. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 15., 1988, Honolulu. **Proceedings [...]**. Honolulu: IEEE, 1988. p. 280-289.

AGARWAL, Tanuj Kumar. **Cache Coherence State Based Replacement Policies**. 2015. M.Sc. Thesis (Engineering) - Indian Institute of Science, Bangalore, 2015.

AHMADI, H.; DENZEL, W.E. A survey of modern high-performance switching techniques. **Journal on Selected Areas in Communications**, [S.l.], v. 7, n. 7, p. 1091-1103, Sep. 1989.

AL-ZOUBI, Hussein; MILENKOVIC, Aleksandar; MILENKOVIC, Milena. Performance evaluation of cache replacement policies for the SPEC CPU2000 benchmark suite. *In:* SOUTHEAST REGIONAL CONFERENCE, 42., 2004, Huntsville. **Proceedings [...]**. New York: ACM, 2004. p. 267–272.

ANDERSON, George A.; JENSEN, E. Douglas. Computer interconnection structures: Taxonomy, characteristics, and examples. **ACM Computing Surveys**, New York, v. 7, n. 4, p. 197–213, Dec. 1975.

ARUNKUMAR, Akhil; WU, Carole-Jean. Remap: Reuse and memory access cost aware eviction policy for last level cache management. *In:* INTERNATIONAL CONFERENCE ON COMPUTER DESIGN, 32., 2014, Seoul. **Proceedings [...]**. [S.l.]: IEEE, 2014. p. 110-117.

ASANOVIC, Krste et al. A view of the parallel computing landscape. **Communications of the ACM**, New York, v. 52, n. 10, p. 56-67, Oct. 2009.

BANG, Duk-Jun et al. An adaptive cache replacement policy based on fine-grain reusability monitor. **IEICE Electronics Express**, [S.l.], v. 15, n. 2, p. 1-6, Jan. 2018. Available at: <https://doi.org/10.1587/elex.14.20171099>. Accessed: 09 sep. 2021.

BECKMANN, Nathan. **Design and Analysis of Spatially-Partitioned Shared Caches**. 2015. Electrical Engineering and Computer Science - Massachusetts Institute of Technology, Cambridge, 2015.

BECKMANN, Nathan; SANCHEZ, Daniel. Maximizing cache performance under uncertainty. *In:* INTERNATIONAL SYMPOSIUM ON HIGH PERFORMANCE COMPUTER ARCHITECTURE, 23., 2008, Austin. **Proceedings [...]**. [S.l.]: IEEE, 2017. p. 109-120.

BÉLÁDY, László A. A study of replacement algorithms for a virtual-storage computer. **IBM Systems journal**, [S.l.], v. 5, n. 2, p. 78-101, 1966.

BELLMAN, Richard. A markovian decision process. **Journal of Mathematics and Mechanics**, Bloomington, v. 6, n. 5, p. 679-684, 1957.

BELLMAN, Richard. Dynamic programming and stochastic control processes. **Information and Control**, [S.l.], v. 1, n. 3, p. 228-239, 1958.

BENINI, L.; BERTOZZI, D. Network-on-chip architectures and design methods. **IET Computers and Digital Techniques**, [S.l.], v. 152, n. 2, p. 261-272, Mar. 2005.

BENINI, L.; MICHELI, G. De. Networks on chips: a new soc paradigm. **IEEE Computer**, [S.l.], v. 35, n. 1, p. 70-78, Jan. 2002.

BIENIA, Christian et al. The PARSEC benchmark suite: Characterization and architectural implications. *In:* INTERNATIONAL CONFERENCE ON PARALLEL ARCHITECTURES AND COMPILATION TECHNIQUES, 17., 2008, Toronto. **Proceedings [...]**. New York: ACM, 2008. p. 72–81.

BRAUN, Peter; LITZ, Heiner. Understanding memory access patterns for prefetching. *In:* INTERNATIONAL WORKSHOP ON AI-ASSISTED DESIGN FOR ARCHITECTURE, 2., 2019, Phoenix. **Proceedings [...]**. [S.l.: s.n.], 2019.

CARLSON, Trevor E. et al. An evaluation of high-level mechanistic core models. **ACM Transactions on Architecture and Code Optimization**, New York, v. 11, n. 3, p. 5:1-25, Oct. 2014.

CASTRO, Márcio et al. Analysis of computing and energy performance of multicore, NUMA, and manycore platforms for an irregular application. *In:* WORKSHOP ON IRREGULAR APPLICATIONS: ARCHITECTURES AND ALGORITHMS, 3., 2013, Denver. **Proceedings [...]**. New York: ACM, 2013. p. 5:1-8.

CENSIER; FEAUTRIER. A new solution to coherence problems in multicache systems. **IEEE Transactions on Computers**, [S.l.], v. C-27, n. 12, p. 1112-1118, Dec. 1978.

CHE, Shuai et al. Rodinia: A benchmark suite for heterogeneous computing. *In:* IEEE INTERNATIONAL SYMPOSIUM ON WORKLOAD CHARACTERIZATION, 5., 2009, Austin. **Proceedings [...]**. [S.l.]: IEEE, 2009. p. 44-54.

CONTI, C. J.; GIBSON, D. H.; PITKOWSKY, S. H. Structural aspects of the system/360 model 85, i: General organization. **IBM Systems Journal**, [S.l.], v. 7, n. 1, p. 2-14, 1968.

CRUZ, Eduardo Henrique Molina da et al. Using memory access traces to map threads and data on hierarchical multi-core platforms. *In:* IEEE INTERNATIONAL SYMPOSIUM ON PARALLEL AND DISTRIBUTED PROCESSING WORKSHOPS AND PHD FORUM, 5., 2011, Anchorage. **Proceedings [...]**. [S.l.]: IEEE, 2011. p. 551-558.

DALLY, W. J.; TOWLES, B. Route packets, not wires: on-chip interconnection networks. *In:* DESIGN AUTOMATION CONFERENCE, 38., 2001, Las Vegas. **Proceedings [...]**. [S.l.]: IEEE, 2001. p. 684-689.

DINECHIN, B. D. de et al. A clustered manycore processor architecture for embedded and accelerated applications. *In:* IEEE HIGH PERFORMANCE EXTREME COMPUTING CONFERENCE, 17., 2013, Waltham. **Proceedings [...]**. [S.l.]: IEEE, 2013. p. 1-6.

DUONG, Nam et al. Improving cache management policies using dynamic reuse distances. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 45., 2012, Vancouver. **Proceedings [...]**. [S.l.]: IEEE, 2012. p. 389-400.

FALDU, Priyank; GROT, Boris. Leeway: Addressing variability in dead-block prediction for last-level caches. *In:* INTERNATIONAL CONFERENCE ON PARALLEL ARCHITECTURES AND COMPILATION TECHNIQUES, 26., 2017, Portland. **Proceedings [...]**. [S.l.]: IEEE, 2017. p. 180-193.

FERNANDEZ-PASCUAL, Ricardo; ROS, Alberto; ACACIO, Manuel E. To be silent or not: On the impact of evictions of clean data in cache-coherent multicores. **The Journal of Supercomputing**, New York, v. 73, n. 10, p. 4428–4443, Oct. 2017.

FLYNN, M. Very high-speed computing systems. **Proceedings of the IEEE**, [S.l.], v. 54, n. 12, p. 1901-1909, Dec. 1966.

FRANCESQUINI, Emilio et al. On the energy efficiency and performance of irregular application executions on multicore, NUMA and manycore platforms. **Journal of Parallel and Distributed Computing**, [S.l.], v. 76, p. 32-48, Feb. 2015.

GAUR, Jayesh; ALAMELDEEN, Alaa R.; SUBRAMONEY, Sreenivas. Base-victim compression: An opportunistic cache compression architecture. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 43., 2016, Seoul. **Proceedings [...]**. [S.l.]: IEEE, 2016. p. 317-328.

GHARACHORLOO, Kourosh. **Memory Consistency Models for Shared-Memory Multiprocessors**. Tech. report CSL-TR-95-685, Stanford University. Stanford, 1995. Accessed: 09 sep. 2021. Available at: <http://i.stanford.edu/pub/cstr/reports/csl/tr/95/685/CSL-TR-95-685.pdf>.

GOODMAN, James R. Using cache memory to reduce processor-memory traffic. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 10., 1983, Stockholm. **Proceedings [...]**. New York: ACM, 1983. p. 124–131.

HASHEMI, Milad et al. Learning memory access patterns. *In:* INTERNATIONAL CONFERENCE ON MACHINE LEARNING, 35., 2018, Stockholm. **Proceedings [...]**. [S.l.]: PMLR, 2018. p. 1919-1928.

HENNESSY, John L.; PATTERSON, David A. **Computer Architecture**: A quantitative approach. 6. ed. Cambridge: Morgan Kaufmann, 2019.

HO, Ron; MAI, Kenneth W; HOROWITZ, Mark A. The future of wires. **Proceedings of the IEEE**, [S.l.], v. 89, n. 4, p. 490-504, Apr. 2001.

HU, Zhigang; KAXIRAS, S.; MARTONOSI, M. Timekeeping in the memory system: predicting and optimizing memory behavior. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 29., 2002, San Francisco. **Proceedings [...]**. [S.l.]: IEEE, 2002. p. 209-220.

ISHII, Yasuo; INABA, Mary; HIRAKI, Kei. Access map pattern matching for data cache prefetch. *In:* INTERNATIONAL CONFERENCE ON SUPERCOMPUTING, 23., 2009, Yorktown Heights. **Proceedings [...]**. New York: ACM, 2009. p. 499–500.

ISHII, Yasuo; INABA, Mary; HIRAKI, Kei. Unified memory optimizing architecture: Memory subsystem control with a unified predictor. *In:* PROCEEDINGS OF THE 26TH ACM INTERNATIONAL CONFERENCE ON SUPERCOMPUTING, 22., 2017, San Servolo Island. **Proceedings [...]**. New York: ACM, 2012. p. 267–278.

JAIN, Akanksha; LIN, Calvin. Back to the future: Leveraging belady's algorithm for improved cache replacement. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 43., 2016, Seoul. **Proceedings [...]**. [S.l.]: IEEE, 2016. p. 78-89.

JAIN, Akanksha; LIN, Calvin. Rethinking belady's algorithm to accommodate prefetching. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 45., 2018, Los Angeles. **Proceedings [...]**. [S.l.]: IEEE, 2018. p. 110-123.

JAIN, Akanksha; LIN, Calvin. **Cache Replacement Policies**. Madison: Morgan & Claypool Publishers, 2019. (Synthesis Lectures on Computer Architecture, v. 14, n. 1).

JAIN, Raj. **The art of computer systems performance analysis**: techniques for experimental design, measurement, simulation, and modeling. New York: Wiley, 1991.

JALEEL, Aamer et al. Achieving non-inclusive cache performance with inclusive caches: Temporal locality aware (tla) cache management policies. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 43., 2010, Atlanta. **Proceedings [...]**. [S.l.]: IEEE, 2010. p. 151–162.

JALEEL, Aamer et al. Adaptive insertion policies for managing shared caches. *In:* INTERNATIONAL CONFERENCE ON PARALLEL ARCHITECTURES AND COMPILATION TECHNIQUES, 17., 2008, Toronto. **Proceedings [...]**. [S.l.]: ACM, 2008. p. 208-219.

JALEEL, Aamer et al. High performance cache replacement using re-reference interval prediction (RRIP). **ACM SIGARCH Computer Architecture News**, New York, v. 38, n. 3, p. 60-71, Jun. 2010.

JEONG, J.; DUBOIS, M. Cache replacement algorithms with nonuniform miss costs. **IEEE Transactions on Computers**, [S.l.], v. 55, n. 4, p. 353-365, Apr. 2006.

JERGER, Natalie D. Enright; KRISHNA, Tushar; PEH, Li-Shiuan. **On-Chip Networks**. 2. ed. Madison: Morgan & Claypool Publishers, 2017. (Synthesis Lectures on Computer Architecture).

JIMéNEZ, Daniel A. Insertion and promotion for tree-based pseudolru last-level caches. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 46., 2013, Davis. **Proceedings [...]**. New York: ACM, 2013. p. 284-296.

JIMéNEZ, Daniel A.; TERAN, Elvira. Multiperspective reuse prediction. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 50., 2017, Boston. **Proceedings [...]**. [S.l.]: IEEE, 2017. p. 436-448.

KAELBLING, Leslie Pack; LITTMAN, Michael L; MOORE, Andrew W. Reinforcement learning: A survey. **Journal of Artificial Intelligence Research**, El Segundo, v. 4, n. 1, p. 237-285, Jan 1996.

KAREDLA, R.; LOVE, J.S.; WHERRY, B.G. Caching strategies to improve disk system performance. **IEEE Computer**, [S.l.], v. 27, n. 3, p. 38-46, 1994.

KERAMIDAS, Georgios; PETOUMENOS, Pavlos; KAXIRAS, Stefanos. Cache replacement based on reuse-distance prediction. *In:* INTERNATIONAL CONFERENCE ON COMPUTER DESIGN, 25., 2007, Lake Tahoe. **Proceedings [...]**. [S.l.]: IEEE, 2007. p. 245-250.

KHAN, Samira Manabi; TIAN, Yingying; JIMENEZ, Daniel A. Sampling dead block prediction for last-level caches. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 43., 2010, Atlanta. **Proceedings [...]**. [S.l.]: IEEE, 2010. p. 175–186.

KHARBUTLI, Mazen; SHEIKH, Rami. Lacs: A locality-aware cost-sensitive cache replacement algorithm. **IEEE Transactions on Computers**, [S.l.], v. 63, n. 8, p. 1975-1987, Aug. 2014.

KHARBUTLI, M.; SOLIHIN, Y. Counter-based cache replacement algorithms. *In:* 2005 INTERNATIONAL CONFERENCE ON COMPUTER DESIGN, 23., 2005, San Jose. **Proceedings [...]**. [S.l.]: IEEE, 2005. p. 61-68.

KIM, Jinchun et al. Kill the program counter: Reconstructing program behavior in the processor cache hierarchy. *In:* INTERNATIONAL CONFERENCE ON ARCHITECTURAL SUPPORT FOR PROGRAMMING LANGUAGES AND OPERATING SYSTEMS, 22., 2017, Xi'an. **Proceedings [...]**. New York: ACM, 2017. p. 737–749.

KIM, Jang Hyun et al. Computing the user experience via big data analysis: A case of uber services. **Computers, Materials & Continua**, [S.l.], v. 67, n. 3, p. 2819-2829, 2021.

KUMAR, R.; ZYUBAN, V.; TULLSEN, D.M. Interconnections in multi-core architectures: understanding mechanisms, overheads and scaling. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 32., 2005, Madison. **Proceedings [...]**. [S.l.]: IEEE, 2005. p. 408-419.

LATHIGARA, P.; BALACHANDRAN, S.; SINGH, V. Application behavior aware re-reference interval prediction for shared LLC. *In:* INTERNATIONAL CONFERENCE ON COMPUTER DESIGN, 33., 2015, New york. **Proceedings [...]**. [S.l.]: IEEE, 2015. p. 172-179.

LEE, Donghee et al. Lrfu: a spectrum of policies that subsumes the least recently used and least frequently used policies. **IEEE Transactions on Computers**, [S.l.], v. 50, n. 12, p. 1352-1361, 2001.

LEUNG, Carson K. et al. Data science for healthcare predictive analytics. *In:* **Proceedings of the 24th Symposium on International Database Engineering and Applications**. New York, NY, USA: Association for Computing Machinery, 2020. (IDEAS '20). ISBN 9781450375030. Available at: <https://doi.org/10.1145/3410566.3410598>.

Li, S. et al. Mcpat: An integrated power, area, and timing modeling framework for multicore and manycore architectures. *In:* **IEEE/ACM International Symposium on Microarchitecture**. [S.l.: s.n.], 2009. p. 469-480. ISSN 1072-4451.

LIU, Evan et al. An imitation learning approach for cache replacement. *In:* INTERNATIONAL CONFERENCE ON MACHINE LEARNING, 37., 2020, Online. **Proceedings [...]**. [S.l.]: PMLR, 2020. p. 6237-6247.

LUK, Chi-Keung et al. Pin: Building customized program analysis tools with dynamic instrumentation. **ACM SIGPLAN Notices**, New York, v. 40, n. 6, p. 190–200, Jun. 2005.

MALAMY, Adam; PATEL, Rajiv N; HAYES, Norman M. **Methods and apparatus for implementing a pseudo-lru cache memory replacement scheme with a locking feature**. Assignee: Sun Microsystems, Inc., US Patent n. 5353425. Date: Oct. 4, 1994.

MANIVANNAN, Madhavan et al. Radar: Runtime-assisted dead region management for last-level caches. *In:* INTERNATIONAL SYMPOSIUM ON HIGH PERFORMANCE COMPUTER ARCHITECTURE, 22., 2016, Barcelona. **Proceedings [...]**. [S.l.]: IEEE, 2016. p. 644-656.

MEGIDDO, Nimrod; MODHA, Dharmendra S. ARC: A self-tuning, low overhead replacement cache. *In:* USENIX CONFERENCE ON FILE AND STORAGE TECHNOLOGIES, 2., 2003, San Francisco. **Proceedings [...]**. [S.l.]: USENIX Association, 2003. p. 115-130.

MIRANSKYY, A. V. et al. Sift: A scalable iterative-unfolding technique for filtering execution traces. *In:* **Proceedings of the 2008 Conference of the Center for Advanced Studies on Collaborative Research: Meeting of Minds**. New York, NY, USA: Association for Computing Machinery, 2008. (CASCON '08). ISBN 9781450378826. Available at: <https://doi.org/10.1145/1463788.1463817>.

MOTTIN, Julien; CARTRON, Mickael; URLINI, Giulio. The STHORM platform. **Smart Multicore Embedded Systems**, New York, v. 1, n. 1, p. 35-43, 2014.

MOUNES-TOUSSI, F.; LILJA, D. J. The effect of using state-based priority information in a shared-memory multiprocessor cache replacement policy. *In:* INTERNATIONAL CONFERENCE ON PARALLEL PROCESSING, 27., 1998, Minneapolis. **Proceedings [...]**. [S.l.]: IEEE, 1998. p. 214-224.

MURALIMANOHAR, Naveen; BALASUBRAMONIAN, Rajeev; JOUPPI, Norman P. Cacti 6.0: A tool to model large caches. **HP laboratories**, [S.l.], HPL-2009-85, p. 24, Apr. 2009.

PAI, Suhit; SINGH, Newton; SINGH, Virendra. AB-Aware: application behavior aware management of shared last level caches. *In:* GREAT LAKES SYMPOSIUM ON VLSI, 28., 2018, Chicago. **Proceedings [...]**. New York: ACM, 2018. p. 237-242.

PATTERSON, David A.; HENNESSY, John L. **Computer Organization and Design MIPS Edition**: The hardware/software interface. 6. ed. Waltham: M. Kaufmann, 2020.

PEKHIMENKO, Gennady et al. Exploiting compressed block size as an indicator of future reuse. *In:* INTERNATIONAL SYMPOSIUM ON HIGH PERFORMANCE COMPUTER ARCHITECTURE, 21., 2015, Burlingame. **Proceedings [...]**. [S.l.]: IEEE, 2015. p. 51-63.

PELED, Leeor et al. Semantic locality and context-based prefetching using reinforcement learning. **ACM SIGARCH Computer Architecture News**, New York, v. 43, n. 3, p. 285–297, Jun. 2015.

PENNEY, Drew D.; CHEN, Lizhong. A survey of machine learning applied to computer architecture design. *In:* INTERNATIONAL WORKSHOP ON AI-ASSISTED DESIGN FOR ARCHITECTURE, 2., 2019, Phoenix. **Proceedings [...]**. [S.l.]: Cornell University, 2019.

POURHABIBI, Tahereh et al. Fraud detection: A systematic literature review of graph-based anomaly detection approaches. **Decision Support Systems**, [S.l.], v. 133, p. 113303, Apr. 2020.

PUTERMAN, Martin L. **Markov Decision Processes: Discrete Stochastic Dynamic Programming**. 1. ed. Hoboken: John Wiley & Sons, Inc., 2005. (Wiley Series in Probability and Statistics).

QURESHI, Moinuddin K. et al. Adaptive insertion policies for high performance caching. **SIGARCH Computer Architecture News**, New York, v. 35, n. 2, p. 381–391, 2007.

QURESHI, Moinuddin K. et al. A case for mlp-aware cache replacement. **ACM SIGARCH Computer Architecture News**, New York, v. 34, n. 2, p. 167–178, May. 2006.

RAJAN, Kaushik; RAMASWAMY, Govindarajan. Emulating optimal replacement with a shepherd cache. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 40., 2007, Chicago. **Proceedings [...]**. [S.l.]: IEEE, 2007. p. 445-454.

RAO, Gururaj S. Performance analysis of cache memories. **Journal of the ACM**, [S.l.], v. 25, n. 3, p. 378-395, 1978.

ROBINSON, John T.; DEVARAKONDA, Murthy V. Data cache management using frequency-based replacement. **SIGMETRICS Performance Evaluation Reviews**, New York, v. 18, n. 1, p. 134–142, 1990.

ROSE, César A. F. De; NAVAUX, Philippe O. A. **Arquiteturas Paralelas**. Porto Alegre: Bookman: Instituto de Informática da UFRGS, 2008. (Série Livros Didáticos, v. 15).

ROSSI, D. et al. Energy efficient parallel computing on the pulp platform with support for openmp. *In:* IEEE CONVENTION OF ELECTRICAL ELECTRONICS ENGINEERS IN ISRAEL, 28., 2014, Eilat. **Proceedings [...]**. [S.l.]: IEEE, 2014. p. 1-5.

RUBINSTEIN, Reuven Y. **Simulation and the Monte Carlo Method**. 3. ed. Hoboken: John Wiley & Sons, Inc., 1981.

SANCHEZ, Daniel; KOZYRAKIS, Christos. Vantage: Scalable and efficient fine-grain cache partitioning. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 38., 2011, San Jose. **Proceedings [...]**. New York: ACM, 2011. p. 57-68.

SESHADRI, Vivek et al. The evicted-address filter: A unified mechanism to address both cache pollution and thrashing. *In:* INTERNATIONAL CONFERENCE ON PARALLEL ARCHITECTURES AND COMPILATION TECHNIQUES, 21., 2012, Minneapolis. **Proceedings [...]**. New York: ACM, 2012. p. 355–366.

SESHADRI, Vivek et al. Mitigating prefetcher-caused pollution using informed caching policies for prefetched blocks. **ACM Transactions on Architecture and Code Optimization**, New York, v. 11, n. 4, p. 51:1-22, Jan. 2015.

SETHUMURUGAN, Subhash; YIN, Jieming; SARTORI, John. Designing a cost-effective cache replacement policy using machine learning. *In:* INTERNATIONAL SYMPOSIUM ON HIGH-PERFORMANCE COMPUTER ARCHITECTURE, 27., 2021, Seoul. **Proceedings [...]**. [S.l.]: IEEE, 2021. p. 291-303.

SHALF, John; DOSANJH, Sudip; MORRISON, John. Exascale computing technology challenges. *In:* **Proceedings of the 9th International Conference on High Performance Computing for Computational Science**. Berlin, Heidelberg: Springer-Verlag, 2011. (VECPAR'10), p. 1–25. ISBN 978-3-642-19327-9. Available at: <http://dl.acm.org/citation.cfm?id=1964238.1964240>.

SHI, Zhan et al. Applying deep learning to the cache replacement problem. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 42., 2019, Columbus. **Proceedings [...]**. New York: CM, 2019. p. 413-425.

SIMON, Horst D. Barriers to exascale computing. *In:* HIGH PERFORMANCE COMPUTING FOR COMPUTATIONAL SCIENCE, 10., 2012, Kope. **Proceedings [...]**. Kope: Springer, 2012. p. 1-3.

SMARAGDAKIS, Yannis; KAPLAN, Scott; WILSON, Paul. Eelru: simple and effective adaptiev page replacement. **ACM SIGMETRICS Performance Evaluation Review**, New York, v. 27, n. 1, p. 122-133, 1999.

SMITH, Alan Jay. A comparative study of set associative memory mapping algorithms and their use for cache and main memory. **IEEE Transactions on Software Engineering**, [S.l.], n. 2, p. 121-130, 1978.

SO, K.; RECHTSCHAFFEN, R.N. Cache operations by mru change. **IEEE Transactions on Computers**, [S.l.], v. 37, n. 6, p. 700-709, Jun. 1988.

SORIN, Daniel J.; HILL, Mark D.; WOOD, David A. **A Primer on Memory Consistency and Cache Coherence**. 1. ed. [S.l.]: Morgan & Claypool Publishers, 2011.

SOUZA, Matheus; FREITAS, Henrique; PÉTROT, Frédéric. Coherence state awareness in way-replacement algorithms for multicore processors. *In:* **Anais do XX Simpósio em Sistemas Computacionais de Alto Desempenho**. Porto Alegre, RS, Brasil: SBC, 2019. p. 240–251. ISSN 0000-0000. Available at: <https://sol.sbc.org.br/index.php/wscad/article/view/8672>.

SOUZA, Matheus A.; FREITAS, Henrique C.; MéHAUT, Jean-François. Design space exploration of energy efficient noc-and cache-based many-core architecture. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE AND HIGH PERFORMANCE COMPUTING, 30., 2018, Lyon. **Proceedings [...]**. [S.l.]: IEEE, 2018. p. 402-409.

SOUZA, Matheus A. et al. Cap bench: a benchmark suite for performance and energy evaluation of low-power many-core processors. **Concurrency and Computation: Practice and Experience**, [S.l.], v. 29, n. 4, p. e3892, 2017.

SRINATH, Santhosh et al. Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers. *In:* INTERNATIONAL SYMPOSIUM ON HIGH PERFORMANCE COMPUTER ARCHITECTURE, 13., 2007, Scottsdale. **Proceedings [...]**. [S.l.]: IEEE, 2007. p. 63-74.

SUBRAMANIAN, Lavanya et al. The application slowdown model: Quantifying and controlling the impact of inter-application interference at shared caches and main memory. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 48., 2015, Waikiki. **Proceedings [...]**. New York: ACM, 2015. p. 62-75.

SUTTON, Richard S.; BARTO, Andrew G. **Reinforcement Learning: An Introduction**. 2. ed. Cambridge: A Bradford Book, 2018.

TADA, Jubee; SATO, Masayuki; EGAWA, Ryusuke. An adaptive demotion policy for high-associativity caches. *In:* INTERNATIONAL SYMPOSIUM ON HIGHLY EFFICIENT ACCELERATORS AND RECONFIGURABLE TECHNOLOGIES, 8., 2017, Bochum. **Proceedings [...]**. New York: ACM, 2017. p. 4:1-6.

TANG, C. K. Cache system design in the tightly coupled multiprocessor system. *In:* NATIONAL COMPUTER CONFERENCE AND EXPOSITION, 1976, New York. **Proceedings [...]**. New York: ACM, 1976. p. 749-753.

TARSA, Stephen J et al. Improving branch prediction by modeling global history with convolutional neural networks. *In:* INTERNATIONAL WORKSHOP ON AI ASSISTED DESIGN FOR ARCHITECTURE, 2., 2019, Phoenix. **Proceedings [...]**. [S.l.]: Cornell University, 2019. p. 1-8.

Vakil-Ghahani, A. et al. Cache replacement policy based on expected hit count. **IEEE Computer Architecture Letters**, [S.l.], v. 17, n. 1, p. 64-67, Jan. 2018.

VIETRI, Giuseppe et al. Driving cache replacement with ml-based lecar. *In:* CONFERENCE ON HOT TOPICS IN STORAGE AND FILE SYSTEMS, 10., 2018, Boston. **Proceedings [...]**. [S.l.]: USENIX Association, 2018. p. 3.

VILLA, Oreste et al. Scaling the power wall: A path to exascale. *In:* **SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis**. [S.l.: s.n.], 2014. p. 830-841.

WANG, Ke et al. High-performance, energy-efficient, fault-tolerant network-on-chip design using reinforcement learning. *In:* DESIGN, AUTOMATION TEST IN EUROPE, 23., 2019, Florence. **Proceedings [...]**. [S.l.]: IEEE, 2019. p. 1166-1171.

WATKINS, Christopher J. C. H.; DAYAN, Peter. Q-learning. **Machine Learning**, [S.l.], v. 8, n. 3, p. 279-292, May 1992.

WILKINSON, Barry; ALLEN, Michael. **Parallel Programming**: Techniques and applications using networked workstations and parallel computers. 2. ed. Upper Saddle River: Prentice-Hall, Inc., 2004.

WILLIAMS, Ronald J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. **Machine Learning**, [S.l.], v. 8, n. 3, p. 229–256, May 1992.

WOO, Steven Cameron et al. The splash-2 programs: Characterization and methodological considerations. **ACM SIGARCH Computer Architecture News**, New York, v. 23, n. 2, p. 24-36, May 1995.

WU, Carole-Jean et al. SHiP: Signature-based hit predictor for high performance caching. *In:* INTERNATIONAL SYMPOSIUM ON MICROARCHITECTURE, 44., 2011, Porto Alegre. **Proceedings [...]**. [S.l.]: IEEE, 2011. p. 430-441.

WU, Nan; XIE, Yuan. A survey of machine learning for computer architecture and system. **ArXiv**, Online, p. 1-37, Feb. 2021. Available at: <https://doi.org/10.1587/elex.14.20171099>. Accessed: 09 sep. 2021.

WULF, Wm A; MCKEE, Sally A. Hitting the memory wall: Implications of the obvious. **ACM SIGARCH computer architecture news**, New York, v. 23, n. 1, p. 20-24, Mar. 1995.

XIE, Yuejian; LOH, Gabriel H. Pipp: Promotion/insertion pseudo-partitioning of multi-core shared caches. *In:* INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 36., 2009, Austin. **Proceedings [...]**. New York: ACM, 2009. p. 174–183.

YE, John M. et al. Regional cache organization for noc based many-core processors. **Journal of Computer and System Sciences**, Orlando, v. 79, n. 2, p. 175-186, Mar. 2013.

ZAHRAN, Mohamed. Cache replacement policy revisited. *In:* WORKSHOP ON DUPLICATING, DECONSTRUCTING, AND DEBUNKING, 6., 2007, San Diego. **Proceedings [...]**. [S.l.]: Citeseer, 2007. p. 1-8.

ZHENG, Hao; LOURI, Ahmed. An energy-efficient network-on-chip design using reinforcement learning. *In:* DESIGN AUTOMATION CONFERENCE, 56., 2019, Las Vegas. **Proceedings [...]**. New York: ACM, 2019. p. 47:1-6.

ZOUZIAS, Anastasios; KALAITZIDIS, Kleovoulos; GROT, Boris. Branch prediction as a reinforcement learning problem: Why, how and case studies. *In:* WORKSHOP ON MACHINE LEARNING FOR COMPUTER ARCHITECTURE AND SYSTEMS, 2., 2021, Online. **Proceedings [...]**. [S.l.]: Cornell University, 2021.

APPENDIX A - DATA FROM EXPERIMENTAL RESULTS

Table A.1 – L1 cache miss rate vs. associativity - FAST

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	8.956%	8.936%	8.967%	8.983%	8.966%	8.960%	8.456%	8.934%	8.931%
	8	9.114%	9.122%	9.089%	9.359%	9.092%	9.082%	8.104%	8.936%	8.952%
	16	0.093%	0.116%	0.131%	0.115%	0.094%	0.144%	8.926%	8.940%	8.958%
LRU	4	9.087%	9.066%	9.075%	9.092%	9.074%	9.050%	8.012%	8.886%	8.887%
	8	9.058%	9.079%	9.048%	9.080%	9.071%	9.020%	7.889%	8.880%	8.900%
	16	0.075%	0.085%	0.076%	0.085%	0.075%	0.075%	8.901%	8.735%	8.878%
MRU	4	8.872%	8.886%	8.900%	8.872%	8.727%	8.896%	7.961%	8.879%	8.705%
	8	8.878%	8.896%	8.886%	8.904%	8.815%	8.885%	7.771%	8.899%	8.882%
	16	8.893%	8.921%	8.902%	8.902%	8.763%	8.883%	8.867%	8.726%	8.885%
NRU	4	9.049%	0.075%	9.089%	0.076%	9.063%	9.060%	7.689%	8.735%	8.881%
	8	9.059%	8.436%	9.047%	8.491%	9.053%	9.059%	7.952%	8.712%	8.903%
	16	0.076%	0.085%	0.075%	0.075%	9.063%	0.075%	8.884%	8.752%	8.905%
PLRU	4	9.061%	9.099%	9.088%	9.068%	9.077%	9.080%	8.371%	8.885%	8.754%
	8	9.097%	9.074%	9.049%	9.092%	9.092%	9.105%	8.372%	8.904%	8.893%
	16	0.085%	0.080%	0.076%	0.094%	0.077%	0.086%	8.376%	8.912%	8.883%
SRRIP	4	9.060%	9.075%	9.049%	9.075%	9.075%	9.075%	8.389%	8.769%	8.720%
	8	9.073%	8.436%	9.048%	8.491%	9.055%	9.082%	8.386%	8.585%	8.885%
	16	0.075%	0.075%	0.075%	0.085%	0.076%	0.075%	8.656%	8.598%	8.904%

Table A.2 – L2 cache miss rate vs. associativity - FAST

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	99.418%	99.437%	99.317%	99.379%	99.123%	99.393%	98.979%	99.178%	99.202%
	8	99.289%	99.201%	99.391%	94.725%	99.390%	99.467%	98.926%	99.266%	99.153%
	16	73.348%	69.814%	72.751%	70.681%	72.997%	71.510%	99.047%	99.396%	99.214%
LRU	4	99.851%	99.805%	99.826%	99.853%	99.847%	99.797%	99.610%	99.796%	99.799%
	8	99.809%	99.825%	99.797%	99.826%	99.826%	99.834%	99.573%	99.761%	99.832%
	16	80.555%	85.289%	81.493%	85.341%	80.480%	80.501%	99.802%	99.776%	99.752%
MRU	4	99.693%	99.796%	99.770%	99.725%	99.710%	99.769%	99.545%	99.737%	99.738%
	8	99.662%	99.824%	99.680%	99.750%	99.814%	99.674%	99.562%	99.813%	99.770%
	16	99.579%	99.818%	99.573%	99.737%	99.799%	99.673%	99.707%	99.803%	99.783%
NRU	4	99.797%	80.510%	99.852%	81.479%	99.809%	99.809%	99.622%	99.806%	99.800%
	8	99.809%	99.692%	99.795%	99.704%	99.798%	99.810%	99.647%	99.735%	99.825%
	16	81.466%	84.997%	80.546%	80.479%	99.814%	80.505%	99.756%	99.813%	99.824%
PLRU	4	99.810%	99.542%	99.850%	99.775%	99.881%	99.663%	99.684%	99.766%	99.698%
	8	99.885%	99.703%	99.798%	99.837%	99.869%	99.691%	99.707%	99.731%	99.713%
	16	85.366%	77.775%	81.420%	79.379%	79.602%	74.435%	99.707%	99.615%	99.698%
SRRIP	4	99.808%	99.825%	99.797%	99.826%	99.826%	99.824%	99.647%	99.819%	99.765%
	8	99.821%	99.692%	99.797%	99.704%	99.811%	99.870%	99.685%	99.795%	99.780%
	16	80.525%	80.578%	80.559%	85.342%	81.490%	80.539%	99.812%	99.773%	99.835%

Table A.3 – L1 cache miss rate vs. associativity - FN

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	0.168%	0.167%	0.168%	0.168%	0.168%	0.168%	0.151%	0.154%	0.152%
	8	0.168%	0.168%	0.169%	0.168%	0.168%	0.168%	0.151%	0.153%	0.152%
	16	0.163%	0.163%	0.163%	0.164%	0.163%	0.163%	0.150%	0.154%	0.154%
LRU	4	0.168%	0.167%	0.168%	0.168%	0.168%	0.168%	0.150%	0.154%	0.156%
	8	0.169%	0.168%	0.167%	0.167%	0.167%	0.168%	0.151%	0.156%	0.153%
	16	0.165%	0.164%	0.163%	0.163%	0.164%	0.163%	0.151%	0.152%	0.151%
MRU	4	0.153%	0.153%	0.152%	0.153%	0.154%	0.153%	0.151%	0.153%	0.153%
	8	0.155%	0.154%	0.156%	0.156%	0.153%	0.152%	0.152%	0.154%	0.154%
	16	0.151%	0.153%	0.154%	0.155%	0.155%	0.153%	0.152%	0.154%	0.155%
NRU	4	0.168%	0.163%	0.169%	0.154%	0.168%	0.168%	0.153%	0.153%	0.154%
	8	0.168%	0.163%	0.167%	0.168%	0.168%	0.167%	0.151%	0.156%	0.155%
	16	0.165%	0.163%	0.164%	0.164%	0.169%	0.163%	0.150%	0.153%	0.152%
PLRU	4	0.167%	0.167%	0.167%	0.168%	0.167%	0.168%	0.152%	0.151%	0.152%
	8	0.170%	0.167%	0.169%	0.168%	0.167%	0.168%	0.150%	0.153%	0.154%
	16	0.164%	0.163%	0.163%	0.163%	0.163%	0.164%	0.152%	0.154%	0.155%
SRRIP	4	0.167%	0.168%	0.168%	0.169%	0.168%	0.167%	0.152%	0.154%	0.155%
	8	0.168%	0.168%	0.168%	0.168%	0.167%	0.169%	0.151%	0.154%	0.155%
	16	0.164%	0.165%	0.163%	0.163%	0.164%	0.164%	0.153%	0.153%	0.154%

Table A.4 – L2 cache miss rate vs. associativity - FN

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	94.434%	94.449%	94.211%	94.448%	94.153%	94.226%	96.699%	96.169%	96.520%
	8	94.059%	94.343%	94.177%	94.271%	94.281%	94.239%	96.288%	96.449%	96.233%
	16	95.078%	95.236%	95.304%	95.176%	95.147%	95.146%	96.966%	96.623%	95.811%
LRU	4	94.307%	94.391%	94.383%	94.242%	94.341%	93.927%	97.042%	96.584%	95.877%
	8	94.311%	94.294%	94.420%	94.438%	94.481%	94.349%	96.354%	95.390%	96.478%
	16	95.045%	95.089%	95.248%	95.182%	95.138%	95.272%	96.637%	96.446%	96.528%
MRU	4	96.485%	96.425%	95.969%	96.473%	96.573%	96.518%	96.906%	95.857%	96.573%
	8	95.821%	95.926%	96.314%	96.360%	96.327%	96.534%	96.698%	95.970%	96.094%
	16	96.470%	96.221%	96.510%	95.936%	96.323%	96.271%	96.800%	96.225%	96.325%
NRU	4	94.233%	95.217%	94.103%	96.111%	94.481%	94.305%	96.168%	96.406%	96.252%
	8	94.040%	95.232%	94.428%	94.201%	94.284%	94.385%	96.646%	95.730%	96.094%
	16	94.801%	95.323%	95.128%	95.109%	94.254%	95.207%	96.704%	96.370%	96.591%
PLRU	4	94.431%	94.343%	94.378%	94.069%	94.382%	94.355%	96.680%	96.739%	95.726%
	8	94.069%	94.536%	94.204%	94.268%	94.502%	94.383%	96.873%	96.313%	96.363%
	16	95.176%	95.217%	95.299%	95.151%	95.206%	95.141%	96.790%	96.081%	95.265%
SRRIP	4	94.597%	94.304%	94.307%	94.436%	94.295%	94.225%	96.638%	96.370%	96.320%
	8	94.434%	94.445%	94.348%	94.290%	94.503%	94.367%	96.724%	96.129%	96.318%
	16	95.159%	94.949%	95.261%	95.126%	95.137%	95.164%	96.735%	96.529%	96.575%

Table A.5 – L1 cache miss rate vs. associativity - GF

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	0.756%	0.750%	0.764%	0.762%	0.762%	0.757%	0.516%	0.545%	0.536%
	8	0.763%	0.762%	0.767%	0.757%	0.759%	0.758%	0.492%	0.552%	0.577%
	16	0.763%	0.763%	0.764%	0.758%	0.758%	0.759%	0.458%	0.545%	0.518%
LRU	4	0.576%	0.758%	0.756%	0.757%	0.762%	0.760%	0.524%	0.557%	0.496%
	8	0.759%	0.755%	0.762%	0.753%	0.754%	0.761%	0.446%	0.504%	0.548%
	16	0.761%	0.763%	0.758%	0.761%	0.756%	0.754%	0.488%	0.589%	0.538%
MRU	4	0.584%	0.537%	0.589%	0.562%	0.573%	0.581%	0.465%	0.544%	0.495%
	8	0.509%	0.537%	0.604%	0.558%	0.570%	0.534%	0.447%	0.555%	0.526%
	16	0.506%	0.496%	0.582%	0.582%	0.504%	0.549%	0.526%	0.553%	0.502%
NRU	4	0.754%	0.760%	0.760%	0.505%	0.763%	0.761%	0.511%	0.536%	0.541%
	8	0.756%	0.770%	0.753%	0.761%	0.756%	0.756%	0.488%	0.519%	0.545%
	16	0.763%	0.768%	0.764%	0.754%	0.757%	0.763%	0.493%	0.518%	0.576%
PLRU	4	0.552%	0.761%	0.759%	0.755%	0.760%	0.750%	0.465%	0.570%	0.539%
	8	0.763%	0.759%	0.759%	0.753%	0.762%	0.758%	0.455%	0.542%	0.514%
	16	0.771%	0.767%	0.763%	0.760%	0.757%	0.759%	0.565%	0.519%	0.574%
SRRIP	4	0.497%	0.759%	0.761%	0.756%	0.757%	0.759%	0.566%	0.569%	0.581%
	8	0.754%	0.760%	0.753%	0.754%	0.764%	0.764%	0.478%	0.565%	0.586%
	16	0.760%	0.755%	0.767%	0.756%	0.765%	0.756%	0.488%	0.569%	0.596%

Table A.6 – L2 cache miss rate vs. associativity - GF

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	95.599%	95.541%	95.546%	95.560%	95.418%	95.357%	95.546%	95.637%	95.596%
	8	95.507%	95.534%	95.422%	95.613%	95.557%	95.634%	95.958%	95.502%	95.356%
	16	95.578%	95.581%	95.567%	95.469%	95.462%	95.446%	95.835%	95.613%	95.826%
LRU	4	95.403%	95.506%	95.498%	95.594%	95.540%	95.520%	95.565%	95.283%	95.656%
	8	95.417%	95.543%	95.535%	95.569%	95.561%	95.504%	96.066%	95.815%	95.536%
	16	95.617%	95.485%	95.420%	95.535%	95.553%	95.585%	95.781%	95.439%	95.796%
MRU	4	95.452%	95.767%	95.422%	95.342%	95.392%	95.375%	95.942%	95.665%	95.661%
	8	95.729%	95.323%	95.377%	95.501%	95.503%	95.573%	96.062%	95.650%	95.759%
	16	95.436%	95.644%	95.365%	95.452%	95.608%	95.491%	95.598%	95.535%	95.521%
NRU	4	95.572%	95.533%	95.594%	95.474%	95.407%	95.504%	95.566%	95.572%	95.615%
	8	95.513%	95.537%	95.598%	95.540%	95.505%	95.511%	95.852%	95.826%	95.537%
	16	95.464%	95.526%	95.446%	95.535%	95.462%	95.532%	95.849%	95.675%	95.469%
PLRU	4	95.513%	95.372%	95.431%	95.422%	95.579%	95.595%	95.820%	95.397%	95.513%
	8	95.450%	95.551%	95.536%	95.553%	95.494%	95.524%	96.105%	95.600%	95.474%
	16	95.534%	95.430%	95.520%	95.586%	95.542%	95.533%	95.445%	95.599%	95.568%
SRRIP	4	95.456%	95.562%	95.494%	95.583%	95.514%	95.572%	95.398%	95.519%	95.442%
	8	95.526%	95.484%	95.556%	95.634%	95.540%	95.459%	95.732%	95.388%	95.522%
	16	95.600%	95.561%	95.453%	95.506%	95.436%	95.614%	95.896%	95.471%	95.465%

Table A.7 – L1 cache miss rate vs. associativity - IS

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	8.590%	8.706%	8.608%	8.645%	8.708%	8.567%	8.776%	8.860%	8.869%
	8	9.088%	8.879%	8.900%	9.151%	9.078%	8.742%	8.900%	9.138%	9.151%
	16	10.687%	10.890%	10.997%	10.741%	11.071%	11.018%	10.086%	9.756%	10.238%
LRU	4	8.622%	8.619%	8.524%	8.619%	8.523%	8.620%	8.773%	8.674%	8.677%
	8	8.619%	8.505%	8.616%	8.505%	8.620%	8.504%	8.669%	8.685%	8.671%
	16	9.630%	9.642%	9.631%	9.626%	9.626%	9.653%	8.671%	8.685%	8.771%
MRU	4	8.774%	8.875%	8.783%	8.630%	8.725%	8.742%	8.770%	8.864%	8.772%
	8	8.799%	9.034%	8.808%	8.491%	8.810%	8.795%	8.814%	8.904%	8.921%
	16	8.512%	9.435%	8.551%	8.064%	9.047%	8.591%	8.521%	9.283%	9.164%
NRU	4	8.527%	9.645%	8.620%	8.672%	8.531%	8.781%	8.689%	8.691%	8.700%
	8	8.526%	9.697%	8.510%	8.772%	8.531%	8.550%	8.674%	8.686%	8.700%
	16	9.632%	9.773%	9.632%	8.674%	8.627%	9.628%	8.676%	8.687%	8.686%
PLRU	4	8.674%	8.762%	8.522%	8.576%	8.658%	8.583%	8.674%	8.679%	8.669%
	8	8.502%	8.634%	8.549%	8.664%	8.556%	8.715%	8.675%	8.657%	8.653%
	16	9.758%	9.711%	9.649%	9.688%	9.588%	9.684%	8.666%	8.652%	8.658%
SRRIP	4	8.515%	8.518%	8.620%	8.522%	8.518%	8.503%	8.670%	8.674%	8.671%
	8	8.623%	8.621%	8.584%	8.512%	8.619%	8.612%	8.682%	8.675%	8.670%
	16	9.769%	9.681%	9.759%	9.649%	9.652%	9.753%	8.676%	8.675%	8.669%

Table A.8 – L2 cache miss rate vs. associativity - IS

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	54.778%	63.062%	54.575%	64.283%	59.425%	54.493%	55.270%	61.256%	61.257%
	8	55.364%	60.134%	55.815%	60.544%	58.788%	55.964%	53.678%	59.107%	60.178%
	16	60.097%	61.646%	62.006%	61.161%	61.999%	61.941%	61.964%	61.012%	62.397%
LRU	4	53.231%	61.941%	51.379%	62.045%	53.803%	53.171%	53.824%	56.442%	56.369%
	8	51.673%	53.871%	51.666%	53.711%	52.184%	49.461%	49.988%	54.262%	53.851%
	16	49.134%	51.635%	49.127%	51.201%	49.241%	49.477%	49.752%	52.228%	54.080%
MRU	4	66.937%	74.432%	67.081%	59.830%	68.379%	66.896%	67.124%	66.345%	64.533%
	8	59.077%	79.069%	59.090%	55.632%	71.666%	59.067%	59.072%	78.526%	78.568%
	16	57.543%	62.063%	57.645%	55.384%	60.031%	57.828%	57.573%	74.409%	73.971%
NRU	4	52.889%	53.637%	54.452%	53.044%	53.194%	55.158%	53.375%	54.063%	54.334%
	8	51.325%	53.812%	51.083%	55.622%	52.785%	51.986%	51.602%	53.305%	53.673%
	16	50.903%	55.787%	50.888%	55.561%	55.021%	50.947%	51.472%	53.711%	53.727%
PLRU	4	52.014%	64.927%	51.644%	62.887%	51.061%	65.272%	52.171%	55.601%	55.576%
	8	49.923%	58.385%	50.791%	56.724%	48.749%	58.613%	50.477%	56.028%	56.086%
	16	52.057%	56.353%	50.083%	54.237%	46.190%	57.194%	50.313%	55.949%	56.024%
SRRIP	4	54.077%	49.696%	51.449%	49.727%	49.731%	49.581%	50.073%	50.060%	50.085%
	8	55.191%	49.553%	48.826%	47.072%	49.556%	49.556%	48.069%	47.678%	47.667%
	16	55.334%	47.737%	49.271%	47.084%	47.084%	49.311%	47.316%	47.311%	47.309%

Table A.9 – L1 cache miss rate vs. associativity - KM

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	0.106%	0.104%	0.103%	0.106%	0.101%	0.109%	0.111%	0.108%	0.090%
	8	0.119%	0.112%	0.110%	0.108%	0.123%	0.374%	0.105%	0.126%	0.117%
	16	0.136%	0.134%	0.134%	0.134%	0.132%	0.152%	0.147%	0.148%	0.149%
LRU	4	0.104%	0.103%	0.104%	0.104%	0.104%	0.104%	0.091%	0.090%	0.091%
	8	0.106%	0.106%	0.106%	0.106%	0.106%	0.106%	0.089%	0.090%	0.091%
	16	0.102%	0.101%	0.101%	0.102%	0.102%	0.102%	0.089%	0.090%	0.091%
MRU	4	0.105%	0.091%	0.095%	0.091%	0.091%	0.091%	0.093%	0.090%	0.091%
	8	0.104%	0.104%	0.099%	0.091%	0.092%	0.091%	0.089%	0.091%	0.091%
	16	0.129%	0.092%	0.124%	0.094%	0.092%	0.093%	0.091%	0.091%	0.092%
NRU	4	0.103%	0.102%	0.104%	0.099%	0.106%	0.099%	0.091%	0.090%	0.091%
	8	0.106%	0.101%	0.106%	0.091%	0.106%	0.106%	0.088%	0.091%	0.091%
	16	0.102%	0.102%	0.101%	0.099%	0.106%	0.102%	0.090%	0.090%	0.091%
PLRU	4	0.103%	0.103%	0.103%	0.104%	0.103%	0.119%	0.091%	0.090%	0.091%
	8	0.106%	0.106%	0.106%	0.106%	0.106%	0.126%	0.082%	0.090%	0.091%
	16	0.102%	0.102%	0.102%	0.102%	0.102%	0.130%	0.082%	0.091%	0.092%
SRRIP	4	0.104%	0.103%	0.103%	0.103%	0.104%	0.103%	0.091%	0.090%	0.091%
	8	0.106%	0.106%	0.106%	0.106%	0.106%	0.106%	0.082%	0.090%	0.091%
	16	0.102%	0.102%	0.102%	0.102%	0.102%	0.102%	0.082%	0.090%	0.091%

Table A.10 – L2 cache miss rate vs. associativity - KM

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	83.456%	84.137%	84.334%	83.732%	84.913%	83.631%	74.880%	76.415%	83.961%
	8	88.120%	82.113%	84.290%	83.512%	86.736%	55.895%	84.241%	73.914%	73.904%
	16	76.358%	77.309%	77.035%	77.415%	77.626%	71.442%	77.653%	77.587%	77.624%
LRU	4	85.336%	85.292%	85.308%	85.334%	85.355%	85.255%	84.185%	83.989%	84.259%
	8	85.850%	85.890%	85.809%	85.825%	85.848%	85.815%	83.976%	84.025%	84.193%
	16	85.347%	85.311%	85.333%	85.253%	85.321%	85.352%	83.949%	84.069%	84.277%
MRU	4	79.288%	84.427%	83.422%	84.241%	84.203%	84.388%	83.849%	84.096%	84.105%
	8	80.015%	89.213%	80.248%	84.161%	83.999%	84.812%	84.007%	84.048%	84.401%
	16	77.033%	85.489%	77.501%	81.618%	83.862%	83.978%	83.588%	83.702%	84.070%
NRU	4	85.305%	85.179%	85.304%	85.083%	85.809%	85.075%	84.241%	84.016%	84.198%
	8	85.852%	85.253%	85.852%	84.308%	85.871%	85.827%	83.762%	84.270%	84.249%
	16	85.380%	85.278%	85.223%	85.096%	85.885%	85.295%	84.016%	84.059%	84.203%
PLRU	4	85.302%	85.227%	85.283%	85.243%	85.287%	88.442%	84.264%	83.977%	84.365%
	8	85.825%	85.627%	85.828%	85.848%	85.818%	86.993%	82.840%	83.916%	84.066%
	16	85.382%	84.960%	85.358%	85.182%	85.304%	76.757%	82.895%	83.455%	83.933%
SRRIP	4	85.337%	85.237%	85.257%	85.172%	85.335%	85.292%	84.218%	83.970%	84.222%
	8	85.864%	85.873%	85.828%	85.814%	85.813%	85.836%	82.834%	84.064%	84.250%
	16	85.304%	85.310%	85.288%	85.267%	85.384%	85.318%	82.897%	84.048%	84.253%

Table A.11 – L1 cache miss rate vs. associativity - LU

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	5.318%	5.160%	5.291%	5.269%	5.358%	4.993%	5.657%	5.739%	5.795%
	8	4.731%	4.826%	4.777%	4.691%	4.784%	4.881%	4.942%	4.869%	4.854%
	16	5.037%	5.222%	4.992%	5.913%	5.175%	5.728%	5.537%	5.499%	5.775%
LRU	4	6.102%	6.066%	6.112%	6.070%	6.107%	6.124%	6.071%	6.111%	6.052%
	8	6.127%	6.061%	6.111%	6.118%	6.104%	6.129%	6.057%	6.088%	6.079%
	16	5.768%	5.787%	5.767%	5.776%	5.783%	5.792%	6.061%	6.098%	6.073%
MRU	4	6.003%	6.017%	6.014%	5.928%	5.912%	5.807%	5.725%	5.920%	5.899%
	8	4.963%	6.060%	4.959%	5.076%	6.049%	4.942%	4.886%	5.212%	5.188%
	16	5.948%	5.559%	5.871%	5.695%	5.517%	5.500%	5.579%	5.504%	5.464%
NRU	4	6.121%	5.788%	6.098%	5.868%	6.124%	5.906%	6.133%	6.131%	6.094%
	8	6.157%	5.807%	6.122%	6.089%	6.136%	6.131%	6.098%	6.087%	6.080%
	16	5.812%	5.798%	5.804%	5.826%	6.135%	5.810%	6.079%	6.070%	6.067%
PLRU	4	6.108%	5.981%	6.089%	5.626%	6.090%	5.821%	6.132%	6.066%	6.097%
	8	6.109%	5.905%	6.124%	4.591%	6.094%	4.707%	5.602%	5.487%	5.557%
	16	5.825%	5.510%	5.806%	5.170%	5.690%	5.706%	5.944%	5.628%	5.972%
SRRIP	4	6.104%	6.049%	6.044%	6.078%	6.110%	6.071%	6.060%	6.078%	5.997%
	8	6.128%	6.048%	6.045%	6.042%	6.026%	6.119%	5.986%	6.085%	6.062%
	16	5.785%	5.721%	5.776%	5.707%	5.710%	5.772%	5.998%	6.026%	6.061%

Table A.12 – L2 cache miss rate vs. associativity - LU

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	18.336%	17.320%	18.262%	21.671%	17.622%	20.114%	16.594%	18.809%	18.965%
	8	23.405%	22.016%	22.712%	31.539%	22.023%	27.542%	24.294%	28.947%	29.800%
	16	30.395%	30.891%	30.260%	39.581%	31.563%	36.945%	33.466%	38.131%	38.264%
LRU	4	11.721%	11.702%	11.729%	12.165%	11.722%	12.042%	11.269%	11.092%	11.250%
	8	11.884%	11.541%	11.888%	12.054%	11.770%	12.150%	10.871%	11.088%	11.100%
	16	11.498%	11.480%	11.479%	11.474%	11.482%	11.469%	10.842%	11.103%	11.009%
MRU	4	19.467%	10.859%	19.270%	14.081%	11.553%	15.134%	14.640%	13.387%	12.275%
	8	38.818%	12.016%	38.504%	32.905%	12.394%	33.013%	30.903%	28.222%	28.390%
	16	41.957%	18.933%	42.303%	39.406%	20.314%	39.609%	39.194%	35.510%	35.439%
NRU	4	11.766%	11.414%	11.815%	11.133%	11.893%	11.408%	11.401%	11.323%	11.126%
	8	12.119%	11.586%	11.911%	11.425%	12.190%	12.011%	11.184%	11.213%	11.491%
	16	11.518%	11.477%	11.519%	11.446%	11.980%	11.607%	11.032%	11.397%	11.344%
PLRU	4	11.843%	12.910%	11.769%	19.429%	11.946%	18.861%	15.272%	15.363%	15.998%
	8	11.887%	15.722%	11.852%	31.925%	12.289%	29.981%	23.833%	23.388%	26.250%
	16	11.672%	24.124%	11.684%	40.744%	11.735%	38.676%	33.727%	33.112%	35.332%
SRRIP	4	11.763%	11.255%	11.193%	11.353%	11.438%	11.969%	10.562%	10.626%	10.258%
	8	11.960%	11.025%	11.112%	11.064%	10.998%	12.130%	9.915%	10.613%	10.611%
	16	11.551%	10.682%	11.052%	10.762%	10.709%	11.882%	10.154%	10.382%	10.679%

Table A.13 – L1 cache miss rate vs. associativity - TSP

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	8	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	16	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
LRU	4	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	8	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	16	0.004%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
MRU	4	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	8	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	16	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
NRU	4	0.005%	0.004%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	8	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	16	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
PLRU	4	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	8	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	16	0.005%	0.005%	0.005%	0.004%	0.005%	0.005%	0.005%	0.005%	0.005%
SRRIP	4	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	8	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	16	0.005%	0.005%	0.005%	0.004%	0.005%	0.005%	0.005%	0.005%	0.005%

Table A.14 – L2 cache miss rate vs. associativity - TSP

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	86.558%	93.267%	93.176%	92.572%	93.017%	93.375%	88.520%	95.006%	95.517%
	8	93.304%	93.238%	92.908%	93.642%	92.073%	93.046%	88.416%	96.211%	95.092%
	16	93.030%	92.844%	93.459%	93.046%	93.406%	85.882%	95.686%	95.912%	95.463%
LRU	4	93.040%	93.720%	93.011%	92.952%	92.641%	93.447%	95.789%	95.159%	95.785%
	8	92.943%	93.738%	92.793%	93.319%	92.845%	93.489%	95.768%	95.598%	95.120%
	16	86.271%	93.531%	93.393%	93.168%	92.776%	93.619%	95.018%	95.916%	95.752%
MRU	4	95.561%	95.768%	95.552%	95.838%	95.598%	88.577%	94.907%	95.033%	95.342%
	8	96.267%	88.673%	94.658%	95.430%	94.850%	95.327%	95.330%	95.894%	95.456%
	16	95.555%	95.529%	96.336%	95.077%	95.795%	95.827%	95.814%	88.545%	95.988%
NRU	4	92.990%	92.726%	92.932%	95.462%	92.924%	93.144%	95.773%	88.475%	95.372%
	8	93.051%	92.996%	92.427%	93.275%	93.545%	93.343%	95.413%	95.552%	95.575%
	16	93.246%	93.221%	93.355%	93.550%	92.551%	85.759%	95.747%	95.410%	95.650%
PLRU	4	92.928%	93.190%	93.115%	92.230%	93.429%	92.604%	95.303%	95.311%	96.101%
	8	93.178%	93.175%	92.858%	93.404%	93.046%	92.527%	96.212%	95.148%	96.361%
	16	93.674%	92.687%	93.173%	93.363%	86.054%	93.236%	95.972%	94.736%	95.719%
SRRIP	4	92.437%	93.392%	93.449%	93.745%	92.897%	92.599%	95.324%	95.566%	95.766%
	8	92.819%	93.273%	93.058%	93.238%	93.205%	93.037%	96.585%	95.285%	95.577%
	16	92.705%	93.226%	92.947%	92.456%	93.398%	92.819%	95.472%	95.493%	95.498%

Table A.15 – L1 cache miss rate vs. block size - FAST

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	9.114%	9.122%	9.089%	9.359%	9.092%	9.082%	8.104%	8.936%	8.952%
	128	8.950%	8.974%	8.842%	8.836%	8.961%	8.847%	8.941%	8.998%	8.962%
	256	9.080%	9.040%	8.916%	9.044%	9.096%	9.081%	9.057%	9.023%	8.901%
LRU	64	9.058%	9.079%	9.048%	9.080%	9.071%	9.020%	7.889%	8.880%	8.900%
	128	8.924%	8.927%	8.900%	8.926%	8.893%	8.906%	8.981%	8.880%	8.893%
	256	8.930%	8.927%	8.670%	8.928%	8.749%	8.556%	8.875%	8.716%	8.922%
MRU	64	8.878%	8.896%	8.886%	8.904%	8.815%	8.885%	7.771%	8.899%	8.882%
	128	8.917%	8.929%	8.914%	8.757%	8.930%	8.926%	8.928%	8.768%	8.725%
	256	8.954%	8.758%	8.956%	8.570%	8.758%	8.911%	8.775%	8.936%	8.866%
NRU	64	9.059%	8.436%	9.047%	8.491%	9.053%	9.059%	7.952%	8.712%	8.903%
	128	8.925%	8.883%	8.901%	8.928%	8.910%	8.866%	8.587%	8.929%	8.885%
	256	8.682%	8.913%	8.723%	8.772%	8.560%	8.601%	8.836%	8.594%	8.786%
PLRU	64	9.097%	9.074%	9.049%	9.092%	9.092%	9.105%	8.372%	8.904%	8.893%
	128	8.861%	8.907%	8.732%	8.913%	8.885%	8.896%	8.861%	8.625%	8.908%
	256	8.665%	8.774%	8.814%	8.739%	8.936%	8.981%	8.611%	8.943%	8.972%
SRRIP	64	9.073%	8.436%	9.048%	8.491%	9.055%	9.082%	8.386%	8.585%	8.885%
	128	8.891%	8.731%	8.890%	8.613%	8.823%	8.880%	8.859%	8.897%	8.766%
	256	8.600%	8.830%	8.830%	8.785%	8.681%	8.752%	8.900%	8.791%	8.831%

Table A.16 – L2 cache miss rate vs. block size - FAST

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	99.289%	99.201%	99.391%	94.725%	99.390%	99.467%	98.926%	99.266%	99.153%
	128	99.271%	99.013%	98.898%	98.751%	98.732%	98.970%	98.981%	98.873%	99.019%
	256	98.374%	97.911%	98.548%	98.783%	98.260%	98.423%	97.897%	98.885%	98.151%
LRU	64	99.809%	99.825%	99.797%	99.826%	99.826%	99.834%	99.573%	99.761%	99.832%
	128	99.897%	99.897%	99.813%	99.897%	99.817%	99.838%	98.997%	99.772%	99.791%
	256	99.903%	99.903%	99.698%	99.902%	99.737%	99.646%	98.479%	99.630%	99.896%
MRU	64	99.662%	99.824%	99.680%	99.750%	99.814%	99.674%	99.562%	99.813%	99.770%
	128	99.639%	99.898%	99.654%	99.778%	99.874%	99.583%	99.761%	99.868%	99.780%
	256	99.588%	99.831%	99.614%	99.659%	99.547%	99.353%	99.621%	99.849%	99.883%
NRU	64	99.809%	99.692%	99.795%	99.704%	99.798%	99.810%	99.647%	99.735%	99.825%
	128	99.900%	99.773%	99.814%	99.897%	99.847%	99.882%	99.779%	99.898%	99.804%
	256	99.888%	99.899%	99.685%	99.881%	99.870%	99.713%	99.877%	99.713%	99.875%
PLRU	64	99.885%	99.703%	99.798%	99.837%	99.869%	99.691%	99.707%	99.731%	99.713%
	128	99.761%	99.438%	99.880%	99.699%	99.801%	99.604%	99.756%	99.643%	99.794%
	256	99.887%	99.407%	99.878%	99.572%	99.727%	99.381%	99.637%	99.734%	99.406%
SRRIP	64	99.821%	99.692%	99.797%	99.704%	99.811%	99.870%	99.685%	99.795%	99.780%
	128	99.885%	99.884%	99.886%	99.777%	99.787%	99.801%	99.761%	99.887%	99.865%
	256	99.668%	99.892%	99.892%	99.538%	99.887%	99.869%	99.899%	99.922%	99.893%

Table A.17 – L1 cache miss rate vs. block size - FN

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	0.168%	0.168%	0.169%	0.168%	0.168%	0.168%	0.151%	0.153%	0.152%
	128	0.122%	0.121%	0.120%	0.120%	0.119%	0.120%	0.120%	0.121%	0.122%
	256	0.105%	0.106%	0.103%	0.104%	0.103%	0.102%	0.104%	0.103%	0.104%
LRU	64	0.169%	0.168%	0.167%	0.167%	0.167%	0.168%	0.151%	0.156%	0.153%
	128	0.120%	0.124%	0.123%	0.120%	0.120%	0.120%	0.120%	0.122%	0.122%
	256	0.104%	0.105%	0.105%	0.103%	0.103%	0.104%	0.105%	0.102%	0.104%
MRU	64	0.155%	0.154%	0.156%	0.156%	0.153%	0.152%	0.152%	0.154%	0.154%
	128	0.123%	0.121%	0.121%	0.120%	0.121%	0.120%	0.120%	0.122%	0.123%
	256	0.104%	0.105%	0.103%	0.106%	0.105%	0.105%	0.104%	0.103%	0.103%
NRU	64	0.168%	0.163%	0.167%	0.168%	0.168%	0.167%	0.151%	0.156%	0.155%
	128	0.121%	0.120%	0.124%	0.122%	0.122%	0.121%	0.120%	0.122%	0.122%
	256	0.106%	0.105%	0.103%	0.103%	0.102%	0.105%	0.104%	0.104%	0.104%
PLRU	64	0.170%	0.167%	0.169%	0.168%	0.167%	0.168%	0.150%	0.153%	0.154%
	128	0.121%	0.120%	0.120%	0.120%	0.121%	0.120%	0.119%	0.122%	0.121%
	256	0.101%	0.105%	0.106%	0.103%	0.105%	0.104%	0.104%	0.103%	0.104%
SRRIP	64	0.168%	0.168%	0.168%	0.168%	0.167%	0.169%	0.151%	0.154%	0.155%
	128	0.119%	0.119%	0.123%	0.119%	0.119%	0.121%	0.121%	0.123%	0.122%
	256	0.102%	0.103%	0.105%	0.104%	0.107%	0.102%	0.103%	0.105%	0.103%

Table A.18 – L2 cache miss rate vs. block size - FN

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	94.059%	94.343%	94.177%	94.271%	94.281%	94.239%	96.288%	96.449%	96.233%
	128	95.618%	96.028%	95.772%	96.049%	95.679%	96.041%	96.107%	96.000%	95.756%
	256	95.550%	95.156%	95.507%	95.410%	95.156%	95.565%	95.087%	94.954%	95.604%
LRU	64	94.311%	94.294%	94.420%	94.438%	94.481%	94.349%	96.354%	95.390%	96.478%
	128	96.086%	95.525%	95.864%	96.099%	96.009%	96.112%	96.302%	96.075%	95.802%
	256	95.197%	95.312%	95.402%	95.288%	95.670%	95.444%	94.974%	95.503%	95.352%
MRU	64	95.821%	95.926%	96.314%	96.360%	96.327%	96.534%	96.698%	95.970%	96.094%
	128	95.799%	96.071%	96.124%	96.188%	96.065%	95.676%	96.176%	95.860%	96.007%
	256	94.833%	95.062%	95.807%	94.724%	95.042%	95.234%	95.597%	95.632%	95.264%
NRU	64	94.040%	95.232%	94.428%	94.201%	94.284%	94.385%	96.646%	95.730%	96.094%
	128	96.226%	96.361%	95.471%	95.890%	95.382%	96.057%	96.086%	96.230%	95.925%
	256	95.018%	95.291%	95.335%	95.799%	95.905%	95.460%	95.553%	95.195%	95.839%
PLRU	64	94.069%	94.536%	94.204%	94.268%	94.502%	94.383%	96.873%	96.313%	96.363%
	128	96.264%	96.176%	96.323%	96.432%	96.124%	96.192%	96.243%	95.876%	96.151%
	256	96.112%	94.995%	94.671%	95.597%	94.889%	94.922%	95.667%	95.867%	95.386%
SRRIP	64	94.434%	94.445%	94.348%	94.290%	94.503%	94.367%	96.724%	96.129%	96.318%
	128	96.066%	96.375%	95.767%	96.092%	96.106%	95.804%	95.717%	95.546%	95.676%
	256	95.987%	95.539%	95.352%	95.741%	95.032%	95.775%	95.359%	95.364%	95.589%

Table A.19 – L1 cache miss rate vs. block size - GF

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	0.763%	0.762%	0.767%	0.757%	0.759%	0.758%	0.492%	0.552%	0.577%
	128	0.392%	0.442%	0.452%	0.455%	0.413%	0.340%	0.443%	0.411%	0.388%
	256	0.288%	0.322%	0.325%	0.385%	0.344%	0.322%	0.320%	0.366%	0.354%
LRU	64	0.759%	0.755%	0.762%	0.753%	0.754%	0.761%	0.446%	0.504%	0.548%
	128	0.410%	0.342%	0.385%	0.415%	0.419%	0.393%	0.424%	0.402%	0.423%
	256	0.332%	0.351%	0.338%	0.332%	0.329%	0.316%	0.374%	0.317%	0.342%
MRU	64	0.509%	0.537%	0.604%	0.558%	0.570%	0.534%	0.447%	0.555%	0.526%
	128	0.366%	0.402%	0.438%	0.434%	0.435%	0.470%	0.462%	0.441%	0.401%
	256	0.319%	0.382%	0.386%	0.300%	0.329%	0.389%	0.283%	0.360%	0.337%
NRU	64	0.756%	0.770%	0.753%	0.761%	0.756%	0.756%	0.488%	0.519%	0.545%
	128	0.440%	0.471%	0.476%	0.402%	0.381%	0.382%	0.356%	0.368%	0.356%
	256	0.343%	0.356%	0.358%	0.331%	0.325%	0.336%	0.352%	0.315%	0.339%
PLRU	64	0.763%	0.759%	0.759%	0.753%	0.762%	0.758%	0.455%	0.542%	0.514%
	128	0.432%	0.473%	0.393%	0.391%	0.410%	0.417%	0.421%	0.451%	0.424%
	256	0.352%	0.358%	0.321%	0.382%	0.341%	0.333%	0.333%	0.335%	0.319%
SRRIP	64	0.754%	0.760%	0.753%	0.754%	0.764%	0.764%	0.478%	0.565%	0.586%
	128	0.433%	0.380%	0.443%	0.397%	0.392%	0.450%	0.437%	0.382%	0.419%
	256	0.302%	0.374%	0.315%	0.338%	0.334%	0.344%	0.313%	0.374%	0.359%

Table A.20 – L2 cache miss rate vs. block size - GF

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	95.507%	95.534%	95.422%	95.613%	95.557%	95.634%	95.958%	95.502%	95.356%
	128	94.284%	94.403%	94.670%	94.403%	94.868%	94.842%	94.675%	94.893%	94.720%
	256	91.030%	91.302%	91.821%	92.708%	92.422%	92.143%	91.617%	91.937%	92.369%
LRU	64	95.417%	95.543%	95.535%	95.569%	95.561%	95.504%	96.066%	95.815%	95.536%
	128	94.853%	94.514%	94.469%	94.543%	94.884%	94.980%	94.387%	94.717%	94.681%
	256	92.311%	92.842%	91.496%	91.900%	91.569%	91.469%	92.423%	91.752%	92.351%
MRU	64	95.729%	95.323%	95.377%	95.501%	95.503%	95.573%	96.062%	95.650%	95.759%
	128	94.179%	94.338%	94.822%	94.871%	94.981%	94.344%	94.657%	94.587%	94.282%
	256	91.212%	92.347%	92.600%	91.196%	90.741%	92.615%	90.725%	92.165%	91.758%
NRU	64	95.513%	95.537%	95.598%	95.540%	95.505%	95.511%	95.852%	95.826%	95.537%
	128	95.011%	94.927%	94.417%	94.944%	94.209%	94.426%	94.162%	94.149%	94.287%
	256	91.998%	92.196%	92.775%	92.542%	92.040%	92.227%	92.324%	91.604%	91.813%
PLRU	64	95.450%	95.551%	95.536%	95.553%	95.494%	95.524%	96.105%	95.600%	95.474%
	128	95.010%	94.978%	94.662%	94.759%	95.039%	95.005%	94.308%	94.978%	94.725%
	256	92.065%	92.017%	92.336%	92.450%	92.405%	91.866%	92.841%	91.743%	92.037%
SRRIP	64	95.526%	95.484%	95.556%	95.634%	95.540%	95.459%	95.732%	95.388%	95.522%
	128	95.050%	95.073%	94.777%	94.703%	94.556%	94.802%	94.938%	94.493%	94.908%
	256	91.024%	92.482%	91.005%	92.461%	91.645%	92.256%	91.406%	92.457%	92.383%

Table A.21 – L1 cache miss rate vs. block size - IS

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	9.088%	8.879%	8.900%	9.151%	9.078%	8.742%	8.900%	9.138%	9.151%
	128	8.621%	8.902%	8.951%	8.867%	8.769%	8.691%	8.679%	8.950%	8.941%
	256	7.191%	7.154%	7.399%	7.426%	7.322%	6.996%	7.079%	7.249%	7.085%
LRU	64	8.619%	8.505%	8.616%	8.505%	8.620%	8.504%	8.669%	8.685%	8.671%
	128	8.186%	8.186%	8.187%	8.198%	8.198%	8.189%	8.665%	8.193%	8.187%
	256	6.127%	6.270%	6.129%	6.128%	6.226%	6.129%	7.186%	6.126%	6.126%
MRU	64	8.799%	9.034%	8.808%	8.491%	8.810%	8.795%	8.814%	8.904%	8.921%
	128	8.532%	8.598%	8.384%	8.022%	8.353%	8.463%	8.434%	8.520%	8.493%
	256	6.633%	6.670%	6.722%	6.265%	6.372%	6.555%	6.523%	6.534%	6.443%
NRU	64	8.526%	9.697%	8.510%	8.772%	8.531%	8.550%	8.674%	8.686%	8.700%
	128	8.271%	8.226%	8.193%	8.208%	8.200%	8.192%	8.200%	8.214%	8.214%
	256	6.132%	6.155%	6.228%	6.125%	6.134%	6.127%	6.127%	6.155%	6.153%
PLRU	64	8.502%	8.634%	8.549%	8.664%	8.556%	8.715%	8.675%	8.657%	8.653%
	128	8.190%	8.377%	8.198%	8.275%	8.170%	8.333%	8.186%	8.244%	8.225%
	256	6.224%	6.583%	6.127%	6.554%	6.106%	6.593%	6.124%	6.350%	6.329%
SRRIP	64	8.623%	8.621%	8.584%	8.512%	8.619%	8.612%	8.682%	8.675%	8.670%
	128	8.224%	8.191%	8.195%	8.199%	8.198%	8.197%	8.198%	8.231%	8.209%
	256	6.143%	6.216%	6.225%	6.226%	6.221%	6.125%	6.126%	6.129%	6.128%

Table A.22 – L2 cache miss rate vs. block size - IS

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	55.364%	60.134%	55.815%	60.544%	58.788%	55.964%	53.678%	59.107%	60.178%
	128	57.516%	60.123%	57.996%	60.407%	59.922%	57.811%	57.603%	60.160%	60.728%
	256	59.661%	61.493%	60.569%	61.939%	61.328%	59.241%	59.853%	62.432%	60.999%
LRU	64	51.673%	53.871%	51.666%	53.711%	52.184%	49.461%	49.988%	54.262%	53.851%
	128	61.049%	64.156%	61.059%	64.044%	61.510%	61.128%	50.478%	64.060%	64.040%
	256	68.938%	71.077%	68.870%	70.815%	69.569%	68.879%	56.368%	70.753%	70.760%
MRU	64	59.077%	79.069%	59.090%	55.632%	71.666%	59.067%	59.072%	78.526%	78.568%
	128	58.831%	83.150%	58.549%	53.674%	73.237%	58.601%	58.732%	82.582%	82.369%
	256	59.022%	85.766%	59.140%	53.567%	74.947%	59.010%	58.892%	84.965%	84.855%
NRU	64	51.325%	53.812%	51.083%	55.622%	52.785%	51.986%	51.602%	53.305%	53.673%
	128	63.239%	65.286%	62.215%	61.812%	64.716%	62.189%	62.173%	65.234%	65.197%
	256	70.059%	73.727%	70.210%	68.197%	73.180%	70.041%	70.045%	73.681%	73.703%
PLRU	64	49.923%	58.385%	50.791%	56.724%	48.749%	58.613%	50.477%	56.028%	56.086%
	128	61.237%	58.190%	61.218%	57.137%	54.140%	58.277%	61.091%	57.319%	57.339%
	256	69.136%	59.092%	69.001%	57.342%	57.999%	59.003%	68.251%	58.178%	58.069%
SRRIP	64	55.191%	49.553%	48.826%	47.072%	49.556%	49.556%	48.069%	47.678%	47.667%
	128	64.220%	59.651%	59.658%	59.661%	59.631%	59.710%	59.660%	60.277%	59.963%
	256	72.142%	67.822%	67.750%	67.753%	67.784%	67.721%	67.637%	67.633%	67.650%

Table A.23 – L1 cache miss rate vs. block size - KM

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	0.119%	0.112%	0.110%	0.108%	0.123%	0.374%	0.105%	0.126%	0.117%
	128	0.101%	0.098%	0.102%	0.100%	0.099%	0.106%	0.099%	0.104%	0.103%
	256	0.087%	0.088%	0.085%	0.085%	0.095%	0.085%	0.094%	0.085%	0.096%
LRU	64	0.106%	0.106%	0.106%	0.106%	0.106%	0.106%	0.089%	0.090%	0.091%
	128	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%	0.091%	0.089%	0.089%
	256	0.084%	0.084%	0.084%	0.084%	0.084%	0.084%	0.085%	0.085%	0.084%
MRU	64	0.104%	0.104%	0.099%	0.091%	0.092%	0.091%	0.089%	0.091%	0.091%
	128	0.103%	0.089%	0.116%	0.089%	0.089%	0.090%	0.092%	0.089%	0.089%
	256	0.097%	0.084%	0.092%	0.084%	0.084%	0.085%	0.085%	0.084%	0.084%
NRU	64	0.106%	0.101%	0.106%	0.091%	0.106%	0.106%	0.088%	0.091%	0.091%
	128	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%
	256	0.084%	0.084%	0.084%	0.085%	0.084%	0.084%	0.084%	0.084%	0.084%
PLRU	64	0.106%	0.106%	0.106%	0.106%	0.106%	0.126%	0.082%	0.090%	0.091%
	128	0.089%	0.089%	0.089%	0.089%	0.089%	0.144%	0.089%	0.089%	0.089%
	256	0.084%	0.087%	0.084%	0.093%	0.083%	0.091%	0.084%	0.084%	0.087%
SRRIP	64	0.106%	0.106%	0.106%	0.106%	0.106%	0.106%	0.082%	0.090%	0.091%
	128	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%	0.089%
	256	0.084%	0.084%	0.084%	0.084%	0.084%	0.084%	0.084%	0.084%	0.084%

Table A.24 – L2 cache miss rate vs. block size - KM

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	88.120%	82.113%	84.290%	83.512%	86.736%	55.895%	84.241%	73.914%	73.904%
	128	81.180%	87.146%	82.308%	81.339%	82.993%	80.397%	81.367%	81.776%	80.971%
	256	84.277%	83.757%	89.973%	89.801%	80.944%	90.282%	81.151%	89.866%	80.502%
LRU	64	85.850%	85.890%	85.809%	85.825%	85.848%	85.815%	83.976%	84.025%	84.193%
	128	88.666%	88.701%	88.678%	88.693%	88.665%	88.683%	87.032%	88.709%	88.704%
	256	90.731%	90.761%	90.750%	90.750%	90.749%	90.762%	90.021%	90.757%	90.722%
MRU	64	80.015%	89.213%	80.248%	84.161%	83.999%	84.812%	84.007%	84.048%	84.401%
	128	85.422%	88.802%	81.295%	88.475%	88.692%	88.520%	87.578%	88.685%	88.795%
	256	80.590%	90.754%	83.885%	90.625%	90.707%	90.233%	90.341%	90.664%	90.746%
NRU	64	85.852%	85.253%	85.852%	84.308%	85.871%	85.827%	83.762%	84.270%	84.249%
	128	88.694%	88.774%	88.771%	88.691%	88.715%	88.681%	88.663%	88.694%	88.690%
	256	90.644%	90.775%	90.740%	90.745%	90.743%	90.704%	90.700%	90.686%	90.703%
PLRU	64	85.825%	85.627%	85.828%	85.848%	85.818%	86.993%	82.840%	83.916%	84.066%
	128	88.720%	88.591%	88.677%	88.570%	88.714%	70.987%	88.657%	88.634%	88.477%
	256	90.739%	87.076%	90.746%	82.448%	90.480%	82.266%	90.712%	90.635%	88.241%
SRRIP	64	85.864%	85.873%	85.828%	85.814%	85.813%	85.836%	82.834%	84.064%	84.250%
	128	88.707%	88.715%	88.567%	88.680%	88.663%	88.716%	88.691%	88.671%	88.675%
	256	90.742%	90.753%	90.739%	90.730%	90.757%	90.688%	90.744%	90.716%	90.748%

Table A.25 – L1 cache miss rate vs. block size - LU

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	4.731%	4.826%	4.777%	4.691%	4.784%	4.881%	4.942%	4.869%	4.854%
	128	3.669%	3.558%	3.651%	3.595%	3.571%	3.584%	3.586%	3.566%	3.632%
	256	2.714%	2.529%	2.669%	2.764%	2.552%	2.855%	2.738%	2.869%	2.782%
LRU	64	6.127%	6.061%	6.111%	6.118%	6.104%	6.129%	6.057%	6.088%	6.079%
	128	3.782%	3.789%	3.785%	3.796%	3.799%	3.797%	3.735%	3.832%	3.821%
	256	2.532%	2.538%	2.517%	2.551%	2.534%	2.539%	2.660%	2.537%	2.545%
MRU	64	4.963%	6.060%	4.959%	5.076%	6.049%	4.942%	4.886%	5.212%	5.188%
	128	3.689%	3.790%	3.698%	3.364%	3.804%	3.346%	3.363%	3.462%	3.466%
	256	2.735%	2.496%	2.866%	2.357%	2.578%	2.284%	2.419%	2.521%	2.389%
NRU	64	6.157%	5.807%	6.122%	6.089%	6.136%	6.131%	6.098%	6.087%	6.080%
	128	3.819%	3.804%	3.813%	3.800%	3.788%	3.780%	3.804%	3.792%	3.775%
	256	2.542%	2.527%	2.520%	2.525%	2.519%	2.527%	2.535%	2.640%	2.529%
PLRU	64	6.109%	5.905%	6.124%	4.591%	6.094%	4.707%	5.602%	5.487%	5.557%
	128	3.806%	3.774%	3.805%	3.441%	3.742%	3.727%	3.877%	3.762%	3.869%
	256	2.524%	2.510%	2.532%	2.775%	2.564%	2.744%	2.786%	2.719%	2.781%
SRRIP	64	6.128%	6.048%	6.045%	6.042%	6.026%	6.119%	5.986%	6.085%	6.062%
	128	3.791%	3.745%	3.726%	3.727%	3.725%	3.796%	3.717%	3.707%	3.716%
	256	2.675%	2.547%	2.509%	2.536%	2.627%	2.539%	2.542%	2.516%	2.553%

Table A.26 – L2 cache miss rate vs. block size - LU

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	23.405%	22.016%	22.712%	31.539%	22.023%	27.542%	24.294%	28.947%	29.800%
	128	29.333%	26.943%	29.227%	35.020%	27.391%	31.087%	30.926%	34.318%	35.534%
	256	40.642%	35.760%	39.526%	41.938%	37.365%	40.631%	39.002%	42.035%	41.556%
LRU	64	11.884%	11.541%	11.888%	12.054%	11.770%	12.150%	10.871%	11.088%	11.100%
	128	17.898%	17.844%	17.959%	17.925%	17.908%	18.018%	25.437%	18.504%	18.547%
	256	27.602%	27.623%	27.311%	27.677%	27.165%	27.725%	35.169%	27.560%	27.582%
MRU	64	38.818%	12.016%	38.504%	32.905%	12.394%	33.013%	30.903%	28.222%	28.390%
	128	42.301%	18.520%	42.494%	34.990%	19.508%	35.221%	33.389%	31.673%	31.254%
	256	46.277%	29.992%	47.504%	40.779%	31.774%	40.522%	40.754%	40.201%	37.845%
NRU	64	12.119%	11.586%	11.911%	11.425%	12.190%	12.011%	11.184%	11.213%	11.491%
	128	18.575%	18.186%	18.189%	18.019%	17.937%	17.673%	18.008%	18.027%	17.661%
	256	27.374%	27.258%	27.219%	27.573%	27.313%	27.393%	27.322%	29.709%	27.468%
PLRU	64	11.887%	15.722%	11.852%	31.925%	12.289%	29.981%	23.833%	23.388%	26.250%
	128	17.833%	21.602%	17.974%	36.698%	18.062%	34.056%	30.162%	29.617%	32.212%
	256	27.328%	32.023%	27.207%	41.348%	28.116%	41.680%	37.524%	36.919%	39.420%
SRRIP	64	11.960%	11.025%	11.112%	11.064%	10.998%	12.130%	9.915%	10.613%	10.611%
	128	17.983%	16.877%	16.755%	16.499%	16.358%	18.016%	16.496%	16.088%	16.199%
	256	30.112%	27.576%	26.560%	27.532%	28.901%	27.515%	27.269%	26.634%	27.629%

Table A.27 – L1 cache miss rate vs. block size - TSP

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	128	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
	256	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
LRU	64	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	128	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
	256	0.003%	0.003%	0.002%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
MRU	64	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	128	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
	256	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
NRU	64	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	128	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
	256	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
PLRU	64	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	128	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
	256	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
SRRIP	64	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%
	128	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%	0.003%
	256	0.003%	0.003%	0.002%	0.003%	0.003%	0.003%	0.002%	0.003%	0.003%

Table A.28 – L2 cache miss rate vs. block size - TSP

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	64	93.304%	93.238%	92.908%	93.642%	92.073%	93.046%	88.416%	96.211%	95.092%
	128	94.204%	94.143%	89.223%	94.719%	88.985%	93.908%	93.531%	94.752%	94.245%
	256	90.849%	91.718%	91.061%	88.029%	91.602%	90.419%	91.422%	92.282%	91.679%
LRU	64	92.943%	93.738%	92.793%	93.319%	92.845%	93.489%	95.768%	95.598%	95.120%
	128	93.973%	94.846%	89.125%	93.962%	94.540%	94.410%	93.069%	94.304%	94.320%
	256	91.318%	90.632%	91.040%	91.367%	91.617%	91.451%	91.239%	90.388%	90.686%
MRU	64	96.267%	88.673%	94.658%	95.430%	94.850%	95.327%	95.330%	95.894%	95.456%
	128	94.272%	94.224%	93.786%	93.881%	94.688%	94.264%	94.323%	93.731%	93.132%
	256	91.767%	91.570%	91.152%	91.117%	91.239%	91.894%	91.505%	91.194%	91.378%
NRU	64	93.051%	92.996%	92.427%	93.275%	93.545%	93.343%	95.413%	95.552%	95.575%
	128	94.734%	94.156%	93.542%	93.735%	94.560%	93.810%	93.546%	94.368%	93.915%
	256	91.001%	91.153%	88.054%	91.132%	91.291%	91.798%	91.274%	91.449%	90.101%
PLRU	64	93.178%	93.175%	92.858%	93.404%	93.046%	92.527%	96.212%	95.148%	96.361%
	128	93.748%	93.756%	89.190%	93.822%	94.948%	93.983%	93.961%	94.241%	93.389%
	256	91.520%	91.380%	90.649%	91.642%	88.762%	91.838%	91.183%	91.804%	90.078%
SRRIP	64	92.819%	93.273%	93.058%	93.238%	93.205%	93.037%	96.585%	95.285%	95.577%
	128	93.466%	94.248%	94.086%	93.368%	94.227%	89.080%	88.858%	94.481%	94.014%
	256	91.816%	91.479%	91.949%	91.417%	91.539%	90.626%	90.733%	91.196%	91.040%

Table A.29 – L2 cache miss rate vs. sharing strategy - FAST

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Private	99.289%	99.201%	99.391%	94.725%	99.390%	99.467%	98.926%	99.266%	99.153%
	Shared	0.297%	15.720%	0.362%	0.384%	15.724%	27.023%	16.144%	26.765%	26.902%
LRU	Private	99.809%	99.825%	99.797%	99.826%	99.826%	99.834%	99.573%	99.761%	99.832%
	Shared	0.020%	0.068%	0.023%	0.111%	0.090%	0.048%	0.042%	0.104%	0.116%
MRU	Private	99.662%	99.824%	99.680%	99.750%	99.814%	99.674%	99.562%	99.813%	99.770%
	Shared	7.828%	0.462%	5.225%	11.140%	0.242%	4.239%	0.791%	0.432%	0.485%
NRU	Private	99.809%	99.692%	99.795%	99.704%	99.798%	99.810%	99.647%	99.735%	99.825%
	Shared	0.034%	0.024%	0.030%	0.039%	0.014%	0.031%	0.034%	0.018%	0.023%
PLRU	Private	99.885%	99.703%	99.798%	99.837%	99.869%	99.691%	99.707%	99.731%	99.713%
	Shared	0.018%	0.226%	0.020%	0.147%	0.395%	0.256%	0.075%	0.195%	3.554%
SRRIP	Private	99.821%	99.692%	99.797%	99.704%	99.811%	99.870%	99.685%	99.795%	99.780%
	Shared	0.036%	0.024%	0.051%	0.039%	0.047%	0.061%	0.056%	0.053%	0.047%

Table A.30 – L2 cache miss rate vs. sharing strategy - FN

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Private	94.059%	94.343%	94.177%	94.271%	94.281%	94.239%	96.288%	96.449%	96.233%
	Shared	9.827%	9.790%	9.666%	9.814%	9.651%	9.941%	9.673%	9.856%	9.689%
LRU	Private	94.311%	94.294%	94.420%	94.438%	94.481%	94.349%	96.354%	95.390%	96.478%
	Shared	9.580%	9.714%	9.613%	9.864%	9.916%	9.612%	9.842%	9.735%	9.674%
MRU	Private	95.821%	95.926%	96.314%	96.360%	96.327%	96.534%	96.698%	95.970%	96.094%
	Shared	9.700%	9.626%	9.795%	9.740%	9.762%	9.794%	9.666%	9.645%	9.940%
NRU	Private	94.040%	95.232%	94.428%	94.201%	94.284%	94.385%	96.646%	95.730%	96.094%
	Shared	9.717%	9.659%	9.640%	9.844%	9.205%	9.718%	9.617%	9.881%	9.795%
PLRU	Private	94.069%	94.536%	94.204%	94.268%	94.502%	94.383%	96.873%	96.313%	96.363%
	Shared	9.878%	9.712%	9.718%	9.762%	9.564%	9.922%	9.666%	9.775%	9.873%
SRRIP	Private	94.434%	94.445%	94.348%	94.290%	94.503%	94.367%	96.724%	96.129%	96.318%
	Shared	9.699%	9.848%	9.859%	9.689%	9.655%	9.762%	9.856%	9.795%	9.785%

Table A.31 – L2 cache miss rate vs. sharing strategy - GF

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Private	95.507%	95.534%	95.422%	95.613%	95.557%	95.634%	95.958%	95.502%	95.356%
	Shared	36.053%	34.571%	32.852%	32.597%	35.026%	32.423%	34.235%	32.441%	36.585%
LRU	Private	95.417%	95.543%	95.535%	95.569%	95.561%	95.504%	96.066%	95.815%	95.536%
	Shared	32.922%	34.546%	36.329%	33.499%	33.396%	34.098%	36.416%	32.207%	35.995%
MRU	Private	95.729%	95.323%	95.377%	95.501%	95.503%	95.573%	96.062%	95.650%	95.759%
	Shared	34.073%	32.132%	38.550%	34.426%	33.449%	34.545%	37.011%	33.151%	32.843%
NRU	Private	95.513%	95.537%	95.598%	95.540%	95.505%	95.511%	95.852%	95.826%	95.537%
	Shared	35.780%	35.474%	34.485%	37.313%	28.096%	33.189%	35.130%	33.851%	33.618%
PLRU	Private	95.450%	95.551%	95.536%	95.553%	95.494%	95.524%	96.105%	95.600%	95.474%
	Shared	36.286%	35.250%	35.764%	34.744%	36.726%	33.990%	33.998%	33.226%	32.808%
SRRIP	Private	95.526%	95.484%	95.556%	95.634%	95.540%	95.459%	95.732%	95.388%	95.522%
	Shared	35.137%	35.468%	33.999%	33.833%	32.407%	34.779%	34.793%	35.044%	35.425%

Table A.32 – L2 cache miss rate vs. sharing strategy - IS

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Private	55.364%	60.134%	55.815%	60.544%	58.788%	55.964%	53.678%	59.107%	60.178%
	Shared	20.229%	27.844%	19.882%	29.343%	25.582%	20.500%	20.151%	22.703%	22.871%
LRU	Private	51.673%	53.871%	51.666%	53.711%	52.184%	49.461%	49.988%	54.262%	53.851%
	Shared	16.258%	16.286%	15.986%	16.317%	16.295%	16.261%	16.271%	16.295%	16.308%
MRU	Private	59.077%	79.069%	59.090%	55.632%	71.666%	59.067%	59.072%	78.526%	78.568%
	Shared	35.104%	21.523%	35.251%	32.131%	20.880%	35.205%	35.127%	22.175%	21.946%
NRU	Private	51.325%	53.812%	51.083%	55.622%	52.785%	51.986%	51.602%	53.305%	53.673%
	Shared	16.212%	15.666%	16.502%	17.614%	15.939%	16.497%	16.507%	16.755%	16.741%
PLRU	Private	49.923%	58.385%	50.791%	56.724%	48.749%	58.613%	50.477%	56.028%	56.086%
	Shared	16.326%	24.805%	16.326%	24.913%	21.224%	25.773%	16.350%	21.368%	21.803%
SRRIP	Private	55.191%	49.553%	48.826%	47.072%	49.556%	49.556%	48.069%	47.678%	47.667%
	Shared	16.902%	16.267%	16.282%	16.002%	16.271%	16.286%	16.003%	16.285%	16.270%

Table A.33 – L2 cache miss rate vs. sharing strategy - KM

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Private	88.120%	82.113%	84.290%	83.512%	86.736%	55.895%	84.241%	73.914%	73.904%
	Shared	0.295%	0.291%	0.292%	0.296%	0.297%	0.291%	0.297%	0.295%	0.296%
LRU	Private	85.850%	85.890%	85.809%	85.825%	85.848%	85.815%	83.976%	84.025%	84.193%
	Shared	0.291%	0.293%	0.294%	0.297%	0.296%	0.295%	0.296%	0.296%	0.296%
MRU	Private	80.015%	89.213%	80.248%	84.161%	83.999%	84.812%	84.007%	84.048%	84.401%
	Shared	0.295%	0.292%	0.290%	0.291%	0.296%	0.294%	0.295%	0.298%	0.298%
NRU	Private	85.852%	85.253%	85.852%	84.308%	85.871%	85.827%	83.762%	84.270%	84.249%
	Shared	0.294%	0.250%	0.290%	0.296%	0.252%	0.293%	0.293%	0.293%	0.296%
PLRU	Private	85.825%	85.627%	85.828%	85.848%	85.818%	86.993%	82.840%	83.916%	84.066%
	Shared	0.293%	0.293%	0.290%	0.293%	0.295%	0.293%	0.293%	0.293%	0.293%
SRRIP	Private	85.864%	85.873%	85.828%	85.814%	85.813%	85.836%	82.834%	84.064%	84.250%
	Shared	0.295%	0.290%	0.294%	0.296%	0.295%	0.293%	0.291%	0.292%	0.297%

Table A.34 – L2 cache miss rate vs. sharing strategy - LU

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Private	23.405%	22.016%	22.712%	31.539%	22.023%	27.542%	24.294%	28.947%	29.800%
	Shared	0.552%	0.552%	0.553%	0.552%	0.551%	0.552%	0.551%	0.552%	0.552%
LRU	Private	11.884%	11.541%	11.888%	12.054%	11.770%	12.150%	10.871%	11.088%	11.100%
	Shared	0.551%	0.552%	0.551%	0.552%	0.551%	0.553%	0.550%	0.552%	0.552%
MRU	Private	38.818%	12.016%	38.504%	32.905%	12.394%	33.013%	30.903%	28.222%	28.390%
	Shared	0.552%	0.552%	0.551%	0.551%	0.553%	0.552%	0.552%	0.552%	0.552%
NRU	Private	12.119%	11.586%	11.911%	11.425%	12.190%	12.011%	11.184%	11.213%	11.491%
	Shared	0.552%	0.550%	0.551%	0.550%	0.550%	0.552%	0.552%	0.552%	0.552%
PLRU	Private	11.887%	15.722%	11.852%	31.925%	12.289%	29.981%	23.833%	23.388%	26.250%
	Shared	0.553%	0.550%	0.552%	0.552%	0.552%	0.552%	0.550%	0.552%	0.552%
SRRIP	Private	11.960%	11.025%	11.112%	11.064%	10.998%	12.130%	9.915%	10.613%	10.611%
	Shared	0.551%	0.553%	0.551%	0.552%	0.552%	0.552%	0.550%	0.550%	0.553%

Table A.35 – L2 cache miss rate vs. sharing strategy - TSP

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Private	93.304%	93.238%	92.908%	93.642%	92.073%	93.046%	88.416%	96.211%	95.092%
	Shared	19.610%	19.471%	19.602%	19.651%	19.488%	19.680%	19.447%	19.408%	19.566%
LRU	Private	92.943%	93.738%	92.793%	93.319%	92.845%	93.489%	95.768%	95.598%	95.120%
	Shared	19.664%	19.634%	19.599%	19.672%	19.578%	19.747%	19.357%	19.211%	19.497%
MRU	Private	96.267%	88.673%	94.658%	95.430%	94.850%	95.327%	95.330%	95.894%	95.456%
	Shared	19.695%	19.600%	19.383%	19.581%	19.744%	19.383%	19.582%	19.582%	19.711%
NRU	Private	93.051%	92.996%	92.427%	93.275%	93.545%	93.343%	95.413%	95.552%	95.575%
	Shared	19.452%	19.492%	19.438%	19.965%	17.356%	19.596%	19.396%	19.564%	19.684%
PLRU	Private	93.178%	93.175%	92.858%	93.404%	93.046%	92.527%	96.212%	95.148%	96.361%
	Shared	19.414%	19.506%	19.795%	19.459%	19.621%	19.558%	19.369%	19.441%	19.415%
SRRIP	Private	92.819%	93.273%	93.058%	93.238%	93.205%	93.037%	96.585%	95.285%	95.577%
	Shared	19.339%	19.740%	19.739%	19.542%	19.570%	19.644%	19.481%	19.536%	19.758%

Table A.36 – L1 cache miss rate - Overall comparison - FAST

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	9.114%	9.122%	9.089%	9.359%	9.092%	9.082%	**8.104%**	8.936%	8.952%
LRU	9.058%	9.079%	9.048%	9.080%	9.071%	9.020%	**7.889%**	8.880%	8.900%
MRU	8.878%	8.896%	8.886%	8.904%	8.815%	8.885%	**7.771%**	8.899%	8.882%
NRU	9.059%	8.436%	9.047%	8.491%	9.053%	9.059%	**7.952%**	8.712%	8.903%
PLRU	9.097%	9.074%	9.049%	9.092%	9.092%	9.105%	**8.372%**	8.904%	8.893%
SRRIP	9.073%	8.436%	9.048%	8.491%	9.055%	9.082%	**8.386%**	8.585%	8.885%

Table A.37 – L2 cache miss rate - Overall comparison - FAST

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	99.289%	99.201%	99.391%	**94.725%**	99.390%	99.467%	98.926%	99.266%	99.153%
LRU	99.809%	99.825%	99.797%	99.826%	99.826%	99.834%	**99.573%**	99.761%	99.832%
MRU	99.662%	99.824%	99.680%	99.750%	99.814%	99.674%	**99.562%**	99.813%	99.770%
NRU	99.809%	99.692%	99.795%	99.704%	99.798%	99.810%	**99.647%**	99.735%	99.825%
PLRU	99.885%	99.703%	99.798%	99.837%	99.869%	**99.691%**	99.707%	99.731%	99.713%
SRRIP	99.821%	99.692%	99.797%	99.704%	99.811%	99.870%	**99.685%**	99.795%	99.780%

Table A.38 – L1 cache miss rate - Overall comparison - FN

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	0.168%	0.168%	0.169%	0.168%	0.168%	0.168%	**0.151%**	0.153%	0.152%
LRU	0.169%	0.168%	0.167%	0.167%	0.167%	0.168%	**0.151%**	0.156%	0.153%
MRU	0.155%	0.154%	0.156%	0.156%	0.153%	0.152%	**0.152%**	0.154%	0.154%
NRU	0.168%	0.163%	0.167%	0.168%	0.168%	0.167%	**0.151%**	0.156%	0.155%
PLRU	0.170%	0.167%	0.169%	0.168%	0.167%	0.168%	**0.150%**	0.153%	0.154%
SRRIP	0.168%	0.168%	0.168%	0.168%	0.167%	0.169%	**0.151%**	0.154%	0.155%

Table A.39 – L2 cache miss rate - Overall comparison - FN

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	**94.059%**	94.343%	94.177%	94.271%	94.281%	94.239%	96.288%	96.449%	96.233%
LRU	94.311%	**94.294%**	94.420%	94.438%	94.481%	94.349%	96.354%	95.390%	96.478%
MRU	**95.821%**	95.926%	96.314%	96.360%	96.327%	96.534%	96.698%	95.970%	96.094%
NRU	**94.040%**	95.232%	94.428%	94.201%	94.284%	94.385%	96.646%	95.730%	96.094%
PLRU	**94.069%**	94.536%	94.204%	94.268%	94.502%	94.383%	96.873%	96.313%	96.363%
SRRIP	94.434%	94.445%	94.348%	**94.290%**	94.503%	94.367%	96.724%	96.129%	96.318%

Table A.40 – L1 cache miss rate - Overall comparison - GF

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	0.763%	0.762%	0.767%	0.757%	0.759%	0.758%	**0.492%**	0.552%	0.577%
LRU	0.759%	0.755%	0.762%	0.753%	0.754%	0.761%	**0.446%**	0.504%	0.548%
MRU	0.509%	0.537%	0.604%	0.558%	0.570%	0.534%	**0.447%**	0.555%	0.526%
NRU	0.756%	0.770%	0.753%	0.761%	0.756%	0.756%	**0.488%**	0.519%	0.545%
PLRU	0.763%	0.759%	0.759%	0.753%	0.762%	0.758%	**0.455%**	0.542%	0.514%
SRRIP	0.754%	0.760%	0.753%	0.754%	0.764%	0.764%	**0.478%**	0.565%	0.586%

Table A.41 – L2 cache miss rate - Overall comparison - GF

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	95.507%	95.534%	95.422%	95.613%	95.557%	95.634%	95.958%	95.502%	**95.356%**
LRU	**95.417%**	95.543%	95.535%	95.569%	95.561%	95.504%	96.066%	95.815%	95.536%
MRU	95.729%	**95.323%**	95.377%	95.501%	95.503%	95.573%	96.062%	95.650%	95.759%
NRU	95.513%	95.537%	95.598%	95.540%	**95.505%**	95.511%	95.852%	95.826%	95.537%
PLRU	**95.450%**	95.551%	95.536%	95.553%	95.494%	95.524%	96.105%	95.600%	95.474%
SRRIP	95.526%	95.484%	95.556%	95.634%	95.540%	95.459%	95.732%	**95.388%**	95.522%

Table A.42 – L1 cache miss rate - Overall comparison - IS

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	9.088%	8.879%	8.900%	9.151%	9.078%	**8.742%**	8.900%	9.138%	9.151%
LRU	8.619%	8.505%	8.616%	8.505%	8.620%	**8.504%**	8.669%	8.685%	8.671%
MRU	8.799%	9.034%	8.808%	**8.491%**	8.810%	8.795%	8.814%	8.904%	8.921%
NRU	8.526%	9.697%	**8.510%**	8.772%	8.531%	8.550%	8.674%	8.686%	8.700%
PLRU	**8.502%**	8.634%	8.549%	8.664%	8.556%	8.715%	8.675%	8.657%	8.653%
SRRIP	8.623%	8.621%	8.584%	**8.512%**	8.619%	8.612%	8.682%	8.675%	8.670%

Table A.43 – L2 cache miss rate - Overall comparison - IS

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	55.364%	60.134%	55.815%	60.544%	58.788%	55.964%	**53.678%**	59.107%	60.178%
LRU	51.673%	53.871%	51.666%	53.711%	52.184%	**49.461%**	49.988%	54.262%	53.851%
MRU	59.077%	79.069%	59.090%	**55.632%**	71.666%	59.067%	59.072%	78.526%	78.568%
NRU	51.325%	53.812%	**51.083%**	55.622%	52.785%	51.986%	51.602%	53.305%	53.673%
PLRU	49.923%	58.385%	50.791%	56.724%	**48.749%**	58.613%	50.477%	56.028%	56.086%
SRRIP	55.191%	49.553%	48.826%	**47.072%**	49.556%	49.556%	48.069%	47.678%	47.667%

Table A.44 – L1 cache miss rate - Overall comparison - KM

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	0.119%	0.112%	0.110%	0.108%	0.123%	0.374%	**0.105%**	0.126%	0.117%
LRU	0.106%	0.106%	0.106%	0.106%	0.106%	0.106%	**0.089%**	0.090%	0.091%
MRU	0.104%	0.104%	0.099%	0.091%	0.092%	0.091%	**0.089%**	0.091%	0.091%
NRU	0.106%	0.101%	0.106%	0.091%	0.106%	0.106%	**0.088%**	0.091%	0.091%
PLRU	0.106%	0.106%	0.106%	0.106%	0.106%	0.126%	**0.082%**	0.090%	0.091%
SRRIP	0.106%	0.106%	0.106%	0.106%	0.106%	0.106%	**0.082%**	0.090%	0.091%

Table A.45 – L2 cache miss rate - Overall comparison - KM

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	88.120%	82.113%	84.290%	83.512%	86.736%	**55.895%**	84.241%	73.914%	73.904%
LRU	85.850%	85.890%	85.809%	85.825%	85.848%	85.815%	**83.976%**	84.025%	84.193%
MRU	**80.015%**	89.213%	80.248%	84.161%	83.999%	84.812%	84.007%	84.048%	84.401%
NRU	85.852%	85.253%	85.852%	84.308%	85.871%	85.827%	**83.762%**	84.270%	84.249%
PLRU	85.825%	85.627%	85.828%	85.848%	85.818%	86.993%	**82.840%**	83.916%	84.066%
SRRIP	85.864%	85.873%	85.828%	85.814%	85.813%	85.836%	**82.834%**	84.064%	84.250%

Table A.46 – L1 cache miss rate - Overall comparison - LU

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4.731%	4.826%	4.777%	**4.691%**	4.784%	4.881%	4.942%	4.869%	4.854%
LRU	6.127%	6.061%	6.111%	6.118%	6.104%	6.129%	**6.057%**	6.088%	6.079%
MRU	4.963%	6.060%	4.959%	5.076%	6.049%	4.942%	**4.886%**	5.212%	5.188%
NRU	6.157%	**5.807%**	6.122%	6.089%	6.136%	6.131%	6.098%	6.087%	6.080%
PLRU	6.109%	5.905%	6.124%	**4.591%**	6.094%	4.707%	5.602%	5.487%	5.557%
SRRIP	6.128%	6.048%	6.045%	6.042%	6.026%	6.119%	**5.986%**	6.085%	6.062%

Table A.47 – L2 cache miss rate - Overall comparison - LU

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	23.405%	**22.016%**	22.712%	31.539%	22.023%	27.542%	24.294%	28.947%	29.800%
LRU	11.884%	11.541%	11.888%	12.054%	11.770%	12.150%	**10.871%**	11.088%	11.100%
MRU	38.818%	**12.016%**	38.504%	32.905%	12.394%	33.013%	30.903%	28.222%	28.390%
NRU	12.119%	11.586%	11.911%	11.425%	12.190%	12.011%	**11.184%**	11.213%	11.491%
PLRU	11.887%	15.722%	**11.852%**	31.925%	12.289%	29.981%	23.833%	23.388%	26.250%
SRRIP	11.960%	11.025%	11.112%	11.064%	10.998%	12.130%	**9.915%**	10.613%	10.611%

Table A.48 – L1 cache miss rate - Overall comparison - TSP

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	**0.005%**	0.005%	0.005%
LRU	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	**0.005%**	0.005%	0.005%
MRU	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	**0.005%**	0.005%	0.005%
NRU	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	**0.005%**
PLRU	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	**0.005%**
SRRIP	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	0.005%	**0.005%**

Table A.49 – L2 cache miss rate - Overall comparison - TSP

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	93.304%	93.238%	92.908%	93.642%	92.073%	93.046%	**88.416%**	96.211%	95.092%
LRU	92.943%	93.738%	**92.793%**	93.319%	92.845%	93.489%	95.768%	95.598%	95.120%
MRU	96.267%	**88.673%**	94.658%	95.430%	94.850%	95.327%	95.330%	95.894%	95.456%
NRU	93.051%	92.996%	**92.427%**	93.275%	93.545%	93.343%	95.413%	95.552%	95.575%
PLRU	93.178%	93.175%	92.858%	93.404%	93.046%	**92.527%**	96.212%	95.148%	96.361%
SRRIP	**92.819%**	93.273%	93.058%	93.238%	93.205%	93.037%	96.585%	95.285%	95.577%

Table A.50 – Coherence operations - FAST

Policy	Operation	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Invalidates	767k	767k	768k	749k	767k	765k	**513k**	694k	696k
	Write-backs	32761k	32761k	32716k	31884k	32715k	32714k	**29204k**	32163k	32170k
LRU	Invalidates	766k	771k	766k	771k	771k	745k	**532k**	707k	694k
	Write-backs	32732k	32809k	32695k	32810k	32779k	32619k	**28617k**	32115k	32208k
MRU	Invalidates	709k	697k	703k	700k	695k	702k	**500k**	696k	700k
	Write-backs	32088k	32193k	32119k	32203k	31887k	32119k	**28208k**	32209k	32135k
NRU	Invalidates	766k	535k	764k	541k	766k	766k	**445k**	684k	697k
	Write-backs	32737k	30644k	32692k	30841k	32710k	32737k	**28935k**	31518k	32220k
PLRU	Invalidates	786k	766k	766k	778k	785k	777k	**503k**	693k	685k
	Write-backs	32880k	32765k	32695k	32854k	32861k	32853k	**30438k**	32191k	32161k
SRRIP	Invalidates	765k	771k	765k	765k	767k	764k	**528k**	616k	701k
	Write-backs	32792k	32789k	32693k	32693k	32724k	32816k	**30467k**	31130k	32146k

Table A.51 – Coherence operations - FN

Policy	Operation	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Invalidates	518	577	534	531	562	555	444	482	**436**
	Write-backs	8782	8706	8813	8692	8666	8705	**8210**	8354	8396
LRU	Invalidates	591	520	480	531	524	542	393	**390**	496
	Write-backs	8744	8704	8688	8645	8647	8681	**8185**	8307	8392
MRU	Invalidates	486	523	548	**441**	460	523	466	488	485
	Write-backs	8355	8382	8441	8335	8354	8412	**8218**	8363	8401
NRU	Invalidates	533	**326**	500	504	541	503	455	438	442
	Write-backs	8811	8474	8677	8685	8730	8678	**8244**	8300	8326
PLRU	Invalidates	636	515	573	524	492	569	**423**	474	471
	Write-backs	8832	8665	8836	8692	8671	8710	**8256**	8299	8370
SRRIP	Invalidates	570	547	562	568	539	646	458	468	**425**
	Write-backs	8723	8674	8677	8726	8662	8751	**8261**	8362	8341

Table A.52 – Coherence operations - GF

Policy	Operation	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Invalidates	206	196	207	194	202	207	**87**	121	122
	Write-backs	278	281	284	274	278	274	**228**	248	255
LRU	Invalidates	194	195	187	165	165	199	**89**	93	98
	Write-backs	282	271	282	265	273	285	**228**	235	242
MRU	Invalidates	118	109	123	118	126	95	**89**	121	96
	Write-backs	246	232	256	241	247	232	**225**	251	242
NRU	Invalidates	191	135	185	202	194	202	**92**	93	111
	Write-backs	274	286	271	274	279	276	**231**	240	240
PLRU	Invalidates	197	206	195	169	200	178	105	**98**	123
	Write-backs	286	275	273	280	279	289	**226**	247	240
SRRIP	Invalidates	165	193	188	162	199	207	**110**	124	128
	Write-backs	277	288	282	271	283	281	**228**	241	252

Table A.53 – Coherence operations - IS

Policy	Operation	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Invalidates	14668	15640	15118	15039	14827	**3129**	14691	14992	14963
	Write-backs	15591	15722	15677	15688	15723	15748	**15085**	15641	15588
LRU	Invalidates	4819	**4278**	4816	4408	4823	4445	12792	15838	16296
	Write-backs	16439	16584	16162	15969	16109	16503	**10593**	15193	15716
MRU	Invalidates	15216	**14734**	14752	16287	16403	15506	14996	16349	15547
	Write-backs	15701	15270	15363	15719	15645	15739	15170	15690	**15125**
NRU	Invalidates	4478	4426	**4259**	16938	4554	4602	16064	15561	16387
	Write-backs	16418	16135	15960	15848	16192	15903	15279	**15073**	15650
PLRU	Invalidates	4421	4502	4704	3460	3619	**3265**	15984	14534	14446
	Write-backs	16436	16439	15929	15903	16410	16342	15254	15496	**15198**
SRRIP	Invalidates	4901	4924	4712	**4171**	4809	4636	16160	16369	15788
	Write-backs	16382	16439	15958	15784	16433	15966	**15297**	15702	15387

Table A.54 – Coherence operations - KM

Policy	Operation	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Invalidates	324k	334k	336k	327k	324k	338k	**253k**	264k	266k
	Write-backs	381k	394k	394k	381k	382k	392k	**301k**	343k	347k
LRU	Invalidates	343k	341k	339k	338k	340k	338k	264k	**262k**	266k
	Write-backs	396k	397k	396k	398k	396k	397k	**339k**	344k	350k
MRU	Invalidates	**263k**	268k	264k	265k	267k	269k	264k	265k	266k
	Write-backs	349k	350k	351k	351k	351k	351k	**337k**	345k	349k
NRU	Invalidates	340k	265k	340k	267k	341k	338k	**261k**	266k	266k
	Write-backs	397k	348k	398k	350k	398k	397k	**333k**	350k	350k
PLRU	Invalidates	338k	340k	339k	340k	339k	335k	**254k**	263k	266k
	Write-backs	397k	395k	397k	397k	397k	396k	**290k**	344k	349k
SRRIP	Invalidates	340k	341k	338k	339k	340k	340k	**253k**	265k	267k
	Write-backs	397k	397k	397k	397k	397k	398k	**290k**	345k	349k

Table A.55 – Coherence operations - LU

Policy	Operation	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Invalidates	74k	70k	**61k**	93k	67k	87k	66k	82k	92k
	Write-backs	251k	250k	249k	251k	249k	251k	**207k**	230k	231k
LRU	Invalidates	150k	131k	151k	157k	143k	161k	**123k**	124k	124k
	Write-backs	258k	252k	258k	257k	258k	258k	**217k**	242k	241k
MRU	Invalidates	95k	122k	**88k**	131k	123k	105k	118k	104k	127k
	Write-backs	241k	249k	239k	244k	247k	240k	**221k**	244k	245k
NRU	Invalidates	156k	125k	151k	126k	161k	151k	132k	**121k**	130k
	Write-backs	259k	262k	258k	239k	259k	258k	**218k**	236k	238k
PLRU	Invalidates	151k	148k	150k	116k	146k	131k	**112k**	118k	114k
	Write-backs	259k	256k	259k	241k	259k	252k	**215k**	239k	236k
SRRIP	Invalidates	150k	104k	108k	107k	98k	141k	**79k**	105k	104k
	Write-backs	259k	253k	252k	252k	253k	253k	**215k**	235k	235k

Table A.56 – Coherence operations - TSP

Policy	Operation	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	Invalidates	3164	3308	3232	3300	3148	3249	**1963**	2282	2247
	Write-backs	2358	2542	2376	2444	2391	2582	2051	2005	**1974**
LRU	Invalidates	3144	3300	3137	3209	3191	3293	**2222**	2260	2238
	Write-backs	2401	2633	2235	2517	2374	2567	**1977**	2042	2085
MRU	Invalidates	2288	**1955**	2284	2240	2207	2341	2195	2271	2204
	Write-backs	2041	2047	2022	2023	2000	2083	**1958**	2071	2045
NRU	Invalidates	3160	3172	3189	3185	3176	3273	**2195**	2218	2228
	Write-backs	2476	2315	2390	2411	2426	2437	**1950**	2043	1975
PLRU	Invalidates	3179	3193	3296	3215	3138	3240	2265	**2216**	2233
	Write-backs	2480	2483	2565	2423	2393	2434	**2014**	2033	2021
SRRIP	Invalidates	3276	3143	3204	3177	3204	3220	2277	**2196**	2226
	Write-backs	2470	2382	2456	2380	2411	2369	2020	**1995**	2008

Table A.57 – Instructions per cycle - FAST

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	13.038	13.046	13.068	7.204	13.039	13.056	**13.239**	13.119	13.107
LRU	13.052	13.040	13.058	13.053	13.060	13.141	**13.262**	13.132	13.124
MRU	13.124	13.114	13.127	13.099	13.126	13.121	**13.286**	13.116	13.115
NRU	13.051	13.230	13.074	13.211	13.061	13.056	**13.270**	13.149	13.124
PLRU	13.085	13.041	13.059	13.054	13.018	13.057	**13.231**	13.120	13.115
SRRIP	13.075	**13.230**	13.058	13.211	13.096	13.156	13.226	13.173	13.123

Table A.58 – Instructions per cycle - FN

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	44.099	44.095	44.098	44.095	44.095	44.096	44.228	44.223	**44.229**
LRU	44.097	44.099	44.101	44.096	44.097	44.095	**44.231**	44.228	44.227
MRU	44.221	44.223	44.224	44.223	44.223	44.226	44.225	44.225	**44.226**
NRU	44.100	43.752	44.101	44.100	44.095	44.099	**44.227**	44.222	44.223
PLRU	44.092	44.097	44.098	44.093	44.100	44.097	**44.232**	44.227	44.223
SRRIP	44.095	44.098	44.095	44.096	44.096	44.094	**44.222**	44.221	44.221

Table A.59 – Instructions per cycle - GF

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	5.659	5.638	5.630	5.672	5.653	5.666	7.008	**7.077**	7.037
LRU	5.643	5.661	5.649	5.655	5.666	5.667	**7.185**	7.100	7.091
MRU	7.121	6.985	7.024	6.909	7.030	7.072	**7.167**	7.087	7.073
NRU	5.660	5.212	5.670	5.666	5.631	5.640	**7.144**	7.107	7.027
PLRU	5.652	5.667	5.647	5.652	5.641	5.650	7.002	**7.112**	7.060
SRRIP	5.660	5.663	5.655	5.655	5.643	5.665	**7.170**	7.080	7.036

Table A.60 – Instructions per cycle - IS

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	1.225	1.067	1.209	1.031	1.110	1.190	**1.278**	1.112	1.068
LRU	1.404	1.393	1.404	1.395	1.389	1.425	**1.437**	1.401	1.407
MRU	1.084	1.086	1.085	**1.304**	1.116	1.084	1.083	1.125	1.122
NRU	1.391	1.259	1.393	**1.430**	1.336	1.385	1.407	1.344	1.339
PLRU	1.420	1.130	1.412	1.187	1.368	1.116	**1.430**	1.209	1.214
SRRIP	1.329	1.445	1.451	1.466	1.445	1.446	1.474	1.478	**1.478**

Table A.61 – Instructions per cycle - KM

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	**30.178**	29.118	28.954	29.001	29.953	21.610	29.990	29.058	29.791
LRU	28.793	28.795	28.800	28.794	28.801	28.798	28.843	**28.845**	28.839
MRU	29.453	**29.962**	29.548	28.844	28.879	28.849	28.845	28.849	28.844
NRU	28.796	**31.680**	28.795	28.841	28.794	28.796	28.849	28.840	28.844
PLRU	28.802	28.804	28.797	28.798	28.798	**29.892**	28.864	28.846	28.841
SRRIP	28.795	28.794	28.800	28.797	28.797	28.795	**28.869**	28.840	28.841

Table A.62 – Instructions per cycle - LU

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	**32.265**	32.082	32.155	31.272	32.223	31.329	31.561	31.200	31.120
LRU	30.744	30.846	30.772	30.793	30.777	30.764	**31.038**	30.954	30.981
MRU	29.327	30.816	29.292	29.102	**30.835**	29.467	29.883	29.642	29.636
NRU	30.782	29.289	30.803	30.734	30.782	30.805	**30.953**	30.764	30.775
PLRU	30.740	30.642	30.744	**31.295**	30.676	31.272	30.282	30.706	30.218
SRRIP	30.801	30.715	30.689	30.688	30.719	30.589	**31.018**	30.767	30.805

Table A.63 – Instructions per cycle - TSP

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	12.077	**12.080**	12.075	12.073	12.079	12.074	12.059	12.054	12.056
LRU	12.080	12.076	12.079	12.077	**12.081**	12.073	12.059	12.056	12.055
MRU	12.054	12.054	12.054	12.053	12.054	12.053	12.054	12.053	**12.056**
NRU	12.076	**13.631**	12.080	12.081	12.076	12.076	12.055	12.055	12.054
PLRU	12.077	**12.081**	12.079	12.072	12.080	12.076	12.054	12.056	12.058
SRRIP	12.079	12.082	12.080	12.078	**12.085**	12.078	12.054	12.054	12.053

Table A.64 – Energy consumption (Joules) - FAST

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	2.878	2.877	2.875	3.919	2.877	2.875	**2.849**	2.873	2.874
LRU	2.875	2.877	2.875	2.875	2.874	2.865	**2.843**	2.870	2.872
MRU	2.871	2.873	2.871	2.874	2.870	2.872	**2.839**	2.873	2.872
NRU	2.875	2.855	2.873	2.857	2.874	2.875	**2.843**	2.866	2.872
PLRU	2.873	2.877	2.874	2.876	2.879	2.875	**2.854**	2.872	2.872
SRRIP	2.873	2.877	2.874	2.874	2.870	2.863	**2.854**	2.862	2.872

Table A.65 – Energy consumption (Joules) - FN

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	0.388	0.388	0.388	0.388	0.388	0.388	0.388	0.388	**0.388**
LRU	0.388	0.388	0.388	0.388	0.388	0.388	0.388	**0.388**	0.388
MRU	0.388	0.388	0.388	0.388	0.388	**0.388**	0.388	0.388	0.388
NRU	0.388	**0.376**	0.388	0.388	0.388	0.388	0.388	0.388	0.388
PLRU	0.388	0.388	0.388	0.388	0.388	0.388	**0.388**	0.388	0.388
SRRIP	0.388	0.388	0.388	0.388	0.388	0.388	0.388	0.388	**0.388**

Table A.66 – Energy consumption (Joules) - GF

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	1.610	1.609	1.609	1.611	1.609	1.610	**1.283**	1.287	1.288
LRU	1.610	1.609	1.610	1.609	1.609	1.611	1.288	**1.286**	1.288
MRU	1.288	1.283	1.289	**1.283**	1.288	1.286	1.286	1.288	1.288
NRU	1.610	1.715	1.610	1.610	1.609	1.610	1.288	1.287	**1.285**
PLRU	1.610	1.610	1.609	1.609	1.609	1.610	**1.282**	1.287	1.286
SRRIP	1.609	1.612	1.610	1.609	1.610	1.610	1.288	**1.287**	1.288

Table A.67 – Energy consumption (Joules) - IS

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4.987	5.611	4.441	5.697	5.447	4.480	**4.235**	4.783	4.943
LRU	3.866	3.900	3.866	3.891	3.902	3.819	**3.811**	3.893	3.882
MRU	4.884	4.874	4.884	**4.157**	4.753	4.884	4.886	4.721	4.737
NRU	3.898	**3.739**	3.894	3.830	4.034	3.909	3.884	4.035	4.045
PLRU	3.833	4.681	3.849	4.480	3.954	4.730	**3.825**	4.434	4.422
SRRIP	4.051	3.773	3.757	3.730	3.772	3.771	3.728	3.722	**3.721**

Table A.68 – Energy consumption (Joules) - KM

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4.898	4.593	**4.550**	4.560	4.908	5.377	4.898	4.937	4.906
LRU	4.509	4.509	4.509	4.508	4.510	4.509	4.499	**4.498**	4.499
MRU	4.662	4.807	4.688	**4.500**	4.510	4.503	4.500	4.502	4.500
NRU	4.509	4.507	4.509	4.499	4.508	4.508	**4.499**	4.499	4.500
PLRU	4.510	4.512	4.509	4.510	4.509	4.910	**4.497**	4.499	4.500
SRRIP	4.509	4.509	4.509	4.509	4.509	4.509	**4.498**	4.498	4.499

Table A.69 – Energy consumption (Joules) - LU

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	1.016	0.996	1.005	1.018	1.002	0.992	**0.972**	0.987	0.991
LRU	0.845	0.847	0.845	0.847	0.845	0.845	0.844	**0.844**	0.845
MRU	0.968	**0.855**	0.964	0.930	0.858	0.947	0.950	0.923	0.924
NRU	0.844	**0.811**	0.844	0.847	0.844	0.844	0.843	0.846	0.848
PLRU	0.845	0.873	**0.845**	1.041	0.849	1.026	0.905	0.923	0.920
SRRIP	0.844	0.848	0.848	0.848	0.849	0.848	0.848	**0.844**	0.846

Table A.70 – Energy consumption (Joules) - TSP

Policy	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	1.006	1.006	1.006	1.006	**1.006**	1.006	1.008	1.009	1.009
LRU	1.006	1.006	1.006	1.006	**1.006**	1.006	1.008	1.009	1.009
MRU	1.009	1.009	1.009	1.009	1.009	1.009	1.009	1.009	**1.009**
NRU	1.006	1.096	1.006	**1.006**	1.006	1.006	1.009	1.009	1.009
PLRU	1.006	1.006	1.006	1.006	**1.006**	1.006	1.009	1.009	1.008
SRRIP	1.006	1.006	1.006	1.006	**1.006**	1.006	1.009	1.009	1.009

APPENDIX B - SHARED L2 CACHE MISS RATE RESULTS

Figure B.1 – L2 cache miss rate vs. associativity - FAST

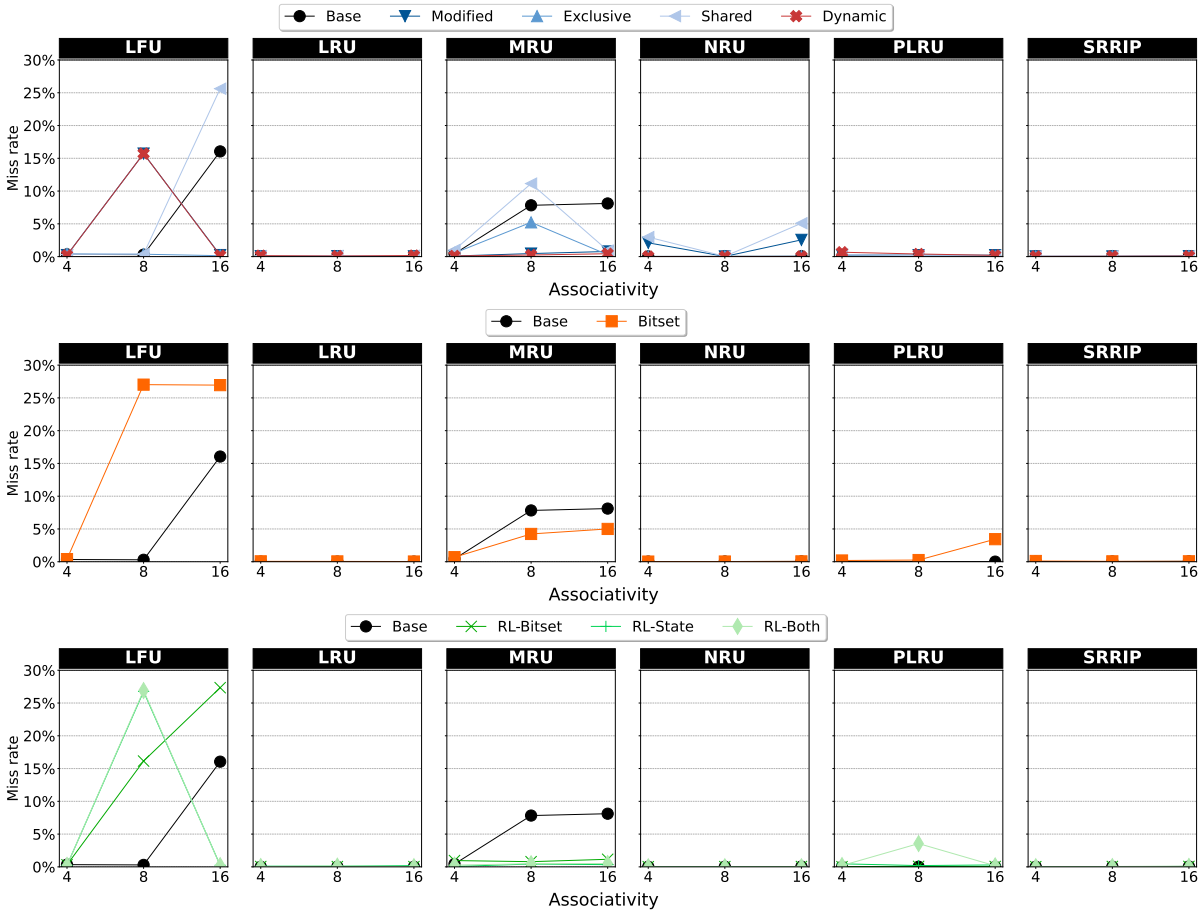


Table B.1 – L2 cache miss rate vs. associativity - FAST

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	0.338%	0.234%	0.416%	0.334%	0.208%	0.400%	0.314%	0.235%	0.231%
	8	0.297%	15.720%	0.362%	0.384%	15.724%	27.023%	16.144%	26.765%	26.902%
	16	16.052%	0.234%	0.138%	25.622%	0.171%	26.948%	27.342%	0.239%	0.195%
LRU	4	0.016%	0.049%	0.018%	0.080%	0.142%	0.061%	0.051%	0.101%	0.052%
	8	0.020%	0.068%	0.023%	0.111%	0.090%	0.048%	0.042%	0.104%	0.116%
	16	0.017%	0.033%	0.017%	0.133%	0.163%	0.031%	0.037%	0.183%	0.034%
MRU	4	0.434%	0.090%	0.542%	1.019%	0.103%	0.696%	0.960%	0.059%	0.268%
	8	7.828%	0.462%	5.225%	11.140%	0.242%	4.239%	0.791%	0.432%	0.485%
	16	8.109%	0.784%	0.288%	0.926%	0.439%	5.004%	1.156%	0.338%	0.509%
NRU	4	0.024%	2.102%	0.025%	2.974%	0.017%	0.023%	0.025%	0.018%	0.018%
	8	0.034%	0.024%	0.030%	0.039%	0.014%	0.031%	0.034%	0.018%	0.023%
	16	0.049%	2.553%	0.058%	5.089%	0.012%	0.054%	0.052%	0.014%	0.028%
PLRU	4	0.021%	0.242%	0.021%	0.310%	0.667%	0.188%	0.025%	0.463%	0.201%
	8	0.018%	0.226%	0.020%	0.147%	0.395%	0.256%	0.075%	0.195%	3.554%
	16	0.016%	0.215%	0.017%	0.113%	0.189%	3.442%	0.056%	0.294%	0.162%
SRRIP	4	0.023%	0.032%	0.031%	0.047%	0.034%	0.112%	0.032%	0.032%	0.031%
	8	0.036%	0.024%	0.051%	0.039%	0.047%	0.061%	0.056%	0.053%	0.047%
	16	0.062%	0.068%	0.063%	0.063%	0.075%	0.065%	0.068%	0.062%	0.067%

Figure B.2 – L2 cache miss rate vs. associativity - FN

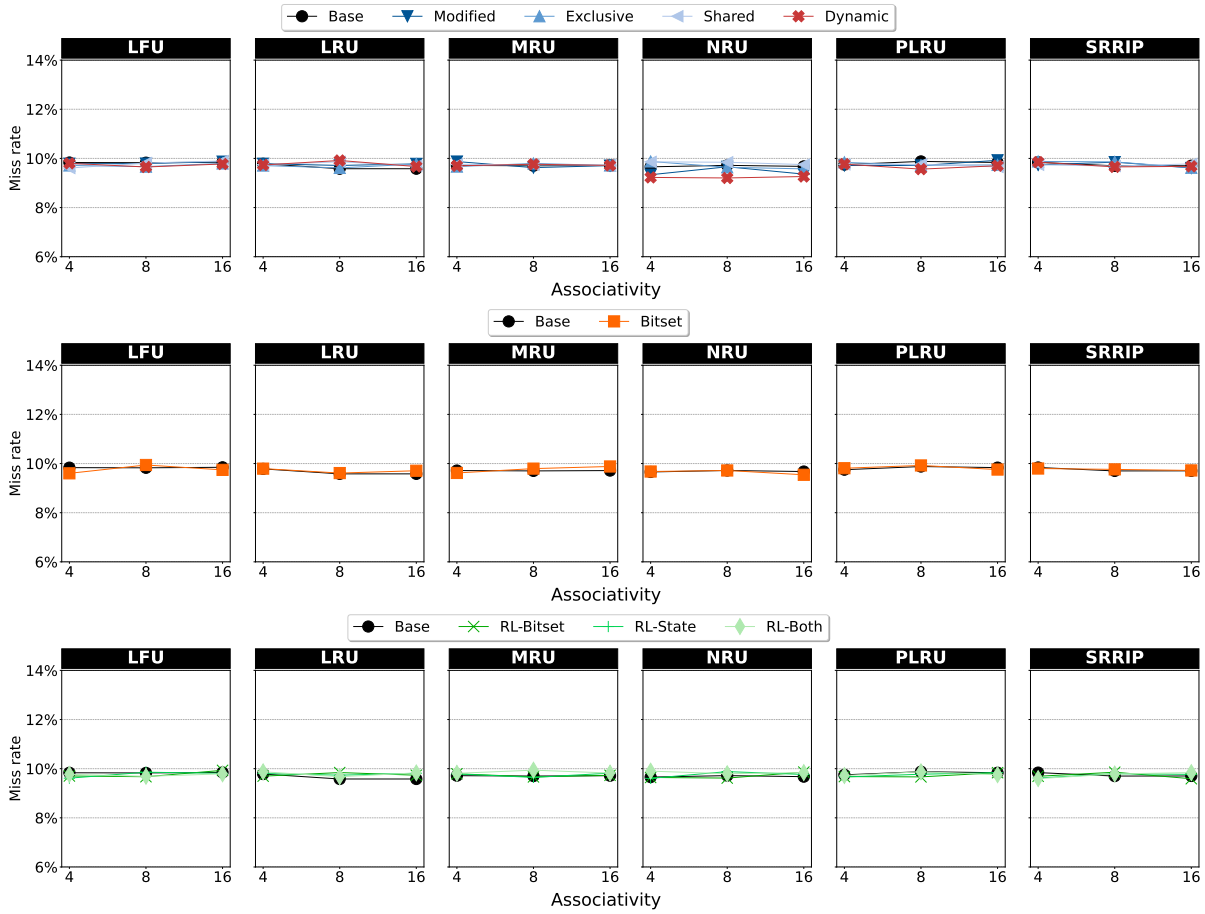


Table B.2 – L2 cache miss rate vs. associativity - FN

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	9.833%	9.785%	9.714%	9.609%	9.799%	9.602%	9.703%	9.614%	9.769%
	8	9.827%	9.790%	9.666%	9.814%	9.651%	9.941%	9.673%	9.856%	9.689%
	16	9.847%	9.869%	9.795%	9.894%	9.773%	9.743%	9.932%	9.813%	9.789%
LRU	4	9.782%	9.783%	9.710%	9.743%	9.733%	9.793%	9.695%	9.804%	9.874%
	8	9.580%	9.714%	9.613%	9.864%	9.916%	9.612%	9.842%	9.735%	9.674%
	16	9.580%	9.777%	9.771%	9.701%	9.669%	9.707%	9.733%	9.812%	9.829%
MRU	4	9.715%	9.872%	9.668%	9.761%	9.699%	9.617%	9.792%	9.787%	9.811%
	8	9.700%	9.626%	9.795%	9.740%	9.762%	9.794%	9.666%	9.645%	9.940%
	16	9.714%	9.706%	9.714%	9.723%	9.716%	9.880%	9.730%	9.818%	9.822%
NRU	4	9.658%	9.335%	9.876%	9.861%	9.227%	9.676%	9.643%	9.606%	9.900%
	8	9.717%	9.659%	9.640%	9.844%	9.205%	9.718%	9.617%	9.881%	9.795%
	16	9.675%	9.361%	9.574%	9.749%	9.263%	9.543%	9.861%	9.764%	9.842%
PLRU	4	9.749%	9.711%	9.841%	9.756%	9.766%	9.810%	9.678%	9.662%	9.720%
	8	9.878%	9.712%	9.718%	9.762%	9.564%	9.922%	9.666%	9.775%	9.873%
	16	9.835%	9.922%	9.736%	9.653%	9.702%	9.756%	9.837%	9.838%	9.754%
SRRIP	4	9.840%	9.737%	9.877%	9.741%	9.847%	9.798%	9.691%	9.620%	9.596%
	8	9.699%	9.848%	9.859%	9.689%	9.655%	9.762%	9.856%	9.795%	9.785%
	16	9.703%	9.611%	9.620%	9.777%	9.677%	9.719%	9.591%	9.767%	9.840%

Figure B.3 – L2 cache miss rate vs. associativity - GF

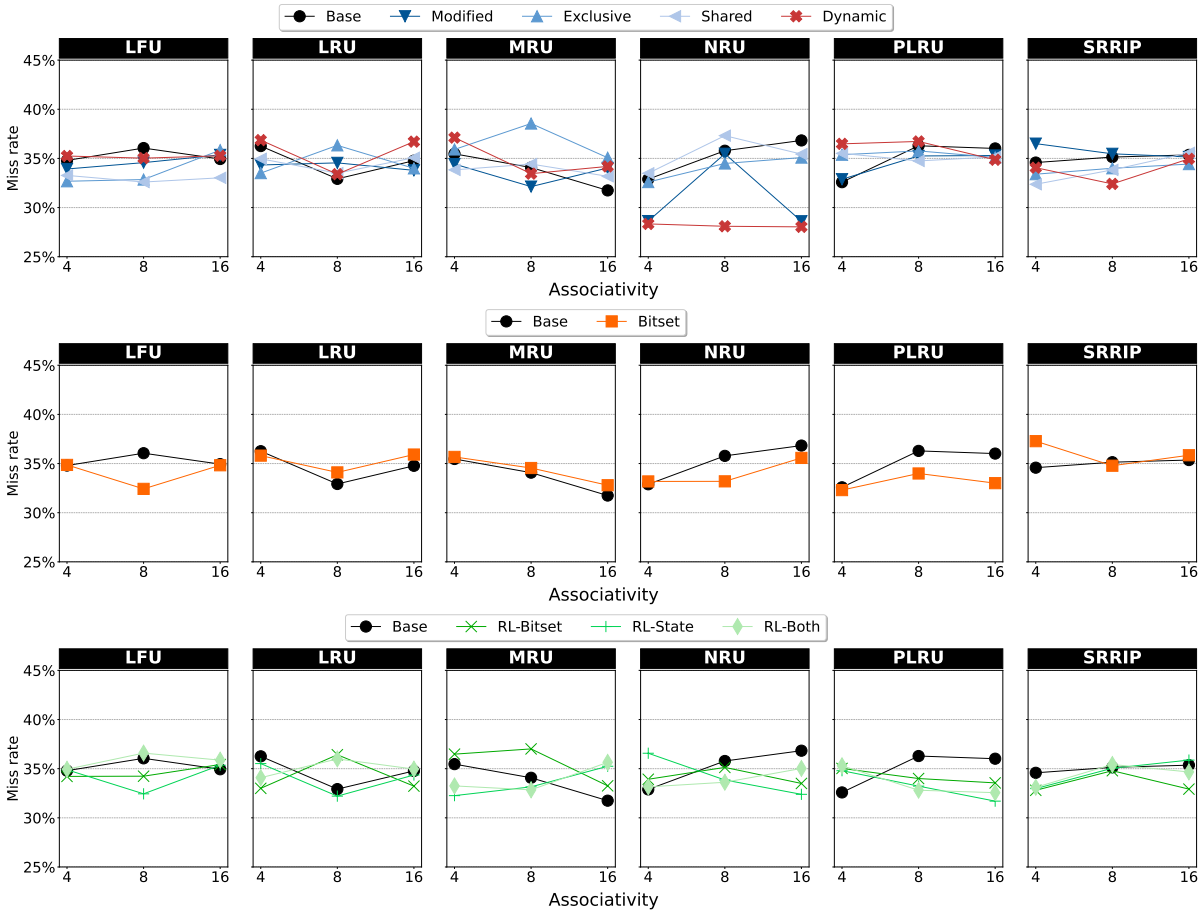


Table B.3 – L2 cache miss rate vs. associativity - GF

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	34.797%	33.925%	32.676%	33.272%	35.244%	34.861%	34.214%	34.890%	34.943%
	8	36.053%	34.571%	32.852%	32.597%	35.026%	32.423%	34.235%	32.441%	36.585%
	16	34.940%	35.357%	35.840%	33.028%	35.265%	34.825%	35.422%	35.353%	35.867%
LRU	4	36.257%	34.332%	33.506%	34.961%	36.860%	35.806%	33.001%	35.517%	34.081%
	8	32.922%	34.546%	36.329%	33.499%	33.396%	34.098%	36.416%	32.207%	35.995%
	16	34.771%	33.787%	33.989%	35.071%	36.706%	35.915%	33.226%	34.355%	34.961%
MRU	4	35.465%	34.426%	35.912%	33.835%	37.112%	35.673%	36.478%	32.258%	33.247%
	8	34.073%	32.132%	38.550%	34.426%	33.449%	34.545%	37.011%	33.151%	32.843%
	16	31.745%	34.030%	35.070%	33.179%	34.186%	32.793%	33.256%	35.247%	35.645%
NRU	4	32.890%	28.625%	32.596%	33.489%	28.338%	33.179%	33.923%	36.570%	33.149%
	8	35.780%	35.474%	34.485%	37.313%	28.096%	33.189%	35.130%	33.851%	33.618%
	16	36.829%	28.608%	35.085%	35.438%	28.037%	35.574%	33.499%	32.390%	35.023%
PLRU	4	32.583%	32.882%	35.375%	35.514%	36.482%	32.300%	35.026%	34.797%	35.299%
	8	36.286%	35.250%	35.764%	34.744%	36.726%	33.990%	33.998%	33.226%	32.808%
	16	36.014%	35.296%	35.101%	35.157%	34.861%	33.004%	33.554%	31.698%	32.559%
SRRIP	4	34.579%	36.503%	33.395%	32.370%	34.057%	37.277%	32.801%	32.971%	33.103%
	8	35.137%	35.468%	33.999%	33.833%	32.407%	34.779%	34.793%	35.044%	35.425%
	16	35.356%	35.145%	34.423%	35.566%	34.957%	35.845%	32.922%	35.873%	34.666%

Figure B.4 – L2 cache miss rate vs. associativity - IS

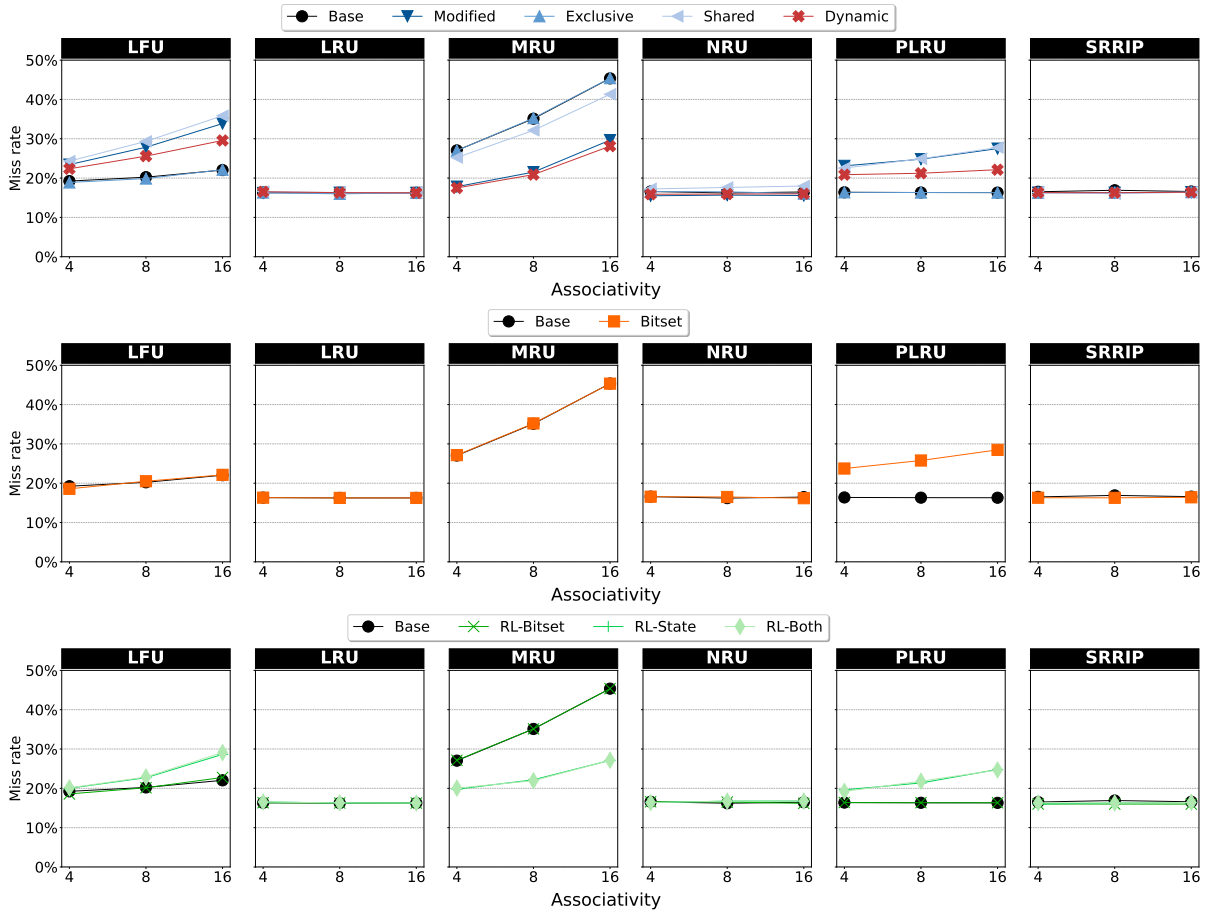


Table B.4 – L2 cache miss rate vs. associativity - IS

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	19.235%	23.425%	18.882%	24.308%	22.361%	18.576%	18.583%	20.035%	20.109%
	8	20.229%	27.844%	19.882%	29.343%	25.582%	20.500%	20.151%	22.703%	22.871%
	16	22.039%	33.860%	22.141%	35.851%	29.566%	22.125%	22.765%	28.628%	29.094%
LRU	4	16.331%	16.220%	16.324%	16.570%	16.511%	16.332%	16.333%	16.530%	16.537%
	8	16.258%	16.286%	15.986%	16.317%	16.295%	16.261%	16.271%	16.295%	16.308%
	16	16.255%	16.253%	16.258%	16.274%	16.263%	16.258%	16.257%	16.263%	16.263%
MRU	4	27.038%	17.819%	26.989%	25.219%	17.488%	27.140%	27.136%	19.789%	20.050%
	8	35.104%	21.523%	35.251%	32.131%	20.880%	35.205%	35.127%	22.175%	21.946%
	16	45.353%	29.649%	45.418%	41.311%	28.121%	45.323%	45.342%	27.116%	27.144%
NRU	4	16.582%	15.501%	16.579%	17.238%	15.884%	16.578%	16.583%	16.380%	16.385%
	8	16.212%	15.666%	16.502%	17.614%	15.939%	16.497%	16.507%	16.755%	16.741%
	16	16.467%	15.564%	16.052%	17.976%	15.999%	16.188%	16.185%	16.779%	16.758%
PLRU	4	16.380%	23.104%	16.380%	22.555%	20.863%	23.734%	16.381%	19.623%	19.216%
	8	16.326%	24.805%	16.326%	24.913%	21.224%	25.773%	16.350%	21.368%	21.803%
	16	16.307%	27.491%	16.303%	27.770%	22.130%	28.483%	16.338%	24.803%	24.669%
SRRIP	4	16.516%	16.304%	16.269%	16.311%	16.308%	16.304%	16.027%	16.023%	16.311%
	8	16.902%	16.267%	16.282%	16.002%	16.271%	16.286%	16.003%	16.285%	16.270%
	16	16.582%	16.417%	16.416%	16.410%	16.410%	16.414%	15.995%	16.412%	16.414%

Figure B.5 – L2 cache miss rate vs. associativity - KM

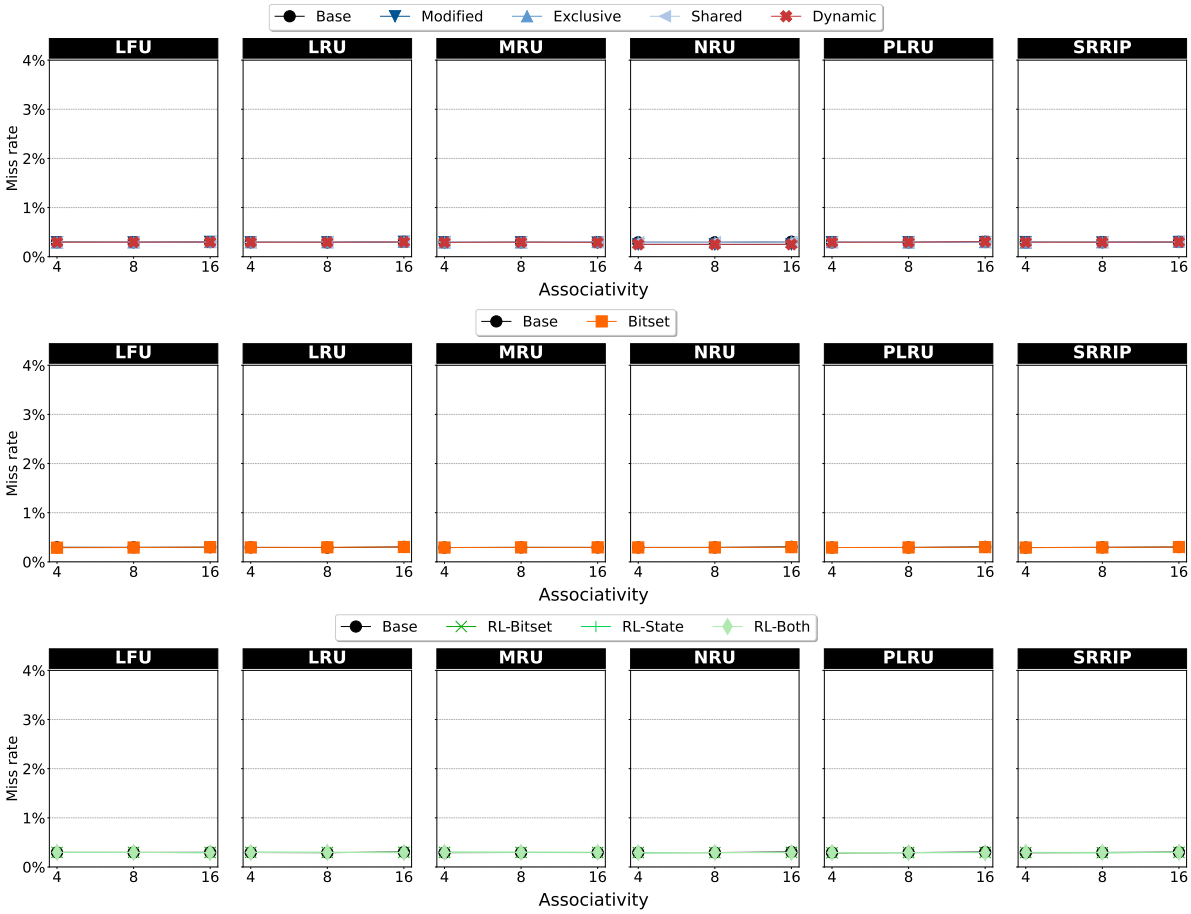


Table B.5 – L2 cache miss rate vs. associativity - KM

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	0.296%	0.295%	0.289%	0.291%	0.296%	0.288%	0.296%	0.295%	0.297%
	8	0.295%	0.291%	0.292%	0.296%	0.297%	0.291%	0.297%	0.295%	0.296%
	16	0.298%	0.303%	0.300%	0.298%	0.299%	0.299%	0.294%	0.292%	0.289%
LRU	4	0.294%	0.292%	0.294%	0.291%	0.295%	0.294%	0.297%	0.297%	0.295%
	8	0.291%	0.293%	0.294%	0.297%	0.296%	0.295%	0.296%	0.296%	0.296%
	16	0.304%	0.305%	0.305%	0.304%	0.300%	0.302%	0.298%	0.292%	0.295%
MRU	4	0.292%	0.293%	0.290%	0.295%	0.292%	0.291%	0.298%	0.297%	0.297%
	8	0.295%	0.292%	0.290%	0.291%	0.296%	0.294%	0.295%	0.298%	0.298%
	16	0.294%	0.288%	0.298%	0.295%	0.293%	0.294%	0.296%	0.295%	0.298%
NRU	4	0.293%	0.251%	0.296%	0.294%	0.251%	0.293%	0.293%	0.290%	0.291%
	8	0.294%	0.250%	0.290%	0.296%	0.252%	0.293%	0.293%	0.293%	0.296%
	16	0.305%	0.250%	0.303%	0.305%	0.252%	0.299%	0.297%	0.294%	0.292%
PLRU	4	0.292%	0.294%	0.296%	0.292%	0.295%	0.292%	0.288%	0.296%	0.294%
	8	0.293%	0.293%	0.290%	0.293%	0.295%	0.293%	0.293%	0.293%	0.293%
	16	0.304%	0.299%	0.297%	0.297%	0.306%	0.299%	0.295%	0.295%	0.290%
SRRIP	4	0.290%	0.298%	0.287%	0.290%	0.295%	0.290%	0.294%	0.295%	0.298%
	8	0.295%	0.290%	0.294%	0.296%	0.295%	0.293%	0.291%	0.292%	0.297%
	16	0.302%	0.301%	0.302%	0.304%	0.300%	0.301%	0.297%	0.296%	0.298%

Figure B.6 – L2 cache miss rate vs. associativity - LU

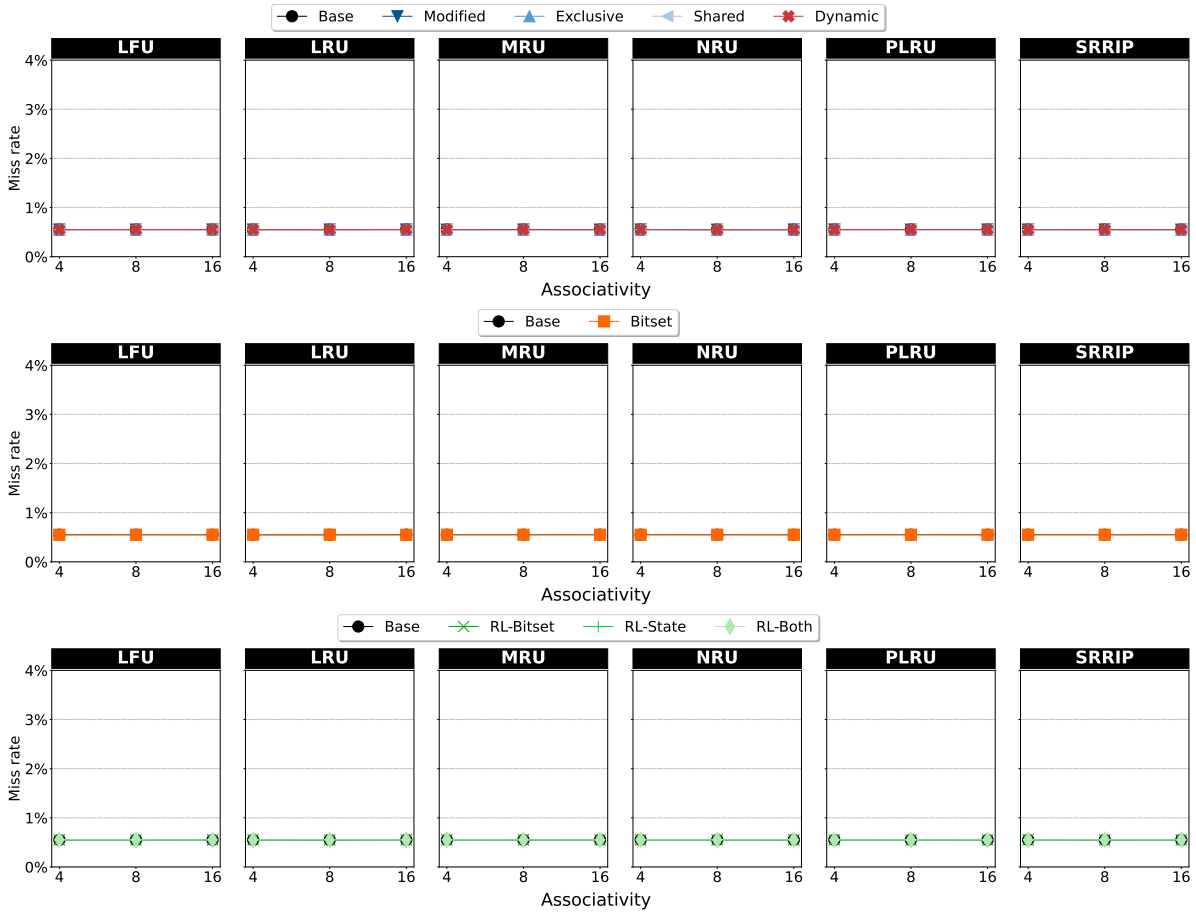


Table B.6 – L2 cache miss rate vs. associativity - LU

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	0.552%	0.552%	0.551%	0.552%	0.551%	0.551%	0.552%	0.549%	0.553%
	8	0.552%	0.552%	0.553%	0.552%	0.551%	0.552%	0.551%	0.552%	0.552%
	16	0.551%	0.552%	0.553%	0.552%	0.552%	0.553%	0.552%	0.552%	0.551%
LRU	4	0.550%	0.552%	0.552%	0.552%	0.551%	0.553%	0.553%	0.550%	0.552%
	8	0.551%	0.552%	0.551%	0.552%	0.551%	0.553%	0.550%	0.552%	0.552%
	16	0.553%	0.553%	0.553%	0.551%	0.551%	0.550%	0.552%	0.552%	0.552%
MRU	4	0.552%	0.550%	0.552%	0.549%	0.552%	0.552%	0.553%	0.551%	0.552%
	8	0.552%	0.552%	0.551%	0.551%	0.553%	0.552%	0.552%	0.552%	0.552%
	16	0.552%	0.551%	0.551%	0.550%	0.551%	0.553%	0.552%	0.550%	0.551%
NRU	4	0.553%	0.552%	0.552%	0.550%	0.550%	0.552%	0.552%	0.552%	0.552%
	8	0.552%	0.550%	0.551%	0.550%	0.550%	0.552%	0.552%	0.552%	0.552%
	16	0.551%	0.553%	0.551%	0.552%	0.548%	0.553%	0.552%	0.552%	0.553%
PLRU	4	0.552%	0.553%	0.553%	0.552%	0.551%	0.552%	0.552%	0.553%	0.552%
	8	0.553%	0.550%	0.552%	0.552%	0.552%	0.552%	0.550%	0.552%	0.552%
	16	0.552%	0.552%	0.552%	0.550%	0.552%	0.552%	0.550%	0.552%	0.552%
SRRIP	4	0.553%	0.552%	0.550%	0.550%	0.550%	0.552%	0.552%	0.552%	0.552%
	8	0.551%	0.553%	0.551%	0.552%	0.552%	0.552%	0.550%	0.550%	0.553%
	16	0.553%	0.550%	0.551%	0.552%	0.552%	0.553%	0.551%	0.552%	0.552%

Figure B.7 – L2 cache miss rate vs. associativity - TSP

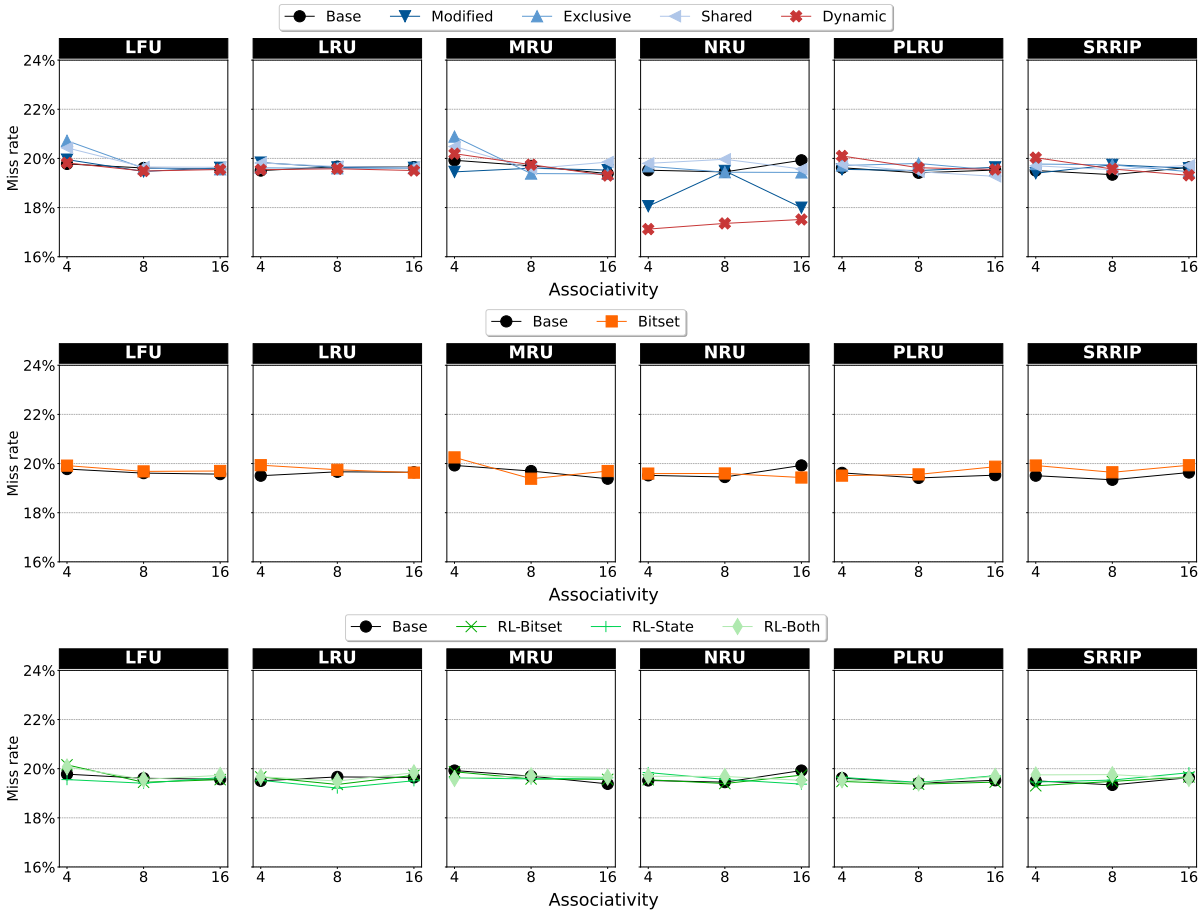


Table B.7 – L2 cache miss rate vs. associativity - TSP

Policy	Ways	Base	Modified	Exclusive	Shared	Dynamic	Bitset	RL-Bitset	RL-State	RL-Both
LFU	4	19.776%	19.951%	20.719%	20.431%	19.817%	19.915%	20.156%	19.556%	20.072%
	8	19.610%	19.471%	19.602%	19.651%	19.488%	19.680%	19.447%	19.408%	19.566%
	16	19.570%	19.619%	19.557%	19.639%	19.548%	19.697%	19.554%	19.636%	19.726%
LRU	4	19.505%	19.830%	19.616%	19.814%	19.542%	19.933%	19.664%	19.527%	19.670%
	8	19.664%	19.634%	19.599%	19.672%	19.578%	19.747%	19.357%	19.211%	19.497%
	16	19.645%	19.619%	19.608%	19.632%	19.507%	19.630%	19.731%	19.508%	19.834%
MRU	4	19.924%	19.453%	20.887%	20.484%	20.193%	20.250%	19.876%	19.625%	19.599%
	8	19.695%	19.600%	19.383%	19.581%	19.744%	19.383%	19.582%	19.582%	19.711%
	16	19.382%	19.525%	19.370%	19.851%	19.303%	19.694%	19.565%	19.630%	19.654%
NRU	4	19.520%	18.057%	19.679%	19.799%	17.124%	19.592%	19.549%	19.842%	19.692%
	8	19.452%	19.492%	19.438%	19.965%	17.356%	19.596%	19.396%	19.564%	19.684%
	16	19.924%	17.996%	19.430%	19.568%	17.516%	19.432%	19.736%	19.372%	19.525%
PLRU	4	19.620%	19.555%	19.711%	19.767%	20.100%	19.509%	19.480%	19.646%	19.542%
	8	19.414%	19.506%	19.795%	19.459%	19.621%	19.558%	19.369%	19.441%	19.415%
	16	19.527%	19.636%	19.516%	19.270%	19.550%	19.871%	19.453%	19.715%	19.688%
SRRIP	4	19.506%	19.398%	19.771%	19.701%	20.030%	19.919%	19.304%	19.458%	19.747%
	8	19.339%	19.740%	19.739%	19.542%	19.570%	19.644%	19.481%	19.536%	19.758%
	16	19.634%	19.602%	19.459%	19.718%	19.312%	19.931%	19.667%	19.835%	19.599%